Track B: Python for data science

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 4

B0 Start here: foundations for data science 5

B0-L1 Python essentials for data work 5

B0-L2 pandas essentials 9

B0-L3 The statistics you need for this track 13

B1 Project setup 17

B1-L1 One pyproject.toml and a src layout 17

B1-L2 One virtual environment per project, and python -m pip 22

B1-L3 ruff and mypy in daily work 26

B2 Code design 31

B2-L1 Pure functions, side effects at the edges 31

B2-L2 Dataclasses as configuration objects 35

B2-L3 When a class earns its place 40

B3 Clean pandas 45

B3-L1 Dtypes in pandas 3, nullable and string 45

B3-L2 Vectorise instead of apply 49

B3-L3 Method chaining and Copy-on-Write 54

B4 Polars and DuckDB 59

B4-L1 Polars expressions and lazy frames 59

B4-L2 DuckDB, SQL over files and DataFrames 63

B4-L3 Choosing pandas, Polars, DuckDB or Spark 67

B5 Data validation 72

B5-L1 DataFrame schemas with pandera 72

B5-L2 pydantic models for config and input records 76

B5-L3 Validate at the boundaries, with severity 81

B6 Reproducibility 87

B6-L1 Seeds and random generators 87

B6-L2 Record the data, config and code behind every result 91

B6-L3 Pinned dependencies and lock files 95

B7 Exploratory analysis done well 99

B7-L1 Profile a dataset before you analyse it 99

B7-L2 Plot before you summarise 103
B7-L3 Write assumptions down, then turn them into checks 107
B8 Statistical correctness 112
B8-L1 Survey weights and the effective sample size 112
B8-L2 Confidence intervals and effect sizes 116
B8-L3 Many tests, and the false positives they bring 120
B9 Experimental design 125
B9-L1 Power and sample size, before fieldwork 125
B9-L2 Analyse an A/B test the way it was randomised 129
B9-L3 Peeking, novelty and other ways tests mislead 133
B10 Feature engineering without leakage 139
B10-L1 Target leakage, features that know the answer 139
B10-L2 Fit on training data only 143
B10-L3 Time-safe features and point-in-time joins 148
B11 Machine learning workflow 153
B11-L1 Train, validation and test, split by the right unit 153
B11-L2 Pipeline and ColumnTransformer 156
B11-L3 Cross-validation and honest hyperparameter search 161
B12 Model evaluation 166
B12-L1 Choose the metric from the decision 166
B12-L2 Calibration, when the probability itself matters 170
B12-L3 Imbalanced classes and error analysis by segment 173
B13 Interpretability and fairness 178
B13-L1 Permutation importance on held-out data 178
B13-L2 SHAP values, and what they do not tell you 182
B13-L3 Performance by subgroup, and the fairness trade-offs 186
B14 Experiment tracking with MLflow 191
B14-L1 Track every run with MLflow 191
B14-L2 Registry versions and aliases, not stages 195
B14-L3 Reproduce a run from its record 199
B15 Embeddings and clustering 204
B15-L1 Sentence embeddings for open-ended answers 204
B15-L2 HDBSCAN, clusters with a noise label 208
B15-L3 Validate clusters before you name them 212
B16 PyTorch hygiene 216
B16-L1 Choose the device once, move model and data to it 216
B16-L2 A training loop that does what you think 220
B16-L3 Reproducible PyTorch runs and safe checkpoints 225

B17 Testing and performance 230
B17-L1 Unit tests for data code that catch real bugs 230
B17-L2 Property-based tests with Hypothesis 234
B17-L3 Measure before you optimise 238
B19 Testing in depth 243
B19-L1 How pytest finds, runs and reports your tests 243
B19-L2 Fixtures, shared setup without shared state 248
B19-L3 Parametrize, one test and many readable cases 253
B19-L4 Testing pandas and NumPy code 258
B19-L5 Testing statistical and random code 263
B19-L6 Testing machine learning pipelines 267
B19-L7 Isolate I/O with fakes, monkeypatch and mocks 272
B19-L8 Coverage, markers and a test suite you can trust in CI 277
B18 Documentation and communication 284
B18-L1 Docstrings and READMEs that answer the next person's questions 284
B18-L2 Model cards that state limits as clearly as scores 288
B18-L3 Decision logs, so the reasons outlive the meeting 291
Track B best-practice checklist 296

Appendix: solutions and answer keys 297

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B0 Start here: foundations for data science

Roadmap: Prerequisite for all months. The Python, pandas and statistics this track assumes.

B0-L1 Python essentials for data work

Concept

This track assumes you can read and write everyday Python 3.11 or later. Here is the core it relies on.

Values and collections. Numbers are `int` or `float`, text is `str`, and `True` and `False` are `bool`. `None` means "no value". A `list` holds items in order, `[1.5, 2.0]`. A `dict` maps keys to values, `{"R1": 1.5, "R2": 2.0}`. A `tuple` is a fixed group, such as `(lower, upper)`.

Functions take inputs and return a result. Type hints say what goes in and out, and a docstring says what the function does:

```python
def mean_weight(weights: list[float]) -> float | None:
    """Average weight, or None for an empty list."""
    if not weights:
        return None
    return sum(weights) / len(weights)
```

Loops and comprehensions. `for rid, w in weights.items():` walks through a dict's keys and values together. A comprehension builds a new list in one line: `[w for w in weights if w > 0]`.

f-strings put values into text: `f"{name}: {share:.1%}"` prints `North: 12.5%`. The part after the colon sets the format.

Truthiness. In an `if`, Python treats `0`, `0.0`, `""`, empty collections and `None` as false. That is handy for "is this list empty?", and dangerous for numbers. A share of `0.0` is real data, so test for a missing value with `is None`, never with `if not share`.

Errors. `raise ValueError("negative weight for R2")` stops the function with a message. `try: ... except ValueError:` catches it where you can handle it. Raise early, with a message that names the bad input.

Imports. `import math` or `from pathlib import Path` bring in standard library modules. Install other packages, such as pandas, with pip (lesson B1-L2).

Why it matters

A brand tracker counted brands "with no data" using `if not share:`. Brands with a real 0% share counted as missing, so the report said 14 brands lacked data when only 9 did. The client asked why five brands had vanished from the chart. One `is None` fixed both the count and the chart.

Worked example: weak code

```
1 weights = {"R1": 1.5, "R2": 0.0, "R3": None}
2 total = 0
3 n = 0
4 for k in weights:
2     if weights[k]:
6         total = total + weights[k]
7         n = n + 1
3 print("mean", total / n)
```

1. **Line 1.** Data at module level: nothing else can reuse or test this logic.
2. **Line 5.** Truthiness skips the real 0.0 weight as well as the missing one.
3. **Line 8.** Prints instead of returning, and divides by zero when every weight is missing.

Worked example: strong code

```
1 def mean_weight(weights: dict[str, float | None]) -> float | None:
2     """Mean of the recorded weights, or None when none is recorded."""
2     recorded = [w for w in weights.values() if w is not None]
3     if not recorded:
5         return None
6     return sum(recorded) / len(recorded)
7
8
4 print(f"mean: {mean_weight({'R1': 1.5, 'R2': 0.0, 'R3': None}):.2f}")
```

1. **Line 1.** Type hints show that weights may be missing and that the result may be None.
2. **Line 3.** A comprehension that keeps 0.0 and drops only missing values.
3. **Line 4.** Truthiness is right here: it asks whether the list is empty.
4. **Line 9.** An f-string with a format: two decimals, mean: 0.75.

Put logic in typed functions that return results, use comprehensions to filter, and test for missing values with `is None`.

Recap card

- **Test for a missing number** if value is None. Truthiness also treats 0 and 0.0 as false.
- **Filter a list in one line** A comprehension: [w for w in weights if w > 0].

- **Show 0.125 as 12.5%** f"{share:.1%}".
- **Walk through a dict** for key, value in d.items():
- **Signal bad input** raise ValueError("message naming the input"). Catch it with try/except where you can handle it.

Practice

Exercise 1. Multiple choice (level 1)

`share` is a float, or `None` when no answer was recorded. Which condition is true only when the share is missing?

- a) `if not share:`
- b) `if share is None:`
- c) `if share == 0:`
- d) `if share < 0.01:`

Why?

- r1) Only None is None. A real 0.0 share is a value, and `is None` leaves it alone.
- r2) `is` is faster than `not`.

Exercise 2. Predict the output (level 1)

What does this print?

```
1 weights = {"R1": 1.5, "R2": 0.0, "R3": 2.0}
2 kept = [rid for rid, w in weights.items() if w > 0]
3 print(kept, len(weights))
```

Exercise 3. Fill the gap (level 2)

Complete the function so its type hint allows a missing share, and its label shows a share of 0.125 as `North: 12.5%`.

```
1 def label(name: str, share: float | _____(1)) -> str:
2     if share is None:
3         return f"{name}: no data"
4     return f"{name}: {share:_____(2)}"
```

Exercise 4. Write the function (level 3)

Write `summarise_weights(weights: dict[str, float | None]) -> dict[str, float | int | None]`. Keys are respondent IDs. Missing weights are `None`. Return a dict with:

- `"n"`: the number of recorded weights, including 0.0,
- `"total"`: their sum (0.0 when none is recorded),

- `"mean"`: total divided by n, or `None` when n is 0.

Raise `ValueError` naming the respondent ID when any weight is negative.

Your code must pass these checks:

- Skip None, count and sum the rest: n 2, total 4.0, mean 2.0.
- A weight of 0.0 is recorded, so it counts in n. Test for missing values with `is None`.
- With no recorded weights, return n 0, total 0.0 and mean None, without dividing by zero.
- Raise ValueError that names the respondent with the negative weight.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Count, total and mean of the recorded weights.
def summarise_weights(
    weights: dict[str, float | None],
) -> dict[str, float | int | None]: ...
```

Starter code:

```python
def summarise_weights(weights):
    total = 0
    n = 0
    for rid in weights:
        if weights[rid]:
            total += weights[rid]
            n += 1
    return {"n": n, "total": total, "mean": total / n}
```

Exercise 5. Spot the bug (level 3)

This function reports 2 brands with no data for `{"A": 0.0, "B": None, "C": 0.4}`. Only one brand has no data. Tap the line with the bug, then fix it.

```python
def count_missing(shares: dict[str, float | None]) -> int:
    """Number of brands with no share recorded."""
    missing = 0
    for brand, share in shares.items():
        if not share:
            missing += 1
    return missing
```

Write down the line number of the bug. Then write the fix.

Sources

- 0, 0.0, an empty string, an empty list and None are falsy; "0" is truthy; f"{0.125:.1%}" gives 12.5%. (verified: Run on Python 3.11 and 3.13.)

B0-L2 pandas essentials

Concept

pandas holds tables as **DataFrames**: named columns over a row **index**. One column is a **Series**. The examples use pandas 3.0, where text columns get the `str` dtype.

```
import pandas as pd

df = pd.read_csv("respondents.csv")
```

Select columns. `df["w"]` returns one column as a Series. `df[["id", "w"]]` returns a smaller DataFrame.

Filter rows. A comparison on a column gives a Series of `True` and `False`, a **mask**. Indexing with it keeps the `True` rows: `df[df["w"] > 0]`. Combine masks with `&` (and), `|` (or) and `~` (not), and put **each condition in brackets**: `df[(df["w"] > 0) & df["in_scope"]]`. Python's `and` raises an error on a Series.

Add columns. `df.assign(w2=df["w"] * 2)` returns a new DataFrame with the extra column and leaves `df` unchanged.

Missing values show as `NaN` or `<NA>`. `df["w"].isna()` marks them. `sum()` and `mean()` skip them.

Group and summarise. `df.groupby("region")["w"].sum()` adds up the weights per region. For several results at once, name each one:

```
df.groupby("region", as_index=False).agg(
    n=("w", "size"),        # rows, missing or not
    recorded=("w", "count"),  # non-missing values
    total=("w", "sum"),
)
```

`as_index=False` keeps `region` as a normal column.

Join tables. `df.merge(regions, on="region_code", how="left")` adds the columns of `regions` to every row of `df`. `how="left"` keeps rows with no match, filling the new columns with missing values. The default, `how="inner"`, drops them.

Tidy the result. `sort_values("region")` orders rows, and `reset_index(drop=True)` renumbers them 0, 1, 2.

Why it matters

An analyst joined respondents to a region lookup with the default inner merge. 312 respondents had a region code missing from the lookup, and

they disappeared from every regional table without an error. The national total no longer matched the sum of the regions. A left merge, followed by a count of rows with a missing region, shows the problem in one line.

Worked example: weak code

```
1  import pandas as pd
2
3  df = pd.read_csv("respondents.csv")
1  df = df.merge(pd.read_csv("regions.csv"), on="region_code")
5  out = {}
2  for i in range(len(df)):
7      r = df.iloc[i]["region"]
3      out[r] = out.get(r, 0) + df.iloc[i]["w"]
```

1. **Line 4.** The default inner merge silently drops respondents with an unknown region code.
2. **Line 6.** A Python loop over rows: slow, and it hides what the code computes.
3. **Line 8.** A missing weight (NaN) turns that region's total into NaN.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def region_totals(respondents: pd.DataFrame, regions: pd.DataFrame) ->
     pd.DataFrame:
5      """Respondents and weight total per region, keeping unknown codes."""
1      joined = respondents.merge(regions, on="region_code", how="left")
7      return (
2          joined.groupby("region", as_index=False, dropna=False)
3          .agg(n=("w", "size"), total_weight=("w", "sum"))
10         .sort_values("region")
11         .reset_index(drop=True)
12     )
```

1. **Line 6.** A left merge keeps every respondent. Unknown codes get a missing region.
2. **Line 8.** dropna=False keeps the missing region as its own group, so nobody disappears.
3. **Line 9.** Named results in one pass. sum skips missing weights.

Filter with bracketed masks, summarise with groupby and named aggregations, and merge with how="left" when every row must survive.

Recap card

- **Filter rows on two conditions** df[(df["w"] > 0) & df["in_scope"]]. Each comparison in brackets, & not and.

- **Count rows against count values** size counts rows, count counts non-missing values. sum and mean skip missing values.
- **Several results per group** df.groupby("region", as_index=False).agg(n=("w", "size"), total=("w", "sum")).
- **Join and keep every row** df.merge(other, on="key", how="left"). The default inner merge drops rows with no match.

Practice

Exercise 1. Multiple choice (level 1)

Which line keeps only rows with a positive weight that are in scope?

- a) `df[(df["w"] > 0) & df["in_scope"]]`
- b) `df[df["w"] > 0 and df["in_scope"]]`
- c) `df[df["w"] > 0, df["in_scope"]]`
- d) `df.filter(df["w"] > 0)`

Why?

- r1) `&` combines two masks row by row, and the brackets make the comparison happen first.
- r2) `&` is shorter to type than `and`.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

What you need to know.

`series.to_dict()` turns a Series into a plain dict, with the index labels as keys. After a groupby, the labels are the group names:

```
pd.Series([3, 5], index=["a", "b"]).to_dict()   # {'a': 3, 'b': 5}
```

```
import pandas as pd

df = pd.DataFrame({"region": ["N", "N", "S"], "w": [1.0, None, 2.0]})
print(df.groupby("region")["w"].sum().to_dict())
```

Exercise 3. Fill the gap (level 2)

Add region names to every respondent, and keep respondents whose region code is missing from the lookup.

```
joined = respondents.merge(regions, on="_____(1)", how="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `region_summary(df: pd.DataFrame) -> pd.DataFrame`. `df` has columns `region` and `w`, where `w` can be missing. Return one row per region, sorted by region, with a default index and the columns:

- `region`,
- `n`: rows in the region, including missing weights,
- `recorded`: rows with a weight,
- `total_weight`: the sum of the weights, 0.0 when none is recorded.

Do not loop over rows.

Your code must pass these checks:

- Return the columns region, n, recorded and total_weight, in that order.
- Count all rows with size, recorded weights with count, and add them with sum. Sort by region and reset the index.
- Leave the input frame unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Rows, recorded weights and weight total per region.
def region_summary(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def region_summary(df):
    out = {}
    for i in range(len(df)):
        r = df.iloc[i]["region"]
        out[r] = out.get(r, 0) + df.iloc[i]["w"]
    return pd.DataFrame({"region": list(out), "total_weight":
    list(out.values())})
```

Exercise 5. Spot the bug (level 3)

For a frame where the only in-scope row has weight 3.0, this function returns two rows, 2.0 and 3.0. Tap the line with the bug, then fix it.

```
import pandas as pd


def in_scope_positive(df: pd.DataFrame) -> pd.DataFrame:
    """Rows that are in scope and have a positive weight."""
    return df[df["w"] > 0 & df["in_scope"]]
```

Write down the line number of the bug. Then write the fix.

Sources

- groupby sum skips NaN; named aggregations with size, count and sum give int64, int64 and float64; a left merge keeps unmatched rows and an inner merge drops them; df[df.w > 0 & df.in_scope] silently ignores in_scope. (verified: Run on pandas 3.0.6 and 3.0.2.)

B0-L3 The statistics you need for this track

Concept

Survey and model results are estimates from a **sample**, used to say something about a **population**. Five ideas carry most of this track.

Mean and spread. The mean is the average. The standard deviation (SD) measures how far values sit from it. The sample SD divides by n - 1. Watch the defaults: NumPy 2's `np.std(x)` divides by n, while pandas 3's `Series.std()` divides by n - 1. For `[2, 4, 4, 4, 5, 5, 7, 9]` they give 2.0 and 2.138.

Standard error (SE). Another sample would give a slightly different estimate. The SE says by how much, on average:

- for a mean: `SD / sqrt(n)`,
- for a share p: `sqrt(p * (1 - p) / n)`.

A share of 50% from 1,000 people has an SE of about 0.0158, 1.6 points. Four times the sample halves the SE.

Confidence interval. A 95% interval is roughly the estimate plus or minus 1.96 SEs: 50% plus or minus 3.1 points. Lesson B8-L2 shows better intervals for small samples and shares near 0 or 100%.

p-value. It is the probability of a difference at least as large as the one you saw, if there were no true difference. It is not the probability that your hypothesis is true. A p-value of 0.03 means "this would be unusual if nothing were going on", nothing more.

Effect size. A tiny difference becomes "significant" in a huge sample. Always report how big the difference is, with its interval, next to any p-value.

Survey weights change all of these. Lesson B8-L1 shows how.

Why it matters

A monthly report flagged every brand whose awareness changed with p < 0.05. With 20,000 respondents a month, a change of 0.6 points was

"significant", and the client reacted to noise-sized moves that no campaign could explain. Reporting each change with its interval, and agreeing in advance on the smallest change worth acting on, ended the false alarms.

Worked example: weak code

```
import numpy as np

scores = np.array([2, 4, 4, 4, 5, 5, 7, 9])
sd = np.std(scores)
print("p < 0.05, so the campaign worked")
```

1. **Line 4.** NumPy's default divides by n, the population SD, not the sample SD.
2. **Line 5.** A p-value says nothing about how big the effect is, or that the hypothesis is true.

Worked example: strong code

```
import math

import numpy as np

scores = np.array([2, 4, 4, 4, 5, 5, 7, 9])
sd = np.std(scores, ddof=1)
se = sd / math.sqrt(len(scores))
low, high = scores.mean() - 1.96 * se, scores.mean() + 1.96 * se
print(f"mean {scores.mean():.2f}, 95% interval {low:.2f} to {high:.2f}")
```

1. **Line 6.** ddof=1 divides by n - 1: the sample standard deviation.
2. **Line 7.** The standard error of the mean.
3. **Line 9.** The estimate with its uncertainty, not a bare yes or no.

Report an estimate with its standard error or interval, know which SD your library computes, and read a p-value for what it is.

Recap card

- **Standard error of a mean and of a share** Mean: SD / sqrt(n). Share: sqrt(p(1 - p) / n). Four times the sample halves the SE.
- **A rough 95% interval** Estimate plus or minus 1.96 standard errors.
- **What a p-value is** The probability of a difference at least this large if there were no true difference. Not the probability that a hypothesis is true.
- **np.std against pandas std** np.std divides by n (population). Series.std divides by n - 1 (sample). Use np.std(x, ddof=1) for the sample SD.

Practice

Exercise 1. Multiple choice (level 1)

An A/B test reports p = 0.03 for the difference in conversion. What does that mean?

- a) There is a 3% chance that the new page is no better.
- b) If the pages truly converted equally, a difference this large or larger would happen about 3% of the time.
- c) The new page converts 3% better.
- d) The result is 97% certain to be correct.

Why?

- r1) A p-value is computed assuming no real difference, and measures how surprising the data would be then.
- r2) A p-value is the probability that the hypothesis is false.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2, pandas 3.0)

```
import numpy as np
import pandas as pd

x = [2, 4, 4, 4, 5, 5, 7, 9]
print(np.std(x), round(pd.Series(x).std(), 3))
```

Exercise 3. Fill the gap (level 2)

Compute the standard error of a share `p` from `n` respondents, then the lower end of a 95% interval.

```
se = math.sqrt(p * (1 - p) / _____(1))
low = p - _____(2) * se
```

Exercise 4. Write the function (level 3)

Write `share_ci(successes: int, n: int, z: float = 1.96) -> tuple[float, float] | None`, the normal-approximation interval for a share. With `p = successes / n` and `se = sqrt(p * (1 - p) / n)`, return `(p - z * se, p + z * se)` clipped to [0, 1].

Return `None` when `n` is 0. Raise `ValueError` when `successes` is negative or above `n`.

Your code must pass these checks:

- 500 of 1,000 gives about (0.469, 0.531).

- Clip the interval to [0, 1]: 1 of 10 gives a lower end of 0.0, not a negative number.
- Return None when n is 0, instead of dividing by zero.
- Raise ValueError when successes is below 0 or above n.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Normal-approximation confidence interval for a share.
def share_ci(
    successes: int,
    n: int,
    z: float = 1.96,
) -> tuple[float, float] | None: ...
```

Starter code:

```python
import math


def share_ci(successes, n, z=1.96):
    p = successes / n
    se = math.sqrt(p * (1 - p) / n)
    return (p - z * se, p + z * se)
```

Exercise 5. Architecture design (level 3)

Match each question to the right tool.

Options: SD / sqrt(n), sqrt(p(1 - p) / n), Estimate plus or minus 1.96 SE, The size of the difference, with its interval.

1. How precise is the mean score of 100 respondents with SD 4?
2. How precise is a 50% awareness share from 1,000 respondents?
3. Give a range that is likely to contain the true share
4. With p < 0.001 from 50,000 respondents, does the change matter?

Sources

- np.std([2, 4, 4, 4, 5, 5, 7, 9]) is 2.0 and pd.Series(...).std() is 2.138; np.std with ddof=1 matches pandas. (verified: Run on NumPy 2.4.6 with pandas 3.0.2 and 3.0.6.)
- SE of a 50% share from 1,000 is 0.0158; from 4,000 it is 0.0079. (verified: Computed in Python 3.11.)

B1 Project setup

Roadmap: Prerequisite for all months. pyproject.toml, virtual environments, ruff, mypy and folder layout.

B1-L1 One pyproject.toml and a src layout

Concept

A data science project starts as a notebook and a CSV. By month three it has helper modules, tests, a model, and three people asking how to run it. The fix is structure from day one.

`pyproject.toml` is the one configuration file for a modern Python project. It holds:

- `[project]`: name, version, `requires-python`, and runtime `dependencies`.
- `[dependency-groups]` (PEP 735): named groups for development only, such as `test` or `lint`. They never ship with the package. Install one with `pip install --group test` (pip 25.1 and later).
- `[tool.*]` tables for tools: `[tool.ruff]`, `[tool.mypy]`, `[tool.pytest.ini_options]`.

One file replaces `setup.py`, `requirements.txt`, `setup.cfg`, `.flake8` and `pytest.ini`.

The src layout puts importable code under `src/`:

```
reach_model/
├── pyproject.toml
├── src/reach_model/        # importable package
│   ├── __init__.py
│   ├── features.py
│   └── train.py
├── tests/                  # pytest, imports reach_model
├── notebooks/              # exploration; imports reach_model, holds no logic
└── data/                   # small samples only; real data stays out of Git
```

With `src/`, Python can only import your package after you install it (`pip install -e .`). Tests then run against the installed package, exactly as a user or a job would. Without it, tests silently import whatever sits in the working directory, and "works on my machine" follows.

Notebooks explore and present. Functions that matter move into `src/`, where tests and type checks reach them. A notebook that defines the feature pipeline inline cannot be tested or reused by the training job.

Pin `requires-python` to the versions you test, for example `">=3.11"`, so pip refuses an interpreter you never tried.

Why it matters

A team trains a churn model from `train.ipynb`, which defines `build_features()` in a cell. The scoring job copies the function into `score.py`. A month later someone fixes a leakage bug in the notebook version only. Training and scoring now compute features differently, and live accuracy drops 9 points while offline metrics look fine.

With `build_features()` in `src/churn/features.py`, imported by both the notebook and the job, one fix reaches both, and one test protects it.

Worked example: weak code

```
churn/
├── train.ipynb            # defines build_features() in cell 7
├── score.py               # copy of build_features(), edited later
├── utils.py
├── requirements.txt       # pandas, scikit-learn (no versions)
├── setup.cfg              # flake8 settings
└── data.csv               # 2 GB client extract, committed
```

1. **Line 2.** Logic trapped in a notebook: untestable, and copied instead of imported.
2. **Line 3.** Two versions of one function drift apart.
3. **Line 5.** No pins: every install resolves different versions.
4. **Line 7.** Client data in Git: a size problem and a data-protection problem.

Worked example: strong code

```
churn/
├── pyproject.toml         # project, dependency groups, ruff, mypy,
  pytest
├── src/churn/
│   ├── __init__.py
│   ├── features.py        # build_features(), used by training and
  scoring
│   ├── train.py
│   └── score.py
├── tests/
│   └── test_features.py
├── notebooks/
│   └── 01_explore.ipynb   # imports churn.features
```

```
└── data/
    └── sample_200_rows.csv
```

1. **Line 2.** One file configures the project and every tool.
2. **Line 5.** One definition, imported everywhere it is needed.
3. **Line 9.** Tests import the installed package, as the jobs do.
4. **Line 13.** A small, anonymised sample for tests. Real data stays in governed storage.

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

Recap card

- **What goes in pyproject.toml?** [project] with dependencies, [dependency-groups] for development tools, and [tool.*] settings for ruff, mypy and pytest.
- **Install a dependency group** pip install --group dev (pip 25.1 and later).
- **Why the src layout?** Only the installed package is importable, so tests run against it exactly as jobs do.
- **Where does shared logic live?** In the package under src/, imported by notebooks, jobs and tests. Never copied.

Practice

Exercise 1. Multiple choice (level 1)

Where do you declare `pytest` and `ruff` so they install for developers but never ship with the package?

- a) `[project] dependencies`
- b) `[dependency-groups]`
- c) `[tool.pytest.ini_options]`
- d) A comment in README.md

Why?

- r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.
- r2) Dependency groups are only for test tools.

Exercise 2. Fill the gap (level 2)

Complete the `pyproject.toml` so runtime needs and developer tools are separate.

What you need to know.

A `pyproject.toml` file is written in TOML. A name in square brackets starts a **table**, a section of settings. Each line below it is `key = value`. A list goes in square brackets.

```toml
[project]
name = "survey-tools"
requires-python = ">=3.11"
dependencies = ["numpy>=2.0,<3", "httpx>=0.28"]

[dependency-groups]
dev = ["pytest>=9"]
```

Read it as: the package `survey-tools` needs Python 3.11 or later, and NumPy and httpx to run. The `dev` group lists tools only developers install.

```toml
[project]
name = "churn"
requires-python = ">=3.11"
_____(1) = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]

[_____(2)]
dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

Exercise 3. Write the function (level 3)

Write `unpinned(pyproject_text)`. Parse the TOML with the standard library and return, sorted, every entry in `[project].dependencies` that has no version constraint at all (no `=`, `<`, `>`, `~` or `!`). Return an empty list when there is no dependencies key.

What you need to know.

Pins and constraints. A dependency string is a package name, optionally followed by a version constraint: `numpy>=2.0,<3` allows 2.x only, `requests==2.32.0` pins one version, `pydantic~=2.12` means 2.12 or a later 2.x. A bare name such as `httpx` has no constraint, so every install takes whatever version is newest. That entry is **unpinned**. Constraints use the characters `=`, `<`, `>`, `~` and `!`.

Reading TOML. `tomllib` is in the standard library from Python 3.11. `tomllib.loads(text)` turns TOML text into nested dicts, one per table:

```python
import tomllib

data = tomllib.loads('[project]\nname =
    "x"\ndependencies = ["numpy>=2.0", "httpx"]\n')
data["project"]["dependencies"]   # ['numpy>=2.0', 'httpx']
data.get("tool", {})              # {}: the table is missing
```

Your code must pass these checks:

- numpy and requests have no constraint. Constrained entries must not be listed.
- Without a dependencies key, return an empty list.
- An extra in brackets is not a version constraint. uvicorn[standard] is still unpinned.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
CONSTRAINT_CHARS = ...

# Runtime dependencies with no version constraint.
def unpinned(pyproject_text: str) -> list[str]: ...
```

Starter code:

```python
import tomllib


def unpinned(pyproject_text: str) -> list[str]:
    ...
```

Exercise 4. Multiple choice (level 4)

With a src layout, a colleague runs `pytest` straight after cloning and gets `ModuleNotFoundError: No module named 'churn'`. What is the right fix?

- a) Add `sys.path.insert(0, 'src')` at the top of every test file.
- b) Run `pip install -e .` in the project's virtual environment.
- c) Move the package out of src/ so imports work from the root.
- d) Copy the package into tests/.

Why?

- r1) The src layout is designed so that only the installed package is importable, which makes tests use it like any user would.
- r2) pytest cannot find packages in folders.

Exercise 5. Architecture design (level 5)

Place each piece of a modelling project.

Options: src/<package>/, A notebook cell, tests/, Outside the repository.

1. build_features(), used by training and by the nightly scoring job
2. Plots comparing feature distributions for a one-off question
3. A check that build_features() gives the right output for 5 hand-made rows

4. The full client data extract

Sources

- PEP 735 defines [dependency-groups]; pip 25.1 (April 2025) added --group. (search-only: https://peps.python.org/pep-0735/ and https://ichard26.github.io/blog/2025/04/whats-new-in-pip-25.1/)
- pip 26.2.1 lists --group in `pip install --help`; tomllib is in the standard library from Python 3.11. (verified: Checked in this repository's build environment.)

B1-L2 One virtual environment per project, and python -m pip

Concept

A **virtual environment** is a folder with its own Python interpreter link and its own `site-packages`. Packages you install there do not touch the system Python or any other project.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e . --group dev
```

Rules that prevent most "it worked yesterday" problems:

- **One environment per project.** Two projects that share an environment share their version conflicts. Upgrading pandas for one breaks the other.
- **Never install into the system Python.** Operating systems and other tools depend on it.
- **Use `python -m pip`, not bare `pip`.** Bare `pip` runs whichever `pip` comes first on your PATH, which may belong to a different interpreter. `python -m pip` always installs into the interpreter you are about to run.
- **Keep `.venv/` out of Git.** It is large, machine-specific and rebuildable. Commit `pyproject.toml` and a lock file instead (lesson B6-L3).

In code, `sys.prefix != sys.base_prefix` is True inside a virtual environment. A script that must not run against the system Python can check it and stop.

Sometimes one project needs **two** environments, because two dependencies cannot live together. A real case from this course's own build: PySpark 4.0.4 fails to import its pandas integration with pandas 3.x, so the Spark test harness runs in its own environment pinned to pandas 2.x, while the rest

uses pandas 3. Separate environments, each fully pinned, beat one environment held together by guesswork.

Tools such as `uv` create and sync environments much faster, and conda manages non-Python libraries. The principles are the same.

Why it matters

An analyst runs `pip install statsmodels` in a terminal, then `python analysis.py` in the same terminal, and gets `ModuleNotFoundError`. `pip` belonged to one Python, and `python` to another. They try `sudo pip install`, which upgrades NumPy in the system Python and breaks a command-line tool the operating system uses. Two hours are gone.

`python -m venv .venv`, activation, and `python -m pip install` make the interpreter that installs and the interpreter that runs the same one.

Worked example: weak code

```
$ pip install pandas scikit-learn
$ sudo pip install --upgrade numpy
$ python train.py
ModuleNotFoundError: No module named 'sklearn'
$ git add .venv/ data/ train.py
```

1. **Line 1.** Bare pip, no environment: installs into whichever Python owns this pip.
2. **Line 2.** Changes the system Python that the operating system relies on.
3. **Line 4.** The python that runs is not the one pip installed into.
4. **Line 5.** Commits a machine-specific environment folder and data.

Worked example: strong code

```
$ python -m venv .venv
$ source .venv/bin/activate
(.venv) $ python -m pip install --upgrade pip
(.venv) $ python -m pip install -e . --group dev
(.venv) $ python -c "import sys; print(sys.prefix != sys.base_prefix)"
True
(.venv) $ python train.py
```

1. **Line 1.** A project-local environment, listed in .gitignore.
2. **Line 4.** The same interpreter installs and runs. Dependencies come from pyproject.toml.
3. **Line 6.** Proof that the active interpreter is the virtual environment.

One environment per project, created from pyproject.toml, with python -m pip so installing and running use the same interpreter.

Recap card

- **Create and use a project environment** python -m venv .venv, activate it, then python -m pip install -e . --group dev.
- **Why python -m pip?** It installs into the interpreter that python runs. Bare pip may belong to another Python.
- **Detect a virtual environment in code** sys.prefix != sys.base_prefix.
- **Two dependencies that cannot coexist** Two pinned environments and two dependency groups, each tested in CI.

Practice

Exercise 1. Multiple choice (level 1)

Why prefer `python -m pip install x` over `pip install x`?

- a) It is faster.
- b) It installs into the interpreter that `python` runs, whatever is first on PATH.
- c) It installs packages globally for all users.
- d) It skips dependency resolution.

Why?

- r1) `python -m pip` runs pip as a module of that exact interpreter.
- r2) Bare pip is deprecated.

Exercise 2. Predict the output (level 2)

You run this with `.venv/bin/python`, a virtual environment created by `python -m venv .venv`. What does it print?

```
import sys

print(sys.prefix != sys.base_prefix)
```

Exercise 3. Fill the gap (level 2)

Create the environment, then install the project in editable mode with its dev group.

```
python -m _____(1) .venv
source .venv/bin/activate
python -m pip install _____(2) . --group dev
```

Exercise 4. Write the function (level 3)

Write `require_virtualenv()`. It raises `RuntimeError` with the message `"activate the project's virtual environment first"` when the running

interpreter is not inside a virtual environment, and returns `None` otherwise. Read `sys.prefix` and `sys.base_prefix` when the function is called, not at import time.

Your code must pass these checks:

- Inside a venv (prefixes differ) the function must return None.
- Outside a venv (prefixes equal) raise RuntimeError that mentions the virtual environment.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Stop when not running inside a virtual environment.
def require_virtualenv() -> None: ...
```

Starter code:

```python
import sys


def require_virtualenv() -> None:
    ...
```

Exercise 5. Architecture design (level 5)

Your project needs pandas 3.0 for analysis, and a Spark test harness on PySpark 4.0.4, which fails to import its pandas integration with pandas 3.x. Choose the set-up.

Options: Two environments: dev (pandas 3) and spark (pandas 2), each pinned, One environment with the latest of everything, One environment, everything downgraded to pandas 2.

1. Environments
2. Where the difference is recorded
3. CI

Sources

- sys.prefix differs from sys.base_prefix inside a venv. (verified: Python venv documentation; checked in this repository's virtual environments.)
- PySpark 4.0.4 fails to import assertDataFrameEqual with pandas 3.x. (verified: Reproduced in this repository (VERSIONS.md).)

B1-L3 ruff and mypy in daily work

Concept

Two tools catch most mistakes before a test runs.

ruff lints and formats. On a ten-line sample it reported three real problems:

```
m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
m.py:1:8: F401 [*] `os` imported but unused
m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

B905 matters for data work: `zip(flags, weights)` silently stops at the shorter list, so a missing weight drops a respondent without an error. `zip(..., strict=True)` raises instead.

mypy checks types. On the same file, strict mode found:

```
m.py:14: error: Unsupported operand types for * ("None" and "int")
    [operator]
m.py:17: error: Function is missing a type annotation  [no-untyped-def]
```

Line 14 multiplies a `float | None` by 100. When the total weight is zero, the function returns `None` and the report crashes. mypy found the zero-weight bug without any test data.

Daily habits:

- Configure both once in `pyproject.toml` (`[tool.ruff]` with the rule sets you want, `[tool.mypy]` with `strict = true`).
- Run `ruff check --fix .` and `ruff format .` before each commit. `--fix` applies only safe fixes.
- Run `mypy` on `src/` and `tests/`. Fix errors by handling the case, not by adding `# type: ignore`.
- Run the same commands in CI (lesson A10-L3), so the rules are not optional.

Libraries without type information show `import-untyped` or missing-stub notes. Install the stub package if one exists, or add a targeted `[[tool.mypy.overrides]]` with `ignore_missing_imports = true` for that module only.

Why it matters

A reporting script formats a weighted share as `f"{share * 100:.1f}%"`. For eleven months every subgroup has respondents. Then a new region with no fieldwork yet appears in the sample frame, its total weight is zero,

`weighted_share` returns `None`, and the monthly client pack fails to build on deadline day.

mypy flags `None * 100` the first time it runs on the file. The fix is two lines, written months before the failure.

Worked example: weak code

```
import os
import pandas as pd


def weighted_share(flags, weights):
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights) if f == 1) / total


def report(flags, weights):
    return f"{weighted_share(flags, weights) * 100:.1f}%"
```

1. **Line 1.** F401: unused import.
2. **Line 5.** No hints: mypy cannot see that the function can return None.
3. **Line 9.** B905: zip silently truncates to the shorter list.
4. **Line 13.** Crashes with TypeError when the share is None.

Worked example: strong code

```
def weighted_share(flags: list[int], weights: list[float]) -> float | None:
    """Weighted share of flagged rows, or None when the total weight is 0."""
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights, strict=True) if f == 1) / total


def report(flags: list[int], weights: list[float]) -> str:
    """Share as a percentage string, or 'n/a' when undefined."""
    share = weighted_share(flags, weights)
    if share is None:
        return "n/a"
    return f"{share * 100:.1f}%"
```

1. **Line 1.** The return type admits None, so mypy makes every caller handle it.
2. **Line 6.** Unequal lengths raise ValueError instead of dropping rows.
3. **Line 12.** The case mypy flagged, handled on purpose.

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

Recap card

- **Why does ruff flag zip() without strict= (B905)?** zip silently truncates to the shorter input. strict=True raises ValueError instead.
- **mypy says None * int is unsupported. The fix?** Handle the None case in code. Never # type: ignore it away.
- **Daily commands before a commit** ruff check --fix ., ruff format ., mypy.
- **Missing stubs for one library** Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
flags = [1, 0, 1]
weights = [2.0, 1.0]
print(list(zip(flags, weights)))
```

Exercise 2. Multiple choice (level 2)

mypy reports `Unsupported operand types for * ("None" and "int")` on `share * 100`. What is the right fix?

- a) Add `# type: ignore` to the line.
- b) Handle the None case before the multiplication.
- c) Change the return type to `float` and keep returning None.
- d) Turn off strict mode.

Why?

- r1) mypy found a real path where the value is None. Handling it removes the crash.
- r2) mypy errors are style issues.

Exercise 3. Fill the gap (level 2)

Complete the pre-commit routine: apply safe lint fixes, format, then type-check.

```
ruff check _____(1) .
ruff _____(2) .
_____(3)
```

Exercise 4. Refactor (level 4)

Make this pass `ruff` (no B905) and `mypy --strict`: add type hints, keep the None result for zero weight, make unequal lengths raise, and make `report` return `"n/a"` instead of crashing.

Your code must pass these checks:

- The share of weight 3 out of 4 is 0.75.
- Zero weight: weighted_share returns None and report returns 'n/a'.
- report must format the share as a percentage with one decimal: 75.0%.
- Unequal lengths must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted share of flagged rows, or None when the total weight is 0.
def weighted_share(
    flags: list[int],
    weights: list[float],
) -> float | None: ...

# Share as a percentage string, or 'n/a' when undefined.
def report(flags: list[int], weights: list[float]) -> str: ...
```

Starter code:

```python
def weighted_share(flags, weights):
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights) if f == 1) / total


def report(flags, weights):
    return f"{weighted_share(flags, weights) * 100:.1f}%"
```

Exercise 5. Architecture design (level 5)

mypy reports `Skipping analyzing "shapely": module is installed, but missing library stubs or py.typed marker`. Choose the response for each case.

Options: Install the stubs in the dev group, Add [[tool.mypy.overrides]] with ignore_missing_imports for that module, Set ignore_missing_imports = true globally.

1. A stub package, types-shapely, exists on PyPI
2. No stubs exist for an internal library
3. A colleague proposes silencing all such messages for every library

Sources

- ruff reports I001, F401 and B905 and mypy --strict reports [operator] and [no-untyped-def] on the sample shown. (verified: ruff 0.16.9 and mypy 2.3.1 run in this repository.)
- zip(strict=True) raises ValueError on unequal lengths (Python 3.10+). (verified: Python standard library; run in Python 3.11.)

B2 Code design

Roadmap: Prerequisite for all months. Pure functions, dataclasses, when to use classes, configuration objects.

B2-L1 Pure functions, side effects at the edges

Concept

A **pure function** returns a value that depends only on its arguments, and changes nothing outside itself: no file writes, no global variables, no mutation of the inputs, no printing, no clock or random calls it does not receive.

```
1 def weighted_mean(values: list[float], weights: list[float]) -> float:
2     return sum(v * w for v, w in zip(values, weights, strict=True)) /
    sum(weights)
```

Pure functions are easy to test (inputs in, answer out), safe to reuse, and safe to run twice. Most analysis logic can be pure: cleaning, feature building, statistics, scoring.

Some work is not pure by nature: reading files, calling APIs, writing tables, logging, reading the clock. Push that work to the **edges** of the program, into a thin layer that gathers inputs, calls the pure core, and writes results.

```
1 read (impure) -> transform, compute (pure) -> write (impure)
```

Three side effects hide easily in data code:

- **Mutating an argument.** `df["x"] = ...` or `rows.append(...)` inside a function changes the caller's object. The caller did not ask for that. Return a new object instead: `df.assign(x=...)`, or build a new list.
- **Reading globals.** A function that reads a module-level `THRESHOLD` or `TODAY` gives different answers after someone changes it. Pass it in.
- **Hidden time and randomness.** `datetime.now()` or `np.random.rand()` inside a function makes its output untestable. Pass a date, or a `numpy.random.Generator`.

In pandas 3, Copy-on-Write is always on, so many operations already return new objects. Assigning a column with `df[col] = ...` still changes `df` itself, which is exactly the mutation to keep out of shared functions.

Why it matters

A feature function adds `age_group` to the DataFrame it receives, in place. The training script calls it, then passes the same DataFrame to a stratified split on `age_group`. Later, someone reorders the calls so the split runs first. The split fails with a KeyError on one laptop and works on another, depending on which notebook cells ran before.

A pure `add_age_group(df) -> DataFrame` that returns a new frame gives the same result in every order, and its test needs three rows.

Worked example: weak code

```
 1 import datetime
 2
 3 THRESHOLD = 0.5
 4
 5
 6 def score_respondents(df):
(1)     df["age"] = datetime.date.today().year - df["birth_year"]
(2)     df["high_reach"] = df["reach"] > THRESHOLD
(3)     df.to_csv("scored.csv", index=False)
(4)     print("scored", len(df))
11     return df
```

1. **Line 7.** Mutates the caller's frame and reads the clock: the answer changes on 1 January.
2. **Line 8.** Reads a module global: change THRESHOLD anywhere and every result changes.
3. **Line 9.** Writes a file from inside the computation.
4. **Line 10.** Output in the middle of logic, not in the program's edge.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
(1) def score_respondents(df: pd.DataFrame, as_of_year: int, threshold:
    float) -> pd.DataFrame:
 5     """Add age and high_reach. Returns a new frame; the input is
    unchanged."""
(2)     return df.assign(
 7         age=as_of_year - df["birth_year"],
 8         high_reach=df["reach"] > threshold,
 9     )
10
11
(3) def main(input_path: str, output_path: str, as_of_year: int, threshold:
    float) -> None:
13     """The edge: read, call the pure core, write."""
```

```
14      scored = score_respondents(pd.read_csv(input_path), as_of_year,
        threshold)
15      scored.to_csv(output_path, index=False)
```

1. **Line 4.** Everything the result depends on is a parameter.
2. **Line 6.** assign returns a new frame. The caller's data is not touched.
3. **Line 12.** Reading, writing and choosing the year happen here, at the edge.

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

Recap card

- **What makes a function pure?** Its result depends only on its arguments, and it changes nothing outside itself.
- **Three hidden side effects in data code** Mutating arguments, reading globals, reading the clock or randomness inside the function.
- **Return a new DataFrame with an extra column** df.assign(col=...). df["col"] = ... changes the caller's frame.
- **Program shape** Read at the edge, compute in a pure core, write at the edge.

Practice

Exercise 1. Multiple choice (level 1)

Which function is pure?

- a) `def add(rows, x): rows.append(x); return rows`
- b) `def share(n, total): return n / total if total else None`
- c) `def stamp(df): return df.assign(ts=pd.Timestamp.now())`
- d) `def save(df, path): df.to_csv(path)`

Why?

- r1) Its result depends only on its arguments, and it changes nothing outside itself.
- r2) It is the shortest.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3?

```
1  import pandas as pd
2
3
4  def add_flag(df):
5      df["flag"] = df["x"] > 1
```

```
    return df


original = pd.DataFrame({"x": [1, 2]})
result = add_flag(original)
print(list(original.columns))
```

Exercise 3. Refactor (level 3)

Make `add_age` pure: take the reference year as a parameter, return a new DataFrame, and leave the input unchanged. Keep the name `add_age` and the new column `age`.

Your code must pass these checks:

- age must be the given year minus birth_year.
- The input frame changed. Return df.assign(age=...) instead.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Return a copy of df with age = as_of_year - birth_year.
def add_age(
    df: pd.DataFrame,
    as_of_year: int,
) -> pd.DataFrame: ...
```

Starter code:

```
import datetime


def add_age(df):
    df["age"] = datetime.date.today().year - df["birth_year"]
    return df
```

Exercise 4. Spot the bug (level 4)

The second call returns a list containing the first call's flags. Tap the line, then fix the function without changing its behaviour otherwise.

```
def flag_low_bases(bases: dict[str, float], minimum: float, flags:
  list[str] = []) -> list[str]:
    """Names of subgroups whose base is below minimum."""
    for name, base in bases.items():
        if base < minimum:
            flags.append(name)
    return flags
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Sort each step of a weekly scoring script into the pure core or the edge.

Options: Edge, Pure core.

1. Read the respondent extract from storage
2. Compute features from the extract and a reference date
3. Score respondents with a model object passed in
4. Write scores to the output table and log the row count

Sources

- In pandas 3 Copy-on-Write is always on, but assigning a column changes the DataFrame it is called on. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository (predict-output exercise).)

B2-L2 Dataclasses as configuration objects

Concept

Analysis code needs settings: a minimum base size, a weight column, a list of waves. Passing them around as a loose `dict` has three costs. A typo in a key (`"min_bse"`) fails late or not at all. Nothing states which keys exist. Any function can change a value half way through a run.

A **dataclass** (Python 3.7 and later) turns the settings into a named, typed object.

```
from dataclasses import dataclass, field


@dataclass(frozen=True)
class ReportConfig:
    weight_col: str
    min_base: int = 50
    waves: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if self.min_base <= 0:
            raise ValueError(f"min_base must be positive, got
    {self.min_base}")
```

What you get:

- **Named fields.** `cfg.min_bse` raises `AttributeError` at once. Your editor and mypy see the fields.
- `frozen=True`**.** Assigning a field raises `dataclasses.FrozenInstanceError`. Settings stay fixed for the run. Use `dataclasses.replace(cfg, min_base=30)` to build a changed copy.

- `__post_init__`**.** Runs after the generated `__init__`. Put checks here so a bad config fails at load time, not in hour three.

Two traps:

- **Type hints are not checked at runtime.** `ReportConfig(weight_col="w", min_base="30")` builds without error. Convert or check types in `__post_init__`, or use pydantic (lesson B5-L2) when input comes from files.
- **Mutable defaults.** `waves: list[str] = []` raises `ValueError` when the class is defined. Use `field(default_factory=list)`, or an immutable tuple.

Load the config once at the edge, from YAML, JSON or arguments, and pass the object to the pure core.

Why it matters

A weighting script reads `config["min_base"]` from a dict built in a notebook cell. An analyst edits the cell to `config["minbase"] = 30` for a test and forgets to revert. The script keeps using the old default of 50, so 14 subgroups the client asked to see are suppressed in the published tables. Nobody notices for two waves.

With a frozen dataclass, `ReportConfig(minbase=30)` raises `TypeError: unexpected keyword argument` at the first run.

Worked example: weak code

```
1 CONFIG = {"weight_col": "weight", "min_base": 50, "waves": []}
2
3
4 def suppress_small(counts, config=CONFIG):
5     config["min_base"] = config.get("minbase", 50)
6     return {k: v for k, v in counts.items() if v >= config["min_base"]}
```

1. **Line 1.** A module-level dict: no fixed keys, no types, shared and mutable.
2. **Line 4.** A mutable default that every call shares.
3. **Line 5.** A typo in the key silently falls back to 50, and the write changes the shared dict.

Worked example: strong code

```
1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     """Settings for one reporting run."""
7
```

```
 8      weight_col: str
 9      min_base: int = 50
10
11      def __post_init__(self) -> None:
(2)         if not isinstance(self.min_base, int) or self.min_base <= 0:
13              raise ValueError(f"min_base must be a positive int, got
        {self.min_base!r}")
14
15
(3) def suppress_small(counts: dict[str, int], config: ReportConfig) ->
      dict[str, int]:
17      """Keep subgroups whose base reaches config.min_base."""
18      return {k: v for k, v in counts.items() if v >= config.min_base}
```

1. **Line 4.** Fixed fields, and no assignment after construction.
2. **Line 12.** Validation at load time. Hints alone are not checked at runtime.
3. **Line 16.** The function receives the config and reads it. It never changes it.

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

Recap card

- **Why a frozen dataclass for settings?** Named, typed fields, unknown keys rejected, and no assignment after construction.
- **Change one setting on a frozen config** dataclasses.replace(cfg, min_base=30) returns a new object.
- **Are dataclass type hints checked at runtime?** No. Validate or convert in **post_init**, or use pydantic for external input.
- **Empty list default on a dataclass field** field(default_factory=list). A plain [] raises ValueError at class definition.

Practice

Exercise 1. Multiple choice (level 1)

`cfg` is an instance of a `@dataclass(frozen=True)` class with a field `min_base`. What does `cfg.min_base = 30` do?

- a) Sets min_base to 30.
- b) Raises dataclasses.FrozenInstanceError.
- c) Creates a copy with min_base 30.
- d) Does nothing, silently.

Why?

- r1) frozen=True makes the generated **setattr** raise on every field assignment.

- r2) Integer fields are read-only in every dataclass.

Exercise 2. Predict the output (level 2)

What does this print?

```python
from dataclasses import dataclass


@dataclass
class Settings:
    min_base: int = 50


print(Settings("30"))
```

Exercise 3. Fill the gap (level 2)

Give `waves` an empty list as its default, safely.

```python
from dataclasses import dataclass, field


@dataclass
class Run:
    waves: list[str] = field(_____(1)=list)
```

Exercise 4. Write the function (level 3)

Complete `ReportConfig` and write `load_config(raw)`.

- `ReportConfig` is a frozen dataclass with `weight_col: str` and `min_base: int = 50`.
- In `__post_init__`, raise `ValueError` if `min_base` is not an `int` or is not positive.
- `load_config(raw: dict[str, object]) -> ReportConfig` builds the config from a dict. An unknown key must raise `TypeError`, a missing `weight_col` must raise `TypeError`.

Your code must pass these checks:

- load_config must build a ReportConfig from the dict, with min_base defaulting to 50.
- The config must be frozen.
- min_base of 0, a negative number or a string must raise ValueError.
- A misspelt key such as minbase must raise TypeError, not fall back to the default.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Settings for one reporting run.
class ReportConfig: ...

# Build a ReportConfig from a parsed YAML or JSON mapping.
def load_config(raw: dict[str, object]) -> ReportConfig: ...
```

Starter code:

```python
from dataclasses import dataclass


class ReportConfig:
    ...


def load_config(raw):
    ...
```

Exercise 5. Architecture design (level 4)

A weighting pipeline reads its settings from `weighting.yaml`. Choose how the settings flow through the code.

Options: A frozen dataclass with typed fields, The dict returned by yaml.safe_load, Module-level constants set from the file at import.

1. Shape of the settings in code
2. When values are checked
3. How functions get the settings

Sources

- Assigning a field of a frozen dataclass raises dataclasses.FrozenInstanceError, a subclass of AttributeError. (verified: Python dataclasses documentation; run on Python 3.11 and 3.14 in this repository.)
- A list default on a dataclass field raises ValueError at class definition. (verified: Python dataclasses documentation; run on Python 3.11 in this repository.)
- Dataclasses do not check or convert field types at runtime. (verified: Python dataclasses documentation; predict-output exercise run in this repository.)

B2-L3 When a class earns its place

Concept

Python does not require classes. Many data science classes are functions in disguise: an `__init__` that stores arguments, and one method that uses them.

```python
class Normaliser:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

Every caller writes `Normaliser("spend").run(df)`. A function, `normalise(df, col)`, says the same thing in one call, with nothing to construct.

A class earns its place when at least one of these holds:

- **State that outlives one call.** A weighting model learns targets in `fit` and applies them in `transform`. scikit-learn's `StandardScaler` stores `mean_` and `scale_` between the two calls.
- **An invariant to protect.** A `Weights` object guarantees values are positive and sum to the sample size. Code that builds one through its constructor cannot break that rule.
- **Several operations on the same data.** A `SurveyDesign` with `mean()`, `ci()` and `subgroup()` that all need weights, strata and clusters.
- **A shared interface.** Several classes with the same method, such as `load()`, so the caller swaps one for another.

Settings with no behaviour belong in a dataclass (lesson B2-L2). A single step belongs in a function (lesson B2-L1).

Test a design by asking: if you delete the class and keep its methods as functions, what breaks? If nothing, keep the functions.

Why it matters

A team wraps each cleaning step in a class: `DropDuplicates`, `FixDates`, `RenameColumns`, each with `__init__` and `run`. A new analyst adds a `self.seen_ids` set to `DropDuplicates` to speed up a second call. The pipeline object is reused across two tracker waves, so wave 2 silently drops every respondent who also took part in wave 1: 38% of the sample.

Pure functions have no `self` to keep stale state between calls.

Worked example: weak code

```
1  class AgeBander:
2      def __init__(self, bins, labels):
3          self.bins = bins
4          self.labels = labels
5
6      def run(self, df):
7          return df.assign(age_band=pd.cut(df["age"], self.bins,
       labels=self.labels))
8
9
10 banded = AgeBander([0, 34, 54, 120], ["18-34", "35-54", "55+"]).run(df)
```

1. **Line 1.** No state outlives a call, no invariant, one operation: a function in disguise.
2. **Line 6.** A single method named run is the usual sign.
3. **Line 10.** Every caller builds an object only to call one method.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def add_age_band(df: pd.DataFrame, bins: list[int], labels: list[str]) ->
       pd.DataFrame:
5      """Return df with age_band cut from age."""
6      return df.assign(age_band=pd.cut(df["age"], bins, labels=labels))
7
8
9  class RakingWeights:
10     """Learn raking targets on one wave, apply them to later waves."""
11
12     def __init__(self, targets: dict[str, dict[str, float]]) -> None:
13         self.targets = targets
14         self.factors_: dict[str, float] | None = None
15
16     def fit(self, df: pd.DataFrame) -> "RakingWeights":
17         ...
18         return self
19
20     def transform(self, df: pd.DataFrame) -> pd.Series:
21         if self.factors_ is None:
22             raise RuntimeError("call fit before transform")
23         ...
```

1. **Line 4.** One step, no state: a function.
2. **Line 14.** State that outlives one call, learned in fit and used in transform.
3. **Line 22.** The class protects an order of operations a function could not.

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

Recap card

- **A class with init and one run method** A function in disguise. Replace it with a function.
- **Four reasons a class earns its place** State across calls, an invariant to protect, several operations on the same data, a shared interface.
- **The deletion test** Turn the methods into functions. If nothing breaks, keep the functions.
- **Settings with no behaviour** A frozen dataclass.

Practice

Exercise 1. Multiple choice (level 1)

Which of these most needs a class?

- a) Rename columns using a mapping.
- b) Learn category frequencies on training data and encode test data with them.
- c) Convert a date column to UTC.
- d) Hold three settings with no behaviour.

Why?

- r1) State learned in one call must be reused in a later call.
- r2) It has the most lines of code.

Exercise 2. Predict the output (level 2)

What does this print?

```
class Deduper:
    def __init__(self) -> None:
        self.seen: set[int] = set()

    def run(self, ids: list[int]) -> list[int]:
        out = [i for i in ids if i not in self.seen]
        self.seen.update(ids)
        return out


d = Deduper()
d.run([1, 2, 3])
print(d.run([2, 3, 4]))
```

Exercise 3. Refactor (level 3)

Replace the class with a function `add_share(df, col)` that returns a new DataFrame with a column `share` equal to `df[col] / df[col].sum()`. If the column sums to zero, set `share` to `0.0` for every row.

Your code must pass these checks:

- share must be each value divided by the column total.
- A zero total must give share 0.0, not NaN.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Return df with share = col / column total, or 0.0 when the total is zero.
def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame: ...
```

Starter code:

```python
class ShareAdder:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(share=df[self.col] / df[self.col].sum())
```

Exercise 4. Spot the bug (level 4)

The same `Scaler` object is fitted on wave 1 and used for waves 2 and 3. Wave 3 results match a scaler fitted on wave 2 only. Tap the line that causes it.

```python
class Scaler:
    """Standardise with the mean and sd learned in fit."""

    def fit(self, values: list[float]) -> "Scaler":
        n = len(values)
        self.mean_ = sum(values) / n
        self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
        return self

    def transform(self, values: list[float]) -> list[float]:
        self.fit(values)
        return [(v - self.mean_) / self.sd_ for v in values]
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Choose a form for each piece of a survey analysis package.

Options: Function, Frozen dataclass, Class with methods.

1. Recode a Likert column to numbers using a mapping
2. Confidence level, minimum base and weight column
3. Weights, strata and clusters used by mean, CI and subgroup methods

Sources

- scikit-learn's StandardScaler stores mean_ and scale_ after fit. (verified: scikit-learn 1.8 API reference; checked in this repository's dev environment.)

B3 Clean pandas

Roadmap: Prerequisite for all months. Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L1 Dtypes in pandas 3, nullable and string

Concept

Every pandas column has a **dtype**. The wrong dtype gives wrong answers without an error: IDs that lose leading zeros, counts that turn into floats, text that sorts as numbers.

Three facts about pandas 3.0:

- **Text columns default to the `str` dtype.** `pd.Series(["a", None]).dtype` prints `str`, not `object`. Missing values in it are `NaN`. Only strings are allowed, so a stray integer no longer hides in a text column.
- **A missing value turns an integer column into `float64`**, unless you ask for a nullable dtype. `pd.Series([1, None])` is `float64`. `pd.Series([1, None], dtype="Int64")` keeps integers and shows the gap as `<NA>`.
- **Setting a value that does not fit raises.** Assigning `1.5` into an `int64` Series raises `TypeError` instead of silently upcasting the column. (Assigning `NaN` still upcasts to `float64`.)

The capital letter matters. `int64` is NumPy's integer with no missing values. `Int64`, `Float64` and `boolean` are pandas **nullable** dtypes that hold `pd.NA`.

Set dtypes where data enters, not later:

```
df = pd.read_csv(
    path,
    dtype={"respondent_id": "str", "postcode": "str", "household_size": "Int64"},
)
```

Without the `dtype` map, `read_csv` reads a postcode such as `01234` as the number `1234.0` once any row is blank. With it, `"01234"` stays a string.

Check the result with `df.dtypes` right after loading. `df.convert_dtypes()` switches every column to the nullable equivalents in one call, useful for exploration. An explicit map is better in pipelines, because it fails loudly when the source changes.

Why it matters

A panel file stores respondent IDs such as `00731`. One wave has a blank ID row, so `read_csv` infers `float64` and the ID becomes `731.0`. The join to the previous wave, where IDs were read as text, matches nothing. The longitudinal report shows 0% retention for a panel that kept 82% of its members.

`dtype={"respondent_id": "str"}` at load time keeps `00731` as written in every wave.

Worked example: weak code

```
1 df = pd.read_csv("panel_w2.csv")
2 df["respondent_id"] = df["respondent_id"].astype(int).astype(str)
3 df["household_size"] = df["household_size"].fillna(0).astype(int)
```

1. **Line 1.** Dtypes inferred: a blank row turns IDs and counts into float64.
2. **Line 2.** Leading zeros were lost at load time. This cannot bring them back.
3. **Line 3.** Replaces 'unknown' with 0, a real value that biases every mean.

Worked example: strong code

```
 1 import pandas as pd
 2
 3 PANEL_DTYPES: dict[str, str] = {
 1     "respondent_id": "str",
 2     "household_size": "Int64",
 6 }
 7
 8
 9 def read_panel(path: str) -> pd.DataFrame:
10     """Read one panel wave with explicit dtypes."""
 3     return pd.read_csv(path, dtype=PANEL_DTYPES)
```

1. **Line 4.** IDs are labels, not numbers: 00731 stays 00731.
2. **Line 5.** Nullable integer: a gap stays <NA>, the rest stay integers.
3. **Line 11.** Dtypes fixed where data enters. In a pipeline, load the map from config.

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

Recap card

- **Default text dtype in pandas 3.0** str, with NaN for missing values. Only strings are allowed.
- **int64 vs Int64** int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
- **Keep leading zeros in IDs** Read the column with dtype str in read_csv.

- **Assign 1.5 into an int64 Series on pandas 3** Raises TypeError instead of silently upcasting.
- **Where to set dtypes** At load time, from a dtype map in config. Check df.dtypes right after.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, what is the dtype of `pd.Series([4, None, 2])`?

- a) int64
- b) Int64
- c) float64
- d) object

Why?

- r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.
- r2) pandas 3 stores every number as float.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

What you need to know.

`io.StringIO(text)` wraps a string in a file-like object. `pd.read_csv` reads it as if it were a file on disk, which keeps small examples and tests free of real files.

```
import io

import pandas as pd

pd.read_csv(io.StringIO("a,b\n1,x\n"))   # one row: a=1, b='x'
```

`df["col"].iloc[0]` returns the first value of a column by position.

```
import io

import pandas as pd

csv = "id,postcode\n1,01234\n2,\n"
df = pd.read_csv(io.StringIO(csv))
print(df["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Keep IDs as text and let the count column hold missing values as integers.

```
df = pd.read_csv(path, dtype={"respondent_id": "_____(1)",
    "household_size": "_____(2)"})
```

Exercise 4. Write the function (level 3)

Write `read_panel(text: str) -> pd.DataFrame`. It reads CSV text with columns `respondent_id` and `household_size`. Keep `respondent_id` as text with leading zeros, and `household_size` as a nullable integer where blanks become `<NA>`. Take the dtype map from a module constant `PANEL_DTYPES`.

Your code must pass these checks:

- respondent_id must keep its leading zeros. Read it as str.
- household_size must be Int64, with the blank as <NA>.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
PANEL_DTYPES = ...

# Read panel CSV text with explicit dtypes.
def read_panel(text: str) -> pd.DataFrame: ...
```

Starter code:

```
import io

import pandas as pd


def read_panel(text):
    return pd.read_csv(io.StringIO(text))
```

Exercise 5. Spot the bug (level 4)

The mean household size reported for wave 3 dropped from 2.6 to 2.1. Nothing changed in the sample. Tap the line that caused it.

```
import pandas as pd


def mean_household(df: pd.DataFrame) -> float:
    """Mean household size among respondents who answered."""
    sizes = df["household_size"].astype("Int64")
    sizes = sizes.fillna(0)
    return float(sizes.mean())
```

Write down the line number of the bug. Then write the fix.

Sources

- In pandas 3.0 text columns default to the str dtype, with NaN as the missing value. (verified: pandas 3.0 release notes; run on pandas 3.0.6 and 3.0.2 in this repository.)
- In pandas 3.0 setting an incompatible value such as 1.5 into an int64 Series raises TypeError. (verified: pandas 3.0 release notes (setitem upcasting removed); run on pandas 3.0.6 and 3.0.2 in this repository.)
- read_csv infers float64 for an integer-like column with a blank value. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository.)

B3-L2 Vectorise instead of apply

Concept

pandas stores each column as an array. A **vectorised** operation, such as `df["a"] * df["b"]`, `np.where()` or `pd.cut()`, runs one compiled loop over the whole array. `df.apply(func, axis=1)` does the opposite. It builds a Series for each row and calls your Python function once per row.

The gap is large. On one million rows, a row-wise `apply` that picks a weight when a flag is set took 4.0 seconds. `np.where` did the same job in 7.5 milliseconds, about 500 times faster. (pandas 3.0.6, NumPy 2.4.6, one laptop core.)

Speed is not the only gain. Vectorised code states the rule for the whole column in one line, so a reviewer checks one expression instead of a branching function.

Reach for these first:

Task	Vectorised tool
Arithmetic between columns	`+`, `-`, `*`, `/` on Series
Two-way choice	`np.where(cond, a, b)`
Many-way choice	`np.select(conditions, choices, default)`
Numeric bands	`pd.cut(values, bins, labels, right=False)`
Lookup from a dict	`Series.map(mapping)`
Limits	`Series.clip(lower, upper)`

Watch missing values. `NaN > 60` is `False`, so `np.where(minutes > 60, "heavy", "light")` labels a missing value `"light"`. Handle missing values on purpose:

`pd.cut` keeps `NaN` as missing, and `Series.where(values.notna())` puts it back after `np.where`.

Keep `apply` for work with no vectorised form, such as calling an external parser on each string. Even then, try `Series.map` or a list comprehension on one column before `apply(axis=1)`.

Why it matters

A weighting script derives 25 columns with `apply(axis=1)`. It ran in 8 minutes on 2 million rows, so nobody noticed. The programme then merged three trackers into one file of 40 million rows. At the measured rate of about 4 seconds per million rows per column, the derivations alone need roughly 67 minutes. The job hit its 60-minute timeout every night for a week, and weighted results for all three trackers were late.

The same script also labelled respondents with a missing `minutes` value as `"light"` viewers, because the row function used `if row["minutes"] > 60`. The vectorised rewrite fixed the speed, and review of the one-line rule exposed the missing-value bug.

Worked example: weak code

```
import pandas as pd


def derive(df):
    def per_row(row):
        weighted_reach = row["final_weight"] if row["reach_flag"] == 1 else 0.0
        if row["age"] < 35:
            band = "16-34"
        elif row["age"] < 55:
            band = "35-54"
        else:
            band = "55+"
        return pd.Series({"weighted_reach": weighted_reach, "age_group": band})

    df[["weighted_reach", "age_group"]] = df.apply(per_row, axis=1)
    return df
```

1. **Line 4.** No type hints and no docstring. The reader cannot tell what the frame must contain.
2. **Line 7.** `NaN < 35` is False, so a missing age falls through to `55+` without a warning.
3. **Line 15.** Builds a Series and calls Python for every row. About 500 times slower than a vectorised rule.

4. **Line 15.** Mutates the caller's frame. A pure function returns a new frame.

Worked example: strong code

```
 1 import numpy as np
 2 import pandas as pd
 3
 1 AGE_BINS = [16, 35, 55, np.inf]
 5 AGE_LABELS = ["16-34", "35-54", "55+"]
 6
 7
 8 def derive(df: pd.DataFrame) -> pd.DataFrame:
 9     """Add weighted reach and age group.
10
11     Args:
12         df: Respondents with reach_flag, final_weight and age.
13
14     Returns:
15         A new frame. Missing ages give a missing age_group.
16     """
 2     return df.assign(
 3         weighted_reach=np.where(df["reach_flag"] == 1,
   df["final_weight"], 0.0),
 4         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
   right=False),
20     )
```

1. **Line 4.** Band edges live in one constant, so a change to the bands is a one-line change.
2. **Line 17.** `assign` returns a new frame, so the caller's data is untouched.
3. **Line 18.** One compiled pass over the column.
4. **Line 19.** `right=False` makes each band include its lower edge: 35 goes to `35-54`. Missing ages stay missing.

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Recap card

- **Why is df.apply(func, axis=1) slow?** It calls a Python function once per row. np.where measured about 500 times faster on one million rows.
- **Vectorised if-else in NumPy** np.where(cond, value_if_true, value_if_false). For several conditions, np.select.
- **What does NaN > 60 return?** False. np.where and np.select send missing values to a real category unless you handle them.
- **Numeric bands that keep missing values missing** pd.cut(values, bins, labels, right=False).

Practice

Exercise 1. Multiple choice (level 1)

Which line computes `weighted_reach` for a frame with one million rows in a single compiled pass?

- a) `df.apply(lambda r: r["final_weight"] if r["reach_flag"] == 1 else 0.0, axis=1)`
- b) `[w if f == 1 else 0.0 for w, f in zip(df["final_weight"], df["reach_flag"])]`
- c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
- d) `for i, row in df.iterrows(): ...`

Why?

- r1) It evaluates the condition and the choice on whole arrays in compiled code.
- r2) It uses fewer characters than the other options.
- r3) NumPy runs the lambda on several cores.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`series.tolist()` returns the values of a pandas Series as a plain Python list, so `print` shows them without the index:

```
pd.Series([3, 4]).tolist()   # [3, 4]
```

```
import pandas as pd

s = pd.Series([0.0, 1.5, None])
print((s > 1).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete the line so rows with `reach_flag == 1` keep their weight and every other row gets zero.

```
df["weighted_reach"] = np._____(1)(df["reach_flag"] == 1,
    df["final_weight"], _____(2))
```

Exercise 4. Write the function (level 3)

Write `weighted_reach_pct(df)`. It returns the weighted share of rows with `reach_flag == 1`: the sum of `final_weight` where the flag is 1, divided by the

sum of all `final_weight`. Return `None` when the total weight is zero or the frame is empty. Do not use `apply` or a loop over rows.

Your code must pass these checks:

- For weights 2, 1, 1 with flags 1, 0, 1 the answer is 3 / 4 = 0.75. Divide the flagged weight by the total weight.
- An empty frame must return None, not raise or return NaN. Guard the division.
- When the total weight is zero, return None instead of dividing by zero.
- Return a Python float (wrap the result in `float()`), not a NumPy scalar.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted share of respondents with reach_flag == 1.
def weighted_reach_pct(df: pd.DataFrame) -> float | None: ...
```

Starter code:

```
import numpy as np
import pandas as pd


def weighted_reach_pct(df: pd.DataFrame) -> float | None:
    ...
```

Exercise 5. Refactor (level 4)

Refactor `add_age_group` to vectorised code. Keep the bands: under 35 is `16-34`, under 55 is `35-54`, anything else is `55+`. A missing age must give a missing age group, not `55+`. Do not change the input frame.

Your code must pass these checks:

- The bands are wrong at an edge. 35 belongs to 35-54 and 55 belongs to 55+. With pd.cut, set right=False.
- A missing age must give a missing age group. `np.where` and `np.select` turn NaN into the default band. `pd.cut` keeps it missing.
- Your function changed the caller's frame. Return a new frame with `df.assign(...)`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
AGE_BINS = ...

# Module constant.
AGE_LABELS = ...
```

```
6
7 # Return a copy of df with an age_group column. Missing ages stay missing.
8 def add_age_group(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def add_age_group(df):
    def band(row):
        if row["age"] < 35:
            return "16-34"
        if row["age"] < 55:
            return "35-54"
        return "55+"

    df["age_group"] = df.apply(band, axis=1)
    return df
```

Exercise 6. Spot the bug (level 5)

The docstring promises that missing minutes stay missing. A respondent with no minutes recorded shows as `"light"`. Tap the line with the bug, then fix the function.

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
    minutes stay missing."""
3     minutes = df["minutes"]
4     band = np.where(minutes >= 60, "heavy", "light")
5     return df.assign(viewing_band=band)
```

Write down the line number of the bug. Then write the fix.

Sources

- Row-wise apply was about 500 times slower than np.where on one million rows. (verified: Measured in this repo's build environment: pandas 3.0.6, NumPy 2.4.6, 4.03 s against 7.5 ms.)
- pd.cut keeps NaN as missing and supports right=False. (verified: Run in pandas 3.0.6.)

B3-L3 Method chaining and Copy-on-Write

Concept

Copy-on-Write (CoW) is always on in pandas 3.0. Every indexing result, such as `df[cols]`, `df[mask]` or `df["a"]`, behaves as a separate object.

Changing it never changes `df`. pandas copies data only when you write to a shared object, so reads stay cheap.

One consequence: **chained assignment never works**.

```
df[df["age"] < 18]["weight"] = 0       # df unchanged,
    ChainedAssignmentError warning
df.loc[df["age"] < 18, "weight"] = 0   # one step: df changed
```

The first line writes to a temporary copy, which Python then discards. pandas 3 emits a `ChainedAssignmentError` warning, but the code keeps running. `SettingWithCopyWarning` no longer exists.

CoW rewards a second habit: **method chaining**. Build the result as one expression of steps that each return a new frame.

```
result = (
    raw
    .rename(columns=COLUMN_MAP)
    .loc[lambda d: d["age"] >= 18]
    .assign(weight=lambda d: d["weight"].clip(upper=5.0))
    .pipe(add_age_band, bins=BINS, labels=LABELS)
)
```

- `assign(col=lambda d: ...)` refers to the frame at that step, not the original `raw`.
- `.loc[lambda d: ...]` filters on the current step.
- `.pipe(func, **kwargs)` calls your own pure function (lesson B2-L1) inside the chain.

Chains read top to bottom, leave no half-finished variables (`df2`, `df_tmp`, `df_final`) in the namespace, and do not mutate the input. Keep each chain to about ten steps. Split longer ones into named functions and join them with `pipe`.

To debug a chain, comment out steps from the bottom up, or add `.pipe(lambda d: print(d.shape) or d)` between steps.

Why it matters

An analyst caps extreme weights with `df[df["weight"] > 5]["weight"] = 5`. On pandas 2 it printed a SettingWithCopyWarning that everyone had learned to ignore. The weights were never capped. Three respondents with weights above 12 moved the reported brand share by 4 points. After the upgrade to pandas 3 the assignment still does nothing, with a different warning.

`df.assign(weight=df["weight"].clip(upper=5.0))` or a single `df.loc[mask, "weight"] = 5` does what the analyst meant.

Worked example: weak code

```
1  df = raw.rename(columns=COLUMN_MAP)
2  df2 = df[df["age"] >= 18]
3  df2[df2["weight"] > 5]["weight"] = 5.0
4  df3 = add_age_band(df2, BINS, LABELS)
5  result = df3
```

1. **Line 2.** Intermediate names that outlive their use and invite edits to the wrong one.
2. **Line 3.** Chained assignment: writes to a temporary copy. The weights are not capped.
3. **Line 5.** Which of df, df2, df3 is the final state? The reader has to trace every line.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def prepare(raw: pd.DataFrame, column_map: dict[str, str], bins:
       list[int], labels: list[str]) -> pd.DataFrame:
5      """Rename, keep adults, cap weights at 5 and add age bands."""
6      return (
7          raw
8          .rename(columns=column_map)
9          .loc[lambda d: d["age"] >= 18]
10         .assign(weight=lambda d: d["weight"].clip(upper=5.0))
11         .pipe(add_age_band, bins=bins, labels=labels)
12     )
```

1. **Line 9.** Filters the renamed frame at this step.
2. **Line 10.** Returns a new frame with capped weights. No assignment to a copy.
3. **Line 11.** Your own pure function, called as one step of the chain.

Under Copy-on-Write, write with .loc in one step, or better, build a new frame with a chain of assign, loc and pipe.

Recap card

- **Copy-on-Write in pandas 3.0** Always on. Indexing results behave as separate objects, copied only on write.
- **df[mask]["col"] = value on pandas 3** Changes nothing and emits ChainedAssignmentError. Use df.loc[mask, "col"] = value.
- **Refer to the current step inside a chain** A lambda: assign(x=lambda d: ...) or loc[lambda d: ...].
- **Call your own function inside a chain** .pipe(func, **kwargs).

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, `df[df["age"] < 18]["weight"] = 0` runs. What happens to `df`?

- a) Weights of under-18s become 0.
- b) Nothing. pandas emits a ChainedAssignmentError warning.
- c) It raises an exception and stops.
- d) Every weight becomes 0.

Why?

- r1) The first indexing step returns a new object, so the write lands on a temporary copy.
- r2) pandas 3 makes every DataFrame read-only.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```python
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3]})
s = df["a"]
s.iloc[0] = 100
print(df["a"].tolist())
```

Exercise 3. Put in order (level 2)

Put the steps of the chain in order. The raw columns are `wt` and `age`. Weights must average 1.0 across the adults in the output.

- .pipe(add_age_band, bins=BINS, labels=LABELS)
- .assign(weight=lambda d: d["weight"] / d["weight"].mean())
- .rename(columns={"wt": "weight"})
- .loc[lambda d: d["age"] >= 18]

Exercise 4. Refactor (level 3)

Rewrite `cap_weights` as one method chain that returns a new frame. Keep only rows where `age >= 18`, then cap `weight` at `cap`. Leave the input unchanged. No chained assignment.

Your code must pass these checks:

- Weights above cap must become cap, among adults only.
- Only rows with age 18 or over must remain.
- The input frame changed.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Keep adults and cap their weights at cap.
def cap_weights(
    df: pd.DataFrame,
    cap: float,
) -> pd.DataFrame: ...
```

Starter code:

```python
def cap_weights(df, cap):
    adults = df[df["age"] >= 18]
    adults[adults["weight"] > cap]["weight"] = cap
    return adults
```

Exercise 5. Spot the bug (level 4)

Weights for invalid interviews should be zero. The check after the function shows they are unchanged. Tap the line to fix, and rewrite the function.

```python
import pandas as pd


def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
    """Set weight to 0 where valid is False."""
    out = df.copy()
    out[~out["valid"]]["weight"] = 0.0
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- Copy-on-Write is always enabled in pandas 3.0 and the mode.copy_on_write option has no effect. (verified: pandas 3.0 release notes; Pandas4Warning text observed on pandas 3.0.6 and 3.0.2 in this repository.)
- Chained assignment never updates the original in pandas 3.0 and emits ChainedAssignmentError as a warning. (verified: pandas Copy-on-Write user guide; run on pandas 3.0.6 and 3.0.2 in this repository.)
- SettingWithCopyWarning no longer exists in pandas 3.0. (verified: pandas 3.0 release notes; pandas.errors has no SettingWithCopyWarning on 3.0.6.)

B4 Polars and DuckDB

Roadmap: Prerequisite for all months. Tools for larger local datasets.

B4-L1 Polars expressions and lazy frames

Concept

Polars is a DataFrame library written in Rust. It runs on all your cores and has no row index. You describe work with **expressions**: `pl.col("weight").sum()` is a recipe, not a result. Polars runs it inside a context.

```
import polars as pl

shares = (
    df.group_by("wave", "brand")
    .agg(pl.col("weight").sum().alias("brand_weight"))
    .with_columns(total=pl.col("brand_weight").sum().over("wave"))
)
```

- `select` returns only the expressions you list. `with_columns` adds or replaces columns. `filter` keeps rows. `group_by(...).agg(...)` aggregates.
- `.over("wave")` computes a group total and broadcasts it back to each row, like a window function in SQL.
- `pl.when(...).then(...).otherwise(...)` replaces `np.where`.

Polars has two modes. A `DataFrame` runs each step at once. A `LazyFrame`, from `df.lazy()` or `pl.scan_parquet(path)`, records the steps and runs nothing until `.collect()`. Before running, the optimiser rewrites the plan. It pushes filters down to the scan, so rows you drop are never read, and reads only the columns you use. `lf.explain()` prints the plan.

Two behaviours to know (Polars 1.33):

- `sum()` **of a group with only nulls returns** `0`, not null. Check `null_count()` before you trust a total.
- `mean()` **skips nulls.** `sum() / pl.len()` counts null rows in the denominator and gives a different answer.

Use lazy mode for files. Scan, filter, select, aggregate, then collect once at the end.

Why it matters

A weekly job reads a 9 GB Parquet extract with `pl.read_parquet`, then filters to the current wave. Memory peaks at 14 GB and the job fails on a 16 GB runner on the week the extract grows.

`pl.scan_parquet(path).filter(pl.col("wave") == wave).select(cols).collect()` lets Polars skip other waves' row groups and unused columns at read time, so memory scales with one wave, not the whole file.

Worked example: weak code

```
1 df = pl.read_parquet("responses.parquet")
2 df = df.filter(pl.col("wave") == "2026-09")
3 totals = df.group_by("brand").agg(pl.col("weight").sum())
4 totals = totals.with_columns(mean_w=pl.col("weight") / df.height)
```

1. **Line 1.** Eager read: every wave and every column loads before the filter.
2. **Line 4.** Divides a brand total by the whole wave's row count: not the brand mean, and null rows count.

Worked example: strong code

```
1  import polars as pl
2
3
4  def brand_weights(path: str, wave: str) -> pl.DataFrame:
5      """Weighted totals and means per brand for one wave. Grain: one row per brand."""
6      return (
1          pl.scan_parquet(path)
2          .filter(pl.col("wave") == wave)
9          .group_by("brand")
10         .agg(
11             total=pl.col("weight").sum(),
12             mean_w=pl.col("weight").mean(),
3              missing=pl.col("weight").null_count(),
14         )
15         .sort("brand")
4          .collect()
17     )
```

1. **Line 7.** Lazy scan: nothing is read yet.
2. **Line 8.** Pushed down to the scan: other waves are skipped at read time.
3. **Line 13.** Reports nulls, because an all-null group sums to 0.
4. **Line 16.** One collect at the end runs the optimised plan.

Scan lazily, filter and select early, aggregate with expressions, and collect once.

Recap card

- **Polars eager vs lazy** DataFrame runs each step at once. LazyFrame (scan_*, .lazy()) builds a plan and runs on collect().
- **Group total on every row in Polars** pl.col("x").sum().over("group").
- **Polars sum() of an all-null group** 0, not null. Report null_count() with totals.
- **Mean that skips nulls in Polars** pl.col("x").mean(). sum() / pl.len() counts null rows.

Practice

Exercise 1. Multiple choice (level 1)

`lf = pl.scan_parquet("big.parquet").filter(pl.col("wave") == "w9")`. What has happened after this line runs?

- a) The whole file is in memory, filtered to wave w9.
- b) Nothing is read yet. lf holds a query plan.
- c) Only the w9 rows are in memory.
- d) Polars wrote a filtered copy of the file.

Why?

- r1) A LazyFrame records steps and runs them only on collect().
- r2) Parquet files cannot be read without DuckDB.

Exercise 2. Predict the output (level 2)

What does this print on Polars 1.33?

What you need to know.

`series.to_list()` returns the values of a Polars Series as a plain Python list. `out["w"]` picks one column of a Polars DataFrame as a Series.

```
pl.DataFrame({"n": [1, 2]})["n"].to_list()   # [1, 2]
```

```
import polars as pl

df = pl.DataFrame({"wave": ["w1", "w1", "w2"], "w": [1.0, 2.0, None]})
out = df.group_by("wave").agg(pl.col("w").sum()).sort("wave")
print(out["w"].to_list())
```

Exercise 3. Fill the gap (level 2)

Add a `share` column with each brand's weight over its wave's total, then run the lazy plan.

```
out = (
```

```
    lf.with_columns(share=pl.col("weight") /
    pl.col("weight").sum().over("wave"))
    .pipe(2)()
)
```

Exercise 4. Write the function (level 3)

Write `brand_share(df: pl.DataFrame) -> pl.DataFrame`. Input columns: `wave`, `brand`, `weight`. Return one row per wave and brand with columns `wave`, `brand`, `share`, where `share` is the brand's weight total over the wave's weight total. When a wave's total is 0, `share` is null. Sort by `wave`, then `brand`.

Your code must pass these checks:

- Each brand's share must be its weight over the wave total, one row per wave and brand.
- A wave with zero total weight must give a null share, not NaN or an error.
- Return exactly wave, brand, share, sorted by wave then brand.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted share of each brand within its wave. Grain: one row per wave
    and brand.
def brand_share(df: pl.DataFrame) -> pl.DataFrame: ...
```

Starter code:

```
import polars as pl


def brand_share(df):
    ...
```

Exercise 5. Spot the bug (level 4)

The mean satisfaction score per region looks too low in regions with many skipped answers. Tap the line with the bug and fix it.

```
import polars as pl


def mean_score(df: pl.DataFrame) -> pl.DataFrame:
    """Mean score per region among respondents who answered."""
    return (
        df.group_by("region")
        .agg(mean_score=pl.col("score").sum() / pl.len())
        .sort("region")
    )
```

Write down the line number of the bug. Then write the fix.

Sources

- Polars sum() of a group with only nulls returns 0. (verified: Run on Polars 1.33.1 and 1.44.2 in this repository.)
- LazyFrame optimisation pushes filters and column selection down to the scan. (verified: Polars user guide, lazy API and optimisations; plan checked with explain() on Polars 1.33.1.)

B4-L2 DuckDB, SQL over files and DataFrames

Concept

DuckDB is an analytical SQL database that runs inside your Python process. There is no server to install. It reads Parquet, CSV and JSON files directly, and it queries pandas and Polars DataFrames that are in scope.

```
import duckdb

con = duckdb.connect()   # in-memory database
rel = con.sql("""
    SELECT wave, brand, sum(weight) AS total
    FROM read_parquet('extracts/*.parquet')
    GROUP BY wave, brand
""")
df = rel.df()    # pandas; .pl() for Polars, .fetchall() for tuples
```

- A glob such as `'extracts/*.parquet'` reads many files as one table. Filters and column lists are pushed down to the reader, as in Polars lazy mode.
- `FROM responses` finds a DataFrame variable named `responses` in the calling scope. `con.register("responses", df)` names it explicitly, which is clearer inside functions.
- Results come back as a relation. Nothing is materialised until you call `.df()`, `.pl()`, `.fetchall()` or `.fetchone()`.

Pass values as parameters, never with f-strings.

```
con.execute("SELECT count(*) FROM responses WHERE wave = ?", [wave])
```

With `f"... WHERE wave = '{wave}'"`, a value that holds a quote changes the query. In this course's own test, the input `w1' or '1'='1` turned a filter for one wave into a count of every row. A `?` placeholder (or `$name` with a dict) sends the value as data, so quotes cannot change the SQL. File paths and table names cannot be parameters. Check those against a list of allowed values.

Create the connection at the edge of your program and pass `con` to functions. The module-level `duckdb.sql()` uses one shared default connection, a global that tests cannot replace.

Why it matters

A dashboard helper built its filter with `f"WHERE region = '{region}'"`. The region list came from a client file. One entry was `Hawke's Bay`. The apostrophe broke the SQL, and the job failed at 06:00 before the client's morning meeting. A second entry with a crafted value could have read other clients' rows.

`con.execute("... WHERE region = ?", [region])` handles `Hawke's Bay` as a plain string.

Worked example: weak code

```
1  import duckdb
2
3
4  def wave_total(wave):
1      q = f"SELECT sum(weight) FROM read_parquet('data/*.parquet') WHERE
       wave = '{wave}'"
2      return duckdb.sql(q).fetchone()[0]
```

1. **Line 5.** Value pasted into SQL: a quote in wave breaks or rewrites the query.
2. **Line 6.** Shared default connection, and None when no rows match.

Worked example: strong code

```
1  import duckdb
2
1  TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE wave = ?"
4
5
2  def wave_total(con: duckdb.DuckDBPyConnection, wave: str) -> float:
7      """Total weight for one wave, 0.0 when the wave has no rows."""
8      row = con.execute(TOTAL_SQL, [wave]).fetchone()
3      if row is None or row[0] is None:
10         return 0.0
11     return float(row[0])
```

1. **Line 3.** The value goes in as a parameter. The SQL text never changes.
2. **Line 6.** The connection is passed in, so a test gives it a small in-memory table.
3. **Line 9.** SQL sum over no rows is NULL. Test with `is None`.

One connection passed in, parameters for values, and a None check on aggregates.

Recap card

- **Query many Parquet files in DuckDB** FROM read_parquet('dir/*.parquet').
- **Pass a value into DuckDB SQL** con.execute("... WHERE x = ?", [value]). Never an f-string.
- **Table names from user input** Placeholders cannot bind identifiers. Check the name against an allowlist.
- **SQL sum over no rows** NULL, returned as None. Check with is None.

Practice

Exercise 1. Multiple choice (level 1)

Which call safely filters by a `region` value read from a client file?

- a) `con.execute(f"... WHERE region = '{region}'")`
- b) `con.execute("... WHERE region = ?", [region])`
- c) `con.execute("... WHERE region = '" + region + "'")`
- d) `con.execute("... WHERE region = '%s'" % region)`

Why?

- r1) The value is sent as data, separate from the SQL text.
- r2) It is shorter.

Exercise 2. Predict the output (level 2)

What does this print? (DuckDB 1.5)

```
import duckdb

con = duckdb.connect()
con.execute("CREATE TABLE t AS SELECT * FROM (VALUES ('w1', 1.0), ('w2', 2.0)) v(wave, weight)")
print(con.execute("SELECT sum(weight) FROM t WHERE wave = 'w9'").fetchone())
```

Exercise 3. Fill the gap (level 2)

Count rows for one wave from all Parquet files in `extracts/`, with the wave as a parameter.

```
con.execute(
    "SELECT count(*) FROM _____(1)('extracts/*.parquet') WHERE wave = _____(2)",
    [wave],
).fetchone()
```

Exercise 4. Write the function (level 3)

Write `region_total(con, region)`. It returns the sum of `weight` in table `responses` for the given `region`, as a `float`. Return `0.0` when no rows match. Pass `region` as a parameter. The checker runs it with regions such as `Hawke's Bay`.

Your code must pass these checks:

- Sum the weights for the region.
- A region with an apostrophe must work. Pass it as a parameter.
- No matching rows must give 0.0, not None.
- A crafted value must match nothing. Pass it as a parameter.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
REGION_TOTAL_SQL = ...

# Total weight for one region.
def region_total(
    con: duckdb.DuckDBPyConnection,
    region: str,
) -> float: ...
```

Starter code:

```python
import duckdb


def region_total(con, region):
    return con.execute(f"SELECT sum(weight) FROM responses WHERE region =
    '{region}'").fetchone()[0]
```

Exercise 5. Architecture design (level 4)

A reporting script lets users pick a table to summarise and a wave to filter on. Choose how each input reaches DuckDB.

Options: A ? parameter, An f-string inside the SQL.

1. The wave value
2. The table name
3. The connection

Sources

- DuckDB binds ? and $name parameters with con.execute(sql, params). (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)

- DuckDB queries pandas DataFrames in the calling scope by variable name and reads Parquet globs with read_parquet. (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)
- Pasting w1' or '1'='1 into an f-string filter returned every row; the same value as a parameter matched none. (verified: Run on DuckDB 1.5.1 in this repository.)

B4-L3 Choosing pandas, Polars, DuckDB or Spark

Concept

All four tools answer the same questions. Pick by where the data lives and how big it is next to your memory, not by fashion.

A measurement from this course's build: a group-by with sum and mean over 10 million rows and 1,000 groups, best of three runs on a 4-core machine. pandas 3.0.6 took 0.22 s, Polars 1.44 took 0.11 s, DuckDB 1.5 took 0.06 s. At this size every tool is fast enough. The choice matters when data outgrows memory.

Situation	Good default
Data fits comfortably in memory, you need the scikit-learn, statsmodels or plotting ecosystem	pandas
Files larger than you want in memory, heavy transforms, one machine	Polars lazy (`scan_parquet`)
SQL is the clearest way to express it, or you join many files ad hoc	DuckDB
Data already in a lakehouse, or larger than one machine, or a scheduled production job	Spark (Track A)

The tools share the **Apache Arrow** memory format, so you mix them at the boundaries:

```
lazy_result = pl.scan_parquet("extracts/*.parquet").filter(...).group_by(...).agg(...)
summary = lazy_result.collect().to_pandas()     # Polars to pandas for statsmodels
rel = con.sql("SELECT ... FROM read_parquet('extracts/*.parquet')")
top = rel.pl()                                  # DuckDB to Polars
```

Rules for mixing:

- **Reduce first, convert last.** Filter and aggregate in the engine that reads the files. Convert the small result.

- **Watch the index.** Polars has no index. `pl.from_pandas(df)` drops a pandas index unless you pass `include_index=True`, or call `reset_index()` first.
- **Check dtypes after each hop.** Text becomes `str` in pandas 3 and `String` in Polars. Nullable integers need care in both directions.

One project, one primary engine. A second tool at a clear boundary is fine. Four tools in one function is not.

Why it matters

An analyst sets `respondent_id` as the pandas index to deduplicate, then hands the frame to a Polars function for a fast join. `pl.from_pandas` drops the index. The join key is gone, so the analyst joins on row position with `hstack`. Two waves sorted differently, and 11% of respondents got another person's scores.

`pl.from_pandas(df.reset_index())` keeps the ID as a column, and the join uses it.

Worked example: weak code

```
❶ big = pl.read_parquet("extracts/*.parquet").to_pandas()
❷ big = big[big["wave"] == WAVE].set_index("respondent_id")
❸ stats = pl.from_pandas(big).group_by("brand").agg(pl.col("weight").sum())
```

1. **Line 1.** Reads every wave eagerly, then copies all of it into pandas before filtering.
2. **Line 2.** The ID moves into the index.
3. **Line 3.** Back to Polars: the index, and so the ID, is dropped. Two conversions of the full data.

Worked example: strong code

```
 1 import pandas as pd
 2 import polars as pl
 3
 4
 5 def brand_totals(path_glob: str, wave: str) -> pd.DataFrame:
 6     """Weight per brand for one wave, as pandas for modelling. Grain: one
       row per brand."""
 7     return (
 8         pl.scan_parquet(path_glob)
 ❶       .filter(pl.col("wave") == wave)
10         .group_by("brand")
11         .agg(total=pl.col("weight").sum())
12         .sort("brand")
13         .collect()
```

```
    .to_pandas()
)
```

1. **Line 9.** Reduce in the engine that reads the files.
2. **Line 14.** Convert once, at the end, when the result has one row per brand.

One primary engine. Reduce where the data is read. Convert small results at a clear boundary.

Recap card

- **Group-by over 10M rows: pandas, Polars, DuckDB** 0.22 s, 0.11 s, 0.06 s on 4 cores. All fast enough. Choose by size against memory and where data lives.
- **Mixing engines** Reduce first in the engine that reads the files, convert the small result last.
- **pl.from_pandas and the index** Dropped by default. reset_index() first, or include_index=True.
- **When Spark** Data already in the lakehouse, larger than one machine, or a scheduled production job.

Practice

Exercise 1. Multiple choice (level 1)

You have 40 GB of Parquet on a laptop with 16 GB of RAM. You need one table of weighted totals by wave and brand, once. Which is the best first choice?

- a) pandas read_parquet, then groupby
- b) DuckDB or Polars lazy, aggregating while reading
- c) Set up a Spark cluster
- d) Split the files by hand and run pandas on each

Why?

- r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.
- r2) They are newer than pandas.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0, Polars 1.33)

```
import pandas as pd
import polars as pl

```

```
pdf = pd.DataFrame({"respondent_id": ["r1", "r2"], "score": [3,
    4]}).set_index("respondent_id")
print(pl.from_pandas(pdf).columns)
```

Exercise 3. Fill the gap (level 2)

Hand a DuckDB result to Polars, then a Polars result to pandas.

```
top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand")._____(1)()
for_model = top_brands._____(2)()
```

Exercise 4. Spot the bug (level 3)

This function runs out of memory on the full extract. Tap the line that loads far more than the result needs, and fix the function.

What you need to know.

`df.item()` returns the single value of a Polars DataFrame that has exactly one row and one column, as a plain Python value. A query that ends in one aggregate, such as a mean, gives that shape:

```
pl.DataFrame({"x": [4.0]}).item()   # 4.0
```

```
import pandas as pd
import polars as pl


def wave_mean(path: str, wave: str) -> float:
    """Mean weight for one wave."""
    df = pl.read_parquet(path).to_pandas()
    return float(df.loc[df["wave"] == wave, "weight"].mean())
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 4)

Choose the primary engine for each job.

Options: pandas, Polars lazy, DuckDB, Spark.

1. Fit a logistic regression with statsmodels on a 50,000-row survey file
2. Ad hoc SQL joins across 30 CSV and Parquet exports on your laptop
3. Nightly job over tables that already live in the Databricks lakehouse
4. Heavy column transforms over 25 GB of Parquet on one 32 GB machine

Sources

- Group-by sum and mean over 10M rows: pandas 3.0.6 0.22 s, Polars 1.44.2 0.11 s, DuckDB 1.5.5 0.06 s (best of 3, 4 cores). (verified: Measured in this repository's dev environment.)

- pl.from_pandas drops the pandas index unless include_index=True. (verified: Polars API reference; run on Polars 1.33.1 and 1.44.2 in this repository.)

B5 Data validation

Roadmap: Prerequisite for all months. Schema checks with pandera and input models with pydantic.

B5-L1 DataFrame schemas with pandera

Concept

A **schema** states what a DataFrame must look like: which columns exist, their dtypes, and rules on their values. **pandera** checks a pandas DataFrame against a schema and raises when it does not match. Examples here use pandera 0.33 with pandas 3.

```python
import pandera.pandas as pa

RESPONSES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "age": pa.Column(int, pa.Check.in_range(16, 99)),
        "weight": pa.Column(float, pa.Check.gt(0)),
    },
    strict=True,
    coerce=True,
)
clean = RESPONSES.validate(raw, lazy=True)
```

- `pa.Column(dtype, checks, nullable=False)` is the default: nulls fail unless you set `nullable=True`.
- `unique=True` rejects duplicate keys.
- `strict=True` rejects columns the schema does not name. `coerce=True` converts dtypes before checking.
- `lazy=True` collects **every** failure into one `SchemaErrors` exception. Its `failure_cases` attribute is a DataFrame with the column, check and failing value. Without `lazy`, validation stops at the first `SchemaError`.

`SchemaErrors` is not a subclass of `SchemaError`. Catch both when you handle failures. A `strict=True` violation raises `SchemaErrors` even without `lazy=True`.

The class syntax, `pa.DataFrameModel`, does the same with type hints. Add `@pa.check_types` to a function whose parameters are annotated `DataFrame[Model]`, and pandera validates inputs and outputs on every call.

Import from `pandera.pandas`. The plain `import pandera` still works in 0.33, but the pandas-specific module is the documented path.

pandera does not run in this app's browser runtime. The exercises check your code's structure. Run `pytest` locally with the `dev` group to execute schemas for real.

Why it matters

A supplier changes a weight export so that non-responders get weight `0` instead of being left out. The weighting script divides by the sum of weights per cell. Three cells now contain only zeros. The script writes `NaN` shares to the client dashboard, and nobody sees an error.

`pa.Column(float, pa.Check.gt(0))` on the input fails the run on day one, with the failing rows listed in `failure_cases`.

Worked example: weak code

```
1  def load_weights(path):
2      df = pd.read_csv(path)
1      assert "weight" in df.columns
2      df = df.dropna()
5      return df
```

1. **Line 3.** Checks one column exists. Not its type, its values or other columns. And python -O strips asserts.
2. **Line 4.** Silently drops rows with any null, in any column. The count of dropped rows is never reported.

Worked example: strong code

```
1  import pandas as pd
2  import pandera.pandas as pa
3
4  WEIGHTS = pa.DataFrameSchema(
5      {
1          "respondent_id": pa.Column(str, unique=True),
7          "cell": pa.Column(str),
2          "weight": pa.Column(float, pa.Check.gt(0)),
9      },
3      strict=True,
11     coerce=True,
12 )
13
14
15 def load_weights(path: str) -> pd.DataFrame:
16     """Read and validate a weights file. Grain: one row per respondent.
17
18     Raises:
19         pa.errors.SchemaErrors: Listing every failing row and check.
20     """
```

```
④     return WEIGHTS.validate(pd.read_csv(path, dtype={"respondent_id":
    "str"}), lazy=True)
```

1. **Line 6.** The grain, enforced: one row per respondent.
2. **Line 8.** Not null (the default) and positive: zero weights fail loudly.
3. **Line 10.** A new or renamed column in the supplier file fails the run.
4. **Line 21.** lazy=True reports every failure at once, not just the first.

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

Recap card

- **pandera Column defaults** nullable=False. Set nullable=True where nulls are expected.
- **strict=True and coerce=True** strict rejects unknown columns. coerce converts dtypes before checks.
- **validate(df, lazy=True)** Runs every check and raises SchemaErrors with failure_cases.
- **SchemaError vs SchemaErrors** Separate classes. SchemaErrors is not a subclass. Catch both, or better, let them raise.

Practice

Exercise 1. Multiple choice (level 1)

In pandera 0.33, what does `lazy=True` change in `schema.validate(df, lazy=True)`?

- a) Validation runs later, when the DataFrame is first used.
- b) Every failure is collected and raised together as SchemaErrors.
- c) Failing rows are dropped instead of raising.
- d) Only dtypes are checked, not values.

Why?

- r1) You see all problems in one run, with failure_cases listing each one.
- r2) It is faster because it skips checks.

Exercise 2. Predict the output (level 2)

What does this print? (pandera 0.33)

```
1 import pandas as pd
2 import pandera.pandas as pa
3
4 schema = pa.DataFrameSchema({"w": pa.Column(float, pa.Check.gt(0))})
5 try:
```

```
    schema.validate(pd.DataFrame({"w": [1.0, -2.0, -3.0]}), lazy=True)
except pa.errors.SchemaErrors as err:
    print(err.failure_cases["failure_case"].tolist())
```

Exercise 3. Fill the gap (level 2)

Reject duplicate IDs, and reject any column the schema does not name.

```
schema = pa.DataFrameSchema(
    {"respondent_id": pa.Column(str, _____(1)=True), "score":
    pa.Column(int)},
    _____(2)=True,
)
```

Exercise 4. Write the function (level 3)

Define a module constant `SCORES` as a pandera `DataFrameSchema` with:

- `respondent_id`: str, unique.
- `nps`: int, between 0 and 10 inclusive.
- `comment`: str, nulls allowed.
- Unknown columns rejected, dtypes coerced.

Then write `validate_scores(df: pd.DataFrame) -> pd.DataFrame` that returns `SCORES.validate(df, lazy=True)`.

pandera does not run in the browser, so the checker reads your code. Run it locally with `pytest` to see it validate.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SCORES = ...

# Validate an NPS extract. Grain: one row per respondent.
def validate_scores(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd
import pandera.pandas as pa


def validate_scores(df):
    return df
```

Exercise 5. Spot the bug (level 4)

A bad weights file passed through this loader and broke the dashboard. Tap the line that let it through.

```
import pandas as pd
import pandera.pandas as pa
```

```python

WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
    pa.Check.gt(0))}, strict=True)


def load_weights(path: str) -> pd.DataFrame:
    """Read and validate weights."""
    df = pd.read_csv(path)
    try:
        return WEIGHTS.validate(df, lazy=True)
    except pa.errors.SchemaError:
        return df
```

Write down the line number of the bug. Then write the fix.

Sources

- pandera 0.33: lazy=True raises SchemaErrors with a failure_cases DataFrame. (verified: pandera documentation; run on pandera 0.33.1 with pandas 3.0.6 in this repository.)
- SchemaErrors is not a subclass of SchemaError in pandera 0.33. (verified: issubclass check on pandera 0.33.1 in this repository.)
- A strict=True violation raises SchemaErrors even without lazy=True. (verified: Run on pandera 0.33.1 in this repository.)
- import pandera.pandas as pa is the documented import for pandas schemas. (search-only: pandera documentation (docs site not reachable from this environment).)

B5-L2 pydantic models for config and input records

Concept

A dataclass (lesson B2-L2) does not check types. **pydantic** does. A `BaseModel` validates and converts data when you build it, and raises one `ValidationError` that lists every problem. Examples use pydantic 2.12.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)

```

```
12 cfg = WeightingConfig.model_validate(yaml.safe_load(text))
```

- `Field(gt=0, le=10)` adds bounds. `@field_validator("name")` adds your own rule.
- `extra="forbid"` rejects unknown keys, so `minbase: 30` in a YAML file fails instead of being ignored. The default, `"ignore"`, drops them silently.
- `frozen=True` rejects assignment after construction.
- `e.errors()` returns a list of dicts with `loc`, `type` and `msg`. Log it as structured data.

pydantic runs in **lax mode** by default. It converts `"30"` to `30` and `30.0` to `30`, but rejects `"thirty"` and `30.5`. It also converts `True` to `1` for an `int` field. Set `ConfigDict(strict=True)` when inputs must already have the right type.

One gap: `model_copy(update={...})` does **not** validate. `cfg.model_copy(update={"min_base": -1})` returns a config with `min_base = -1`. To change a validated model, build a new one: `WeightingConfig.model_validate({**cfg.model_dump(), "min_base": 30})`.

Use pydantic for records and settings that arrive from outside: YAML, JSON, API payloads. Use pandera (lesson B5-L1) for whole DataFrames. For a list of records, `TypeAdapter(list[Model]).validate_python(rows)` validates them in one call.

Why it matters

A weighting config file had `cap: 0.5` typed where `5` was meant, and `minbase` spelt without the underscore. The loader used a plain dict. The cap shrank every weight above 0.5 down to 0.5, which flattened the weighting, and `min_base` stayed at its default. The report went out with unweighted-looking results.

`WeightingConfig.model_validate` with `gt=1.0` and `extra="forbid"` rejects both mistakes in one error message, before any data is read.

Worked example: weak code

```
1 def load_config(raw):
2     return {
3         "weight_col": raw.get("weight_col", "weight"),
4         "min_base": int(raw.get("min_base", 50)),
5         "cap": float(raw.get("cap", 5.0)),
6     }
```

1. **Line 3.** A missing key falls back to a guess.
2. **Line 4.** A typo such as minbase is ignored. int() accepts negatives.
3. **Line 5.** No bounds: 0.5 passes.

Worked example: strong code

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    """Settings for one weighting run, loaded from YAML."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


def load_config(raw: dict[str, object]) -> WeightingConfig:
    """Validate a parsed YAML mapping.

    Raises:
        pydantic.ValidationError: Listing every invalid or unknown key.
    """
    return WeightingConfig.model_validate(raw)
```

1. **Line 7.** Unknown keys fail. No changes after loading.
2. **Line 9.** Required: no default, so a missing key fails.
3. **Line 11.** A bound that encodes what a cap means.

Validate external settings and records once, at the boundary, with every rule in the model.

Recap card

- **pydantic extra keys, default behaviour** Ignored silently. Use ConfigDict(extra="forbid") for config.
- **pydantic lax mode** Converts "30" and 30.0 to 30, True to 1. Use strict=True to refuse conversions.
- **model_copy(update=...)** Does not validate. Rebuild with model_validate({**m.model_dump(), ...}).
- **pydantic or pandera** pydantic for records and settings from outside. pandera for whole DataFrames.

Practice

Exercise 1. Multiple choice (level 1)

A pydantic 2 model has the default config. A YAML file adds an unknown key `minbase: 30`. What happens on `model_validate`?

- a) ValidationError: extra_forbidden

- b) The key is ignored silently.
- c) min_base is set to 30.
- d) A warning is printed.

Why?

- r1) The default extra setting is "ignore". Set extra="forbid" to reject unknown keys.
- r2) pydantic never checks keys.

Exercise 2. Predict the output (level 2)

What does this print? (pydantic 2.12)

```python
from pydantic import BaseModel, Field


class Cfg(BaseModel):
    min_base: int = Field(default=50, gt=0)


cfg = Cfg(min_base="30")
print(cfg.model_copy(update={"min_base": -1}).min_base)
```

Exercise 3. Fill the gap (level 2)

Reject unknown keys, stop changes after loading, and require `nps` between 0 and 10.

```python
class Response(BaseModel):
    model_config = ConfigDict(extra="_____(1)", _____(2)=True)

    respondent_id: str
    nps: int = Field(ge=0, _____(3)=10)
```

Exercise 4. Write the function (level 3)

Write a pydantic model `WeightingConfig` and a function `load_config(raw)`.

- Fields: `weight_col: str` (required), `min_base: int` default 50 and greater than 0, `cap: float` default 5.0 and greater than 1.0.
- Reject unknown keys. Reject assignment after construction.
- `load_config(raw: dict[str, object]) -> WeightingConfig` validates the mapping.

Your code must pass these checks:

- A valid mapping must build a config with defaults filled in.
- cap must be greater than 1.0 and min_base greater than 0.
- Unknown keys such as minbase must fail. Set extra="forbid".

- weight_col is required. Give it no default.
- Assignment after construction must fail. Set frozen=True.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Settings for one weighting run.
class WeightingConfig(BaseModel): ...

# Validate a parsed YAML or JSON mapping.
def load_config(raw: dict[str, object]) -> WeightingConfig: ...
```

Starter code:

```python
from pydantic import BaseModel, ConfigDict, Field


def load_config(raw):
    return raw
```

Exercise 5. Spot the bug (level 4)

A job lowers the minimum base for a small market. The run then reports subgroups of size 0 as valid. Tap the line with the bug and fix the function so the new value is validated.

```python
from pydantic import BaseModel, ConfigDict, Field


class Cfg(BaseModel):
    """Run settings."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    min_base: int = Field(default=50, gt=0)


def with_min_base(cfg: Cfg, value: int) -> Cfg:
    """Return a copy of cfg with a new min_base."""
    return cfg.model_copy(update={"min_base": value})
```

Write down the line number of the bug. Then write the fix.

Sources

- pydantic 2 default extra behaviour is ignore; extra="forbid" raises extra_forbidden. (verified: pydantic documentation; run on pydantic 2.12.5 and 2.13.5 in this repository.)
- Lax mode converts "30" and 30.0 to int 30 and True to 1, and rejects 30.5 and "thirty". (verified: pydantic conversion table; run on pydantic 2.12.5 in this repository.)

- model_copy(update=...) does not validate the update. (verified: pydantic documentation for model_copy; run on pydantic 2.12.5 in this repository.)

B5-L3 Validate at the boundaries, with severity

Concept

Validation belongs at **boundaries**, where data crosses from one owner to another:

- **On the way in.** After reading a file, API response or config. Check schema, keys and value ranges (lessons B5-L1 and B5-L2).
- **On the way out.** Before writing a table or sending a report. Check the promises you make to the next consumer: grain, row counts, totals, shares that sum to 1.

Checks in the middle of a pure transformation add noise. A function that receives validated input and is unit tested does not need to re-check it.

Not every failed check should stop a run. Give each check a **severity**:

- **CRITICAL** stops the run. Wrong grain, missing required column, empty input, weights that do not sum to the sample size.
- **WARN** records the problem and continues. Missing share above its usual level, a subgroup below its reporting base.

Record results as **structured data**, not prose, so a dashboard or alert filters them:

```
Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    check: str
    severity: Severity
    passed: bool
    observed: float | None
```

Run all checks, collect the results, log them, **then** raise if any CRITICAL check failed. The log keeps every result, even on the run that stops.

Two coding rules make checks trustworthy:

- **Guard every division.** An empty input has no missing share. Return `None`, not a `ZeroDivisionError`.
- **Test `None` with `is None`.** `if share:` treats a real share of `0.0` like a missing one, and the check silently never runs.

Why it matters

A tracker QA step printed "WARNING: low base in 3 cells" to the notebook output and carried on. Nobody reads notebook output for scheduled jobs. The same step never checked for an empty input. The week a supplier delivered an empty file, the job computed shares from zero rows, divided by zero, and wrote `NaN` to every cell of the client dashboard.

Structured results in a table get alerts. A CRITICAL `min_rows` check stops the run before the empty file reaches the dashboard.

Worked example: weak code

```
def qa(df):
    share = df["weight"].isna().sum() / len(df)
    if share:
        if share > 0.1:
            print("WARNING: many missing weights")
    if len(df) == 0:
        print("empty file")
```

1. **Line 2.** Divides by zero on an empty file, before the emptiness check runs.
2. **Line 3.** Truthiness: a share of 0.0 skips the check.
3. **Line 5.** Prose on stdout. Nothing stores or alerts on it.
4. **Line 7.** An empty file is critical, but the run carries on.

Worked example: strong code

```
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome, stored in the run log."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) ->
    list[QaResult]:
    """Evaluate every input check. Never raises."""
    n = len(df)
```

```
    share = None if n == 0 else float(df["weight"].isna().sum()) / n
    return [
        QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
        QaResult("missing_weight", "WARN", share is not None and share <=
max_missing, share),
    ]


def enforce(results: list[QaResult]) -> None:
    """Raise when any CRITICAL check failed."""
    failed = [r.check for r in results if r.severity == "CRITICAL" and
not r.passed]
    if failed:
        raise RuntimeError(f"critical checks failed: {failed}")
```

1. **Line 6.** A closed set of severities, checked by mypy.
2. **Line 22.** Guarded division: no share on an empty frame.
3. **Line 25.** is None, so a real 0.0 passes the check instead of skipping it.
4. **Line 33.** Raised after all checks ran, so the log has the full picture.

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

Recap card

- **Where to validate** At boundaries: after reading input, before writing output.
- **CRITICAL vs WARN** CRITICAL stops the run (raise). WARN records and continues.
- **Order of QA operations** Run every check, log all results, then raise if any CRITICAL failed.
- **Two coding rules for checks** Guard every division. Test for None with is None, never truthiness.

Practice

Exercise 1. Multiple choice (level 1)

A pipeline reads a supplier file, runs five pure transforms, then writes a client table. Where do validation checks belong?

- a) After every transform.
- b) After reading, and before writing.
- c) Only before writing.
- d) Nowhere, if the transforms have unit tests.

Why?

- r1) Those are the boundaries where data changes owner. Tests cover the pure middle.
- r2) Fewer checks run faster.

Exercise 2. Predict the output (level 2)

No weights are missing this week. What does this print?

```python
def missing_check(share: float | None, limit: float) -> str:
    if share:
        return "pass" if share <= limit else "fail"
    return "no data"


print(missing_check(0.0, 0.1))
```

Exercise 3. Put in order (level 2)

Order the steps of one pipeline run.

- Run input checks and collect results
- Write the client table
- Run output checks on the result
- Run the pure transforms
- Log all results, then raise if any CRITICAL failed
- Read the supplier file

Exercise 4. Write the function (level 3)

Write `run_checks(df, min_rows, max_missing) -> list[QaResult]` and `enforce(results) -> None`. `QaResult` and `Severity` are given.

- `min_rows`, CRITICAL: passes when the row count is at least `min_rows`. `observed` is the row count as a float.
- `missing_weight`, WARN: the share of null `weight` values. Passes when the share is at most `max_missing`. On an empty frame the share is `None` and the check fails. `observed` is the share.
- `run_checks` never raises. `enforce` raises `RuntimeError` naming each failed CRITICAL check, and ignores WARN failures.

Your code must pass these checks:

- Check names, severities, observed values and pass flags must match the brief.
- A missing share of 0.0 must pass, with observed 0.0.

- An empty frame must fail min_rows, and give a None share without ZeroDivisionError.
- enforce must raise RuntimeError naming the failed CRITICAL check.
- A WARN failure alone must not raise.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One check outcome.
class QaResult: ...

# Evaluate input checks. Never raises.
def run_checks(
    df: pd.DataFrame,
    min_rows: int,
    max_missing: float,
) -> list[QaResult]: ...

# Stop the run when any CRITICAL check failed.
def enforce(results: list[QaResult]) -> None: ...
```

Starter code:

```python
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df, min_rows, max_missing):
    ...


def enforce(results):
    ...
```

Exercise 5. Architecture design (level 4)

Assign a severity to each check on a weekly brand-tracking run.

Options: CRITICAL, WARN.

1. respondent_id is unique in the input
2. A brand's subgroup base is below 50
3. Weights sum to the sample size within 0.1%
4. Share of missing age is above last month's level

B6 Reproducibility

Roadmap: Prerequisite for all months. Seeds, config, data versioning and pinned dependencies.

B6-L1 Seeds and random generators

Concept

A result you cannot rerun is a result you cannot defend. Randomness enters analysis through sampling, train/test splits, bootstrap intervals, model initialisation and synthetic data. Control each source.

Create a generator and pass it in. NumPy 1.17 introduced `numpy.random.Generator`. Examples use NumPy 2.4.

```
rng = np.random.default_rng(20260925)      # at the edge, once
sample = draw_sample(ids, k=500, rng=rng)  # the function receives rng
```

- `np.random.default_rng(seed)` gives the same stream of numbers for the same seed, on every run and machine with the same NumPy version.
- Functions take `rng: np.random.Generator` as a parameter. They never call `np.random.seed()` or the legacy `np.random.rand()`, which share one hidden global state across your whole program and every library you import.
- One generator, used twice, gives two **different** draws. That is correct: two samples should differ. Rerunning the whole script gives the same pair.
- Need independent streams for parallel workers? `rng.spawn(n)` returns `n` child generators.

Libraries take the seed or generator as a parameter too: `train_test_split(..., random_state=42)` in scikit-learn, `df.sample(n, random_state=rng)` in pandas.

Two hidden sources of non-determinism:

- `hash()` **on strings changes between Python processes** (hash randomisation, `PYTHONHASHSEED`). `hash(respondent_id) % 10` puts the same respondent in a different bucket on each run. Use `hashlib.sha256` for stable buckets.
- **Set iteration order** for strings also changes between runs. Sort before you rely on order.

Record the seed in the run log with the results, so anyone reruns the exact analysis.

Why it matters

A holdout set was defined as `hash(respondent_id) % 10 == 0`. The model scored AUC 0.81 in the notebook. The scheduled retraining job, a new Python process, put a different 10% in the holdout. Some of the old test respondents were now in training, and the reported AUC rose to 0.84 with no change to the model. The team spent a week chasing an improvement that did not exist.

A bucket from `hashlib.sha256` of the ID is the same in every process.

Worked example: weak code

```
 1 import numpy as np
 2
 3 np.random.seed(1)
 4
 5
 6 def draw_sample(ids, k):
 7     return list(np.random.choice(ids, size=k, replace=False))
 8
 9
10 def holdout(ids):
11     return [i for i in ids if hash(i) % 10 == 0]
```

1. **Line 3.** Global state, set at import: any other call to np.random changes what draw_sample returns.
2. **Line 7.** Reads the hidden global generator.
3. **Line 11.** String hash() is randomised per process: a different holdout every run.

Worked example: strong code

```
 1 import hashlib
 2
 3 import numpy as np
 4
 5
 6 def draw_sample(ids: list[str], k: int, rng: np.random.Generator) ->
     list[str]:
 7     """Draw k distinct IDs using the generator passed in."""
 8     return [str(x) for x in rng.choice(ids, size=k, replace=False)]
 9
10
11 def bucket(respondent_id: str, n_buckets: int) -> int:
12     """Stable bucket for an ID, the same in every process."""
13     digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
14     return int(digest, 16) % n_buckets
15
16
17 def main(seed: int) -> None:
18     """The edge: create the generator once and log the seed."""
```

```
19      rng = np.random.default_rng(seed)
20      ...
```

1. **Line 6.** The randomness is a parameter: tests pass a seeded generator.
2. **Line 13.** A cryptographic hash is stable across processes and machines.
3. **Line 19.** One generator, created at the edge from a logged seed.

Seed once at the edge, pass the generator in, and never use hash() for anything that must be stable.

Recap card

- **Reproducible randomness in NumPy 2** rng = np.random.default_rng(seed), created once at the edge and passed in.
- **Why not np.random.seed?** It seeds one global generator that every library shares and advances.
- **Stable bucket for an ID** int(hashlib.sha256(id.encode()).hexdigest(), 16) % n. Never hash(), which is randomised per process.
- **Independent streams for workers** rng.spawn(n).

Practice

Exercise 1. Multiple choice (level 1)

Which is the recommended way to get reproducible random numbers in NumPy 2?

- a) `np.random.seed(42)` at the top of the script
- b) `rng = np.random.default_rng(42)`, passed to the functions that need it
- c) `random.seed(42)` from the standard library
- d) No seed. Results average out.

Why?

- r1) Each Generator owns its state, so no other code can change the stream it gives your function.
- r2) np.random.seed was removed in NumPy 2.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
1 import numpy as np
2
3 rng = np.random.default_rng(1)
4 first = rng.choice(10, size=3, replace=False).tolist()
5 second = rng.choice(10, size=3, replace=False).tolist()
6 print(first == second)
```

Exercise 3. Fill the gap (level 2)

Create a seeded generator and give four parallel workers independent streams.

```
rng = np.random._____(1)(20260925)
worker_rngs = rng._____(2)(4)
```

Exercise 4. Write the function (level 3)

Write `bucket(respondent_id: str, n_buckets: int) -> int`. It returns a number from 0 to `n_buckets - 1` that is the same for the same ID in every Python process. Use `hashlib.sha256` on the UTF-8 bytes of the ID, read the hex digest as an integer, and take it modulo `n_buckets`. Do not use `hash()`.

Your code must pass these checks:

- Buckets must come from int(sha256(id.encode('utf-8')).hexdigest(), 16) % n_buckets.
- Every bucket must be between 0 and n_buckets - 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Stable bucket for an ID, the same in every process.
def bucket(respondent_id: str, n_buckets: int) -> int: ...
```

Starter code:

```
def bucket(respondent_id, n_buckets):
    return hash(respondent_id) % n_buckets
```

Exercise 5. Spot the bug (level 4)

Two analysts run the same bootstrap with seed 7 and get different confidence intervals. One of them imports a plotting helper that draws random jitter with `np.random`. Tap the line that makes the result depend on other code, and fix the function.

What you need to know.

`np.asarray(values)` turns a Python list into a NumPy array, so array methods such as `.mean()` work on it. An input that is already an array passes through without a copy.

```
np.asarray([1.0, 2.0]).mean()   # 1.5
```

```
import numpy as np


def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
    list[float]:
    """Bootstrap means of values."""
```

```
    np.random.seed(seed)
    out = []
    for _ in range(n_boot):
        draw = np.random.choice(values, size=len(values), replace=True)
        out.append(float(draw.mean()))
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- np.random.default_rng(seed) returns a Generator; Generator.spawn(n) returns n independent child generators. (verified: NumPy random documentation; run on NumPy 2.4.6 in this repository.)
- Python's hash() of a str differs between processes unless PYTHONHASHSEED is fixed. (verified: Python data model documentation (object.**hash**); observed across two processes in this repository.)

B6-L2 Record the data, config and code behind every result

Concept

To rerun a result you need three things exactly as they were: the **data**, the **config** and the **code**. Record all three with every run, in a small **run manifest**.

```
manifest = {
    "run_id": run_id,
    "started_utc": datetime.now(timezone.utc).isoformat(),
    "input_sha256": file_sha256(input_path),
    "config_sha256": config_sha256(config),
    "git_commit": git_commit,
    "seed": seed,
    "packages": {"pandas": version("pandas"), "numpy": version("numpy")},
}
```

Data. A file name and a date do not identify content. Suppliers resend files under the same name. A **content hash** does: `hashlib.sha256` over the file bytes changes if any byte changes. `hashlib.file_digest(f, "sha256")` (Python 3.11 and later) reads the file in chunks, so large files do not load into memory. Keep raw inputs immutable: write each delivery to a new path and never overwrite it. For lakehouse tables, record the Delta table version instead (lesson A3-L3).

Config. Hash the config you loaded, not the file on disk, so defaults count. Serialise with `json.dumps(config, sort_keys=True)` first. Without `sort_keys`, the same settings in a different key order give a different hash.

Code. Record the Git commit. Refuse to produce client output from a working tree with uncommitted changes, or record that it was dirty.

Environment. `importlib.metadata.version("pandas")` returns the installed version. The lock file (lesson B6-L3) covers the rest.

Store the manifest next to the output, or as a row in a run log table. When someone asks "why did this number change?", compare two manifests. The field that differs is where to look.

Why it matters

A client queried a brand share that moved 3 points between two versions of the same wave's report. The analysis folder held `wave12.csv`, the config and the notebook, all edited since. Nobody knew whether the supplier had resent the file, a setting had changed, or the code had. Reconstructing the answer took four days.

With manifests, the two runs differed in one field: `input_sha256`. The supplier had resent the file with 212 late completes.

Worked example: weak code

```
1 df = pd.read_csv("data/wave12.csv")
2 results = analyse(df, CONFIG)
3 results.to_csv(f"out/wave12_{date.today()}.csv")
```

1. **Line 1.** The name says nothing about the content. A resent file has the same name.
2. **Line 3.** A date, but no record of the input, config, code or seed behind it.

Worked example: strong code

```
1  import hashlib
2  import json
3  from pathlib import Path
4
5
6  def file_sha256(path: Path) -> str:
7      """SHA-256 of a file's bytes, read in chunks."""
8      with path.open("rb") as f:
1          return hashlib.file_digest(f, "sha256").hexdigest()
10
11
12 def config_sha256(config: dict[str, object]) -> str:
13     """SHA-256 of a config, independent of key order."""
```

```
(2)     text = json.dumps(config, sort_keys=True)
15      return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

1. **Line 9.** Content identity: any changed byte changes the hash.
2. **Line 14.** Stable serialisation: key order does not change the hash.

Hash the input and the loaded config, record the commit and the seed, and store the manifest with the output.

Recap card

- **Three things to record for a rerun** Data (content hash or table version), config (hash of the loaded config), code (Git commit).
- **Hash a large file** with path.open("rb") as f: hashlib.file_digest(f, "sha256") (Python 3.11+).
- **Stable config hash** sha256 of json.dumps(config, sort_keys=True).encode("utf-8").
- **Raw inputs** Immutable: each delivery at a new path, never overwritten.

Practice

Exercise 1. Multiple choice (level 1)

Which value best identifies the exact input file a result came from?

- a) The file name
- b) The file's modified time
- c) The SHA-256 of the file's bytes
- d) The row count

Why?

- r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.
- r2) Hashes are shorter than file names.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import json
2
3 print(json.dumps({"min_base": 50, "cap": 5.0}, sort_keys=True))
```

Exercise 3. Fill the gap (level 2)

Hash a large file without loading it into memory.

```
1 with path.open("_____(1)") as f:
```

```
    digest = hashlib._____(2)(f, "sha256").hexdigest()
```

Exercise 4. Write the function (level 3)

Write two functions.

- `file_sha256(path: Path) -> str` returns the hex SHA-256 of the file's bytes.
- `config_sha256(config: dict[str, object]) -> str` returns the hex SHA-256 of `json.dumps(config, sort_keys=True)` encoded as UTF-8, so key order does not matter.

Your code must pass these checks:

- file_sha256 must hash the file's raw bytes with SHA-256.
- Key order must not change the config hash. Use json.dumps(..., sort_keys=True).
- Hash json.dumps(config, sort_keys=True) encoded as UTF-8.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# SHA-256 of a file's bytes, read in chunks.
def file_sha256(path: Path) -> str: ...

# SHA-256 of a config, independent of key order.
def config_sha256(config: dict[str, object]) -> str: ...
```

Starter code:

```python
import hashlib
import json
from pathlib import Path


def file_sha256(path):
    ...


def config_sha256(config):
    return hashlib.sha256(str(config).encode()).hexdigest()
```

Exercise 5. Architecture design (level 4)

Choose what the run manifest records for a monthly client report built from a supplier CSV.

Options: SHA-256 of the input file, with the raw file kept unchanged, The input file name.

1. Input data
2. Settings

3. Code
4. Where the manifest lives

Sources

- hashlib.file_digest was added in Python 3.11. (verified: Python hashlib documentation; run on Python 3.11 in this repository.)

B6-L3 Pinned dependencies and lock files

Concept

`pyproject.toml` states what your project **accepts**: `pandas>=3.0,<4`. A **lock file** records what was **installed**: `pandas 3.0.6`, `numpy 2.4.6`, and every indirect dependency, each with a file hash. You need both.

- The ranges in `pyproject.toml` say which versions your code supports. Keep them honest: a lower bound you have tested, an upper bound at the next major version when the library breaks things across majors.
- The lock file makes every install identical: your laptop, a colleague's, CI and the scheduled job. Commit it.

With **uv** (examples use uv 0.8):

```
1 uv lock                  # resolve pyproject.toml ranges, write uv.lock
2 uv sync --locked         # install exactly uv.lock; fail if it is out of
     date
3 uv lock --upgrade-package pandas   # move one package on purpose
```

`uv sync --locked` exits with an error when `pyproject.toml` has changed since the lock was written: "The lockfile at `uv.lock` needs to be updated, but `--locked` was provided." Use it in CI, so a dependency change without a new lock fails the build instead of resolving something new on the runner.

Without uv, `pip-compile --generate-hashes` (pip-tools) writes a fully pinned `requirements.txt` with hashes, and `pip install --require-hashes -r requirements.txt` refuses any file whose hash differs.

`pip freeze > requirements.txt` is a weaker substitute. It lists what happens to be in your environment, including packages you installed by hand for a one-off test, with no hashes and no record of why each is there.

Upgrade on purpose: one dependency at a time, on a branch, with tests. Record the reason in the commit message. Never let a fresh install pick up new versions silently.

Why it matters

A forecasting job ran on a fresh container every night. `requirements.txt` said `pandas>=2.0`. The night pandas 3.0 was released, the container installed it. The job's text columns switched to the new `str` dtype, a `.str.contains` call behaved differently on missing values, and the job filtered out 6% of rows. Nobody had changed a line of code.

A committed lock file, installed with `uv sync --locked`, keeps the old pandas until someone upgrades it deliberately, with tests.

Worked example: weak code

```
  1 # requirements.txt
❶ 2 pandas
❷ 3 scikit-learn>=1.0
  4 numpy
❸ 5 jupyter
```

1. **Line 2.** No bounds: any future major version installs on the next fresh build.
2. **Line 3.** A lower bound only. Every build resolves whatever is newest that day.
3. **Line 5.** A development tool mixed into the runtime requirements.

Worked example: strong code

```
  1 # pyproject.toml
  2 [project]
  3 requires-python = ">=3.12"
❶ 4 dependencies = ["pandas>=3.0,<4", "scikit-learn>=1.8,<2", "numpy>=2.4,<3"]
  5
  6 [dependency-groups]
❷ 7 dev = ["pytest>=9", "ruff>=0.13", "jupyterlab>=4"]
  8
  9 # CI
❸ 10 uv sync --locked --group dev
```

1. **Line 4.** Honest ranges: tested lower bounds, next major excluded.
2. **Line 7.** Development tools in their own group, not in the runtime install.
3. **Line 10.** Installs exactly uv.lock. A stale lock fails the build.

Ranges in pyproject.toml, exact versions and hashes in a committed lock file, and --locked in CI.

Recap card

- **pyproject.toml vs lock file** pyproject.toml: ranges you support. Lock: exact versions and hashes of everything installed.
- **uv sync --locked** Installs exactly uv.lock and fails if pyproject.toml changed without a new lock.

- **Hashes without uv** pip-compile --generate-hashes, then pip install --require-hashes -r requirements.txt.
- **Why not pip freeze?** It captures one environment's accidents, with no hashes and no reasons.

Practice

Exercise 1. Multiple choice (level 1)

What does a lock file add that `pandas>=3.0,<4` in pyproject.toml does not?

- a) The exact version of every direct and indirect dependency, with file hashes.
- b) A list of packages your code imports.
- c) Faster imports at runtime.
- d) Nothing. They hold the same information.

Why?

- r1) Every install then gets identical packages, on every machine.
- r2) It lets pip skip dependency resolution entirely on first install.

Exercise 2. Predict the output (level 2)

You add `polars>=1.30` to `dependencies` in pyproject.toml and push without running `uv lock`. CI runs `uv sync --locked`. What happens?

```
1 $ uv sync --locked
```

Exercise 3. Fill the gap (level 2)

Write a pinned requirements file with hashes using pip-tools, then install it so any mismatched file is refused.

```
1 pip-compile --_____(1) requirements.in
2 python -m pip install --_____(2) -r requirements.txt
```

Exercise 4. Put in order (level 3)

Order the steps to upgrade scikit-learn in a locked project.

- Commit pyproject.toml and uv.lock with the reason
- Create a branch
- uv lock --upgrade-package scikit-learn
- Raise the bound in pyproject.toml if the new version is outside the range
- Run the test suite and compare key outputs with the last release

Exercise 5. Architecture design (level 4)

Set up dependency management for a nightly scoring job.

Options: Tested lower bounds, next major excluded, in pyproject.toml, Package names only, no versions, Exact == pins in pyproject.toml, no lock.

1. Declared dependencies
2. Exact versions
3. Install in CI and the job

Sources

- uv sync --locked fails when uv.lock needs updating, with the message quoted in this lesson. (verified: Run with uv 0.8.17 in this repository's scratch directory.)
- pip-compile --generate-hashes and pip install --require-hashes. (search-only: pip-tools and pip documentation (not run here).)
- Series.str.contains returns None for a missing value on pandas 2.3 (object dtype) and False on pandas 3.0 (str dtype). (verified: Run on pandas 2.3.3 and 3.0.6 in this repository.)

B7 Exploratory analysis done well

Roadmap: Prerequisite for all months. Profiling, visual checks and documented assumptions.

B7-L1 Profile a dataset before you analyse it

Concept

Before any chart or model, answer five questions about a new dataset. Each takes one line of pandas, and each catches a class of error that no later step will.

1. **Shape and grain.** How many rows, and what does one row mean? Check the claimed key is unique: `df.duplicated(subset=["respondent_id", "wave"]).sum()` should be 0.
2. **Dtypes.** `df.dtypes`. IDs read as floats, dates as text and numbers as `str` all show here (lesson B3-L1).
3. **Missingness.** `df.isna().mean()` gives the share missing per column. Look for columns that are nearly empty, and for shares that differ by wave or region.
4. **Cardinality.** `df.nunique()`. A `gender` column with 14 distinct values holds typos. An ID column with fewer distinct values than rows holds duplicates.
5. **Ranges.** `df.describe()`. Ages of 999, negative weights, and dates in 1900 are codes for "missing" or errors.

Put the per-column answers in one table:

```
def profile(df: pd.DataFrame) -> pd.DataFrame:
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing_share": df.isna().mean(),
        "n_unique": df.nunique(),
    })
```

Two pandas defaults to know (pandas 3.0):

- `nunique()` **excludes** missing values. Pass `dropna=False` to count NaN as a value.
- `duplicated()` marks every repeat **except the first** (`keep="first"`). To count every row involved in a duplicate, use `keep=False`.

Save the profile with the analysis. When the next delivery arrives, compare the two profiles. A column whose missing share jumps from 2% to 40% tells you something changed at the source before it changes your results.

Libraries such as ydata-profiling build long HTML reports. They are useful for a first look. The five checks above, in code, are what you rerun on every delivery.

Why it matters

A satisfaction score was coded 1 to 10, with 99 for "don't know". Nobody profiled the file. The mean score for one region came out at 14.2, above the maximum possible value. The analyst assumed a weighting problem and spent a day on the weights. `describe()` would have shown a maximum of 99 in the first minute.

Worked example: weak code

```
df = pd.read_csv("wave12.csv")
df.head()
df.groupby("region")["score"].mean()
```

1. **Line 2.** Five rows show nothing about codes like 99, duplicates or missingness.
2. **Line 3.** Analysis before profiling: 99 codes are averaged in.

Worked example: strong code

```
import pandas as pd


def profile(df: pd.DataFrame) -> pd.DataFrame:
    """One row per column: dtype, missing share and distinct values."""
    return pd.DataFrame(
        {
            "dtype": df.dtypes.astype(str),
            "missing_share": df.isna().mean(),
            "n_unique": df.nunique(),
        }
    )


def duplicate_rows(df: pd.DataFrame, key: list[str]) -> int:
    """Rows whose key appears more than once, all copies counted."""
    return int(df.duplicated(subset=key, keep=False).sum())
```

1. **Line 9.** Share missing per column, comparable across deliveries.
2. **Line 17.** keep=False counts every copy, not only the repeats.

Shape and grain, dtypes, missingness, cardinality and ranges, in code, before any analysis.

Recap card

- **Five profiling questions** Shape and grain, dtypes, missingness, cardinality, ranges.
- **nunique() and missing values** Excluded by default. dropna=False counts NaN as a value.
- **Count every row in a duplicate group** df.duplicated(subset=key, keep=False).sum().
- **Why save the profile?** Compare it with the next delivery to spot source changes early.

Practice

Exercise 1. Multiple choice (level 1)

A satisfaction score should run from 1 to 10. Which call shows at once that some rows hold 99?

- a) `df.head()`
- b) `df["score"].describe()`
- c) `df.dtypes`
- d) `df.shape`

Why?

- r1) describe() reports min and max, so a max of 99 stands out.
- r2) describe() removes invalid values.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
1 import pandas as pd
2
3 ids = pd.Series(["a", "b", "b", None])
4 print(ids.nunique(), ids.duplicated().sum())
```

Exercise 3. Fill the gap (level 2)

Count every row that shares a (respondent_id, wave) key with another row, and the share of missing values per column.

```
1 dupes = df.duplicated(subset=["respondent_id", "wave"],
      keep=_____(1)).sum()
2 missing = df.isna()._____(2)()
```

Exercise 4. Write the function (level 3)

Write `profile(df: pd.DataFrame) -> pd.DataFrame`. Return one row per input column, indexed by column name, with columns:

- `dtype`: the dtype as a string.
- `missing_share`: share of missing values.
- `n_unique`: distinct non-missing values.
- `min` and `max`: for numeric columns, the minimum and maximum. For other columns, missing (`None` or `NaN`).

What you need to know.

Two pandas tools build this kind of summary:

- `pd.api.types.is_numeric_dtype(series)` is `True` for integer, float and boolean columns, and `False` for text.
- `pd.DataFrame.from_dict(rows, orient="index")` turns a dict of dicts into a frame. Each outer key becomes a row label, and each inner key becomes a column.

```
rows = {"a": {"n": 1, "kind": "x"}, "b": {"n": 2, "kind": "y"}}
pd.DataFrame.from_dict(rows, orient="index")
#    n kind
# a  1    x
# b  2    y
```

`series.isna().mean()` is the share of missing values, because `True` counts as 1.

Your code must pass these checks:

- Return one row per column, with columns dtype, missing_share, n_unique, min, max.
- missing_share is isna().mean(). n_unique excludes missing values.
- Numeric columns get min and max. Text columns get a missing value.
- dtype must be the column dtype as a string.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Profile each column: dtype, missing share, distinct values, numeric
    range.
def profile(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def profile(df):
```

```
    return df.describe()
```

Exercise 5. Spot the bug (level 4)

The QA report says 1 duplicated respondent row, but 2 rows in the file share the same ID, and both must be sent back to the supplier. Tap the line with the bug and fix it.

```
import pandas as pd


def rows_with_duplicate_ids(df: pd.DataFrame) -> pd.DataFrame:
    """Every row whose respondent_id appears more than once."""
    mask = df.duplicated(subset=["respondent_id"])
    return df[mask]
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas nunique() excludes missing values by default and duplicated() defaults to keep="first". (verified: pandas API reference; run on pandas 3.0.6 and 3.0.2 in this repository.)

B7-L2 Plot before you summarise

Concept

Summary statistics hide shape. **Anscombe's quartet** (1973) is four small datasets with the same means, variances and correlation. Computed in this course's build for the first three:

Set	mean of y	variance of y	correlation with x
I	7.50	4.13	0.816
II	7.50	4.13	0.816
III	7.50	4.12	0.816

Plotted, set I is a noisy line, set II is a curve, and set III is a perfect line with one outlier. The numbers alone cannot tell them apart.

Four plots catch most problems:

- **Histogram** of each key numeric column. Shows skew, gaps, spikes at codes such as 99 or 0, and more than one peak.
- **Scatter plot** of each pair you will model or correlate. Shows curves, clusters and single points that drive a correlation.

- **Line over time** of counts and key rates by wave. Shows breaks in the series when a questionnaire or supplier changed.
- **Bar chart of missingness** by column and by subgroup.

Make each plot honest:

- Label both axes with units, and give every figure a title that states what it shows.
- Choose histogram bins on purpose. Too few hide a spike at 99. Try two or three bin counts.
- Use a log scale for heavily skewed values such as spend, and say so on the axis.
- Plot the raw data before you plot the smoothed or aggregated version.

Write plotting code as functions that return a `matplotlib.figure.Figure` (matplotlib 3.10). Build with `fig, ax = plt.subplots()`, not the global `plt.plot()` state. A function that returns a figure is testable: a test reads `ax.get_xlabel()` or counts the bars. The caller decides whether to show it or save it.

Why it matters

A team reported a correlation of 0.62 between ad recall and purchase intent across 40 markets, and the client planned a budget shift on it. A scatter plot showed 39 markets in a flat cloud and one market far in the top-right corner. Without that market the correlation was close to zero. The decision was about one market, not 40.

Worked example: weak code

```
❶ print(df["spend"].mean(), df["spend"].std())
❷ plt.hist(df["spend"])
❸ plt.show()
```

1. **Line 1.** Summaries first: a skewed spend column makes the mean misleading.
2. **Line 2.** Global pyplot state, default bins, no labels, no title.
3. **Line 3.** Nothing returned: nothing to test or save.

Worked example: strong code

```
1 import matplotlib.pyplot as plt
2 from matplotlib.figure import Figure
3
4
5 def plot_distribution(values: list[float], title: str, xlabel: str, bins:
     int = 30) -> Figure:
```

```
6     """Histogram of values with labelled axes. The caller shows or saves
      it."""
(1)   fig, ax = plt.subplots(figsize=(6, 4))
(2)   ax.hist(values, bins=bins)
9     ax.set_title(title)
10    ax.set_xlabel(xlabel)
11    ax.set_ylabel("Respondents")
(3)   return fig
```

1. **Line 7.** An explicit figure and axes, not the global pyplot state.
2. **Line 8.** Bins are a parameter, so you try several.
3. **Line 12.** Returned for the caller to show, save or test.

Plot distributions, pairs and time series before summarising, with functions that return labelled figures.

Recap card

- **Anscombe's quartet** Same mean, variance and correlation (0.816), different shapes. Plot before summarising.
- **Four first plots** Histograms, scatter plots of modelled pairs, lines over waves, missingness bars.
- **Testable plotting code** fig, ax = plt.subplots(), set labels on ax, return fig. No plt.show() inside.
- **Histogram bins** Choose on purpose and try several. Too few hide spikes at codes.

Practice

Exercise 1. Multiple choice (level 1)

Two variables have correlation 0.816 in four different datasets. What can you conclude about the relationship without plotting?

- a) It is linear in all four.
- b) It is the same shape in all four.
- c) Very little about its shape.
- d) An outlier is present in all four.

Why?

- r1) Anscombe's quartet has identical correlations with a line, a curve and an outlier-driven fit.
- r2) Correlation is always misleading.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

What you need to know.

`np.corrcoef(x, y)` returns a 2 by 2 matrix of Pearson correlations: each variable with itself on the diagonal (always 1.0), and the correlation between `x` and `y` at `[0, 1]` and `[1, 0]`.

```
np.corrcoef([1, 2, 3], [2, 4, 7])[0, 1]   # 0.993...
```

```
import numpy as np

x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
print(round(np.corrcoef(x, y1)[0, 1], 2) == round(np.corrcoef(x, y3)[0,
    1], 2))
```

Exercise 3. Fill the gap (level 2)

Create an explicit figure and axes, and label the x axis.

```
fig, ax = plt._____(1)()
ax.hist(values, bins=30)
ax._____(2)("Monthly spend (GBP)")
```

Exercise 4. Write the function (level 3)

Write `plot_distribution(values, title, xlabel, bins=30) -> Figure`. Draw a histogram with `bins` bins on one axes, set the title and x label from the arguments, set the y label to `"Respondents"`, and return the figure. Do not call `plt.show()`.

Your code must pass these checks:

- Return a matplotlib Figure.
- Set the title and x label from the arguments, and the y label to Respondents.
- Use the bins argument for the histogram.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Histogram of values with labelled axes.
def plot_distribution(
    values: list[float],
    title: str,
    xlabel: str,
    bins: int = 30,
) -> Figure: ...
```

Starter code:

```python
import matplotlib.pyplot as plt


def plot_distribution(values, title, xlabel, bins=30):
    plt.hist(values)
    plt.show()
```

Exercise 5. Architecture design (level 4)

Choose the first plot for each question.

Options: Histogram, Scatter plot, Line over waves, Bar chart of missing share.

1. Does the age column hold codes such as 999?
2. Is the recall and intent correlation driven by a few markets?
3. Did the supplier change in wave 9 shift the awareness rate?
4. Which questions did the new mobile layout leave unanswered?

Sources

- Anscombe's sets I to III share mean 7.50, variance about 4.12 to 4.13 and correlation 0.816. (verified: Anscombe (1973), The American Statistician 27(1); computed with NumPy 2.4.6 in this repository.)

B7-L3 Write assumptions down, then turn them into checks

Concept

Every analysis rests on assumptions: what a row means, how "don't know" is coded, which respondents are in scope, that weights sum to the sample size. Unwritten, they live in one analyst's head and break silently when the data changes.

Keep an **assumptions log** next to the analysis. One entry per assumption, with the same fields each time:

```yaml
- id: A3
  statement: "score uses 1 to 10; 98 = refused, 99 = don't know"
  evidence: "Questionnaire v4, Q12; profile of waves 1 to 11 shows only
   1-10, 98, 99"
  handling: "98 and 99 become missing before any statistic"
  check: "score in 1..10 or missing, after recoding"
  owner: "analysis team"
  last_verified: 2026-09-01
```

Then make the assumption **executable**. The `handling` becomes code, and the `check` becomes a QA result (lesson B5-L3) that runs on every delivery.

```python
MISSING_CODES: dict[str, list[int]] = {"score": [98, 99]}
    # from config, per A3


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) ->
    pd.DataFrame:
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col,
    vals in codes.items()})
```

Recode **per column**. `df.replace(99, np.nan)` on the whole frame also blanks a real age of 99. `Series.mask(cond)` sets values to missing where the condition holds and leaves the rest.

Three kinds of assumption to write down every time:

- **Structure.** Grain, keys, which columns are required.
- **Coding.** Missing codes, scales, units, category lists.
- **Scope.** Who is included: age limits, completes only, which markets and waves.

When a check fails, the log tells you which belief was wrong and who to ask. When a client asks "why is this respondent excluded?", the answer has an ID.

Why it matters

An analyst knew that 99 meant "don't know" on the satisfaction scale and removed those rows by hand in a notebook. The analyst left. The next wave went through the scripted pipeline, which did not know. The regional mean rose from 7.1 to 11.4. It was published, because 11.4 looked like a large improvement rather than an impossible value on a 1 to 10 scale.

A logged assumption with an executable check would have failed the run on the first 99.

Worked example: weak code

```python
# 99 = don't know, drop them (ask Sam)
df = df[df["score"] != 99]
df = df.replace(98, np.nan)
```

1. **Line 1.** An assumption in a comment, tied to one person, with no check.
2. **Line 2.** Drops whole rows, so other answers from those respondents are lost too.
3. **Line 3.** Whole-frame replace: any column with a real 98 is blanked.

Worked example: strong code

```python
import pandas as pd


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) -> pd.DataFrame:
    """Set listed codes to missing, per column (assumption A3). Other columns untouched."""
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col, vals in codes.items()})


def score_in_range(df: pd.DataFrame, low: int, high: int) -> bool:
    """Check for A3: every non-missing score lies in [low, high]."""
    s = df["score"].dropna()
    return bool(s.between(low, high).all())
```

1. **Line 6.** Per-column recoding from config. The respondent keeps their other answers.
2. **Line 12.** The assumption as a check, run on every delivery.

Log each assumption with evidence and owner, implement its handling per column, and check it on every run.

Recap card

- **Fields of an assumptions log entry** id, statement, evidence, handling, check, owner, last_verified.
- **Recode missing codes safely** Per column: df[col].mask(df[col].isin(codes)). Never a whole-frame replace.
- **Three kinds of assumption** Structure (grain, keys), coding (codes, scales, units), scope (who is included).
- **From assumption to code** Handling in config-driven code, and a QA check that runs on every delivery.

Practice

Exercise 1. Multiple choice (level 1)

Where should the assumption "98 and 99 mean refused and don't know on score" live?

- a) In a code comment where the rows are dropped.
- b) In the assumptions log, in config for the handling, and in a QA check.
- c) In the analyst's notes.
- d) Nowhere: everyone knows the convention.

Why?

- r1) Written, applied and checked in code, it survives staff changes and new deliveries.
- r2) Logs are required by law.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"score": [3, 99, 7], "age": [99, 40, 30]})
print(df.replace(99, np.nan)["age"].tolist())
```

Exercise 3. Put in order (level 2)

Order the life of one assumption.

- Observe the coding in the questionnaire and the profile
- Implement the handling in code, driven by config
- Write the assumption, evidence and owner in the log
- Update last_verified when the check or the source is reviewed
- Add a QA check that runs on every delivery

Exercise 4. Write the function (level 3)

Write `recode_missing(df, codes) -> pd.DataFrame`. `codes` maps a column name to a list of values that mean "missing" in that column. Return a new frame where those values are missing in those columns only. Leave other columns and the input frame unchanged.

Your code must pass these checks:

- Listed codes in the listed column must become missing.
- Other columns must keep their values, including a real 99 or 98.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Set listed codes to missing, per column.
def recode_missing(
    df: pd.DataFrame,
    codes: dict[str, list[int]],
) -> pd.DataFrame: ...
```

Starter code:

```
import numpy as np
```

```
import pandas as pd


def recode_missing(df, codes):
    for vals in codes.values():
        df = df.replace(vals, np.nan)
    return df
```

Exercise 5. Architecture design (level 4)

Classify each assumption and pick its check.

Options: Structure: check key uniqueness, Coding: check values against the codebook, Scope: check inclusion rules.

1. One row per respondent per wave
2. region uses codes 1 to 12 from the lookup table
3. Only respondents aged 18 or over are reported
4. Only completed interviews are in the file

Sources

- DataFrame.replace(99, np.nan) changes every column holding 99; Series.mask(cond) sets values where cond is True to missing. (verified: pandas API reference; run on pandas 3.0.6 in this repository.)

B8 Statistical correctness

Roadmap: Prerequisite for all months. Survey weights, confidence intervals, effect sizes and multiple comparisons.

B8-L1 Survey weights and the effective sample size

Concept

A survey sample rarely matches its population. If young adults are 20% of the population but 10% of your completes, each young respondent gets a larger **weight** so the weighted sample matches the population.

The **weighted mean** divides by the sum of the weights, not by the row count:

```
weighted_mean = (y * w).sum() / w.sum()      # same as np.average(y, weights=w)
```

Weights have a cost. Unequal weights make estimates less precise than the raw sample size suggests. Kish's **effective sample size** measures that cost:

```
n_eff = w.sum() ** 2 / (w ** 2).sum()
design_effect = n / n_eff
```

Equal weights give `n_eff = n`. A sample of 5 with weights 0.5, 0.5, 1, 2 and 4 has `n_eff = 2.98`: it carries about as much information as three unweighted respondents. The design effect is 1.68.

Rules that prevent the common mistakes:

- **Use weights for population estimates:** shares, means and totals you report as "UK adults".
- **Use `n_eff`, not `n`, for precision:** confidence intervals, significance tests and minimum base rules (lesson B8-L2).
- **Report the unweighted base** next to every weighted figure, so readers see how many people stand behind it.
- **Drop rows where the value or the weight is missing** before you compute. A missing weight is not zero, and a missing answer is not a zero answer.
- **Guard the division.** An empty subgroup or a zero weight total has no mean. Return `None`.

Check weights before using them: all positive, none missing, and a sum equal to the sample size or the population total, whichever the weighting

scheme promises. Extreme weights (above about 5 times the mean) are a warning sign. Trimming them is a design decision, logged as an assumption (lesson B7-L3).

Why it matters

A brand tracker reported that awareness among 18 to 24s rose from 41% to 52%, "significant at 95%". The test used the 180 raw respondents in the group. Their weights were uneven, and the effective sample size was 71. With 71, the change was well inside the margin of error. The client moved budget on a result the data did not support.

Worked example: weak code

```
share = (df["aware"] * df["weight"]).mean()
se = (share * (1 - share) / len(df)) ** 0.5
```

1. **Line 1.** Divides by the row count, not the weight total. Only right if weights average exactly 1.
2. **Line 2.** Uses raw n, overstating precision whenever weights vary.

Worked example: strong code

```
import pandas as pd


def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of value, ignoring rows with a missing value or weight."""
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)


def effective_n(weights: pd.Series) -> float | None:
    """Kish effective sample size: (sum w)^2 / sum(w^2)."""
    w = weights.dropna()
    sum_sq = (w**2).sum()
    if sum_sq == 0:
        return None
    return float(w.sum() ** 2 / sum_sq)
```

1. **Line 6.** Rows missing either the answer or the weight do not count.
2. **Line 10.** Divides by the weight total.
3. **Line 19.** The n to use for precision.

Weighted estimates divide by the weight total. Precision uses the effective sample size.

Recap card

- **Weighted mean** sum(y * w) / sum(w), over rows with both present. Same as np.average(y, weights=w).
- **Kish effective sample size** (sum w)^2 / sum(w^2). Design effect = n / n_eff.
- **Which n for intervals and suppression?** n_eff. Report the raw base as well.
- **Missing weight** Not zero. Drop the row from weighted estimates, and investigate.

Practice

Exercise 1. Multiple choice (level 1)

A subgroup has 180 respondents and an effective sample size of 71. Which number belongs in the standard error of its weighted share?

- a) 180
- b) 71
- c) The sum of the weights
- d) 180 minus 71

Why?

- r1) Unequal weights reduce precision, and n_eff measures the information that remains.
- r2) Smaller numbers are always safer.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
import numpy as np

y = np.array([1, 0, 1, 0, 1])
w = np.array([0.5, 0.5, 1.0, 2.0, 4.0])
print(np.average(y, weights=w), y.mean())
```

Exercise 3. Fill the gap (level 2)

Compute Kish's effective sample size and the design effect.

```
n_eff = w.sum() ** 2 / (w ** _____(1)).sum()
deff = _____(2) / n_eff
```

Exercise 4. Write the function (level 3)

Write two functions.

- `weighted_share(df, value, weight) -> float | None`: the weighted mean of `value`, ignoring rows where the value or the weight is missing. Return `None` when no rows remain or the weight total is 0.
- `effective_n(weights: pd.Series) -> float | None`: Kish's effective sample size, ignoring missing weights. Return `None` when the sum of squared weights is 0.

Your code must pass these checks:

- Divide sum(value * weight) by sum(weight), over rows with both present.
- An empty frame or a zero weight total must return None.
- n_eff = sum(w)^2 / sum(w^2), ignoring missing weights. Equal weights give n.
- No weights must return None, not ZeroDivisionError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted mean of value over rows with both value and weight present.
def weighted_share(
    df: pd.DataFrame,
    value: str,
    weight: str,
) -> float | None: ...

# Kish effective sample size, (sum w)^2 / sum(w^2), ignoring missing
  weights.
def effective_n(weights: pd.Series) -> float | None: ...
```

Starter code:

```python
import pandas as pd


def weighted_share(df, value, weight):
    return (df[value] * df[weight]).mean()


def effective_n(weights):
    return len(weights)
```

Exercise 5. Architecture design (level 4)

Choose the right quantity for each part of a weighted subgroup report.

Options: Weighted estimate, Unweighted estimate, Raw base (n), Effective sample size.

1. The share shown in the chart
2. The n in the confidence interval
3. The base printed under the chart
4. The number compared with the minimum reporting base

Sources

- Kish effective sample size is (sum w)^2 / sum(w^2). (verified: Kish (1965), Survey Sampling; computed with NumPy 2.4.6 in this repository.)

B8-L2 Confidence intervals and effect sizes

Concept

A share without an interval is a guess with no stated size. Report every estimate with a **confidence interval**, and every difference with an **effect size**.

Intervals for a proportion. The textbook Wald interval, `p ± 1.96 * sqrt(p(1-p)/n)`, fails at the edges. For 0 out of 20 it gives `[0, 0]`, as if you were certain. The **Wilson** interval behaves well for small `n` and for shares near 0 or 1. For 0 out of 20 it gives `[0, 0.161]`. In statsmodels 0.14:

```
from statsmodels.stats.proportion import proportion_confint
low, high = proportion_confint(count=40, nobs=100, alpha=0.05,
    method="wilson")   # (0.309, 0.498)
```

For a **weighted** share, use the weighted estimate and the effective sample size from lesson B8-L1 in the same formula. The 18 to 24 group from that lesson, with a weighted share of 52% and `n_eff = 71`, has a Wilson interval of 40.6% to 63.2%. With the raw 180 it would be 44.7% to 59.2%, which looks far more precise than the data is.

Effect sizes. A p-value answers "is there any difference?", and with a large sample the answer is nearly always yes. Two groups of 100,000 with shares of 50.5% and 50.0% give p = 0.025, and the difference is too small to matter. State the size:

- The **difference in shares**, in percentage points, with its interval. This is what a client reads.

- **Cohen's h**, `2*asin(sqrt(p1)) - 2*asin(sqrt(p2))`, compares differences across different base rates. Rough guides: 0.2 small, 0.5 medium, 0.8 large.
- For means, the difference with its interval, or **Cohen's d** (difference over pooled standard deviation).

Decide before the analysis what size of difference matters for the decision. A significant difference below that size is reported as "no meaningful change".

Why it matters

A retailer's panel of 90,000 customers showed satisfaction at 7.62 against 7.58 the previous quarter, p = 0.01. The headline read "Satisfaction significantly up". The store managers' bonus scheme was tied to "significant improvement", and bonuses were paid. The change was 0.04 points on a 10-point scale, smaller than the rounding in the published chart.

Worked example: weak code

```
se = (p * (1 - p) / n) ** 0.5
low, high = p - 1.96 * se, p + 1.96 * se
if p_value < 0.05:
    headline = "Significant increase"
```

1. **Line 1.** Wald: zero width at p = 0 or 1, and too narrow for small n.
2. **Line 4.** Significance without size: true for trivial differences in large samples.

Worked example: strong code

```
import math


def wilson_interval(p: float, n: float, z: float = 1.959964) ->
  tuple[float, float] | None:
    """Wilson interval for a share p from n observations (use n_eff when
  weighted)."""
    if n <= 0 or not 0 <= p <= 1:
        return None
    denom = 1 + z**2 / n
    centre = (p + z**2 / (2 * n)) / denom
    half = z / denom * math.sqrt(p * (1 - p) / n + z**2 / (4 * n**2))
    return (max(0.0, centre - half), min(1.0, centre + half))


def cohens_h(p1: float, p2: float) -> float:
    """Effect size for two shares, comparable across base rates."""
    return 2 * math.asin(math.sqrt(p1)) - 2 * math.asin(math.sqrt(p2))
```

1. **Line 4.** n is a float, so n_eff from weights fits.

2. **Line 6.** No interval for an empty group or an impossible share.
3. **Line 16.** An effect size to report beside the difference in points.

Wilson intervals with n_eff for weighted shares, and an effect size beside every p-value.

Recap card

- **Why not the Wald interval?** Zero width at p = 0 or 1, and too narrow for small n. Use Wilson.
- **Interval for a weighted share** Wilson with the weighted p and n_eff.
- **Significant vs meaningful** Large samples make trivial differences significant. Report the size with its interval.
- **Cohen's h** 2asin(sqrt(p1)) - 2asin(sqrt(p2)). About 0.2 small, 0.5 medium, 0.8 large.

Practice

Exercise 1. Multiple choice (level 1)

0 of 20 respondents chose a new brand. What does the Wald interval `p ± 1.96 * sqrt(p(1-p)/n)` give?

- a) [0, 0]
- b) [0, 0.161]
- c) [-0.05, 0.05]
- d) It raises an error.

Why?

- r1) With p = 0 the standard error is 0, so the interval collapses.
- r2) Twenty respondents is too few for any interval.

Exercise 2. Predict the output (level 2)

What does this print? (statsmodels 0.14)

What you need to know.

`proportions_ztest(count, nobs)` from `statsmodels.stats.proportion` tests whether two proportions differ. `count` holds the successes in each group, and `nobs` the group sizes. It returns the z statistic and the two-sided p-value.

```python
from statsmodels.stats.proportion import proportions_ztest

z, p = proportions_ztest([30, 45], [100, 100])   # 30% against 45%
# z = -2.19, p = 0.028
```

```python
from statsmodels.stats.proportion import proportions_ztest
```

```
2
3 stat, p_value = proportions_ztest([50500, 50000], [100000, 100000])
4 print(p_value < 0.05)
```

Exercise 3. Fill the gap (level 2)

Get a 95% Wilson interval for 40 out of 100 with statsmodels.

```
1 low, high = proportion_confint(count=40, nobs=100, alpha=_____(1),
    method="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `wilson_interval(p: float, n: float, z: float = 1.959964) -> tuple[float, float] | None` with the formula:

- `denom = 1 + z**2 / n`
- `centre = (p + z**2 / (2 * n)) / denom`
- `half = z / denom * sqrt(p * (1 - p) / n + z**2 / (4 * n**2))`

Return `(centre - half, centre + half)`, clipped to [0, 1]. Return `None` when `n <= 0` or `p` is outside [0, 1]. `n` is a float so you pass `n_eff` for weighted shares.

Your code must pass these checks:

- 0.4 from 100 must give (0.3094, 0.4980). Check the Wilson formula.
- 0 from 20 must give (0, 0.1611), not a zero-width interval.
- A float n_eff such as 71.0 must work.
- Return None for n <= 0 or p outside [0, 1].

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Wilson score interval for a share.
2 def wilson_interval(
3     p: float,
4     n: float,
5     z: float = 1.959964,
6 ) -> tuple[float, float] | None: ...
```

Starter code:

```
1 import math
2
3
4 def wilson_interval(p, n, z=1.96):
5     se = math.sqrt(p * (1 - p) / n)
6     return (p - z * se, p + z * se)
```

Exercise 5. Architecture design (level 4)

A panel of 90,000 shows satisfaction of 7.62 against 7.58 last quarter, p = 0.01. The team agreed beforehand that 0.2 points matters for decisions. Choose what to report.

Options: No meaningful change in satisfaction, Satisfaction significantly up.

1. Headline
2. Supporting detail
3. Rule for future quarters

Sources

- statsmodels proportion_confint(40, 100, method="wilson") returns (0.3094, 0.4980); method="normal" gives (0, 0) for 0 of 20. (verified: Run on statsmodels 0.14.6 and 0.15.0 in this repository.)
- proportions_ztest([50500, 50000], [100000, 100000]) gives p = 0.025. (verified: Run on statsmodels 0.14.6 in this repository.)
- Cohen's h thresholds of 0.2, 0.5 and 0.8 for small, medium and large. (verified: Cohen (1988), Statistical Power Analysis for the Behavioral Sciences.)

B8-L3 Many tests, and the false positives they bring

Concept

At a 5% significance level, each test of a true null hypothesis has a 5% chance of a false positive. Run 20 such tests and the chance of **at least one** false positive is `1 - 0.95**20 = 0.64`. A simulation in this course's build agreed: 20 t-tests on pure noise, repeated 2,000 times, gave at least one "significant" result 66% of the time.

Survey reports run many tests by default: every brand, every region, every age band, every wave. Some cells will light up by chance every time.

Adjust the p-values for the number of tests in the family. In statsmodels 0.14:

```
from statsmodels.stats.multitest import multipletests
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="holm")
```

- **Bonferroni** multiplies each p-value by the number of tests `m`. Simple and strict.

- **Holm** is a step-down version of Bonferroni. It controls the same error (any false positive in the family) and always rejects at least as many hypotheses. Prefer it to Bonferroni.
- **Benjamini-Hochberg** (`method="fdr_bh"`) controls the **false discovery rate**: the expected share of false positives among the results you flag. It is less strict and suits screening many cells for follow-up.

For p-values 0.01, 0.02, 0.03, 0.04 and 0.20, Bonferroni and Holm reject only the first. Benjamini-Hochberg rejects the first four.

Define the **family** before you look: "the 12 regional comparisons in this chart". Adjust within it. Choosing the family after seeing the results defeats the point.

Two habits that reduce the problem at the source:

- Pre-register the few comparisons that answer the business question. Test those. Label the rest as exploratory.
- Report how many tests you ran. "3 of 40 regions changed significantly" reads very differently from "Region 7 changed significantly".

Why it matters

A monthly tracker tested 14 brands in 9 regions, 126 tests, each at 5%. Every month about six cells were flagged in red on the dashboard. Regional managers were asked to explain their red cells. The following month, different cells turned red. Nobody could explain the pattern, because there was none: 126 tests at 5% produce about six false positives by chance.

Holm adjustment within each brand's family of 9 regions removed nearly all the monthly red cells. The few that remained repeated in the next month.

Worked example: weak code

```
for region, p in p_values.items():
    if p < 0.05:
        flag(region)
```

1. **Line 2.** Each test at 5%, with no adjustment for how many run: false positives are expected.

Worked example: strong code

```
from statsmodels.stats.multitest import multipletests


def flag_regions(p_values: dict[str, float], alpha: float = 0.05) ->
    dict[str, float]:
```

```
5     """Holm-adjusted p-values for one family of regional tests, flagged
      below alpha."""
6     regions = list(p_values)
1     reject, p_adj, _, _ = multipletests([p_values[r] for r in regions],
      alpha=alpha, method="holm")
2     return {r: float(p) for r, p, rej in zip(regions, p_adj, reject,
      strict=True) if rej}
```

1. **Line 7.** One family, adjusted together with Holm.
2. **Line 8.** Report adjusted p-values, so readers see the correction.

Define the family first, adjust within it with Holm or Benjamini-Hochberg, and report how many tests ran.

Recap card

- **Chance of at least one false positive in 20 tests at 5%** 1 - 0.95^20 = 0.64.
- **Holm vs Bonferroni** Same family-wise control. Holm is step-down and rejects at least as many. Prefer Holm.
- **Benjamini-Hochberg** Controls the false discovery rate. Suits screening many cells.
- **Define the family** Before looking at results. Report how many tests ran.

Practice

Exercise 1. Multiple choice (level 1)

You run 20 independent tests at the 5% level. All null hypotheses are true. What is the chance of at least one significant result?

- a) 5%
- b) About 64%
- c) 100%
- d) 0%, because the nulls are true

Why?

- r1) 1 - 0.95^20 = 0.64.
- r2) 20 × 5% = 100%, minus overlap.

Exercise 2. Predict the output (level 2)

What does this print? (statsmodels 0.14)

```
1 from statsmodels.stats.multitest import multipletests
2
3 p = [0.01, 0.02, 0.03, 0.04, 0.20]
```

```
print(multipletests(p, alpha=0.05, method="holm")[0].sum(),
    multipletests(p, alpha=0.05, method="fdr_bh")[0].sum())
```

Exercise 3. Fill the gap (level 2)

Adjust a family of p-values with Holm's method.

```
reject, p_adj, _, _ = _____(1)(p_values, alpha=0.05, method="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `holm_adjust(p_values: list[float]) -> list[float]` without statsmodels. Return adjusted p-values **in the input order**.

1. Sort the p-values ascending. For the k-th smallest (k from 0), compute `min(1, (m - k) * p)`, where `m` is the number of p-values.
2. Walk the sorted list and keep a running maximum, so adjusted values never decrease.
3. Put each adjusted value back at its original position.

Your code must pass these checks:

- Multiply the k-th smallest by (m - k) and keep a running maximum.
- Return adjusted values in the input order, not sorted order.
- Cap at 1.0, and return an empty list for no input.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Holm step-down adjusted p-values, in input order.
def holm_adjust(p_values: list[float]) -> list[float]: ...
```

Starter code:

```
def holm_adjust(p_values):
    m = len(p_values)
    return [min(1.0, p * m) for p in p_values]
```

Exercise 5. Architecture design (level 4)

Choose the approach for each analysis.

Options: Holm, family of the three, Benjamini-Hochberg, No adjustment.

1. Three pre-registered comparisons that decide a campaign's budget
2. Screen 400 item-by-segment cells to pick a few for follow-up research
3. One primary hypothesis, stated before fieldwork

Sources

- multipletests on [0.01, 0.02, 0.03, 0.04, 0.20]: Holm and Bonferroni reject one, fdr_bh rejects four. (verified: Run on statsmodels 0.14.6 and 0.15.0 in this repository.)
- 20 t-tests on noise, 2,000 repetitions: at least one $p < 0.05$ in 66% of repetitions. (verified: Simulated with NumPy 2.4.6 and SciPy 1.17.1 in this repository.)

B9 Experimental design

Roadmap: Prerequisite for all months. Power, sample size and A/B test analysis.

B9-L1 Power and sample size, before fieldwork

Concept

A test with too few people cannot see the effect you care about. It returns "no significant difference", and the team concludes the change did nothing. Decide the sample size **before** fieldwork, from four inputs:

- **Baseline**: the current rate, for example 40% consideration.
- **Minimum detectable effect (MDE)**: the smallest change worth acting on, for example 5 points, to 45%.
- **Significance level** (alpha): usually 0.05, two-sided.
- **Power**: the chance of detecting the MDE if it is real. Usually 0.8.

For two proportions, the sample per group is:

```
z_a = NormalDist().inv_cdf(1 - alpha / 2)
z_b = NormalDist().inv_cdf(power)
p_bar = (p1 + p2) / 2
n = ((z_a * sqrt(2 * p_bar * (1 - p_bar)) + z_b * sqrt(p1 * (1 - p1) + p2
    * (1 - p2))) / (p2 - p1)) ** 2
```

Round up. For 40% to 45% at alpha 0.05 and power 0.8: **1,534 per group**. statsmodels 0.14 agrees: `NormalIndPower().solve_power(effect_size=proportion_effectsize(0.45, 0.40), alpha=0.05, power=0.8)` returns 1,533.05.

The MDE drives everything. Required `n` grows with the inverse square of the effect:

Change	n per group
40% to 45%	1,534
40% to 42%	9,493

Halving the effect you want to detect needs roughly four times the sample.

Two adjustments for survey work:

- **Design effect.** Weighted or clustered samples carry less information (lesson B8-L1). Multiply `n` by the expected design effect: with a design effect of 1.5, 1,533.05 × 1.5 rounds up to 2,300.
- **Subgroups.** If the decision needs the effect within each of four regions, each region needs the full `n`, not a quarter of it.

Write the power calculation into the research plan, with its inputs. If the budget cannot buy the `n`, say so before fieldwork, and agree a larger MDE or a different design.

Why it matters

A packaging test used 300 respondents per cell to compare purchase intent. The new pack scored 43% against 40%. The report said "no significant difference, keep the current pack". With 300 per cell, the test had about a 1 in 4 chance of detecting a real 5-point lift. The result said nothing about the pack. It said the test was too small.

Worked example: weak code

```
1 N_PER_CELL = 300  # what the budget allows
2 result = run_test(N_PER_CELL)
3 if result.p_value > 0.05:
2     conclusion = "No effect"
```

1. **Line 1.** Sample size from budget, with no check of what it can detect.
2. **Line 4.** Absence of evidence from an underpowered test read as evidence of absence.

Worked example: strong code

```
1  import math
2  from statistics import NormalDist
3
4
1  def n_per_group(p1: float, p2: float, alpha: float = 0.05, power: float =
     0.8) -> int:
6      """Sample per group to detect p1 vs p2 with a two-sided test."""
7      if p1 == p2:
2          raise ValueError("p1 and p2 must differ")
9      z_a = NormalDist().inv_cdf(1 - alpha / 2)
10     z_b = NormalDist().inv_cdf(power)
11     p_bar = (p1 + p2) / 2
12     top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 *
     (1 - p1) + p2 * (1 - p2))
```

```
    return math.ceil((top / (p2 - p1)) ** 2)
```

1. **Line 5.** Every input to the decision is a named parameter.
2. **Line 8.** A zero effect needs infinite n. Fail clearly instead of dividing by zero.
3. **Line 13.** Round up: 1,533.05 means 1,534 people.

Fix baseline, MDE, alpha and power first, compute n, adjust for design effect and subgroups, then field.

Recap card

- **Four inputs to a sample size** Baseline, minimum detectable effect, alpha, power.
- **n per group, 40% vs 45%, alpha 0.05, power 0.8** 1,534.
- **Halve the effect you want to detect** About four times the sample.
- **Weighted or subgroup reads** Multiply n by the design effect. Each subgroup needs the full n.

Practice

Exercise 1. Multiple choice (level 1)

A test with 300 per cell finds no significant difference. Power for the effect you care about was 0.25. What can you conclude?

- a) There is no effect.
- b) Very little. The test was unlikely to detect the effect even if it were real.
- c) The effect is negative.
- d) The effect is exactly zero with 75% confidence.

Why?

- r1) Power 0.25 means a real effect of that size is missed 75% of the time.
- r2) Non-significant results are always meaningless.

Exercise 2. Predict the output (level 2)

Required n per group is 1,534 to detect 40% vs 45%. Roughly how many per group to detect 40% vs 42.5%, with the same alpha and power? The code prints the ratio of the two sample sizes, rounded.

```
import math
from statistics import NormalDist


def n_per_group(p1, p2, alpha=0.05, power=0.8):
    z_a = NormalDist().inv_cdf(1 - alpha / 2)
    z_b = NormalDist().inv_cdf(power)
```

```
    p_bar = (p1 + p2) / 2
    top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 * (1 - p1) + p2 * (1 - p2))
    return math.ceil((top / (p2 - p1)) ** 2)


print(round(n_per_group(0.40, 0.425) / n_per_group(0.40, 0.45)))
```

Exercise 3. Fill the gap (level 2)

Compute the sample size with statsmodels for 40% vs 45%.

```
h = proportion_effectsize(0.45, 0.40)
n = NormalIndPower()._____(1)(effect_size=h, alpha=0.05, _____(2)=0.8)
```

Exercise 4. Write the function (level 3)

Write `n_per_group(p1, p2, alpha=0.05, power=0.8, design_effect=1.0) -> int`, using the formula in the concept with `statistics.NormalDist`. Multiply by `design_effect` before rounding up. Raise `ValueError` if `p1 == p2`, or if either share is outside (0, 1).

Your code must pass these checks:

- 40% vs 45% at alpha 0.05, power 0.8 must give 1,534, in either order.
- Use the alpha and power arguments in the z values.
- Multiply by design_effect before rounding up.
- Raise ValueError for equal shares or shares outside (0, 1).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Sample per group to detect p1 vs p2 with a two-sided test.
def n_per_group(
    p1: float,
    p2: float,
    alpha: float = 0.05,
    power: float = 0.8,
    design_effect: float = 1.0,
) -> int: ...
```

Starter code:

```
import math


def n_per_group(p1, p2, alpha=0.05, power=0.8, design_effect=1.0):
    return 300
```

Exercise 5. Architecture design (level 4)

A client wants to know whether a new ad lifts consideration from 40% by at least 5 points, overall and within each of 4 regions. Weighting will give a design effect of about 1.5. The budget buys 4,000 completes. Choose the plan.

Options: 2 × 1,534 × 1.5 = 4,602: slightly above budget, 4,000 is plenty for any read.

1. Sample for the overall read
2. Regional reads
3. Recommendation

Sources

- NormalIndPower().solve_power(effect_size=proportion_effectsize(0.45, 0.40), alpha=0.05, power=0.8) returns 1533.05. (verified: Run on statsmodels 0.15.0 in this repository; matches the closed-form result of 1,534.)
- With 300 per cell, power to detect 40% vs 45% is 0.24. (verified: Computed with statsmodels NormalIndPower in this repository.)

B9-L2 Analyse an A/B test the way it was randomised

Concept

An A/B test compares groups that differ only by chance and by the change you made. Keep that property in the analysis.

1. Check the split first. If you assigned 50/50 and got 50,600 against 49,400, something went wrong: a redirect lost users, a bot filter hit one arm, a crash hit one version. A chi-square test on the counts detects this **sample ratio mismatch** (SRM). For those counts, p = 0.00015. A very small p-value here, below 0.001 by common practice, means stop and find the cause. The treatment effect is not trustworthy until you do.

```
from scipy.stats import chisquare
chisquare([50_600, 49_400]).pvalue        # 0.000148, SciPy 1.17
```

2. Analyse as randomised. Everyone assigned to B stays in B, even if they never saw the new page (intention to treat). Dropping "people who did not see it" from B, but not the matching people from A, breaks the randomisation.

3. Analyse at the unit of randomisation. If you randomised users, compute one outcome per user. Counting page views as independent inflates `n` and shrinks intervals.

4. Report the difference with its interval. For 1,150 of 10,000 in B against 1,000 of 10,000 in A, the lift is 1.5 points, with a 95% interval of 0.6 to 2.4 points. statsmodels 0.14:

```
from statsmodels.stats.proportion import confint_proportions_2indep
confint_proportions_2indep(1150, 10_000, 1000, 10_000)   # (0.0064, 0.0236)
```

5. Compare with the minimum effect you planned for (lesson B9-L1). An interval that lies wholly above the MDE is a clear win. One that straddles it needs more data or a judgement call, stated as such.

Fix the primary metric, the analysis and the stopping rule in writing before launch. Secondary metrics are exploratory and are reported as such (lesson B8-L3).

Why it matters

A checkout test showed a 4% lift in conversion. Nobody checked the split. Arm B had 3% fewer users than arm A, because a slow script in B timed out on older phones, and those users were logged as never arriving. They were also the users least likely to buy. The "lift" came from losing low-converting users from B. When the script was fixed, the lift disappeared.

An SRM check on day one would have flagged the missing 3%.

Worked example: weak code

```
b_seen = b[b["saw_new_page"]]
rate_a = a["converted"].mean()
rate_b = b_seen["converted"].mean()
print("lift", rate_b - rate_a)
```

1. **Line 1.** Filters one arm on a post-assignment event: no longer a fair comparison.
2. **Line 4.** No split check, no interval, no comparison with the planned effect.

Worked example: strong code

```
import math


def srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float:
    """Chi-square p-value (1 df) for the observed split against the planned one."""
    n = n_a + n_b
```

```
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))


def lift_interval(x_a: int, n_a: int, x_b: int, n_b: int, z: float =
  1.959964) -> tuple[float, float, float]:
    """Difference in conversion (B minus A) with a Wald interval, for
  large samples."""
    p_a, p_b = x_a / n_a, x_b / n_b
    se = math.sqrt(p_a * (1 - p_a) / n_a + p_b * (1 - p_b) / n_b)
    diff = p_b - p_a
    return diff, diff - z * se, diff + z * se
```

1. **Line 9.** Chi-square with 1 degree of freedom, in the standard library.
2. **Line 12.** Uses every assigned user, as randomised.

Check the split, keep everyone in their assigned arm, analyse per randomised unit, and report the lift with its interval.

Recap card

- **First step in any A/B analysis** Sample ratio mismatch check: chi-square on arm counts against the plan.
- **Intention to treat** Analyse everyone in the arm they were assigned to, whatever happened next.
- **Unit of analysis** The unit of randomisation, usually the user, not the page view.
- **Report a lift as** The difference with its interval, compared with the planned minimum effect.

Practice

Exercise 1. Multiple choice (level 1)

A 50/50 test ends with 50,600 users in A and 49,400 in B. The SRM check gives p = 0.00015. What next?

- a) Report the treatment effect. A 1% imbalance is small.
- b) Stop and find why B lost users before trusting any result.
- c) Reweight B up to 50,600 and continue.
- d) Drop 1,200 random users from A to balance.

Why?

- r1) A split that differs from the design means something removed users unevenly, and they may differ in outcome.
- r2) Any p-value below 0.05 invalidates a test.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import math


def srm_p_value(n_a, n_b, expected_share_a=0.5):
    n = n_a + n_b
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))


print(srm_p_value(5060, 4940) > 0.05, srm_p_value(50600, 49400) > 0.05)
```

Exercise 3. Put in order (level 2)

Order the analysis of a finished A/B test.

- Compare the interval with the planned minimum effect
- Check the split against the design
- Estimate the primary metric's lift with its interval
- Report secondary metrics as exploratory
- Aggregate to one outcome per randomised user

Exercise 4. Write the function (level 3)

Write `srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float`. Compute the chi-square statistic with expected counts `n * expected_share_a` and `n * (1 - expected_share_a)`, and return the 1-degree-of-freedom p-value `math.erfc(math.sqrt(stat / 2))`. Raise `ValueError` if either count is negative, the total is 0, or `expected_share_a` is not strictly between 0 and 1.

Your code must pass these checks:

- The 50/50 p-values must match chi-square with 1 df: 0.000148 and 0.230.
- Use expected_share_a for the expected counts.
- Raise ValueError for a zero total, a negative count, or a share outside (0, 1).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Sample ratio mismatch p-value: chi-square with 1 degree of freedom.
def srm_p_value(
    n_a: int,
    n_b: int,
    expected_share_a: float = 0.5,
```

```
) -> float: ...
```

Starter code:

```
def srm_p_value(n_a, n_b, expected_share_a=0.5):
    return abs(n_a - n_b) / (n_a + n_b)
```

Exercise 5. Spot the bug (level 4)

The analysis reports a larger lift than the product team expected. Tap the line that breaks the randomisation, and fix the function.

What you need to know.

`series.to_dict()` turns a pandas Series into a dict: the index labels become keys. After a `groupby(...).mean()`, the keys are the group names.

```
pd.Series([0.2, 0.4], index=["A", "B"]).to_dict()   # {'A': 0.2, 'B': 0.4}
```

```
import pandas as pd


def conversion_by_arm(users: pd.DataFrame) -> dict[str, float]:
    """Conversion rate per assigned arm. One row per randomised user."""
    analysed = users[(users["arm"] == "A") | users["saw_new_page"]]
    return analysed.groupby("arm")["converted"].mean().to_dict()
```

Write down the line number of the bug. Then write the fix.

Sources

- chisquare([50600, 49400]).pvalue is 0.000148; for 5060 vs 4940 it is 0.230. (verified: Run on SciPy 1.17.1 in this repository; matches math.erfc(sqrt(stat / 2)).)
- confint_proportions_2indep(1150, 10000, 1000, 10000) gives (0.0064, 0.0236). (verified: Run on statsmodels 0.15.0 in this repository.)
- An SRM p-value below 0.001 is a common threshold for investigation. (search-only: Fabijan et al. (2019), Diagnosing Sample Ratio Mismatch in Online Controlled Experiments, KDD.)

B9-L3 Peeking, novelty and other ways tests mislead

Concept

A test's 5% false positive rate holds only if you analyse once, at the planned sample size. Common habits break it.

Peeking. Checking results every day and stopping the first time $p < 0.05$ gives the noise ten chances to cross the line. A simulation in this course's build: two identical arms with a 10% conversion rate, 1,000 runs, 200 users

per arm per look. One look: 5.2% false positives. Five looks: 12.9%. Ten looks: 18.4%.

Fixes, in order of simplicity:

- **Fixed horizon.** Compute the sample size (lesson B9-L1), run to it, analyse once. Dashboards during the test show health metrics and the split check, not the lift.
- **Planned interim looks** with a group-sequential design, where each look uses a stricter threshold so the overall rate stays at 5%. Plan the number of looks and the boundaries before launch.
- **Always-valid methods** designed for continuous monitoring,, offered by some experimentation platforms. Use them as the platform documents them.

Novelty and learning effects. Users click a new button because it is new. The lift fades after a week. Run for at least one full weekly cycle, and compare the effect in week 1 and week 2.

Too many metrics. Twenty secondary metrics produce about one false positive per test (lesson B8-L3). Name one primary metric and a few guardrails before launch.

Stopping for a good result only. A team that stops early when B looks good but runs longer when B looks bad biases every conclusion towards B.

Interference. If treated users affect control users, through shared inventory, pricing or social features, the arms are not independent. Randomise at a level that contains the interaction, such as city or store.

Write the plan down: primary metric, MDE, sample size, duration, looks, and the decision rule. Then follow it.

Why it matters

A growth team ran 30 tests a quarter, checked each daily, and shipped any variant that reached $p < 0.05$. Their dashboard showed 11 winners, with a combined forecast lift of 9% in sign-ups. Sign-ups over the quarter were flat. With daily peeking over three-week tests, the false positive rate per test was far above 5%, and most of the winners were noise.

Worked example: weak code

```
❶ for day in range(1, 29):
2     p = daily_p_value(day)
❷     if p < 0.05:
4         ship("B")
```

```
        break
```

1. **Line 1.** Up to 28 looks at the same growing data.
2. **Line 3.** The same 5% threshold at every look: the overall false positive rate climbs well above 5%.

Worked example: strong code

```
from dataclasses import dataclass


@dataclass(frozen=True)
class TestPlan:
    """Written before launch and stored with the results."""

    primary_metric: str
    mde_points: float
    n_per_arm: int
    min_days: int
    looks: int = 1


def ready_to_analyse(plan: TestPlan, n_per_arm_now: int, days_run: int) -> bool:
    """True only once the planned sample and duration are both reached."""
    return n_per_arm_now >= plan.n_per_arm and days_run >= plan.min_days
```

1. **Line 5.** The plan is data, fixed before launch.
2. **Line 12.** One analysis unless a sequential design was planned.
3. **Line 17.** The lift is looked at once, when the plan says so.

Plan the sample, duration and looks, analyse once at the horizon, and treat anything else as exploration.

Recap card

- **Why peeking breaks a test** Each look is another chance for noise to cross $p < 0.05$. Ten looks gave 18.4% false positives in simulation.
- **Safe alternatives to daily peeking** Fixed horizon, or planned group-sequential looks with stricter thresholds.
- **Novelty effect** Lift that fades after launch. Run full weeks and compare week 1 with later weeks.
- **Interference** Treated units affect controls. Randomise at a level that contains the interaction.

Practice

Exercise 1. Multiple choice (level 1)

You check an A/A test (no real difference) after each of 10 equal batches and stop at the first p < 0.05. In this lesson's simulation, how often did that declare a winner?

- a) About 5%
- b) About 18%
- c) About 50%
- d) Never, because the arms are identical

Why?

- r1) Each look is another chance for noise to cross the threshold.
- r2) Later batches have more users, so they are always significant.

Exercise 2. Predict the output (level 2)

A test ran 14 days. The lift in conversion was 3.1 points in week 1 and 0.4 points in week 2. What does this print?

```
week_lifts = {"week_1": 3.1, "week_2": 0.4}
fading = week_lifts["week_2"] < week_lifts["week_1"] / 2
print("novelty suspected" if fading else "stable")
```

Exercise 3. Fill the gap (level 2)

Allow analysis only when both the planned sample and the planned duration are reached.

```
return n_per_arm_now >= plan.n_per_arm _____(1) days_run >= plan._____(2)
```

Exercise 4. Write the function (level 3)

Write `peeking_false_positive_rate(rng, n_sims, n_per_look, looks, rate, z_crit=1.959964) -> float`. It simulates `n_sims` A/A tests where both arms convert at `rate`. Each arm gets `n_per_look` new users per look. At each look, run a pooled two-proportion z-test on the cumulative data. Return the share of simulations where `|z| > z_crit` at **any** look. Treat a zero standard error as z = 0. Use only the `rng` passed in.

What you need to know.

The pooled z-test. For two arms with `n` users each and `xa`, `xb` conversions: pooled rate `p = (xa + xb) / (2 * n)`, standard error `se = sqrt(p * (1 - p) * 2 / n)`, and `z = (xb / n - xa / n) / se`.

NumPy tools. These work on whole arrays, with one simulation per row:

```
# True/False draws, shape (2, 6)
hits = rng.random((2, 6)) < 0.5
# Running totals along each row
runs = hits.cumsum(axis=1)
# [1 3 5]: positions 2, 4 and 6
idx = np.arange(1, 4) * 2 - 1
# Those columns from every row
runs[:, idx]

# Divide only where y > 0, else 0: [0.5 0. ]
x, y = np.array([1.0, 2.0]), np.array([2.0, 0.0])
np.divide(x, y, out=np.zeros_like(x), where=y > 0)

# Share of rows with any True: 0.5
np.array([[True, False], [False, False]]).any(axis=1).mean()
```

`.any(axis=1)` asks "true anywhere in this row?", and `.mean()` of booleans is the share of `True`.

Your code must pass these checks:

- With one look the false positive rate must be close to 5%.
- With ten looks, count a simulation if |z| crosses the line at any look. Expect roughly 18%.
- The same seed must give the same result.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Share of A/A simulations declared significant at any of the looks.
def peeking_false_positive_rate(
    rng: np.random.Generator,
    n_sims: int,
    n_per_look: int,
    looks: int,
    rate: float,
    z_crit: float = 1.959964,
) -> float: ...
```

Starter code:

```
import numpy as np


def peeking_false_positive_rate(rng, n_sims, n_per_look, looks, rate,
    z_crit=1.959964):
    return 0.05
```

Exercise 5. Architecture design (level 4)

Stakeholders want to "see results daily" on a four-week pricing test. Choose the plan.

Options: Split check, traffic and error rates. No lift until the horizon., Daily lift with p-values.

1. What the daily dashboard shows
2. Analyses of the primary metric
3. Randomisation unit, since prices are shared in store

Sources

- A/A simulation, 1,000 runs, 200 users per arm per look, rate 10%: 5.2% false positives with 1 look, 12.9% with 5, 18.4% with 10. (verified: Simulated with NumPy 2.4.6 in this repository (seed 1).)

B10 Feature engineering without leakage

Roadmap: Prerequisite for all months. Target leakage, contamination and time-safe features.

B10-L1 Target leakage, features that know the answer

Concept

Target leakage happens when a feature carries information that would not exist at the moment you make the prediction. The model learns the leak, scores brilliantly in validation, and fails in production, where the leak is absent.

Ask one question of every feature: **would I know this value at prediction time?**

Typical leaks in customer and survey data:

- **Consequences of the outcome.** `cancellation_reason` predicts churn perfectly, because only churners have one. `refund_amount` predicts complaints.
- **Values measured after the prediction date.** `calls_to_support_last_30d` computed on the extraction date, not the prediction date, includes calls made while the customer was leaving.
- **Proxies from the labelling process.** A `reviewed_by` column that is only filled in for fraud cases.
- **IDs and timestamps that encode order.** If churners were loaded in a later batch, a row ID predicts churn.

A quick screen catches the worst cases. Compute each feature's AUC on its own against the target. A single feature with an AUC near 1.0, or near 0.0 for an inverse relationship, deserves suspicion before celebration. With scikit-learn 1.8:

```
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y, X[col])
strength = max(auc, 1 - auc)
```

A screen does not prove leakage, and a leak spread across several features will pass it. The real defence is a **feature timeline**: for every feature, the date it becomes known, compared with the prediction date. Keep that table

in the project and review it with someone who knows how the data is produced.

When in doubt, leave the feature out and measure how much the model loses. A feature worth 0.15 AUC on its own that nobody can explain is more likely a leak than a discovery.

Why it matters

A churn model scored AUC 0.97 in validation. The top feature was `days_since_last_login`, computed on the day the training data was extracted. For churned customers, that included the weeks after they had already left. In production, scored on active customers, the model ranked customers little better than chance. The retention campaign built on it spent its budget on the wrong people for two months.

Worked example: weak code

```
features = ["tenure_months", "plan", "days_since_last_login",
  "cancellation_reason"]
model.fit(train[features], train["churned"])
print(roc_auc_score(test["churned"], model.predict_proba(test[features])
  [:, 1]))
```

1. **Line 1.** cancellation_reason exists only for churners. days_since_last_login was measured after the prediction date.
2. **Line 3.** The test set shares the leak, so the score looks superb.

Worked example: strong code

```
import pandas as pd
from sklearn.metrics import roc_auc_score

LEAK_THRESHOLD = 0.95


def leakage_screen(X: pd.DataFrame, y: pd.Series, threshold: float =
  LEAK_THRESHOLD) -> list[str]:
    """Numeric features that alone separate the classes suspiciously
  well."""
    suspects = []
    for col in X.select_dtypes("number").columns:
        known = X[col].notna()
        if known.sum() == 0 or y[known].nunique() < 2:
            continue
        auc = roc_auc_score(y[known], X.loc[known, col])
        if max(auc, 1 - auc) >= threshold:
            suspects.append(col)
```

```
    return suspects
```

1. **Line 10.** One feature at a time: a single near-perfect feature is the classic leak.
2. **Line 12.** No AUC without both classes. Skip rather than crash.
3. **Line 15.** Catches inverse relationships too.

For every feature, know when it becomes available. Screen single-feature AUCs, and treat a near-perfect one as a suspect.

Recap card

- **The one question for every feature** Would I know this value at prediction time?
- **Single-feature leakage screen** AUC of each feature alone. Flag max(auc, 1 - auc) near 1.
- **Common leaks** Consequences of the outcome, values measured after the prediction date, labelling proxies, order-encoding IDs.
- **Feature timeline** A table of when each feature becomes known, reviewed with the data owner.

Practice

Exercise 1. Multiple choice (level 1)

You predict on 1 March whether a customer will churn in March. Which feature leaks?

- a) Tenure in months on 1 March
- b) Number of support calls in February
- c) Number of support calls in March
- d) Plan type on 1 March

Why?

- r1) March calls happen after the prediction is made, and may be caused by leaving.
- r2) Support calls always leak.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import roc_auc_score

churned = [0, 1, 0, 1]
days_inactive = [0.1, 0.9, 0.2, 0.8]
contract_months_left = [0.9, 0.1, 0.8, 0.2]
a = roc_auc_score(churned, days_inactive)
```

```
b = roc_auc_score(churned, contract_months_left)
print(a, max(b, 1 - b))
```

Exercise 3. Put in order (level 2)

Order the review of a proposed feature.

- Explain any high-AUC feature with the data owner, or drop it
- Drop features known only after the prediction date
- Find the date each feature value becomes known
- Write down the prediction date and what the model will see then
- Run the single-feature AUC screen on what remains

Exercise 4. Write the function (level 3)

Write `leakage_screen(X, y, threshold=0.95) -> list[str]`. For each numeric column of `X`, in column order, compute `roc_auc_score` on rows where the column is not missing. Return the names where `max(auc, 1 - auc) >= threshold`. Skip columns where the non-missing rows do not contain both classes.

Your code must pass these checks:

- reason_code separates the classes perfectly. Flag it.
- months_left separates the classes in reverse. Use max(auc, 1 - auc).
- Return only suspicious numeric columns, in column order. Skip text columns and one-class columns.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Numeric features that alone separate the classes suspiciously well.
def leakage_screen(
    X: pd.DataFrame,
    y: pd.Series,
    threshold: float = 0.95,
) -> list[str]: ...
```

Starter code:

```
import pandas as pd
from sklearn.metrics import roc_auc_score


def leakage_screen(X, y, threshold=0.95):
    return [c for c in X.columns if roc_auc_score(y, X[c]) > threshold]
```

Exercise 5. Architecture design (level 4)

A fraud model predicts at transaction time. Decide what to do with each candidate feature.

Options: Keep, Drop: known only after the prediction, Rebuild as of transaction time.

1. Transaction amount
2. Chargeback filed (yes/no)
3. Customer's transaction count, computed on the extract date
4. Analyst who reviewed the case

Sources

- roc_auc_score returns 0.0 for a perfect inverse ranking. (verified: Run on scikit-learn 1.8.0 and 1.9.1 in this repository.)

B10-L2 Fit on training data only

Concept

Every step that **learns from data** must learn from the training rows only: imputation means, scaling parameters, category lists, target encodings, feature selection, even the choice of outliers to drop. If a step sees the test rows, information about them flows into the model, and the test score becomes optimistic. This is **train/test contamination**.

A demonstration from this course's build, with scikit-learn 1.8. Take 100 rows of pure noise: 5,000 random features and a random 0/1 target. Nothing can be predicted.

- Select the 20 features most correlated with the target **on all 100 rows**, then cross-validate a logistic regression on them: accuracy **0.85**.
- Put the same selection **inside** each cross-validation fold: accuracy **0.48**, the chance level it should be.

The first model did not learn anything real. The selection step had already looked at every test fold's labels and picked the features that happened to fit them.

The rule for manual preprocessing:

```
params = fit_preprocessing(train)          # learn from train only
train_ready = apply_preprocessing(train, params)
test_ready = apply_preprocessing(test, params)   # reuse, never refit
```

Split **first**, before any step that learns. In cross-validation, that means refitting the preprocessing inside every fold. Doing that by hand is error-prone. scikit-learn's `Pipeline` does it for you (lesson B11-L2).

Steps that do **not** learn are safe on the full data: fixed recodes from a codebook, unit conversions, parsing dates. Steps that learn include anything with `fit`, anything that computes a mean, a quantile, a list of categories or a correlation with the target.

Why it matters

A propensity model imputed missing income with the median of the full dataset and scaled every feature on the full dataset before the train/test split. It then selected features by their correlation with the target, also on the full data. The validation AUC was 0.79. On the next month's customers it was 0.66. Most of the gap came from the feature selection: it had fitted the noise in the test rows.

Worked example: weak code

```
1 df["income"] = df["income"].fillna(df["income"].median())
2 df["income"] = (df["income"] - df["income"].mean()) / df["income"].std()
3 train, test = train_test_split(df, test_size=0.2, random_state=1)
```

1. **Line 1.** The median includes test rows.
2. **Line 2.** Scaling parameters learned from test rows too.
3. **Line 3.** The split comes after the learning, too late.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 1 def fit_income_prep(train: pd.DataFrame) -> dict[str, float]:
 5     """Learn median, mean and sd of income from training rows only."""
 6     filled = train["income"].fillna(train["income"].median())
 7     return {"median": float(train["income"].median()), "mean":
       float(filled.mean()), "sd": float(filled.std())}
 8
 9
10 def apply_income_prep(df: pd.DataFrame, params: dict[str, float]) ->
     pd.DataFrame:
11     """Impute and scale income with parameters learned elsewhere."""
 2     filled = df["income"].fillna(params["median"])
13     return df.assign(income=(filled - params["mean"]) / params["sd"])
```

1. **Line 4.** Learning happens in one place, on training rows.
2. **Line 12.** Applying reuses the learned values, for train and test alike.

Split first. Fit every learning step on training rows. Apply the fitted parameters to everything else.

Recap card

- **Train/test contamination** A learning step saw test rows: imputation, scaling, encoding, selection.
- **Selection on all rows, noise data** CV accuracy 0.85. Inside the folds: 0.48. The first number is a leak.
- **The manual rule** Split first, fit on train, apply the fitted parameters to train and test.
- **Safe before the split** Steps that learn nothing from the rows: fixed recodes, unit conversions, parsing.

Practice

Exercise 1. Multiple choice (level 1)

Which step is safe to run on the full dataset before the split?

- a) Filling missing income with the median
- b) Converting height from inches to centimetres
- c) Selecting the 20 features most correlated with the target
- d) Standardising features to mean 0 and sd 1

Why?

- r1) A unit conversion uses a fixed constant. It learns nothing from the rows.
- r2) Conversions happen before any modelling.

Exercise 2. Predict the output (level 2)

Pure noise: 100 rows, 5,000 random features, random labels. The 20 best features are selected on all rows, then 5-fold CV is run. Roughly what accuracy does it print? (scikit-learn 1.8, seed 0)

What you need to know.

The code uses four scikit-learn tools (version 1.8):

- `SelectKBest(f_classif, k=20).fit_transform(X, y)` scores each column against the labels `y` and returns only the `k` best-scoring columns.
- `LogisticRegression()` is a linear classifier.

- `cross_val_score(model, X, y, cv=5)` splits the rows into 5 folds. It fits on 4 folds, scores accuracy on the fifth, and repeats for each fold. It returns the 5 scores.
- `rng.normal(size=(rows, cols))` draws random numbers, and `rng.integers(0, 2, n)` draws `n` random labels, 0 or 1.

On pure noise, an honest model scores about 0.5 accuracy: a coin toss.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000))
y = rng.integers(0, 2, 100)
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
print(round(cross_val_score(LogisticRegression(), X_sel, y, cv=5).mean(),
    2))
```

Exercise 3. Fill the gap (level 2)

Fit the scaler on training rows, then apply it to both sets.

```
scaler = StandardScaler().fit(_____(1))
X_train_s = scaler.transform(X_train)
X_test_s = scaler._____(2)(X_test)
```

Exercise 4. Write the function (level 3)

Write `fit_prep(train) -> dict[str, float]` and `apply_prep(df, params) -> pd.DataFrame` for the column `income`.

- `fit_prep` learns `median` of income, then fills missing values with it and learns `mean` and `sd` of the filled column. Training rows only.
- `apply_prep` fills missing income with `params["median"]` and returns a new frame with `income` scaled as `(filled - mean) / sd`. If `sd` is 0, set scaled income to 0.0.

Your code must pass these checks:

- Learn the median, then mean and sd of income after filling with that median.
- Test rows must use the training median, mean and sd, not their own.
- When sd is 0, scaled income must be 0.0.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Learn imputation and scaling parameters for income from training rows.
def fit_prep(train: pd.DataFrame) -> dict[str, float]: ...

# Impute and scale income with parameters learned from training rows.
def apply_prep(
    df: pd.DataFrame,
    params: dict[str, float],
) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def fit_prep(train):
    return {}


def apply_prep(df, params):
    filled = df["income"].fillna(df["income"].median())
    return df.assign(income=(filled - filled.mean()) / filled.std())
```

Exercise 5. Spot the bug (level 4)

Target encoding replaces each region with its churn rate. The model's test AUC is far above its production AUC. Tap the line that leaks, and fix the function so the rates come from training rows only.

What you need to know.

`pd.concat([a, b])` stacks DataFrames on top of each other: the rows of `a`, then the rows of `b`.

`series.map(lookup)` replaces each value with the matching entry of another Series, matched by its index:

```python
rates = pd.Series({"North": 0.1, "South": 0.3})
pd.Series(["North", "South"]).map(rates)   # 0.1, 0.3
```

```python
import pandas as pd


def encode_region(train: pd.DataFrame, test: pd.DataFrame) ->
    tuple[pd.DataFrame, pd.DataFrame]:
    """Replace region with its churn rate."""
    both = pd.concat([train, test])
    rates = both.groupby("region")["churned"].mean()
    return train.assign(region=train["region"].map(rates)),
    test.assign(region=test["region"].map(rates))
```

Write down the line number of the bug. Then write the fix.

Sources

- SelectKBest(k=20) on all rows of 100 x 5,000 noise gives 0.85 CV accuracy; inside a Pipeline, 0.48. (verified: Run on scikit-learn 1.8.0 and 1.9.1 with NumPy 2.4.6 in this repository (seed 0).)
- scikit-learn provides TargetEncoder with internal cross fitting. (verified: scikit-learn 1.8 API reference; sklearn.preprocessing.TargetEncoder imported in this repository.)

B10-L3 Time-safe features and point-in-time joins

Concept

When rows have dates, every feature must be computed **as of** the row's own date, using only earlier data. Three tools in pandas 3.0 cover most cases.

Prior-only aggregates. "Total spend before this order" must exclude the current order. Take the running total and subtract the current value, or shift first:

```
orders = orders.sort_values(["customer", "ordered_at"])
prior = orders.groupby("customer")["spend"].cumsum() - orders["spend"]
```

Point-in-time joins. To attach the latest satisfaction score **before** each prediction date, use `pd.merge_asof`. It matches each left row with the last right row whose key is at or before it, within the same `by` group. Both frames must be sorted on the key.

```
pd.merge_asof(preds.sort_values("as_of"), scores.sort_values("scored_at"),
              left_on="as_of", right_on="scored_at", by="customer",
              allow_exact_matches=False)
```

The default `allow_exact_matches=True` also matches a score recorded **at the same timestamp** as the prediction. If that score could only be known after the prediction was made, set it to `False`. A plain `merge` on customer followed by "take the latest" silently uses scores from the future.

Time-based splits. Random splits put next month's rows in training and last month's in test. For anything you will use forward in time, split by date: train on data up to a cut-off, test on data after it. scikit-learn's `TimeSeriesSplit` gives expanding windows for cross-validation.

Build a small **as-of test** for every time feature: a customer with events before and after the prediction date, and an assertion that only the earlier events count. It catches most mistakes in a few lines.

Why it matters

A next-purchase model used "customer's total orders" as a feature, computed once over the full order table. For each training row, the count included orders placed after the row's date, including the very purchase the model was trying to predict. Validation AUC was 0.91. In production, where the future does not exist yet, it was 0.68.

Recomputing the count as prior orders only brought validation down to 0.70. That was the model's true performance, and the team finally had an honest number to improve.

Worked example: weak code

```
❶ totals = orders.groupby("customer")["spend"].sum().rename("total_spend")
  2 rows = orders.merge(totals, on="customer")
❷ train, test = train_test_split(rows, test_size=0.2)
```

1. **Line 1.** One total per customer across all time: every row sees its own future.
2. **Line 3.** Random split: training rows come from after test rows.

Worked example: strong code

```
  1 import pandas as pd
  2
  3
  4 def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame:
  5     """Spend and order count before each order, per customer. Grain: one
    row per order."""
  6     o = orders.sort_values(["customer", "ordered_at"])
  7     grp = o.groupby("customer")["spend"]
  8     return o.assign(
❶         prior_spend=grp.cumsum() - o["spend"],
❷         prior_orders=o.groupby("customer").cumcount(),
 11     )
 12
 13
 14 def time_split(df: pd.DataFrame, date_col: str, cutoff: pd.Timestamp) ->
    tuple[pd.DataFrame, pd.DataFrame]:
 15     """Train on rows before cutoff, test on rows at or after it."""
❸     before = df[date_col] < cutoff
 17     return df[before], df[~before]
```

1. **Line 9.** Running total minus the current order: only earlier orders count.
2. **Line 10.** cumcount starts at 0, so it counts earlier orders only.
3. **Line 16.** A time split: the test set is the future, as in production.

Compute every feature as of the row's date, join with merge_asof, and split by time.

Recap card

- **Prior-only running total** groupby(key)[col].cumsum() - col, after sorting by key and date.
- **Point-in-time join in pandas** pd.merge_asof(left, right, left_on, right_on, by=..., allow_exact_matches=False), both sorted.
- **merge_asof default for equal timestamps** allow_exact_matches=True: same-time rows match. Set False when they would not be known yet.
- **Split for forward-looking models** By time: train before a cut-off, test after. TimeSeriesSplit for CV.

Practice

Exercise 1. Multiple choice (level 1)

A model will score customers each month, going forward. How should you split the historical data for validation?

- a) Random 80/20 split
- b) Train on months up to a cut-off, test on later months
- c) Test on the earliest months, train on later ones
- d) No split: use training accuracy

Why?

- r1) It matches production, where the model predicts a future it has not seen.
- r2) Recent data is always more accurate.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

What you need to know.

`pd.to_datetime([...])` turns date strings such as `"2026-01-31"` into timestamps, so pandas compares and sorts them as dates, not as text.

```
import pandas as pd

preds = pd.DataFrame({"customer": ["c1"], "as_of":
    pd.to_datetime(["2026-04-01"])})
scores = pd.DataFrame({
    "customer": ["c1", "c1"],
    "scored_at": pd.to_datetime(["2026-02-15", "2026-04-01"]),
    "nps": [3, 9],
})
```

```
out = pd.merge_asof(preds, scores, left_on="as_of", right_on="scored_at",
    by="customer")
print(out["nps"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Attach the latest earlier score to each prediction row, excluding same-timestamp scores.

```
pd._____(1)(preds, scores, left_on="as_of", right_on="scored_at",
    by="customer", allow_exact_matches=_____(2))
```

Exercise 4. Write the function (level 3)

Write `add_prior_spend(orders) -> pd.DataFrame`. `orders` has `customer`, `ordered_at` and `spend`, in any row order. Return the orders sorted by `customer` then `ordered_at`, with two new columns:

- `prior_spend`: total spend by the same customer on **earlier** orders (0 for the first).
- `prior_orders`: number of earlier orders by the same customer (0 for the first).

Your code must pass these checks:

- prior_spend must sum earlier orders only, per customer, in date order.
- prior_orders must count earlier orders only, starting at 0.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Spend and order count before each order, per customer.
def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def add_prior_spend(orders):
    totals = orders.groupby("customer")["spend"].transform("sum")
    return orders.assign(prior_spend=totals)
```

Exercise 5. Spot the bug (level 4)

For each prediction, the feature should be the customer's most recent NPS score strictly **before** `as_of`. Validation looks too good. Tap the line that leaks and fix it.

What you need to know.

`df.groupby(keys).tail(1)` keeps the last row of each group, in the frame's current order. After `sort_values("date")`, that is the latest row per group:

```
df = pd.DataFrame({"k": ["a", "a", "b"], "v": [1, 2, 3]})
df.groupby("k").tail(1)   # rows (a, 2) and (b, 3)
```

```
import pandas as pd


def latest_nps(preds: pd.DataFrame, scores: pd.DataFrame) -> pd.DataFrame:
    """Most recent NPS strictly before each prediction date."""
    joined = preds.merge(scores, on="customer")
    joined = joined.sort_values("scored_at").groupby(["customer", "as_of"]).tail(1)
    return joined[["customer", "as_of", "nps"]].sort_values(["customer", "as_of"]).reset_index(drop=True)
```

Write down the line number of the bug. Then write the fix.

Sources

- pd.merge_asof defaults to direction="backward" and allow_exact_matches=True. (verified: pandas API reference; run on pandas 3.0.6 in this repository.)

B11 Machine learning workflow

Roadmap: Prerequisite for all months. Splits, Pipeline, ColumnTransformer, cross-validation and hyperparameter search.

B11-L1 Train, validation and test, split by the right unit

Concept

Three sets, three jobs:

- **Training** rows fit the model.
- **Validation** rows, or cross-validation folds within training, choose between models and settings.
- **Test** rows measure the final choice, **once**. Every extra look at the test set turns it into a second validation set, and its score stops being an honest estimate.

Split so the test set looks like the data the model will meet.

Stratify on the target when a class is rare. `train_test_split(X, y, stratify=y)` keeps a 3% positive rate at 3% in both sets. Without it, a small test set can end up with very few positives.

Group when rows are not independent. A panel respondent answering five waves appears in five rows. A random row split puts some of their waves in training and others in test. The model then recognises the **person** instead of learning the pattern, and the test score measures memory.

A demonstration from this course's build (scikit-learn 1.8, random forest): 200 simulated respondents, 5 waves each, a label that depends only on the person and is unrelated to the features. Random 5-fold CV reported accuracy **0.995**. `GroupKFold` by respondent reported **0.468**: chance, which is the truth.

```
from sklearn.model_selection import GroupKFold, GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=7)
train_idx, test_idx = next(splitter.split(df, groups=df["respondent_id"]))
```

The group is whatever repeats: respondent, household, store, patient, device. If it will be **new** in production, keep it out of training in validation too. For time-ordered use, split by time as well (lesson B10-L3).

Record the split: the seed, the group column, and the resulting row counts, in the run manifest (lesson B6-L2).

Why it matters

A model predicting survey drop-out was validated with a random row split over a panel dataset. It scored AUC 0.93. Deployed on a newly recruited panel, where every respondent was new, it scored 0.61. Each respondent's answering style had let the model match test rows to the same person's training rows. A group split would have shown 0.61 before deployment.

Worked example: weak code

```
train, test = train_test_split(panel, test_size=0.2)
for params in grid:
    score = evaluate(fit(train, params), test)
```

1. **Line 1.** Random rows: the same respondent sits on both sides.
2. **Line 3.** Tuning on the test set: its score is no longer an estimate of anything.

Worked example: strong code

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit


def split_by_group(
    df: pd.DataFrame, group_col: str, test_size: float, seed: int
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Split so that no group appears in both train and test."""
    splitter = GroupShuffleSplit(n_splits=1, test_size=test_size,
    random_state=seed)
    train_idx, test_idx = next(splitter.split(df, groups=df[group_col]))
    return df.iloc[train_idx], df.iloc[test_idx]
```

1. **Line 9.** test_size is the share of groups, not rows. The seed makes the split reproducible.
2. **Line 10.** Every row of a respondent lands on the same side.

Tune on validation, measure once on test, and split by the unit that will be new in production.

Recap card

- **Roles of train, validation and test** Fit, choose, measure once.
- **Rows repeat per respondent** Split by group (GroupKFold, GroupShuffleSplit), or the model learns people.
- **Group leakage demonstration** Random CV 0.995, GroupKFold 0.468 on a label unrelated to the features.
- **Rare class** stratify=y keeps the class share in each set.

Practice

Exercise 1. Multiple choice (level 1)

You tried 40 model settings and picked the one with the best test score. What is that test score now?

- a) An unbiased estimate of performance on new data.
- b) An optimistic estimate, because the test set chose the model.
- c) A pessimistic estimate.
- d) The same as the training score.

Why?

- r1) Picking the best of 40 on the same data selects for luck on that data.
- r2) Test sets always overstate performance.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit

df = pd.DataFrame({"rid": ["a", "a", "b", "b", "c", "c", "d", "d"], "x":
   range(8)})
splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=0)
train_idx, test_idx = next(splitter.split(df, groups=df["rid"]))
print(len(test_idx), df.iloc[test_idx]["rid"].nunique())
```

Exercise 3. Fill the gap (level 2)

Keep the rare positive class at the same rate in train and test.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
   _____(1)=y, _____(2)=42)
```

Exercise 4. Write the function (level 3)

Write

`split_by_group(df, group_col, test_size, seed) -> tuple[pd.DataFrame, pd.DataFrame]` with `GroupShuffleSplit`, so that no value of `group_col` appears in both returned frames. The same seed must give the same split.

Your code must pass these checks:

- No respondent may appear in both train and test.
- Every row must land in one set, and 20% of the 20 respondents go to test.
- The same seed must give the same split.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Split rows so no group appears in both train and test.
def split_by_group(
    df: pd.DataFrame,
    group_col: str,
    test_size: float,
    seed: int,
) -> tuple[pd.DataFrame, pd.DataFrame]: ...
```

Starter code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split


def split_by_group(df, group_col, test_size, seed):
    return train_test_split(df, test_size=test_size, random_state=seed)
```

Exercise 5. Architecture design (level 4)

Choose the split for each project.

Options: Group split by member, Time split, Stratified random split.

1. Predict drop-out for newly recruited panel members; data has 6 waves per member
2. Fraud flag at 0.8% prevalence, one row per independent transaction, no time drift
3. Forecast next month's store sales

Sources

- Random 5-fold CV gave 0.995 and GroupKFold gave 0.468 on 200 simulated respondents with 5 waves each and a person-level random label. (verified: Run with scikit-learn 1.8.0 and 1.9.1, RandomForestClassifier(n_estimators=100, random_state=0), in this repository.)

B11-L2 Pipeline and ColumnTransformer

Concept

Lesson B10-L2 fitted preprocessing on training rows by hand. scikit-learn's `Pipeline` does it for you, every time, including inside each cross-validation fold. Examples use scikit-learn 1.8.

A `Pipeline` chains steps. Calling `fit` fits each step on the training rows and passes its output to the next. Calling `predict` runs the fitted steps on new rows **without refitting**. Cross-validation, grid search and `cross_val_score` all refit the whole pipeline per fold, so no fold's test rows leak into its preprocessing.

A `ColumnTransformer` applies different preprocessing to different columns:

```
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
    StandardScaler()), NUMERIC),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")),
    CATEGORICAL),
])
model = Pipeline([("prep", prep), ("clf",
    LogisticRegression(max_iter=1000))])
```

Three defaults to know:

- `OneHotEncoder(handle_unknown="error")` is the default. A region that did not appear in training raises `ValueError` at prediction time. Set `handle_unknown="ignore"` to encode it as all zeros.
- `ColumnTransformer(remainder="drop")` is the default. Columns you do not list are **silently dropped**. List every column you mean to use, or set `remainder="passthrough"` on purpose.
- Output feature names come from `get_feature_names_out()`, prefixed with the step name: `num__age`, `cat__region_n`.

Keep the column lists in config (lesson A1-L3), and have one function build the pipeline. The same fitted object is what you save, test and deploy, so preprocessing in production is exactly what was validated.

Why it matters

A lead-scoring model was trained in a notebook. Preprocessing ran in a separate cell: median imputation and scaling by hand. The deployment team rewrote the preprocessing from the notebook for production and imputed with the **mean**. Income was skewed, so the mean was 40% above the median. Scores for every lead with missing income shifted, and the sales team's call list changed without anyone noticing.

A single fitted `Pipeline`, saved and deployed as one object, leaves no second copy of the preprocessing to get wrong.

Worked example: weak code

```
X["age"] = X["age"].fillna(X["age"].median())
```

```
2 X = pd.get_dummies(X, columns=["region"])
3 scores = cross_val_score(LogisticRegression(), X, y, cv=5)
```

1. **Line 1.** Imputation on all rows, before CV: every fold's test rows shape the median.
2. **Line 2.** Dummy columns depend on the data seen. A new region in production changes the columns.

Worked example: strong code

```
1  from sklearn.compose import ColumnTransformer
2  from sklearn.impute import SimpleImputer
3  from sklearn.linear_model import LogisticRegression
4  from sklearn.pipeline import Pipeline, make_pipeline
5  from sklearn.preprocessing import OneHotEncoder, StandardScaler
6
7
8  def build_pipeline(numeric: list[str], categorical: list[str]) ->
     Pipeline:
9      """Preprocessing and model as one object, fitted per fold by CV."""
10     prep = ColumnTransformer(
11         [
12             ("num", make_pipeline(SimpleImputer(strategy="median"),
     StandardScaler()), numeric),
13             (
14                 "cat",
1                  make_pipeline(SimpleImputer(strategy="most_frequent"),
     OneHotEncoder(handle_unknown="ignore")),
16                 categorical,
17             ),
18         ]
19     )
2      return Pipeline([("prep", prep), ("clf",
     LogisticRegression(max_iter=1000))])
```

1. **Line 15.** An unseen category becomes all zeros instead of an error.
2. **Line 20.** One object to cross-validate, save and deploy.

Put every learning step in one Pipeline, split columns with a ColumnTransformer, and deploy the fitted object itself.

Recap card

- **Why Pipeline** Every learning step refits per CV fold, and one fitted object goes to production.
- **OneHotEncoder default for unseen categories** handle_unknown="error": raises ValueError. Use "ignore" for production.
- **ColumnTransformer default for unlisted columns** remainder="drop": silently dropped.

- **Feature names after preprocessing** get_feature_names_out(), prefixed with step names such as num__age.

Practice

Exercise 1. Multiple choice (level 1)

Why does putting a scaler inside a Pipeline prevent leakage in `cross_val_score`?

- a) The Pipeline refits the scaler on each fold's training rows only.
- b) Pipelines do not scale test data.
- c) The scaler is fitted once on all rows, but faster.
- d) Pipelines remove outliers automatically.

Why?

- r1) cross_val_score clones and fits the whole pipeline per fold.
- r2) Scalers inside pipelines have no parameters.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder().fit(pd.DataFrame({"region": ["n", "s"]}))
try:
    enc.transform(pd.DataFrame({"region": ["e"]}))
    print("encoded")
except ValueError:
    print("ValueError")
```

Exercise 3. Fill the gap (level 2)

Keep unlisted columns instead of dropping them, and encode unseen categories as zeros.

```
prep = ColumnTransformer(
    [("cat", OneHotEncoder(handle_unknown="_____(1)"), ["region"])],
    remainder="_____(2)",
)
```

Exercise 4. Write the function (level 3)

Write `build_pipeline(numeric, categorical) -> Pipeline` with two named steps:

- `"prep"`: a `ColumnTransformer` with `"num"` (median `SimpleImputer`, then `StandardScaler`) on `numeric`, and `"cat"` (most-frequent `SimpleImputer`, then `OneHotEncoder(handle_unknown="ignore")`) on `categorical`.
- `"clf"`: `LogisticRegression(max_iter=1000)`.

Your code must pass these checks:

- The pipeline needs steps named prep and clf, in that order.
- Numeric columns need median imputation before scaling: the median of training age is 30.
- An unseen region must not raise. Use OneHotEncoder(handle_unknown="ignore").
- Name the ColumnTransformer parts num and cat.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Preprocessing and classifier as one estimator.
def build_pipeline(
    numeric: list[str],
    categorical: list[str],
) -> Pipeline: ...
```

Starter code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline


def build_pipeline(numeric, categorical):
    return Pipeline([("clf", LogisticRegression())])
```

Exercise 5. Spot the bug (level 4)

The model ignores `tenure`, although it is the strongest predictor in exploration. Tap the line that causes it, and fix the function.

```
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


def build(categorical: list[str]) -> Pipeline:
    """Encode categorical columns and fit a classifier on every
    feature."""
```

```
    prep = ColumnTransformer([("cat",
OneHotEncoder(handle_unknown="ignore"), categorical)])
    return Pipeline([("prep", prep), ("clf",
LogisticRegression(max_iter=1000))])
```

Write down the line number of the bug. Then write the fix.

Sources

- In scikit-learn 1.8, OneHotEncoder defaults to handle_unknown="error" and ColumnTransformer to remainder="drop". (verified: scikit-learn 1.8 API reference; signatures checked on 1.8.0 and 1.9.1 in this repository.)

B11-L3 Cross-validation and honest hyperparameter search

Concept

Cross-validation (CV) splits the training data into k folds, trains on k - 1 and scores on the remaining one, k times. Report the **mean and the spread** across folds. A mean AUC of 0.81 with fold scores from 0.70 to 0.90 is a less certain result than 0.81 with scores from 0.79 to 0.83.

`cross_validate` (scikit-learn 1.8) returns several metrics in one pass:

```
res = cross_validate(pipe, X, y, cv=StratifiedKFold(5, shuffle=True,
    random_state=7),
                     scoring=["roc_auc", "average_precision"])
res["test_roc_auc"].mean(), res["test_roc_auc"].std()
```

Hyperparameter search. `GridSearchCV` or `RandomizedSearchCV` tries settings and keeps the best by CV score. Inside a pipeline, name parameters as `step__param`: `{"clf__C": [0.01, 0.1, 1, 10]}`.

The catch: `best_score_` is the **best of many tries** on the same folds. It is optimistic, for the same reason as picking a model on the test set (lesson B11-L1). A simulation in this course's build: 20 datasets of pure noise, a k-nearest-neighbours grid of 60 settings. The mean `best_score_` was **0.598** accuracy. The honest estimate was **0.515**.

Get the honest estimate with **nested CV**: an outer CV loop scores the **whole search** as if it were one model.

```
search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
honest = cross_val_score(search, X, y, cv=outer, scoring="roc_auc")
```

Or keep a held-out test set that the search never sees, and score the refitted best model on it once.

Practical rules:

- Fix `random_state` in every splitter, so reruns compare like with like.
- Search on a log scale for regularisation strengths: 0.001, 0.01, 0.1, 1, 10.
- Prefer `RandomizedSearchCV` when the grid has more than about 50 combinations.
- Report the chosen settings, the honest score and the fold spread in the run log.

Why it matters

A team compared 300 configurations of a gradient-boosted model and reported the top `best_score_`, AUC 0.842, to the client as expected performance. The model went live and scored 0.80 over the first quarter. Nothing had drifted. The 0.842 had been the luckiest of 300 draws on the same five folds. A nested CV estimate of 0.80 would have set the right expectation from the start.

Worked example: weak code

```
search = GridSearchCV(pipe, {"clf__C": [0.001, 0.01, 0.1, 1, 10, 100]}, cv=5)
search.fit(X, y)
print("expected AUC", search.best_score_)
```

1. **Line 1.** Default scoring is accuracy for classifiers, not the AUC being reported.
2. **Line 3.** The best of several tries on the same folds: optimistic, and the wrong metric.

Worked example: strong code

```
import numpy as np
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline


def nested_cv_auc(pipe: Pipeline, X: np.ndarray, y: np.ndarray, grid: dict[str, list[float]], seed: int) -> float:
    """Honest AUC of the whole search: the search is refitted inside each outer fold."""
    inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
    outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed + 1)
    search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
    return float(cross_val_score(search, X, y, cv=outer, scoring="roc_auc").mean())
```

1. **Line 10.** The metric you will report is the metric you tune on.

2. **Line 11.** Outer folds never take part in choosing the settings.

Report CV mean and spread, tune with the reported metric, and estimate performance with nested CV or an untouched test set.

Recap card

- **Report CV results as** Mean and spread across folds, for the metric you care about.
- **Pipeline parameter names in a grid** step__param, such as clf__C. make_pipeline names steps after their class.
- **Is best_score_ an honest estimate?** No. Noise simulation: 0.598 best_score_ against 0.515 nested. Use nested CV or a held-out test set.
- **Nested CV in scikit-learn** cross_val_score(GridSearchCV(pipe, grid, cv=inner), X, y, cv=outer).

Practice

Exercise 1. Multiple choice (level 1)

After `GridSearchCV` over 80 settings, what is `best_score_`?

- a) An unbiased estimate of performance on new data
- b) The mean CV score of the best setting, optimistic because it was selected as the best
- c) The test-set score
- d) The training score of the refitted model

Why?

- r1) Selecting the maximum of many noisy scores favours settings that were lucky on these folds.
- r2) GridSearchCV always overfits.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

What you need to know.

Every step in a scikit-learn pipeline has a name, and each step's settings are addressed as `name__setting`.

- `Pipeline([("scale", StandardScaler()), ("clf", LogisticRegression())])` uses the names you give.
- `make_pipeline(...)` names each step after its class, in lower case.

`pipe.get_params()` returns a dict of every setting, keyed by those names:

```
pipe = make_pipeline(StandardScaler(), DecisionTreeClassifier())
[name for name, _ in pipe.steps]   # ['standardscaler',
   'decisiontreeclassifier']
"decisiontreeclassifier__max_depth" in pipe.get_params()   # True
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pipe = make_pipeline(StandardScaler(), LogisticRegression())
print("clf__C" in pipe.get_params(), "logisticregression__C" in
   pipe.get_params())
```

Exercise 3. Fill the gap (level 2)

Tune C of the step named clf, scoring by ROC AUC.

```
search = GridSearchCV(pipe, {"_____(1)": [0.01, 0.1, 1, 10]}, cv=inner,
    _____(2)="roc_auc")
```

Exercise 4. Write the function (level 3)

Write `nested_cv_auc(X, y, c_values, seed) -> float`. Build a pipeline with steps `"scale"` (`StandardScaler`) and `"clf"` (`LogisticRegression(max_iter=1000)`). Wrap it in `GridSearchCV` over `{"clf__C": c_values}` with an inner `StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)` and `scoring="roc_auc"`. Return the mean of `cross_val_score` of the search with an outer `StratifiedKFold(n_splits=5, shuffle=True, random_state=seed + 1)`, also scored by ROC AUC.

Your code must pass these checks:

- Use the inner and outer StratifiedKFold settings from the brief, scored by roc_auc.
- On noise the honest estimate must sit near 0.5 (0.5103 with these settings).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Nested CV estimate of ROC AUC for a tuned logistic regression.
def nested_cv_auc(
    X: np.ndarray,
    y: np.ndarray,
    c_values: list[float],
    seed: int,
) -> float: ...
```

Starter code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
```

```
3
4
5 def nested_cv_auc(X, y, c_values, seed):
6     search = GridSearchCV(LogisticRegression(), {"C": c_values},
    cv=5).fit(X, y)
7     return search.best_score_
```

Exercise 5. Architecture design (level 4)

You must tune a model and tell the client what AUC to expect. You have 8,000 labelled rows. Choose the plan.

Options: Hold out 20%, untouched until the end, Use all rows for the search.

1. Held-out test set
2. Search on the other 80%
3. Number given to the client

Sources

- Across 20 noise datasets with a 60-setting kNN grid, mean best_score_ was 0.598 and mean nested CV accuracy 0.515. (verified: Simulated with scikit-learn 1.9.1 in this repository.)
- cross_validate returns test_<metric> keys for each scorer. (verified: Run on scikit-learn 1.8.0 in this repository.)

B12 Model evaluation

Roadmap: Prerequisite for all months; Months 4 to 6: CNNs; Months 10 to 12: Transformers applied. Metric choice, calibration, imbalanced classes and error analysis.

B12-L1 Choose the metric from the decision

Concept

A metric is a summary of the mistakes you care about. Choose it from the decision the model supports, before you train anything.

Accuracy fails on imbalanced data. A test set from this course's build has 2.4% positives. Predicting "negative" for everyone scores accuracy **0.976**. A logistic regression at the default 0.5 threshold scored 0.9755, with **recall 0.0**: it found no positives at all.

Better summaries, all in scikit-learn 1.8:

- **ROC AUC** (`roc_auc_score`): the chance a random positive ranks above a random negative. It ignores the threshold and the class balance. Good for comparing rankers. It hides how many false positives come with each true positive when positives are rare.
- **Average precision**, the area under the precision-recall curve (`average_precision_score`): sensitive to rare positives. A random ranking scores the **prevalence**, 0.025 here. The same logistic regression scored **0.30**, twelve times the baseline. Always quote the baseline next to it.
- **Precision and recall at a threshold**: precision is the share of flagged cases that are real. Recall is the share of real cases that are flagged. These map straight onto work: "of 100 customers we call, 30 were going to churn" and "we reach 64% of the churners".
- **Cost-weighted metrics**: if a missed fraud costs £400 and a false alarm costs £5 of analyst time, compute the total cost at each threshold and pick the cheapest.

Pick the **threshold** from the decision too. 0.5 is a default of the library, not a business rule. If the team can review 200 cases a day, the threshold is whatever flags 200 cases.

Report the metric with the prevalence and the number of positives in the test set. A precision of 0.30 from 12 positives is not the same evidence as from 1,200.

Why it matters

A claims triage model was reported as "97.6% accurate". It flagged nothing. Fraud prevalence was 2.4%, so a model that approved every claim scored the same. The number went into a board paper before anyone asked how many frauds it caught. The answer was zero.

Worked example: weak code

```
pred = model.predict(X_test)
print("accuracy", accuracy_score(y_test, pred))
```

1. **Line 1.** predict() uses a 0.5 threshold that nobody chose.
2. **Line 2.** Accuracy on rare positives rewards predicting the majority class.

Worked example: strong code

```
import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score


def evaluate(y_true: np.ndarray, scores: np.ndarray, threshold: float) -> dict[str, float | None]:
    """Ranking and threshold metrics, with the baseline to compare against."""
    flagged = scores >= threshold
    tp = int((flagged & (y_true == 1)).sum())
    n_flagged, n_pos = int(flagged.sum()), int((y_true == 1).sum())
    return {
        "prevalence": float(y_true.mean()),
        "roc_auc": float(roc_auc_score(y_true, scores)),
        "average_precision": float(average_precision_score(y_true, scores)),
        "precision": tp / n_flagged if n_flagged > 0 else None,
        "recall": tp / n_pos if n_pos > 0 else None,
    }
```

1. **Line 11.** The baseline for average precision, reported alongside it.
2. **Line 14.** No flags, no precision: None, not a division error or a misleading 0.

Choose metrics and thresholds from the decision. Report prevalence and counts beside every number.

Recap card

- **Accuracy at 2.4% prevalence** 0.976 for predicting all negatives. Useless as a headline.
- **Baseline for average precision** The prevalence. Report it beside the score.
- **Where the threshold comes from** The decision: review capacity or costs. Not the 0.5 default.
- **Precision and recall** Precision: share of flags that are real. Recall: share of real cases flagged.

Practice

Exercise 1. Multiple choice (level 1)

Positives are 2.4% of the test set. A model has accuracy 0.976. What do you know?

- a) The model is excellent.
- b) Nothing useful yet. Predicting all negatives scores the same.
- c) The model finds 97.6% of positives.
- d) The model is calibrated.

Why?

- r1) Accuracy is dominated by the 97.6% majority class.
- r2) Accuracy is never useful.

Exercise 2. Predict the output (level 2)

What does this print, to two decimals? (scikit-learn 1.8, NumPy 2.4)

```
import numpy as np
from sklearn.metrics import average_precision_score

rng = np.random.default_rng(0)
y = (rng.random(20000) < 0.05).astype(int)
random_scores = rng.random(20000)
print(round(average_precision_score(y, random_scores), 2))
```

Exercise 3. Fill the gap (level 2)

Flag the top 200 cases a day, whatever their scores.

```
threshold = np.sort(scores)[_____(1)]
flagged = scores >= _____(2)
```

Exercise 4. Write the function (level 3)

Write `evaluate(y_true, scores, threshold) -> dict[str, float | None]` returning:

- `prevalence`: mean of `y_true`.
- `roc_auc` and `average_precision` from scikit-learn.
- `precision` and `recall` at `scores >= threshold`, computed from counts. Return `None` for precision when nothing is flagged, and for recall when there are no positives.

Your code must pass these checks:

- Check prevalence, ROC AUC, and precision and recall from counts at scores >= threshold.
- average_precision must come from average_precision_score(y_true, scores).
- No flagged cases: precision is None, recall is 0.0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Ranking metrics, threshold metrics and the prevalence baseline.
def evaluate(
    y_true: np.ndarray,
    scores: np.ndarray,
    threshold: float,
) -> dict[str, float | None]: ...
```

Starter code:

```python
from sklearn.metrics import accuracy_score


def evaluate(y_true, scores, threshold):
    return {"accuracy": accuracy_score(y_true, scores >= threshold)}
```

Exercise 5. Architecture design (level 4)

Choose the headline metric for each model.

Options: Precision among the top k flagged, Accuracy, ROC AUC, Total cost at the chosen threshold.

1. Fraud: 1% prevalence, analysts review 150 cases a day
2. Compare two ranking models before any threshold exists
3. Loan approval: a default costs £4,000, a wrongly refused customer costs £300 of margin

Sources

- On make_classification data with 2.4% positives, all-negative accuracy was 0.976, logistic regression at 0.5 had recall 0.0, average precision 0.30 against a random 0.025. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository (random_state=0).)

B12-L2 Calibration, when the probability itself matters

Concept

A model is **calibrated** when its probabilities mean what they say: of all customers scored 0.30, about 30% churn. Ranking metrics such as ROC AUC ignore calibration. A model can rank perfectly and still say 0.9 where the truth is 0.1.

Calibration matters whenever someone uses the number itself: expected revenue (`probability × value`), a forecast count of churners, a risk shown to a person, or thresholds agreed in probability terms.

Check it with a calibration table: group predictions into bins, then compare the mean predicted probability with the observed rate in each bin. scikit-learn 1.8 gives the same with `calibration_curve(y, p, n_bins=10)`. Plot observed against predicted. A calibrated model follows the diagonal.

Summarise with the **Brier score** (`brier_score_loss`), the mean squared error of the probabilities. Compare it with a constant model that always predicts the prevalence.

A common cause of bad calibration is **reweighting for imbalance**. In this course's build, a logistic regression on data with 2.4% positives had a mean predicted probability of 0.024 and a Brier score of 0.021. The same model with `class_weight="balanced"` had a mean predicted probability of **0.42**, and a Brier score of **0.199**, worse than predicting 0.024 for everyone (0.024). Its AUC was slightly higher. Its probabilities were useless as probabilities.

Fix it with `CalibratedClassifierCV(model, method="sigmoid", cv=5)`, which fits a calibration map on held-out folds. Use `method="isotonic"` only with plenty of data, roughly a thousand positives or more, because it overfits small samples. Check calibration again on the test set after fitting.

If you only need a ranking or a top-k list, calibration does not matter. Say which one you deliver.

Why it matters

A retention team estimated campaign value as the sum of churn probabilities times customer margin. The model had been trained with balanced class weights. Its average churn probability was 0.41 on a base that churned at 3% a month. The business case promised ten times the saves the campaign could deliver, and budget was approved on it.

Worked example: weak code

```
model = LogisticRegression(class_weight="balanced").fit(X_train, y_train)
expected_churners = model.predict_proba(X_live)[:, 1].sum()
```

1. **Line 1.** Balanced weights push probabilities towards 0.5.
2. **Line 2.** Sums miscalibrated probabilities as if they were real rates.

Worked example: strong code

```
import numpy as np
import pandas as pd


def calibration_table(y: np.ndarray, p: np.ndarray, n_bins: int) -> pd.DataFrame:
    """Mean predicted and observed rate per equal-width bin. Empty bins are left out."""
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    idx = np.searchsorted(edges[1:-1], p, side="right")
    df = pd.DataFrame({"bin": idx, "y": y, "p": p})
    table = df.groupby("bin").agg(mean_predicted=("p", "mean"), observed=("y", "mean"), n=("y", "size"))
    return table.reset_index(drop=True)
```

1. **Line 8.** Assigns each probability to an equal-width bin.
2. **Line 10.** Predicted against observed, with the count behind each row.

When the probability is used as a number, check calibration, compare the Brier score with a constant baseline, and recalibrate on held-out data.

Recap card

- **Calibrated means** Of cases scored p, about p are positive.
- **Check calibration** Calibration table or calibration_curve, plus Brier score against the constant prevalence baseline.
- **class_weight="balanced" and probabilities** Pushes them towards 0.5. Mean p went from 0.024 to 0.42 at 2.4% prevalence.
- **Recalibrate** CalibratedClassifierCV(model, method="sigmoid", cv=5). Isotonic only with plenty of data.

Practice

Exercise 1. Multiple choice (level 1)

A model has ROC AUC 0.85. Of customers it scores between 0.8 and 0.9, 20% churn. What is true?

- a) It ranks well but is poorly calibrated.
- b) It is well calibrated.
- c) The AUC must be wrong.
- d) It ranks badly.

Why?

- r1) AUC measures ordering only. The probabilities can be far off while the order is right.
- r2) Any model with AUC above 0.8 is calibrated.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import brier_score_loss

y = [0, 0, 0, 1]
print(brier_score_loss(y, [0.25, 0.25, 0.25, 0.25]), brier_score_loss(y,
    [0.5, 0.5, 0.5, 0.5]))
```

Exercise 3. Fill the gap (level 2)

Recalibrate a classifier with a sigmoid map fitted on held-out folds.

```
calibrated = _____(1)(model, method="_____(2)", cv=5).fit(X_train,
    y_train)
```

Exercise 4. Write the function (level 3)

Write `calibration_table(y, p, n_bins) -> pd.DataFrame`. Split [0, 1] into `n_bins` equal-width bins. Assign each probability with `np.searchsorted(edges[1:-1], p, side="right")`. Return one row per **non-empty** bin, in bin order, with columns `mean_predicted`, `observed` (mean of `y`) and `n`, and a default 0..k index.

Your code must pass these checks:

- Each bin needs its mean predicted probability, observed rate and count.
- Leave out empty bins, and reset the index.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Mean predicted probability and observed rate per equal-width bin.
def calibration_table(
    y: np.ndarray,
    p: np.ndarray,
    n_bins: int,
) -> pd.DataFrame: ...
```

Starter code:

```python
import numpy as np
import pandas as pd


def calibration_table(y, p, n_bins):
    return pd.DataFrame({"mean_predicted": [p.mean()], "observed":
    [y.mean()]})
```

Exercise 5. Architecture design (level 4)

A churn model will be used two ways. Decide what each use needs.

Options: Good ranking only, Calibrated probabilities.

1. Pick the top 500 customers to call each week
2. Forecast the number of churners next month for finance
3. Class imbalance handling for the forecast model

Sources

- Logistic regression at 2.4% prevalence: mean p 0.024 and Brier 0.021; with class_weight="balanced", mean p 0.42 and Brier 0.199; constant baseline 0.024. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository (make_classification, random_state=0).)
- The table matches sklearn.calibration.calibration_curve with strategy="uniform" on test data. (verified: Compared with np.allclose on scikit-learn 1.8.0 in this repository.)

B12-L3 Imbalanced classes and error analysis by segment

Concept

Imbalance. When positives are rare, a model trained as usual predicts low probabilities, and the default 0.5 threshold flags almost nothing. That is a threshold problem, not always a model problem.

In this course's build (2.4% positives, logistic regression, scikit-learn 1.8), the plain model at 0.5 flagged 3 cases and found none. Lowering the threshold to

0.1 flagged 284 cases with precision 0.36 and recall 0.42. At 0.02 it reached recall 0.63 with precision 0.04. The same model trained with `class_weight="balanced"` reached recall 0.64 and precision 0.05 at 0.5: a very similar trade-off, with probabilities that were no longer calibrated (lesson B12-L2).

Options, in the order to try them:

1. **Train normally and choose the threshold** from capacity or cost (lesson B12-L1). Probabilities stay calibrated.
2. **Class weights** when the model struggles to learn the minority class at all. Recalibrate afterwards if you need probabilities.
3. **Resampling** (over- or under-sampling) **inside the training folds only**. Never resample the test set: its class balance must match production.

Error analysis. One overall metric hides where a model fails. Break the test results down by segment: region, device, customer tenure, survey mode. For each segment report the count, positives, precision and recall. A model with recall 0.60 overall can have recall 0.15 for new customers, the group the campaign most wants to reach.

Then read examples. Take the 20 most confident false positives and the 20 most confident false negatives, and look at them with someone who knows the domain. Patterns in those 40 rows often point to a missing feature, a label error or a leak.

Report segment results with their counts. A recall of 0.20 from 5 positives is a question, not a finding.

Why it matters

A complaint-routing model reported recall 0.71 overall. Complaints arriving by the new web chat were 18% of the volume and had recall 0.22: the model had been trained mostly on email text, and chat messages were shorter and informal. Urgent chat complaints waited in the general queue for three weeks before a supervisor noticed. A per-channel breakdown at validation would have shown it.

Worked example: weak code

```
model = LogisticRegression().fit(X_train, y_train)
print("recall", recall_score(y_test, model.predict(X_test)))
```

1. **Line 2.** The 0.5 default flags almost nothing on rare positives, and one overall number hides weak segments.

Worked example: strong code

```python
import pandas as pd


def metrics_by_segment(df: pd.DataFrame, segment: str) -> pd.DataFrame:
    """Count, positives, precision and recall per segment. Columns y and flagged are 0/1."""
    rows = []
    for name, g in df.groupby(segment, sort=True):
        tp = int(((g["y"] == 1) & (g["flagged"] == 1)).sum())
        n_flag, n_pos = int(g["flagged"].sum()), int(g["y"].sum())
        rows.append({
            segment: name,
            "n": len(g),
            "positives": n_pos,
            "precision": tp / n_flag if n_flag > 0 else None,
            "recall": tp / n_pos if n_pos > 0 else None,
        })
    return pd.DataFrame(rows)
```

1. **Line 7.** One row per segment, in a stable order.
2. **Line 13.** Counts beside every rate, so small segments are read with care.
3. **Line 15.** No positives, no recall: None instead of a crash or a fake 0.

Treat rare positives with a chosen threshold first, then break results down by segment and read the worst errors.

Recap card

- **First response to rare positives** Train normally and choose the threshold from capacity or cost.
- **Resampling rule** Only inside training folds. Never the test set.
- **Error analysis** Metrics per segment with counts, then read the most confident errors.
- **Pooled vs segment recall** Pooling weights large segments. Overall 0.7 can hide a segment at 0.2.

Practice

Exercise 1. Multiple choice (level 1)

A model trained on 2% positives flags nothing at the default threshold. What should you try first?

- a) Oversample the test set to 50% positives.
- b) Lower the threshold to match review capacity or costs.
- c) Switch to a deep neural network.

- d) Report accuracy instead.

Why?

- r1) The model's ranking may be fine. Only the cut-off is wrong for rare positives.
- r2) Lower thresholds always improve models.

Exercise 2. Predict the output (level 2)

What does this print?

```
segments = {"email": (80, 100), "chat": (4, 20)}
for name, (found, positives) in segments.items():
    print(name, found / positives)
print("overall", sum(f for f, _ in segments.values()) / sum(p for _, p in
   segments.values()))
```

Exercise 3. Put in order (level 2)

Order the steps for handling a rare positive class.

- Read the most confident false positives and false negatives
- Break test results down by segment, with counts
- Train the model normally
- Choose the threshold from capacity or cost on validation data
- Split train and test, stratified, test left at its natural balance

Exercise 4. Write the function (level 3)

Write `metrics_by_segment(df, segment) -> pd.DataFrame`. `df` has 0/1 columns `y` and `flagged` and a segment column named by `segment`. Return one row per segment, sorted by segment, with columns `segment` (named as the input column), `n`, `positives`, `precision` and `recall`. Use `None` for precision when a segment has no flags, and for recall when it has no positives.

Your code must pass these checks:

- Return the columns in the order given, one row per segment, sorted by segment.
- Count true positives per segment: precision = tp / flagged, recall = tp / positives.
- No flags gives no precision, and no positives gives no recall: missing, not 0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Count, positives, precision and recall per segment.
```

```
def metrics_by_segment(
    df: pd.DataFrame,
    segment: str,
) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def metrics_by_segment(df, segment):
    return df.groupby(segment)["y"].mean().reset_index()
```

Exercise 5. Spot the bug (level 4)

Validation reports recall 0.78, but production recall is 0.41. Tap the line that makes the evaluation unrepresentative, and fix the function.

What you need to know.

`df.sample(frac=5.0, replace=True, random_state=seed)` draws rows at random, with replacement, five times as many as the frame holds. Each row appears about five times. `pd.concat([a, b])` stacks two frames, rows of `a` first.

```
import pandas as pd


def prepare_sets(train: pd.DataFrame, test: pd.DataFrame, seed: int) ->
   tuple[pd.DataFrame, pd.DataFrame]:
    """Oversample positives so the model sees more of them."""
    pos_train = train[train["y"] == 1].sample(frac=5.0, replace=True,
   random_state=seed)
    pos_test = test[test["y"] == 1].sample(frac=5.0, replace=True,
   random_state=seed)
    return pd.concat([train, pos_train]), pd.concat([test, pos_test])
```

Write down the line number of the bug. Then write the fix.

Sources

- Plain logistic regression at 2.4% prevalence: threshold 0.1 gave precision 0.36, recall 0.42; threshold 0.02 gave recall 0.63, precision 0.04; balanced weights at 0.5 gave recall 0.64, precision 0.05. (verified: Run with scikit-learn 1.9.1 in this repository (make_classification, random_state=0).)

B13 Interpretability and fairness

Roadmap: Prerequisite for all months. Permutation importance, SHAP and subgroup performance.

B13-L1 Permutation importance on held-out data

Concept

"Which features matter?" has a trustworthy answer and a misleading one.

The misleading one is **impurity importance**, `feature_importances_` on scikit-learn tree ensembles. It is computed on training data and favours features with many distinct values. In this course's build (scikit-learn 1.9, random forest, 3,000 rows), a meaningless column of random IDs with 3,000 distinct values got an impurity importance of **0.335**, half as much as the real signal (0.661).

The trustworthy one is **permutation importance** on **held-out** data: shuffle one column, breaking its link with the target, and measure how much the score drops. A feature the model relies on causes a large drop. The same random ID column dropped the test AUC by **0.004**: nothing.

```
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
                                n_repeats=10, random_state=0)
result.importances_mean, result.importances_std
```

Read it with three cautions:

- **It measures the model, not the world.** A large drop means this model uses the feature. It does not mean the feature causes the outcome.
- **Correlated features share credit.** If `age` and `years_since_birth_cohort` carry the same information, shuffling one leaves the other, and both look unimportant. Group correlated features and permute them together, or report them as a group.
- **Use the spread.** Report the mean drop with its standard deviation across repeats. A drop of 0.003 ± 0.004 is zero.

Use the metric you report for the model. An importance in accuracy on imbalanced data says little (lesson B12-L1).

Why it matters

A retention model's slide showed "customer ID" as the third most important driver, from `feature_importances_`. The product team asked what it meant. It meant nothing: the ID had thousands of distinct values, so trees split on it to memorise training rows. Permutation importance on the test set gave it zero. The slide had already been shown to the client.

Worked example: weak code

```
importances = dict(zip(X_train.columns, model.feature_importances_))
top = sorted(importances, key=importances.get, reverse=True)[:5]
```

1. **Line 1.** Impurity importance from training data: biased towards high-cardinality features.

Worked example: strong code

```
import numpy as np
import pandas as pd
from sklearn.base import ClassifierMixin
from sklearn.metrics import roc_auc_score


def permutation_drop(model: ClassifierMixin, X: pd.DataFrame, y: pd.Series, col: str, rng: np.random.Generator, n_repeats: int) -> float:
    """Mean fall in held-out ROC AUC when col is shuffled."""
    base = roc_auc_score(y, model.predict_proba(X)[:, 1])
    drops = []
    for _ in range(n_repeats):
        shuffled = X.assign(**{col: rng.permutation(X[col].to_numpy())})
        drops.append(base - roc_auc_score(y, model.predict_proba(shuffled)[:, 1]))
    return float(np.mean(drops))
```

1. **Line 9.** Score on held-out rows, with the reported metric.
2. **Line 12.** Breaks the link between one feature and the target, keeping its distribution.

Measure importance by shuffling on held-out data, with the reported metric, and report the spread.

Recap card

- **Impurity importance bias** Favours high-cardinality features, from training data. A random ID scored 0.335 in this lesson's example.
- **Permutation importance** Shuffle a column on held-out data and measure the score drop, with its spread.

- **Correlated features** Share credit. Permute and report them as a group.
- **What importance means** What the model relies on. Not what causes the outcome.

Practice

Exercise 1. Multiple choice (level 1)

A random ID column ranks high in a random forest's `feature_importances_`. What is the likely cause?

- a) IDs predict the outcome.
- b) Impurity importance favours features with many distinct values, measured on training data.
- c) The forest has too few trees.
- d) The ID was scaled incorrectly.

Why?

- r1) Many split points let trees memorise training rows, which impurity importance rewards.
- r2) Random forests cannot handle integers.

Exercise 2. Predict the output (level 2)

`age` and `birth_year` hold the same information. A model uses both. You permute only `age` on held-out data. What happens to its importance?

```
drop_age_alone = 0.01        # measured
drop_both_together = 0.12    # measured when both are shuffled together
print(drop_age_alone < drop_both_together)
```

Exercise 3. Fill the gap (level 2)

Compute permutation importance on the test set, scored by ROC AUC, with 10 repeats.

```
result = permutation_importance(model, _____(1), y_test,
    scoring="roc_auc", n_repeats=10, random_state=0)
print(result.importances_mean, result._____(2))
```

Exercise 4. Write the function (level 3)

Write `permutation_drop(model, X, y, col, rng, n_repeats) -> float`. Compute the ROC AUC of `model.predict_proba(X)[:, 1]`. Then, `n_repeats` times, replace `col` with `rng.permutation(X[col].to_numpy())` in a **copy** of `X`, recompute the AUC, and record the drop. Return the mean drop. Leave `X` unchanged.

Your code must pass these checks:

- X changed. Shuffle a copy, for example with X.assign(...).
- Shuffle with rng.permutation each repeat, and average base AUC minus shuffled AUC.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Mean fall in ROC AUC when col is shuffled.
def permutation_drop(
    model: ClassifierMixin,
    X: pd.DataFrame,
    y: pd.Series,
    col: str,
    rng: np.random.Generator,
    n_repeats: int,
) -> float: ...
```

Starter code:

```python
import numpy as np


def permutation_drop(model, X, y, col, rng, n_repeats):
    return float(model.coef_[0][list(X.columns).index(col)])
```

Exercise 5. Architecture design (level 4)

A client asks for "the top drivers of churn" from your model. Choose what to deliver.

Options: Permutation importance on the test set, with the reported metric, feature_importances_ from training.

1. Importance method
2. tenure_months and contract_start_year are 0.97 correlated
3. Wording on the slide

Sources

- A random ID with 3,000 values got impurity importance 0.335 and permutation importance (ROC AUC, test) 0.004 in a 200-tree random forest. (verified: Run with scikit-learn 1.9.1 in this repository (random_state=0).)

B13-L2 SHAP values, and what they do not tell you

Concept

Permutation importance (lesson B13-L1) ranks features for the whole model. **SHAP values** explain **one prediction**: how much each feature pushed this customer's score up or down from a baseline.

For one row, SHAP values are **additive**:

```
prediction = base_value + sum of the row's SHAP values
```

The base value is the model's average output over the background data. In shap 0.51:

```
import shap
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)          # shap.Explanation
explanation.values, explanation.base_values
```

Know which **output space** the values live in. For a scikit-learn `GradientBoostingClassifier`, TreeExplainer explains the log-odds (`decision_function`), not the probability. For a `RandomForestClassifier` it explains probabilities, with one set of values per class, so `.values` has shape `(rows, features, classes)`. This course's build checked both additivity identities on shap 0.51 with scikit-learn 1.9. Say which space you report.

Aggregate carefully. The mean **absolute** SHAP value per feature gives a global ranking. A beeswarm plot shows the direction and spread.

What SHAP does **not** tell you:

- **Causes.** A feature with a large SHAP value is one the model uses. Changing it in the real world may do nothing.
- **Robust truths about correlated features.** When two features carry the same information, the credit split between them depends on the model and the explainer's assumptions.
- **Anything about data the model never saw.** Explanations for out-of-range inputs describe extrapolation.

For customer-facing reasons ("your application was declined because..."), SHAP values are a starting point for wording. They are not a legal explanation on their own.

shap is not available in this app's browser runtime. The exercises use plain Python on SHAP outputs. Run shap itself locally.

Why it matters

A pricing team read a SHAP summary showing that `number_of_support_calls` pushed renewal probability **up**. They proposed encouraging customers to call support more. The model had learned that customers who called support were the engaged ones who had already decided to stay. More calls would not create engagement. An experiment on proactive calls later showed no effect on renewals.

Worked example: weak code

```
explanation = shap.TreeExplainer(rf_classifier)(X_test)
top = X_test.columns[np.argsort(explanation.values.mean(axis=0))[-3:]]
print("Top causes of churn:", list(top))
```

1. **Line 2.** Signed means cancel out, and a random forest's values have a class axis that is ignored here.
2. **Line 3.** Model reliance reported as causes.

Worked example: strong code

```
def top_contributions(names: list[str], values: list[float], k: int) -> list[tuple[str, float]]:
    """The k largest contributions to one prediction, by absolute size, with their sign."""
    if len(names) != len(values):
        raise ValueError("one SHAP value per feature is required")
    ranked = sorted(zip(names, values, strict=True), key=lambda pair: abs(pair[1]), reverse=True)
    return ranked[:k]


def check_additivity(base_value: float, values: list[float], prediction: float, tol: float = 1e-6) -> bool:
    """True when base value plus contributions reproduces the model output."""
    return abs(base_value + sum(values) - prediction) <= tol
```

1. **Line 5.** Rank by size, keep the sign for the wording.
2. **Line 11.** A cheap test that you are reading the right output space.

SHAP explains one prediction additively in a stated output space. It describes the model, not the world.

Recap card

- **SHAP additivity** prediction = base_value + sum of the row's SHAP values, in the explainer's output space.

- **Output space for GradientBoostingClassifier in TreeExplainer** Log-odds (decision_function), not probability.
- **Global ranking from SHAP** Mean absolute SHAP value per feature.
- **What SHAP cannot show** Causes, a stable split between correlated features, behaviour outside the training range.

Practice

Exercise 1. Multiple choice (level 1)

A feature has a large positive SHAP value for most churners. What can you conclude?

- a) Increasing the feature causes churn.
- b) The model uses the feature to push churn scores up.
- c) The feature is measured wrongly.
- d) The model is overfitting.

Why?

- r1) SHAP values attribute the model's output. They say nothing about intervening in the world.
- r2) Positive SHAP values are always spurious.

Exercise 2. Predict the output (level 2)

A GradientBoostingClassifier row has base value -1.2 (log-odds) and SHAP values 0.8, 0.5 and -0.3. What does this print?

What you need to know.

Log-odds convert to a probability with the logistic function: `p = 1 / (1 + e^(-log_odds))`. `math.exp(x)` computes `e^x`.

```
import math

1 / (1 + math.exp(-0.0))   # 0.5: log-odds 0 means even odds
```

```
import math

log_odds = -1.2 + 0.8 + 0.5 - 0.3
print(round(log_odds, 2), round(1 / (1 + math.exp(-log_odds)), 2))
```

Exercise 3. Fill the gap (level 2)

Compute SHAP values for a tree model with shap 0.51.

```
explainer = shap._____(1)(model)
explanation = explainer(X_test)
contributions = explanation._____(2)
```

Exercise 4. Write the function (level 3)

Write `top_contributions(names, values, k) -> list[tuple[str, float]]`. Return the `k` pairs `(name, value)` with the largest **absolute** values, largest first, keeping the sign. Raise `ValueError` if `names` and `values` differ in length.

Your code must pass these checks:

- Rank by absolute size, and keep the sign of each value.
- Mismatched lengths must raise ValueError.
- k larger than the number of features returns all of them.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# The k largest contributions to one prediction, by absolute size.
def top_contributions(
    names: list[str],
    values: list[float],
    k: int,
) -> list[tuple[str, float]]: ...
```

Starter code:

```python
def top_contributions(names, values, k):
    return sorted(zip(names, values), key=lambda p: p[1], reverse=True)
    [:k]
```

Exercise 5. Architecture design (level 4)

Explain a declined loan application to the customer service team. Choose each part.

Options: State the space (log-odds) and translate the final score to a probability, Show log-odds contributions labelled as percentage points.

1. Output space shown
2. What the explanation lists
3. Wording

Sources

- shap 0.51 TreeExplainer: additivity holds in decision_function space for GradientBoostingClassifier and in probability space, per class, for RandomForestClassifier. (verified: Checked with np.allclose on shap 0.51.0 and scikit-learn 1.9.1 in this repository.)

B13-L3 Performance by subgroup, and the fairness trade-offs

Concept

A model can be accurate overall and fail one group. Before release, compute the key rates **per subgroup**: age band, region, sex, ethnicity where you hold it lawfully, and any group the decision affects differently.

Per group, report:

- **n** and **positives**, so readers can judge the evidence.
- **Selection rate**: the share flagged.
- **True positive rate** (recall): the share of real positives flagged.
- **False positive rate**: the share of real negatives flagged.
- **Precision**, and a calibration check (lesson B12-L2).

Then face a hard fact. When groups have **different base rates**, a calibrated score cannot also give equal false positive and true positive rates, except in trivial cases (Kleinberg, Mullainathan and Raghavan, 2016; Chouldechova, 2017).

A simulation in this course's build: a perfectly calibrated score for two groups with base rates of 30% and 10%, one threshold of 0.3. In both groups, cases scored 0.3 to 0.4 were positive about 34% of the time. Yet group A had a true positive rate of 0.65 and a false positive rate of 0.39. Group B had 0.15 and 0.03.

So choose, with the people accountable for the decision, which property matters most for this use:

- Equal **calibration**: a score means the same risk in every group.
- Equal **true positive rates**: each group's genuine cases are found at the same rate. Suits allocating help.
- Equal **false positive rates**: each group bears the same rate of wrong flags. Suits decisions with a cost to the flagged person.

Write the choice, the reasons and the measured gaps in the model card (lesson B18-L2).

Removing a protected attribute from the features does not remove its influence. Postcode, first name and shopping patterns act as **proxies**. Keep the attribute available for **measuring**, where lawful, even if the model never uses it.

Why it matters

A hardship-support model flagged customers for proactive help. Overall recall was 0.62. By age band, customers over 70 had recall 0.28: they used fewer digital channels, so the features the model relied on were sparse for them. The group most likely to need help was the least likely to be offered it. A per-group table at validation showed the gap in one line.

Worked example: weak code

```
features = [c for c in df.columns if c not in ("age_band", "sex")]
print("recall", recall_score(y_test, flagged))
```

1. **Line 1.** Dropping attributes does not remove proxies, and leaves nothing to measure gaps with.
2. **Line 2.** One overall number hides a group at a fraction of it.

Worked example: strong code

```
import pandas as pd


def group_rates(df: pd.DataFrame, group: str) -> pd.DataFrame:
    """Selection rate, TPR and FPR per group, with counts. Columns y and flagged are 0/1."""
    rows = []
    for name, g in df.groupby(group, sort=True):
        pos, neg = g["y"] == 1, g["y"] == 0
        flag = g["flagged"] == 1
        rows.append({
            group: name,
            "n": len(g),
            "positives": int(pos.sum()),
            "selection_rate": float(flag.mean()),
            "tpr": float((flag & pos).sum() / pos.sum()) if pos.sum() > 0 else None,
            "fpr": float((flag & neg).sum() / neg.sum()) if neg.sum() > 0 else None,
        })
    return pd.DataFrame(rows)
```

1. **Line 7.** The group attribute is used to measure, whether or not the model uses it.
2. **Line 15.** Guarded: a group with no positives has no TPR.

Measure rates per group with counts, choose the fairness property that fits the decision, and document the gaps.

Recap card

- **Rates to report per group** n, positives, selection rate, TPR, FPR, precision, calibration.
- **Impossibility result** With different base rates, calibration and equal error rates cannot all hold (Kleinberg et al. 2016; Chouldechova 2017).
- **Dropping a protected attribute** Proxies keep its influence. Keep it for measurement where lawful.
- **Choosing a fairness property** From the harm: missed help (TPR), wrong flags (FPR), numbers used directly (calibration).

Practice

Exercise 1. Multiple choice (level 1)

You remove `sex` from a model's features. What happens to sex-related disparities?

- a) They disappear.
- b) They can persist through proxies, and you lose the ability to measure them if you discard the column.
- c) They always get worse.
- d) The model becomes calibrated.

Why?

- r1) Other features correlate with sex, and measurement needs the attribute.
- r2) Removing features never changes a model.

Exercise 2. Predict the output (level 2)

What does this print?

```
groups = {"A": {"tp": 65, "pos": 100, "fp": 150, "neg": 390}, "B": {"tp": 15, "pos": 100, "fp": 25, "neg": 900}}
for name, g in groups.items():
    print(name, round(g["tp"] / g["pos"], 2), round(g["fp"] / g["neg"], 2))
```

Exercise 3. Fill the gap (level 2)

Compute one group's true positive rate and false positive rate from boolean Series `flag`, `pos` and `neg`.

```
tpr = (flag & _____(1)).sum() / pos.sum()
fpr = (flag & _____(2)).sum() / neg.sum()
```

Exercise 4. Write the function (level 3)

Write `group_rates(df, group) -> pd.DataFrame`. `df` has 0/1 columns `y` and `flagged`, and a group column. Return one row per group, sorted, with columns `<group>`, `n`, `positives`, `selection_rate`, `tpr` and `fpr`. Use `None` for `tpr` when a group has no positives, and for `fpr` when it has no negatives.

Your code must pass these checks:

- Return the columns in order, one row per group, sorted.
- selection_rate is the share flagged. TPR divides by positives, FPR by negatives.
- A group with no negatives has no FPR: return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Selection rate, TPR and FPR per group, with counts.
def group_rates(
    df: pd.DataFrame,
    group: str,
) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def group_rates(df, group):
    return df.groupby(group)["flagged"].mean().reset_index()
```

Exercise 5. Architecture design (level 4)

Choose the fairness property to prioritise for each use.

Options: Equal true positive rates, Equal false positive rates, Equal calibration.

1. Offer proactive hardship support
2. Refer claims for a fraud investigation that delays payment
3. Risk scores used directly as probabilities in pricing

Sources

- Calibrated scores with base rates of 30% and 10% at threshold 0.3 gave TPR 0.65 and FPR 0.39 in one group, 0.15 and 0.03 in the other. (verified: Simulated with NumPy 2.4.6 in this repository (Beta(3,7) and Beta(1,9) risks, 200,000 rows each).)
- Calibration within groups and balance of error rates cannot all hold when base rates differ, except in trivial cases. (verified: Kleinberg, Mullainathan

and Raghavan (2016), Inherent Trade-Offs in the Fair Determination of Risk Scores; Chouldechova (2017), Big Data 5(2).)

B14 Experiment tracking with MLflow

Roadmap: Prerequisite for all months; Months 16 to 18: MLOps and deployment. Tracking, registry and the model lifecycle.

B14-L1 Track every run with MLflow

Concept

After twenty experiments nobody remembers which settings produced the good score. **MLflow tracking** records each run: parameters, metrics, tags and files. Examples use MLflow 3.16. On Databricks the tracking server is built in. Locally, MLflow 3 stores runs in a SQLite file, `mlflow.db`, in the working directory by default.

```python
import mlflow

mlflow.set_experiment("churn")
with mlflow.start_run(run_name="lr-c1"):
    mlflow.log_params({"C": 1.0, "seed": 7, "features": "v3"})
    mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
    model = train(X_train, y_train, C=1.0, seed=7)
    mlflow.log_metric("cv_auc", cv_auc)
    mlflow.sklearn.log_model(model, name="model", input_example=X_train[:5])
```

What to record, every time:

- **Parameters**: every setting that changes the result, including the seed and the feature set version. MLflow stores them as **strings**: `1.0` comes back as `'1.0'`. A parameter cannot be changed once logged in a run. Logging `C` twice with different values raises `MlflowException`.
- **Tags**: the Git commit and the input data hash from the run manifest (lesson B6-L2). They link the run to exact code and data.
- **Metrics**: the honest estimate (lesson B11-L3) and its spread, named consistently across runs: `cv_auc_mean`, `cv_auc_std`.
- **The model**: `log_model` saves the fitted object with a `requirements.txt`, `conda.yaml` and `python_env.yaml` pinned to the versions used. For scikit-learn models MLflow 3.16 saves in the skops format.

Use the `with mlflow.start_run():` block. If training raises, the run ends with status FAILED instead of staying open and collecting the next run's metrics.

Compare runs with `mlflow.search_runs(experiment_names=["churn"])`, which returns a pandas DataFrame with `params.*`, `metrics.*` and `tags.*` columns.

mlflow does not run in this app's browser runtime. The exercises check your code's structure.

Why it matters

A model with AUC 0.83 went to production from a notebook. Three months later it needed retraining. Nobody could reproduce 0.83: the notebook had been edited since, the seed had not been recorded, and the training extract had been refreshed. The team spent two weeks rebuilding a model they had already built once. One tracked run with parameters, commit and data hash would have made the retrain a single command.

Worked example: weak code

```
1  model = LogisticRegression(C=c).fit(X_train, y_train)
1  print(c, roc_auc_score(y_val, model.predict_proba(X_val)[:, 1]))
2  pickle.dump(model, open("model_final_v2.pkl", "wb"))
```

1. **Line 2.** Results live in cell output, lost on the next run.
2. **Line 3.** A file name as version control, with no code, data or environment attached.

Worked example: strong code

```
1  import mlflow
2  import numpy as np
3  from sklearn.linear_model import LogisticRegression
4
5
6  def tracked_fit(X_train: np.ndarray, y_train: np.ndarray, c: float, seed:
       int, commit: str, data_hash: str, cv_auc: float) -> str:
7      """Fit and log one run. Returns the run ID."""
1      with mlflow.start_run(run_name=f"lr-c{c}") as run:
9          mlflow.log_params({"C": c, "seed": seed})
2          mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
11         model = LogisticRegression(C=c, random_state=seed,
       max_iter=1000).fit(X_train, y_train)
12         mlflow.log_metric("cv_auc", cv_auc)
3          mlflow.sklearn.log_model(model, name="model",
       input_example=X_train[:5])
14         return run.info.run_id
```

1. **Line 8.** The run ends cleanly even if fitting raises.
2. **Line 10.** Code and data identity, attached to the run.
3. **Line 13.** The model with pinned requirements and an input example.

One run per fit, inside a with block, with parameters, commit, data hash, metrics and the model.

Recap card

- **What to log per run** Params (incl. seed), tags (git_commit, input_sha256), metrics, the model.
- **MLflow parameters** Stored as strings and immutable within a run.
- **Why with mlflow.start_run()** The run ends with FAILED status if training raises, instead of staying open.
- **log_model in MLflow 3** mlflow.sklearn.log_model(model, name="model"), with pinned requirements saved alongside.

Practice

Exercise 1. Multiple choice (level 1)

You log `mlflow.log_param("C", 1.0)` and later read the run back. What type is the value?

- a) float
- b) str
- c) int
- d) Decimal

Why?

- r1) MLflow stores parameters as strings, so compare and convert carefully.
- r2) Python cannot store floats in a database.

Exercise 2. Predict the output (level 2)

What happens? (MLflow 3.16)

What you need to know.

`mlflow.log_param(key, value)` logs one parameter to the active run. `mlflow.log_params({...})`, from the concept, logs several at once. Both write to the same parameter store, so the concept's rules about parameters apply to each call.

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("C", 1.0)
    mlflow.log_param("C", 2.0)
```

Exercise 3. Fill the gap (level 2)

Log the Git commit as a tag and the model under the name `model`.

What you need to know.

Each plural MLflow call has a singular form for one item: `log_params` and `log_param(key, value)`, `log_metrics` and `log_metric(key, value)`, `set_tags` and `set_tag(key, value)`.

```
mlflow._____(1)("git_commit", commit)
mlflow.sklearn.log_model(model, _____(2)="model")
```

Exercise 4. Write the function (level 3)

Write `tracked_fit(X_train, y_train, c, seed, commit, data_hash, cv_auc) -> str`. Inside `with mlflow.start_run(...) as run:` log params `C` and `seed`, set tags `git_commit` and `input_sha256`, fit `LogisticRegression(C=c, random_state=seed, max_iter=1000)`, log the metric `cv_auc`, log the model with `mlflow.sklearn.log_model(model, name="model")`, and return `run.info.run_id`.

mlflow does not run in the browser. The checker reads your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Fit a logistic regression and record the run in MLflow.
def tracked_fit(
    X_train: np.ndarray,
    y_train: np.ndarray,
    c: float,
    seed: int,
    commit: str,
    data_hash: str,
    cv_auc: float,
) -> str: ...
```

Starter code:

```
import mlflow
import numpy as np
from sklearn.linear_model import LogisticRegression


def tracked_fit(X_train, y_train, c, seed, commit, data_hash, cv_auc):
    mlflow.start_run()
    model = LogisticRegression(C=c).fit(X_train, y_train)
    print(cv_auc)
    return model
```

Exercise 5. Architecture design (level 4)

Decide how each item is recorded in MLflow.

Options: Parameter, Tag, Metric, Artifact.

1. Random seed
2. Git commit
3. Cross-validated AUC
4. Calibration plot

Sources

- MLflow 3.16: default local tracking URI is sqlite:///<cwd>/mlflow.db; params come back as strings; re-logging a param with a new value raises MlflowException; log_model accepts name=; sklearn models are saved as model.skops with requirements.txt, conda.yaml and python_env.yaml. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)

B14-L2 Registry versions and aliases, not stages

Concept

The **model registry** turns logged models into named, numbered versions. Each `log_model(..., registered_model_name="churn_model")` or `mlflow.register_model(...)` call creates a new **version**: 1, 2, 3. A version never changes. It points at one run, one set of parameters and one model file.

Aliases are movable labels on versions. MLflow 3.16:

```
from mlflow import MlflowClient

client = MlflowClient()
client.set_registered_model_alias("churn_model", "champion", "3")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

The scoring job always loads `@champion`. Promotion moves the alias to a new version. Rollback moves it back. The job's code never changes, and every prediction can be traced to a version and its run.

Stages (`Staging`, `Production`, `Archived`) are the older mechanism. `transition_model_version_stage` has been deprecated since MLflow 2.9, and MLflow 3.16 warns that stages will be removed. Models in Unity Catalog on Databricks use aliases only, with three-level names such as `ml.churn.churn_model` after `mlflow.set_registry_uri("databricks-uc")`.

Promote on evidence, not by hand:

1. Train and register a candidate version.

2. Evaluate it on the same held-out data and metric as the current champion.
3. Move `champion` only if the candidate beats it by a margin agreed in advance.
4. Keep the old version reachable, for example with a `previous_champion` alias, so rollback is one call.

Record the decision: which versions were compared, the scores and who approved. On Databricks, Unity Catalog grants (lesson A4-L2) control who can move an alias.

Why it matters

A team kept production models as files named `model_prod.pkl`, overwritten on each release. A new version cut conversion forecasts by 12% overnight. Rolling back meant finding last month's file on someone's laptop. It took a day. With versions and an alias, rollback is `set_registered_model_alias("churn_model", "champion", "4")`, and the scoring job picks it up on its next run.

Worked example: weak code

```
(1) client.transition_model_version_stage("churn_model", new_version,
      "Production")
    model = mlflow.pyfunc.load_model("models:/churn_model/Production")
```

1. **Line 1.** Deprecated since MLflow 2.9, not available in Unity Catalog, and promoted without a comparison.

Worked example: strong code

```
from __future__ import annotations

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from mlflow import MlflowClient


def promote_if_better(client: MlflowClient, name: str, candidate: str,
    metric: str, min_gain: float) -> bool:
    """Move the champion alias to candidate if it beats the champion by
    min_gain."""
(1)  champ = client.get_model_version_by_alias(name, "champion")
    champ_score = client.get_run(champ.run_id).data.metrics[metric]
    cand_score = client.get_run(client.get_model_version(name,
    candidate).run_id).data.metrics[metric]
(2)  if cand_score < champ_score + min_gain:
```

```
15          return False
3       client.set_registered_model_alias(name, "previous_champion",
        champ.version)
17      client.set_registered_model_alias(name, "champion", candidate)
18      return True
```

1. **Line 11.** The current champion, resolved through its alias.
2. **Line 14.** An agreed margin, so noise alone does not trigger a release.
3. **Line 16.** Rollback stays one alias move away.

Versions are fixed. Aliases move. Promote by comparison on the same data and metric, and keep the previous champion addressable.

Recap card

- **Version vs alias** A version is fixed. An alias is a movable label on a version.
- **Load the current production model** mlflow.pyfunc.load_model("models:/name@champion").
- **Model stages** Deprecated since MLflow 2.9, not supported in Unity Catalog. Use aliases.
- **Safe promotion** Compare on the same data and metric, require a margin, keep previous_champion for rollback.

Practice

Exercise 1. Multiple choice (level 1)

What does the scoring job load in an alias-based setup?

- a) `models:/churn_model/7`
- b) `models:/churn_model@champion`
- c) `models:/churn_model/Production`
- d) The latest file in a shared folder

Why?

- r1) The alias moves on promotion and rollback, so the job's code never changes.
- r2) Aliases load faster than versions.

Exercise 2. Predict the output (level 2)

A stub registry in plain Python behaves like MLflow's aliases. What does this print?

```
1 aliases = {"champion": "3"}
2 aliases["previous_champion"] = aliases["champion"]
```

```
aliases["champion"] = "5"
aliases["champion"] = aliases["previous_champion"]
print(aliases)
```

Exercise 3. Fill the gap (level 2)

Point the champion alias at version 4, then load whatever it points to.

```
client._____(1)("churn_model", "champion", "4")
model = mlflow.pyfunc.load_model("models:/churn_model_____(2)champion")
```

Exercise 4. Write the function (level 3)

Write `promote_if_better(client, name, candidate, metric, min_gain) -> bool`. The model already has a `champion` alias.

- Read the champion with `client.get_model_version_by_alias(name, "champion")`, and its score from `client.get_run(champ.run_id).data.metrics[metric]`.
- Read the candidate's score through `client.get_model_version(name, candidate)`.
- If the candidate's score is at least the champion's plus `min_gain`, set `previous_champion` to the champion's version, then `champion` to `candidate`, and return `True`. Otherwise change nothing and return `False`.

The checker passes a stub client with the same methods as `MlflowClient`.

Your code must pass these checks:

- A candidate that beats the champion by min_gain becomes champion, and the old champion becomes previous_champion.
- A gain below min_gain (0.005 < 0.01) must change nothing and return False.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Move the champion alias to candidate if it beats the champion by
    min_gain.
def promote_if_better(
    client: MlflowClient,
    name: str,
    candidate: str,
    metric: str,
    min_gain: float,
) -> bool: ...
```

Starter code:

```
def promote_if_better(client, name, candidate, metric, min_gain):
    client.set_registered_model_alias(name, "champion", candidate)
    return True
```

Exercise 5. Architecture design (level 4)

Set up model releases on Databricks with Unity Catalog.

Options: ml_prod.churn.churn_model, churn_model_final.

1. Model name
2. What the scoring job loads
3. Who can move the champion alias

Sources

- set_registered_model_alias, get_model_version_by_alias and models:/name@alias work as described; transition_model_version_stage warns it is deprecated since 2.9.0. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)
- Models in Unity Catalog use three-level names with mlflow.set_registry_uri("databricks-uc") and do not support stages. (search-only: Databricks documentation, Manage model lifecycle in Unity Catalog (docs site not reachable from this environment).)

B14-L3 Reproduce a run from its record

Concept

A tracked run is only useful if you can rebuild its result from the record. Test that before you need it, not during an incident.

From a run ID, MLflow 3.16 gives you everything the run logged:

```
run = mlflow.get_run(run_id)
run.data.params            # {'C': '1.0', 'seed': '7', 'features': 'v3'}  strings
run.data.tags["git_commit"], run.data.tags["input_sha256"]
run.data.metrics["cv_auc"]
reqs = mlflow.pyfunc.get_model_dependencies(f"runs:/{run_id}/model")   # path to requirements.txt
```

The procedure:

1. **Code.** Check out the logged commit: `git switch --detach <commit>`.
2. **Environment.** Create a fresh environment from the lock file at that commit (lesson B6-L3), or from the model's `requirements.txt`.
3. **Data.** Load the input and compare its SHA-256 with `input_sha256`. If it differs, stop: you are reproducing on different data.

4. **Settings.** Convert the parameters back to their types. `'1.0'` must become `1.0` and `'7'` must become `7`. Passing `'7'` as a seed fails, or worse, works differently.
5. **Rerun and compare.** Recompute the metric and compare with the logged value within a small tolerance. Identical code, data, settings and versions on the same hardware usually reproduce exactly. Different CPUs or thread counts can change the last digits of floating-point results.

If the numbers differ by more than the tolerance, the record is incomplete. Something that changes the result was not logged: a feature list read from a file, a default that changed between library versions, an unseeded shuffle. Add it to the run's parameters and try again.

Schedule a reproduction check: once a quarter, pick the champion model's run and rebuild it from scratch in CI.

Why it matters

An auditor asked a lender to show how a credit model's reported AUC of 0.78 had been produced. The run existed in MLflow with parameters and a metric. The input table had since been overwritten, and no data hash had been logged. The team could not show the model had been validated on the data they claimed. The model was suspended for four weeks while it was rebuilt with full records.

Worked example: weak code

```
params = mlflow.get_run(run_id).data.params
model = LogisticRegression(C=params["C"], random_state=params["seed"])
```

1. **Line 2.** Parameters are strings. '1.0' and '7' are passed through unconverted.

Worked example: strong code

```
PARAM_TYPES: dict[str, type] = {"C": float, "seed": int, "features": str}


def parse_params(params: dict[str, str], types: dict[str, type]) -> dict[str, object]:
    """Convert logged string parameters back to their types. Fails on missing or bad values."""
    missing = sorted(set(types) - set(params))
    if missing:
        raise KeyError(f"run is missing parameters: {missing}")
    return {key: cast(params[key]) for key, cast in types.items()}


def reproduced(logged: float, rerun: float, tol: float = 1e-9) -> bool:
```

```
13      """True when the rerun metric matches the logged one within tol."""
(3)     return abs(logged - rerun) <= tol
```

1. **Line 1.** The expected parameters and their types, in one place.
2. **Line 8.** A run without a needed parameter cannot be reproduced. Say so.
3. **Line 14.** An explicit tolerance, recorded with the check.

Code from the commit, environment from the lock, data checked by hash, settings converted back to types, metric compared within a tolerance.

Recap card

- **Reproduce a run** Commit, environment from lock, data verified by hash, typed params, rerun, compare within tolerance.
- **Parameters from mlflow.get_run** Strings. Convert with the expected types before use.
- **Model dependencies** mlflow.pyfunc.get_model_dependencies(model_uri) returns the path to requirements.txt.
- **Rerun differs beyond tolerance** Something that changes results was not logged. Find it and log it.

Practice

Exercise 1. Multiple choice (level 1)

You are reproducing a run. The input file's SHA-256 differs from the logged `input_sha256`. What next?

- a) Continue. The file name is the same.
- b) Stop. You would be reproducing on different data. Find the original input.
- c) Update the logged tag to the new hash.
- d) Rerun twice and average.

Why?

- r1) Any changed byte changes the hash, so the data is not the data the run used.
- r2) Hashes are unreliable.

Exercise 2. Predict the output (level 2)

Logged parameters come back as strings. What does this print?

```
1 params = {"C": "1.0", "seed": "7"}
2 print(params["seed"] + "1", int(params["seed"]) + 1)
```

Exercise 3. Put in order (level 2)

Order the steps to reproduce a logged run.

- Load the input and verify its SHA-256 against input_sha256
- Check out the logged git commit
- Convert the logged parameters back to their types
- Build a fresh environment from the lock file at that commit
- Rerun and compare the metric within a tolerance

Exercise 4. Write the function (level 3)

Write `parse_params(params: dict[str, str], types: dict[str, type]) -> dict[str, object]`. For each key in `types`, convert `params[key]` with the type, for example `float("1.0")`. Raise `KeyError` naming the missing keys (sorted) when any key in `types` is absent from `params`. Ignore extra keys in `params`. Let a bad value raise the type's own `ValueError`.

Your code must pass these checks:

- Convert each logged string with its type.
- Missing parameters must raise KeyError naming all of them.
- Ignore extra keys, and let a bad value raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Convert logged string parameters back to their types.
def parse_params(
    params: dict[str, str],
    types: dict[str, type],
) -> dict[str, object]: ...
```

Starter code:

```python
def parse_params(params, types):
    return dict(params)
```

Exercise 5. Architecture design (level 4)

A quarterly check rebuilds the champion model's run in CI. The rerun AUC is 0.7812 against a logged 0.7851. Choose the response.

Options: Not reproduced: the gap is far above the tolerance of 1e-6, Close enough.

1. Verdict
2. Next step
3. Lasting fix

Sources

- mlflow.pyfunc.get_model_dependencies returns a path to requirements.txt; get_run(...).data.params values are strings. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)

B15 Embeddings and clustering

Roadmap: Months 7 to 9: NLP with deep learning. SentenceTransformers, HDBSCAN and cluster validation.

B15-L1 Sentence embeddings for open-ended answers

Concept

Open-ended survey answers ("What could we improve?") arrive as thousands of short texts. A **sentence embedding model** turns each text into a fixed-length vector, so that texts with similar meaning get nearby vectors. With the sentence-transformers library:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
vectors = model.encode(answers, batch_size=64, normalize_embeddings=True)
```

`all-MiniLM-L6-v2` returns 384 numbers per text. With `normalize_embeddings=True` every vector has length 1, so the **cosine similarity** of two texts is their dot product: 1 for the same direction, 0 for unrelated.

Rules that keep embeddings usable:

- **One model per analysis.** Vectors from different models, or different versions of one model, live in different spaces. Never compare or mix them. Record the model name and version with the vectors.
- **Cache the vectors.** Encoding is the slow step. Store vectors with the text ID, the model name and a hash of the text, and re-encode only new or changed answers.
- **Clean lightly.** Strip whitespace and drop empty answers. Do not stem or remove stop words: the model was trained on natural text.
- **Check the language.** Many small models are English-only. Use a multilingual model for multi-country surveys, and test it on your own answers.
- **Handle zero vectors.** An empty string can produce a vector you should not compare. Filter before encoding, and guard any division by a vector's length.

Embeddings reflect the text the model was trained on, including its biases. Read samples of what the model calls "similar" before you trust any grouping built on it.

sentence-transformers needs PyTorch, which this app cannot run in the browser. This course's build did not run sentence-transformers, so the API details above come from the library's documentation and are marked as not run. The exercises use NumPy on vectors.

Why it matters

An analyst embedded wave 1 answers with one model and, a year later, wave 2 answers with a newer model under the same short name. Similarity between the waves came out low across the board, and the report said themes had "shifted dramatically". The texts had not changed much. The two sets of vectors came from different spaces.

Worked example: weak code

```
1 def most_similar(query, matrix):
1     scores = matrix @ query
3     return int(scores.argmax())
```

1. **Line 2.** A dot product equals cosine similarity only when all vectors have length 1.

Worked example: strong code

```
1  import numpy as np
2
3
4  def top_k_similar(query: np.ndarray, matrix: np.ndarray, k: int) ->
     list[int]:
5      """Row indices of the k rows most cosine-similar to query, most
     similar first."""
6      q_norm = np.linalg.norm(query)
7      if q_norm == 0:
8          raise ValueError("query vector has zero length")
9      norms = np.linalg.norm(matrix, axis=1)
1      sims = np.full(len(matrix), -np.inf)
11     ok = norms > 0
2      sims[ok] = (matrix[ok] @ query) / (norms[ok] * q_norm)
13     return [int(i) for i in np.argsort(-sims, kind="stable")[:k]]
```

1. **Line 10.** Zero-length rows rank last instead of causing a division by zero.
2. **Line 12.** Cosine similarity that does not assume normalised input.

One model, recorded and cached. Compare with cosine similarity, and guard against zero vectors.

Recap card

- **Encode with sentence-transformers** SentenceTransformer(name).encode(texts, normalize_embeddings=True). Not run in this course's build.
- **Cosine similarity for unit vectors** The dot product. Otherwise divide by both lengths.
- **Mixing embedding models** Never. Each model defines its own space.
- **Cache key for a vector** Text ID, text hash, model name and version.

Practice

Exercise 1. Multiple choice (level 1)

You have vectors for 2025 answers from model A, and for 2026 answers from model B. Can you compare them?

- a) Yes, if both have 384 dimensions.
- b) No. Re-encode both years with the same model.
- c) Yes, after normalising both.
- d) Yes, by averaging them.

Why?

- r1) Each model defines its own space. The same position means different things in each.
- r2) Newer models are always better.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
import numpy as np

a = np.array([3.0, 4.0])
b = np.array([1.0, 0.0])
print(float(a @ b), round(float(a @ b / (np.linalg.norm(a) *
    np.linalg.norm(b))), 2))
```

Exercise 3. Fill the gap (level 2)

Normalise each row of a matrix to length 1.

```
norms = np.linalg.norm(matrix, axis=_____(1), keepdims=True)
unit = matrix / _____(2)
```

Exercise 4. Write the function (level 3)

Write `top_k_similar(query, matrix, k) -> list[int]`. Return the row indices of the `k` rows with the highest cosine similarity to `query`, highest first, as Python ints. Rows with zero length rank last. Raise `ValueError` if `query` has zero length. Do not assume the inputs are normalised.

Your code must pass these checks:

- Rank by cosine similarity, not raw dot product: row 0 is long but points the wrong way.
- Zero-length rows rank last, and indices are Python ints.
- A zero-length query must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row indices of the k rows most cosine-similar to query.
def top_k_similar(
    query: np.ndarray,
    matrix: np.ndarray,
    k: int,
) -> list[int]: ...
```

Starter code:

```
import numpy as np


def top_k_similar(query, matrix, k):
    return list(np.argsort(matrix @ query)[:k])
```

Exercise 5. Architecture design (level 4)

Plan embeddings for a quarterly tracker's open-ended question, in five languages.

Options: One multilingual model, tested on a sample from each language, An English-only model for all languages, A different model per language.

1. Model
2. What is stored with each vector
3. Next quarter

Sources

- SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2").encode(texts, batch_size=..., normalize_embeddings=True) returns 384-dimensional vectors. (unverified: sentence-transformers documentation and model card. Not run in this course's build.)

B15-L2 HDBSCAN, clusters with a noise label

Concept

k-means needs you to choose the number of clusters, and it puts every point in one. Open-ended answers rarely fit that: some themes are large, some small, and many answers belong to no theme at all. **HDBSCAN** finds dense groups of points, lets clusters differ in size and shape, and labels points in sparse regions as **noise**, `-1`.

scikit-learn has included it since version 1.3. With scikit-learn 1.8:

```
from sklearn.cluster import HDBSCAN

labels = HDBSCAN(min_cluster_size=15, copy=True).fit_predict(vectors)
```

- `min_cluster_size` (default 5) is the main setting: the smallest group you would call a theme. Raise it for fewer, larger themes.
- `min_samples` controls how conservative clustering is. Higher values label more points as noise.
- Pass `copy=True`. scikit-learn 1.8 warns that the default will change from `False` to `True` in 1.10, and `False` lets the estimator modify your input array.
- The default metric is Euclidean. For normalised embeddings (lesson B15-L1), Euclidean distance ranks pairs in the same order as cosine similarity.

Read the result before you use it:

- **Noise share.** On 600 points in four blobs plus 60 random points, this course's build labelled 12% as noise. For short survey answers a large noise share is normal, and it is information: those answers do not share a theme with enough others.
- **Number and size of clusters.** One giant cluster and many tiny ones usually means the settings or the embedding need work.
- **Samples.** Read 10 answers from each cluster before you name it.

High-dimensional embeddings are often reduced first, for example with UMAP or PCA, to make density easier to find. Record every setting and the random seed of any reduction step, because the clusters depend on them.

Why it matters

A team clustered 8,000 complaints with k-means and k = 12, because the dashboard had space for 12 tiles. Every complaint was forced into a theme. A cluster labelled "billing" contained 900 complaints about billing and 400

unrelated ones that were nearest to it. Its share of complaints was overstated by 40%. HDBSCAN on the same vectors marked most of the 400 as noise.

Worked example: weak code

```
labels = KMeans(n_clusters=12).fit_predict(vectors)
shares = pd.Series(labels).value_counts(normalize=True)
```

1. **Line 1.** k chosen by the dashboard, every answer forced into a cluster, no seed.
2. **Line 2.** Shares inflated by answers that belong nowhere.

Worked example: strong code

```
import numpy as np


def summarise_labels(labels: np.ndarray) -> dict[str, float | int | None]:
    """Cluster count, noise share and largest cluster share (of clustered points)."""
    n = len(labels)
    clustered = labels[labels != -1]
    if n == 0:
        return {"n_clusters": 0, "noise_share": None, "largest_share": None}
    sizes = np.unique(clustered, return_counts=True)[1]
    return {
        "n_clusters": int(len(sizes)),
        "noise_share": float((labels == -1).mean()),
        "largest_share": float(sizes.max() / len(clustered)) if len(clustered) > 0 else None,
    }
```

1. **Line 7.** Noise is reported separately, never counted as a cluster.
2. **Line 14.** Guarded: all noise means no clusters to compare.

Let density decide the clusters, keep noise as its own answer, and summarise counts, noise share and balance before naming themes.

Recap card

- **HDBSCAN noise label** -1: points in sparse regions. Report the share, never treat it as a cluster.
- **HDBSCAN main setting** min_cluster_size (default 5). min_samples makes it more conservative.
- **HDBSCAN in scikit-learn 1.8** sklearn.cluster.HDBSCAN. Pass copy=True: the default changes in 1.10.
- **Read a clustering** Noise share, number and size of clusters, then samples from each.

Practice

Exercise 1. Multiple choice (level 1)

HDBSCAN labels 30% of survey answers `-1`. What does that mean?

- a) The algorithm failed on those answers.
- b) Those answers are in sparse regions and share no dense theme with enough others.
- c) They form a 31st cluster.
- d) They are duplicates.

Why?

- r1) -1 is the noise label for points that no dense cluster claims.
- r2) HDBSCAN always drops 30%.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

Three NumPy tools count labels:

- `labels[labels != -1]` keeps the elements where the condition is true.
- `np.unique(a)` returns the distinct values, sorted.
- `np.bincount(a)` counts each non-negative integer: position `i` holds how often `i` appears.

```
a = np.array([0, 2, 2, 2])
np.unique(a)     # [0 2]
np.bincount(a)   # [1 0 3]: one 0, no 1s, three 2s
```

`(a == 2).mean()` is the share of elements equal to 2, because `True` counts as 1.

```
import numpy as np

labels = np.array([0, 0, 1, -1, 1, 1, -1, 0, 0, 0])
clustered = labels[labels != -1]
print(len(np.unique(clustered)), float((labels == -1).mean()),
    float(np.bincount(clustered).max() / len(clustered)))
```

Exercise 3. Fill the gap (level 2)

Cluster embeddings with HDBSCAN, smallest theme 20 answers, without modifying the input array.

```
labels = HDBSCAN(_____(1)=20, copy=_____(2)).fit_predict(vectors)
```

Exercise 4. Write the function (level 3)

Write `summarise_labels(labels) -> dict[str, float | int | None]` returning:

- `n_clusters`: distinct labels other than -1.
- `noise_share`: share of all points labelled -1, or `None` for an empty array.
- `largest_share`: size of the largest cluster over the number of clustered (non-noise) points, or `None` if there are none.

Your code must pass these checks:

- Exclude -1 from clusters. Noise share is over all points, largest share over clustered points.
- All noise: no clusters, noise share 1.0, largest share None.
- An empty array must not divide by zero.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Cluster count, noise share and largest cluster share.
def summarise_labels(
    labels: np.ndarray,
) -> dict[str, float | int | None]: ...
```

Starter code:

```
import numpy as np


def summarise_labels(labels):
    return {"n_clusters": len(np.unique(labels))}
```

Exercise 5. Architecture design (level 4)

HDBSCAN on 5,000 complaint embeddings gives one cluster with 70% of clustered points, 40 tiny clusters, and 45% noise. Choose the next steps.

Options: Read samples from the big cluster, a few tiny ones, and the noise, Publish the 41 themes.

1. First step
2. Setting to try
3. Record

Sources

- sklearn.cluster.HDBSCAN defaults to min_cluster_size=5 and metric="euclidean"; scikit-learn 1.8 warns that copy will default to True in 1.10. (verified: Signature and warning observed on scikit-learn 1.8.0 and 1.9.1 in this repository.)

- Four blobs of 600 points plus 60 uniform points: HDBSCAN(min_cluster_size=15) found 4 clusters and 12% noise. (verified: Run with scikit-learn 1.8.0 in this repository (make_blobs random_state=0).)

B15-L3 Validate clusters before you name them

Concept

Any clustering algorithm returns clusters, even on random data. Before a cluster becomes a "theme" in a report, check three things.

1. Separation. The **silhouette score** compares each point's distance to its own cluster with its distance to the nearest other cluster. It runs from -1 to 1, higher is better. scikit-learn 1.8: `silhouette_score(X, labels)`. Compute it **without noise points**. The function treats `-1` as a cluster like any other. In this course's build, four blobs with HDBSCAN noise scored 0.70 with noise removed and 0.555 with noise counted as a cluster.

2. Stability. Rerun the whole pipeline on resampled data, or with different seeds, and compare the labels on the points both runs share. The **adjusted Rand index** (`adjusted_rand_score`) is 1 for identical groupings, around 0 for chance, and ignores label names: `[0, 0, 1, 1]` against `[1, 1, 0, 0]` scores 1.0. Themes that appear in only some runs are artefacts of the settings.

3. Meaning. Read 10 to 20 answers per cluster. Have two people name the clusters independently, then compare. A cluster nobody can name consistently is not a theme yet.

Report with each theme:

- its size and share, with noise reported separately,
- the silhouette and stability figures for the clustering as a whole,
- three verbatim example answers, chosen at random, not the most typical ones.

No single number proves clusters are real. Together, separation, stability and human reading make a defensible case. Keep the validation results in the run record with the settings (lesson B6-L2).

Why it matters

An insight team presented seven "customer mindsets" from a clustering of attitude statements. A year later a new team reran the same method on a fresh sample. Three of the seven mindsets reappeared. The other four had

been unstable from the start: a stability check, rerunning on bootstrap samples, would have shown them splitting and merging from run to run. Two product launches had been planned around mindsets that did not exist.

Worked example: weak code

```
score = silhouette_score(vectors, labels)
print("7 clear segments, silhouette", round(score, 2))
```

1. **Line 1.** Noise (-1) counted as a cluster, and one run taken as the truth.
2. **Line 2.** A single number presented as proof.

Worked example: strong code

```
import numpy as np
from sklearn.metrics import adjusted_rand_score, silhouette_score


def silhouette_without_noise(X: np.ndarray, labels: np.ndarray) -> float | None:
    """Silhouette over clustered points only. None if fewer than two clusters remain."""
    keep = labels != -1
    kept = labels[keep]
    n_clusters = len(np.unique(kept))
    if n_clusters < 2 or n_clusters >= len(kept):
        return None
    return float(silhouette_score(X[keep], kept))


def stability(labels_a: np.ndarray, labels_b: np.ndarray) -> float:
    """Agreement of two labelings of the same points, ignoring label names."""
    return float(adjusted_rand_score(labels_a, labels_b))
```

1. **Line 7.** Noise is not a cluster and does not enter the silhouette.
2. **Line 10.** silhouette_score needs between 2 and n - 1 clusters. Return None instead of raising.
3. **Line 17.** Compare runs on shared points. Label names do not matter.

Check separation without noise, stability across reruns, and meaning by reading, before any cluster gets a name.

Recap card

- **Three checks before naming clusters** Separation (silhouette), stability (ARI across reruns), meaning (read and name independently).
- **Silhouette and HDBSCAN noise** Exclude -1 first. silhouette_score treats it as a cluster (0.70 vs 0.555 in this lesson's example).

- **Adjusted Rand index** 1 for identical groupings, about 0 for chance, can be negative, ignores label names.
- **Examples on a theme slide** Random verbatims, not hand-picked ones.

Practice

Exercise 1. Multiple choice (level 1)

A clustering has silhouette 0.62. Is that enough to report its clusters as themes?

- a) Yes. Above 0.5 is proof.
- b) No. Also check stability across reruns and read samples from each cluster.
- c) No. Silhouette must be exactly 1.
- d) Yes, if the number of clusters matches the dashboard.

Why?

- r1) Separation in one run says nothing about whether clusters recur or mean anything.
- r2) Silhouette is never informative.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import adjusted_rand_score

print(adjusted_rand_score([0, 0, 1, 1], [1, 1, 0, 0]),
      adjusted_rand_score([0, 0, 1, 1], [0, 1, 0, 1]))
```

Exercise 3. Fill the gap (level 2)

Compute the silhouette over clustered points only.

```
keep = labels _____(1) -1
score = silhouette_score(X[keep], labels[_____(2)])
```

Exercise 4. Write the function (level 3)

Write `silhouette_without_noise(X, labels) -> float | None`. Drop points labelled -1. If fewer than 2 clusters remain, or as many clusters as remaining points, return `None`. Otherwise return `silhouette_score` on the remaining points as a float.

Your code must pass these checks:

- Compute the silhouette on non-noise points only.

- Fewer than two clusters must return None, not raise.
- As many clusters as points must return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Silhouette over clustered points only.
def silhouette_without_noise(
    X: np.ndarray,
    labels: np.ndarray,
) -> float | None: ...
```

Starter code:

```python
from sklearn.metrics import silhouette_score


def silhouette_without_noise(X, labels):
    return silhouette_score(X, labels)
```

Exercise 5. Architecture design (level 4)

You will present six themes from open-ended answers. Choose what goes on the methods slide.

Options: Silhouette without noise, with the noise share stated, Silhouette with noise counted as a cluster.

1. Separation
2. Stability
3. Examples per theme

Sources

- Silhouette 0.70 excluding noise and 0.555 including it for HDBSCAN on four blobs plus uniform noise; adjusted_rand_score([0,0,1,1],[1,1,0,0]) = 1.0 and ([0,0,1,1],[0,1,0,1]) = -0.5. (verified: Run with scikit-learn 1.8.0 and 1.9.1 in this repository.)

B16 PyTorch hygiene

Roadmap: Months 1 to 3: PyTorch foundations. Device handling, training loops and reproducibility.

B16-L1 Choose the device once, move model and data to it

Concept

PyTorch runs on the CPU, on NVIDIA GPUs (`cuda`) and on Apple silicon (`mps`). Every tensor and every model parameter lives on one **device**, and an operation needs all its inputs on the same device. Examples use PyTorch 2.14.

Choose the device once, at the edge, and pass it in:

```python
def pick_device(prefer_gpu: bool = True) -> torch.device:
    if prefer_gpu and torch.cuda.is_available():
        return torch.device("cuda")
    if prefer_gpu and torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")
```

Then move the model once, and each batch as you use it.

The trap: `.to()` **behaves differently on modules and tensors.**

- `model.to(device)` moves the parameters **in place** and returns the same model.
- `tensor.to(device)` returns a **new** tensor. The original stays where it was. `batch.to(device)` on its own line does nothing useful. Write `batch = batch.to(device)`.

Mixing devices raises `RuntimeError`, with a message such as "Tensor on device cpu is not on the expected device". This course's build reproduced it with PyTorch's `meta` device, because no GPU was available.

Other habits:

- Never hardcode `"cuda"`. The same code must run on a laptop CPU, in CI and on a GPU cluster.
- Move tensors to the CPU before converting to NumPy: `preds.cpu().numpy()`. NumPy cannot read GPU memory.
- Log the device with every run (lesson B14-L1). Results can differ slightly between devices.

PyTorch does not run in this app's browser runtime. The exercises check your code's structure, and the device logic uses plain Python.

Why it matters

A training script written on a GPU workstation had `.cuda()` calls on 14 lines. It had to run in CI on CPU machines for a smoke test. Every run failed at import of the first batch. The fix took an afternoon of search and replace, and one `.cuda()` hidden in a helper was missed and broke the nightly job a week later. One `device` chosen at the edge and passed in would have needed no changes.

Worked example: weak code

```
1 model = Net().cuda()
2 for x, y in loader:
2     x.to("cuda")
4     loss = loss_fn(model(x), y.cuda())
```

1. **Line 1.** Hardcoded GPU: fails on any machine without CUDA.
2. **Line 3.** Returns a new tensor that is thrown away. x stays on the CPU.

Worked example: strong code

```
1  from collections.abc import Iterable
2
3  import torch
4
5
1  def pick_device(prefer_gpu: bool = True) -> torch.device:
7      """CUDA, then Apple MPS, then CPU."""
8      if prefer_gpu and torch.cuda.is_available():
9          return torch.device("cuda")
10     if prefer_gpu and torch.backends.mps.is_available():
11         return torch.device("mps")
12     return torch.device("cpu")
13
14
15 def train_epoch(
16     model: torch.nn.Module,
17     loader: Iterable[tuple[torch.Tensor, torch.Tensor]],
18     loss_fn: torch.nn.Module,
19     optimizer: torch.optim.Optimizer,
20     device: torch.device,
21 ) -> float:
22     """One pass over loader. Returns the mean batch loss."""
23     model.train()
24     total, batches = 0.0, 0
25     for x, y in loader:
2          x, y = x.to(device), y.to(device)
```

```
27          optimizer.zero_grad()
28          loss = loss_fn(model(x), y)
29          loss.backward()
30          optimizer.step()
3           total += loss.item()
32          batches += 1
33      return total / batches if batches > 0 else float("nan")
```

1. **Line 6.** One decision, made once, testable on any machine.
2. **Line 26.** Reassigned: the moved tensors are the ones used.
3. **Line 31.** A Python float for logging, not a tensor holding the graph.

Pick the device once and pass it in. Reassign tensors after .to(). Never hardcode "cuda".

Recap card

- **Choose a device** Once, at the edge: cuda if available, then mps, then cpu. Pass it in.
- **Tensor.to vs Module.to** Tensor.to returns a new tensor (reassign). Module.to moves in place.
- **GPU tensor to NumPy** t.detach().cpu().numpy().
- **Hardcoded .cuda()** Breaks on CPU machines and CI. Use .to(device).

Practice

Exercise 1. Multiple choice (level 1)

After `x.to(device)` on its own line, where is `x`?

- a) On device.
- b) Where it was before. .to() on a tensor returns a new tensor.
- c) Copied to both devices.
- d) Deleted.

Why?

- r1) Tensor.to returns a new tensor. Only nn.Module.to changes the object in place.
- r2) .to() only works on models.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14; the meta device stands in for a GPU)

What you need to know.

`torch.nn.Linear(n_in, n_out)` is a layer with a weight matrix and a bias, both **parameters** the model learns. `model.parameters()` yields them, so

`next(model.parameters())` is the first one. Every tensor has a `.device` saying where it lives, such as `cpu` or `cuda:0`.

```
layer = torch.nn.Linear(3, 2)
next(layer.parameters()).shape    # torch.Size([2, 3])
next(layer.parameters()).device   # device(type='cpu')
```

`torch.ones(2)` is a tensor of two ones.

```
import torch

t = torch.ones(2)
t.to("meta")
model = torch.nn.Linear(2, 1)
model.to("meta")
print(t.device, next(model.parameters()).device)
```

Exercise 3. Fill the gap (level 2)

Move a batch to the chosen device, and bring predictions back for NumPy.

```
x = x._____(1)(device)
preds = model(x).detach()._____(2)().numpy()
```

Exercise 4. Write the function (level 3)

Write `pick_device_name(prefer_gpu: bool, cuda_ok: bool, mps_ok: bool) -> str` that returns `"cuda"`, `"mps"` or `"cpu"`: CUDA first, then MPS, then CPU, and always `"cpu"` when `prefer_gpu` is False. It takes availability as arguments so it runs and tests anywhere. The caller passes `torch.cuda.is_available()` and `torch.backends.mps.is_available()`.

Your code must pass these checks:

- Prefer CUDA, then MPS, then CPU.
- prefer_gpu=False must always give cpu.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Name of the device to use: cuda, then mps, then cpu.
def pick_device_name(
    prefer_gpu: bool,
    cuda_ok: bool,
    mps_ok: bool,
) -> str: ...
```

Starter code:

```
def pick_device_name(prefer_gpu, cuda_ok, mps_ok):
    return "cuda"
```

Exercise 5. Spot the bug (level 4)

Validation runs without errors on the CPU but raises "Expected all tensors to be on the same device" on the GPU. Tap the line that causes it.

What you need to know.

`with torch.inference_mode():` runs the block without tracking gradients. Evaluation needs no gradients, so this saves memory and time. `model.eval()` switches layers such as dropout to evaluation behaviour.

`tensor.argmax(dim=1)` returns, for each row, the position of the largest value: the predicted class.

```
torch.tensor([[0.1, 0.9], [0.8, 0.2]]).argmax(dim=1)   # tensor([1, 0])
```

```
from collections.abc import Iterable

import torch


def validate(model: torch.nn.Module, loader: Iterable[tuple[torch.Tensor, torch.Tensor]], device: torch.device) -> float:
    """Accuracy on the validation loader."""
    model.eval()
    correct, total = 0, 0
    with torch.inference_mode():
        for x, y in loader:
            x.to(device)
            preds = model(x).argmax(dim=1)
            correct += int((preds.cpu() == y).sum())
            total += len(y)
    return correct / total if total > 0 else float("nan")
```

Write down the line number of the bug. Then write the fix.

Sources

- In PyTorch 2.14, Tensor.to returns a new tensor and leaves the original unchanged; Module.to moves parameters in place and returns the same module; mixing devices raises RuntimeError. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository, using the meta device.)

B16-L2 A training loop that does what you think

Concept

PyTorch leaves the training loop to you. That gives control, and five ways to get it silently wrong. The standard loop, PyTorch 2.14:

```
for epoch in range(max_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
    val_loss = evaluate(model, val_loader, loss_fn, device)   # model.eval() + torch.inference_mode()
```

The five traps:

1. **Missing `optimizer.zero_grad()`.** Gradients **add up** across `backward()` calls. In this course's build, three backward passes on `2 * w` left a gradient of 6.0 instead of 2.0. Training still runs, with steps that grow each batch.
2. **Missing `model.eval()` for validation.** Dropout stays active and batch norm keeps updating its statistics. Two predictions for the same input differed in train mode and matched in eval mode.
3. **Missing `torch.inference_mode()`** (or `torch.no_grad()`) during validation. PyTorch builds a graph it never uses, wasting memory.
4. **Softmax before `CrossEntropyLoss`.** The loss applies log-softmax itself and expects raw **logits**. For logits `[2.0, 0.5]` and class 0, the correct loss is 0.2014. Passing softmax outputs gave 0.4252, a different loss with weaker gradients.
5. **Summing loss tensors for logging.** `total += loss` keeps every batch's graph alive, and memory grows each step. Use `total += loss.item()`.

Early stopping. Track validation loss each epoch. Stop when it has not improved by at least `min_delta` for `patience` epochs, and keep a copy of the best weights (`copy.deepcopy(model.state_dict())`), not the last ones.

Log the loss curve for both training and validation (lesson B14-L1). A validation loss that rises while training loss falls is overfitting. Both flat from the start usually means a bug, such as frozen parameters or a learning rate of zero.

Why it matters

A text classifier trained for 30 epochs showed a smooth, falling training loss and a validation accuracy stuck at 51%. The code applied `softmax` in the model's `forward` and then used `CrossEntropyLoss`. Gradients were small and the model barely learned. Removing one line lifted validation accuracy to

78% in six epochs. The previous two weeks of hyperparameter tuning had been on a broken loss.

Worked example: weak code

```
for x, y in loader:
    out = torch.softmax(model(x), dim=1)
    loss = nn.CrossEntropyLoss()(out, y)
    loss.backward()
    optimizer.step()
    total += loss
```

1. **Line 2.** CrossEntropyLoss expects logits. Softmax here changes the loss and flattens gradients.
2. **Line 4.** No zero_grad before it: gradients accumulate across batches.
3. **Line 6.** Keeps each batch's graph alive. Use loss.item().

Worked example: strong code

```
def should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool:
    """True when the last `patience` epochs did not beat the earlier best by min_delta."""
    if patience < 1:
        raise ValueError("patience must be at least 1")
    if len(val_losses) <= patience:
        return False
    best_before = min(val_losses[:-patience])
    return min(val_losses[-patience:]) > best_before - min_delta
```

1. **Line 5.** Too few epochs to judge: keep training.
2. **Line 8.** Stop only when recent epochs failed to improve on the earlier best.

zero_grad, train and eval modes, inference_mode for validation, logits into CrossEntropyLoss, loss.item() for logs, and early stopping on validation loss.

Recap card

- **The training iteration** to(device), zero_grad, forward and loss, backward, step.
- **Validation mode** model.eval() and torch.inference_mode(). Back to model.train() after.
- **CrossEntropyLoss input** Raw logits. It applies log-softmax itself.
- **Early stopping** Stop when validation loss has not improved by min_delta for patience epochs. Keep the best weights.

Practice

Exercise 1. Multiple choice (level 1)

A loop calls `loss.backward()` and `optimizer.step()` but never `optimizer.zero_grad()`. What happens?

- a) Nothing. PyTorch clears gradients automatically.
- b) Gradients accumulate across batches, so steps use the sum of all earlier gradients.
- c) backward() raises an error on the second batch.
- d) The model stops learning.

Why?

- r1) backward() adds into .grad. Only zero_grad() resets it.
- r2) Optimizers ignore gradients without zero_grad.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14)

```
import torch

w = torch.tensor([1.0], requires_grad=True)
for _ in range(3):
    loss = (w * 2).sum()
    loss.backward()
print(w.grad.item())
```

Exercise 3. Put in order (level 2)

Order the steps inside one training iteration.

- optimizer.step()
- loss.backward()
- loss = loss_fn(model(x), y)
- x, y = x.to(device), y.to(device)
- optimizer.zero_grad()

Exercise 4. Write the function (level 3)

Write `should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool`. Return `True` when the best loss in the last `patience` epochs is not lower than the best loss before them by more than `min_delta`. Return `False` when there are `patience` or fewer losses. Raise `ValueError` if `patience < 1`.

Your code must pass these checks:

- No improvement on 0.9 in the last 2 epochs: stop.
- An improvement in the window, or too few epochs, means continue.
- Improvements smaller than min_delta do not count.
- patience below 1 must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Early-stopping rule on validation losses, one per epoch.
def should_stop(
    val_losses: list[float],
    patience: int,
    min_delta: float = 0.0,
) -> bool: ...
```

Starter code:

```python
def should_stop(val_losses, patience, min_delta=0.0):
    return val_losses[-1] > val_losses[-2]
```

Exercise 5. Spot the bug (level 4)

Validation accuracy is stuck near chance while training loss falls slowly. Tap the line that breaks the loss.

What you need to know.

`nn.Sequential(a, b, c)` chains layers: the output of one is the input of the next. `nn.Linear(n_in, n_out)` is a fully connected layer, and `nn.ReLU()` sets negative values to 0. The class stores the chain as `self.net`, and `forward` decides what the model returns.

```python
net = nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))
net(torch.ones(1, 3)).shape   # torch.Size([1, 2]): one score per class
```

```python
import torch
from torch import nn


class Classifier(nn.Module):
    """Two-layer classifier. Train with nn.CrossEntropyLoss."""

    def __init__(self, n_in: int, n_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, 32), nn.ReLU(),
    nn.Linear(32, n_classes))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.softmax(self.net(x), dim=1)
```

Write down the line number of the bug. Then write the fix.

Sources

- Three backward passes on 2*w leave w.grad = 6.0; dropout outputs differ in train mode and match in eval mode; CrossEntropyLoss on logits [2.0, 0.5] with target 0 is 0.2014, and 0.4252 on their softmax. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository.)

B16-L3 Reproducible PyTorch runs and safe checkpoints

Concept

Neural network training draws random numbers everywhere: weight initialisation, data shuffling, dropout, augmentation. Control every source and two runs with the same seed give the same weights. In this course's build (PyTorch 2.14, CPU), a small network trained twice with seed 1 produced identical parameters. Seed 2 produced different ones.

Seed at the entry point, once. Your own code takes generators as parameters (lesson B6-L1). Libraries and PyTorch internals use global generators, so seed those too, at the start of the training script:

```
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)                          # CPU and all CUDA devices
loader_gen = torch.Generator().manual_seed(seed)
train_loader = DataLoader(ds, batch_size=64, shuffle=True,
    generator=loader_gen)
```

- Pass a seeded `generator` to every shuffling `DataLoader`, so batch order is fixed.
- With `num_workers > 0`, the PyTorch documentation recommends a `worker_init_fn` that seeds NumPy and `random` in each worker from `torch.initial_seed()`.
- `torch.use_deterministic_algorithms(True)` makes PyTorch raise an error for operations that have no deterministic implementation, instead of silently varying. On CUDA it also needs the `CUBLAS_WORKSPACE_CONFIG` environment variable, as the documentation describes.

Exact equality holds for the same code, library versions and hardware. Across GPU models or library versions, expect small differences, and compare metrics within a tolerance (lesson B14-L3).

Checkpoints let a long run resume and let you keep the best epoch. Save a dict of state, not the model object:

```
torch.save({"model": model.state_dict(), "optimizer":
    optimizer.state_dict(),
            "epoch": epoch, "torch_rng": torch.get_rng_state()}, path)
```

`torch.load` loads **weights only** by default in PyTorch 2.14. A checkpoint holding a pickled model object fails with `UnpicklingError`. That default protects you: loading an arbitrary pickle can execute code. Save state dicts, rebuild the model class in code, and load the weights into it.

Why it matters

A team fine-tuned a classifier and reported 0.86 F1. A reviewer reran the notebook and got 0.83, then 0.85. The DataLoader shuffled with the global generator, dropout drew from it too, and an augmentation library used its own. Nobody could say which number was "the" result. Fixing the seeds and the loader generator made the run repeatable: 0.84, every time. The earlier 0.86 had been a lucky draw.

Worked example: weak code

```
model = torch.load("best_model.pt")
loader = DataLoader(ds, batch_size=64, shuffle=True)
```

1. **Line 1.** Expects a pickled model object: refused by the weights-only default, and unsafe if forced.
2. **Line 2.** Shuffles with the global generator: batch order changes with any other random call.

Worked example: strong code

```
import random
from pathlib import Path

import numpy as np
import torch


def seed_everything(seed: int) -> torch.Generator:

    """Seed Python, NumPy and PyTorch global state; return a generator for
    DataLoaders."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    return torch.Generator().manual_seed(seed)


def save_checkpoint(path: Path, model: torch.nn.Module, optimizer:
    torch.optim.Optimizer, epoch: int) -> None:
```

```
    """Save state dicts and RNG state, never the model object."""
    torch.save(
        {
            "model": model.state_dict(),
            "optimizer": optimizer.state_dict(),
            "epoch": epoch,
            "torch_rng": torch.get_rng_state(),
        },
        path,
    )
```

1. **Line 13.** A dedicated generator for DataLoader shuffling.
2. **Line 20.** Weights as tensors: loads under the weights-only default.
3. **Line 23.** Resuming restores the random stream as well as the weights.

Seed every generator at the entry point, give DataLoaders their own generator, and checkpoint state dicts, not objects.

Recap card

- **Seed a PyTorch run** random.seed, np.random.seed, torch.manual_seed at the entry point, plus a seeded Generator for each DataLoader.
- **torch.load default in PyTorch 2.14** Weights only. Pickled objects fail with UnpicklingError.
- **Checkpoint contents** Model and optimizer state dicts, epoch, RNG state, best metric.
- **Deterministic operations** torch.use_deterministic_algorithms(True) raises on operations with no deterministic version.

Practice

Exercise 1. Multiple choice (level 1)

`torch.load("model.pt")` raises `UnpicklingError` in PyTorch 2.14. The file was saved with `torch.save(model, path)`. Best fix?

- a) Always pass weights_only=False.
- b) Save model.state_dict() instead, rebuild the model class in code, and load the weights into it.
- c) Downgrade PyTorch.
- d) Rename the file to .pkl.

Why?

- r1) State dicts hold only tensors and plain values, which load safely under the weights-only default.
- r2) Model objects cannot be saved.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.14)

What you need to know.

`torch.randperm(n, generator=g)` returns the numbers 0 to n - 1 in random order, drawn from generator `g`. Without `generator=`, random functions such as `torch.rand(100)` draw from PyTorch's global generator.

```
g = torch.Generator().manual_seed(1)
torch.randperm(4, generator=g).tolist()   # [1, 3, 2, 0]
```

```
import torch

g1 = torch.Generator().manual_seed(0)
g2 = torch.Generator().manual_seed(0)
a = torch.randperm(5, generator=g1).tolist()
torch.rand(100)
b = torch.randperm(5, generator=g2).tolist()
print(a == b)
```

Exercise 3. Fill the gap (level 2)

Give a shuffling DataLoader its own seeded generator.

```
loader_gen = torch.Generator()._____(1)(seed)
loader = DataLoader(ds, batch_size=64, shuffle=True, _____(2)=loader_gen)
```

Exercise 4. Write the function (level 3)

Write `seed_everything(seed: int) -> torch.Generator`. Seed Python's `random`, NumPy's global generator and PyTorch with `torch.manual_seed`, then return `torch.Generator().manual_seed(seed)` for DataLoaders.

PyTorch does not run in the browser. The checker reads your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Seed Python, NumPy and PyTorch global state at the entry point.
def seed_everything(seed: int) -> torch.Generator: ...
```

Starter code:

```
import torch


def seed_everything(seed):
    torch.manual_seed(seed)
```

Exercise 5. Architecture design (level 4)

Design checkpointing for a 12-hour training job that must resume after interruptions and keep the best epoch.

Options: Model and optimizer state dicts, epoch, RNG state, best validation loss, The whole model object with torch.save(model).

1. Checkpoint content
2. When to save
3. Loading

Sources

- Same-seed training on CPU gave identical parameters; torch.load of a pickled model raised UnpicklingError; a dict of state dicts, epoch and torch.get_rng_state() loaded with the default. (verified: Run with torch 2.14.0 (CPU) in a scratch environment for this repository.)
- PyTorch recommends a worker_init_fn seeding NumPy and random from torch.initial_seed(), and CUBLAS_WORKSPACE_CONFIG for deterministic CUDA. (search-only: PyTorch documentation, Reproducibility notes (not reachable from this environment).)

B17 Testing and performance

Roadmap: Prerequisite for all months. Unit tests, Hypothesis, profiling and memory use.

B17-L1 Unit tests for data code that catch real bugs

Concept

A test is worth writing if it fails when the code is wrong. For data code, most bugs sit at the edges: empty input, missing values, zero denominators, one row, duplicate keys. Test those on purpose.

With pytest 9, a test is a function whose name starts with `test_` and which uses plain `assert`:

```
import pytest


def test_share_zero_total_is_none():
    assert share(3, 0) is None


@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
   (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == expected
```

Habits that make tests catch bugs:

- **Test pure functions** (lesson B2-L1). No files, clocks or databases, so tests are fast and exact.
- **Build tiny, explicit inputs** in the test itself: three rows, written out. A reader sees the input and the expected answer together.
- **One behaviour per test**, named for it: `test_zero_total_returns_none`, not `test_share_2`.
- **Compare DataFrames with** `pd.testing.assert_frame_equal`. It checks values, dtypes, column order and index, and prints a clear difference.
- **Compare floats with** `pytest.approx` when the value comes from arithmetic: `assert mean == pytest.approx(0.3)`.
- **Use** `tmp_path` for anything that writes files. pytest gives each test a fresh temporary directory.

Check that each test can fail. Break the code on purpose, once, and watch the test go red. A test that stays green whatever you do protects nothing.

This course's own quality gate does the same: every seeded bug must fail its checker.

Run the tests in CI on every push (lesson A10-L3), and fix a failing test before merging anything else.

Why it matters

A share function had a test, `assert share(1, 4) == 0.25`, and 100% line coverage. The zero-total branch returned `0.0` instead of `None`. Dashboards showed empty segments as "0% of respondents", which the client read as a real finding. A four-word test, `assert share(3, 0) is None`, would have failed on the first run.

Worked example: weak code

```
1 def test_share():
2     result = share(1, 4)
1     assert result
```

1. **Line 3.** Passes for any truthy value: 0.25, 99 or 'oops'. And 0.0 would fail a correct result.

Worked example: strong code

```
1  import pytest
2
3
4  def share(part: float, total: float) -> float | None:
5      """part / total, or None when total is 0."""
6      if total == 0:
7          return None
8      return part / total
9
10
1  @pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
       (0, 5, 0.0), (5, 5, 1.0)])
12 def test_share_values(part: float, total: float, expected: float) -> None:
13     assert share(part, total) == pytest.approx(expected)
14
15
16 def test_share_zero_total_is_none() -> None:
2      assert share(3, 0) is None
```

1. **Line 11.** Several cases, including 0 and 1, in one readable table.
2. **Line 17.** The edge case that most often ships broken, tested explicitly.

Test edges on purpose, with tiny explicit inputs, one behaviour per test, and prove each test can fail.

Recap card

- **A test worth writing** One that fails when the code is wrong. Break the code once to prove it.
- **Edge cases for data code** Empty input, missing values, zero denominators, one row, duplicate keys.
- **Compare DataFrames in tests** pd.testing.assert_frame_equal: values, dtypes, columns and index.
- **Compare floats in tests** pytest.approx(expected).

Practice

Exercise 1. Multiple choice (level 1)

Which assertion best tests that `share(3, 0)` returns `None`?

- a) `assert not share(3, 0)`
- b) `assert share(3, 0) is None`
- c) `assert share(3, 0) == 0`
- d) `assert share(3, 0) != 1`

Why?

- r1) It fails for every result except None, including 0.0.
- r2) `is` is faster than `==`.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
import pandas as pd

a = pd.DataFrame({"n": [1, 2]})
b = pd.DataFrame({"n": [1.0, 2.0]})
print(a.equals(b), (a == b).all().all())
try:
    pd.testing.assert_frame_equal(a, b)
    print("equal")
except AssertionError:
    print("AssertionError")
```

Exercise 3. Fill the gap (level 2)

Run one test over three cases.

```
@pytest.mark._____(1)(("part", "total", "expected"), [(1, 4, 0.25), (0,
    5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == pytest._____(2)(expected)
```

Exercise 4. Write the function (level 3)

`share(part, total)` should return `part / total`, or `None` when `total` is 0. Write two tests:

- `test_share_basic()`: `share(1, 4)` equals 0.25.
- `test_share_zero_total()`: `share(3, 0)` is `None`.

The checker runs your tests against a correct `share`, which must pass, and against two broken versions, which your tests must catch.

Your code must pass these checks:

- Your tests must pass for a correct share function.
- test_share_zero_total must fail when share returns 0.0 for a zero total. Use `is None`.
- test_share_basic must check the value 0.25, not just that a result exists.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# part / total, or None when total is 0.
def share(part: float, total: float) -> float | None: ...

def test_share_basic() -> None: ...

def test_share_zero_total() -> None: ...
```

Starter code:

```python
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    if total == 0:
        return None
    return part / total


def test_share_basic():
    assert share(1, 4)


def test_share_zero_total():
    assert not share(3, 0)
```

Exercise 5. Architecture design (level 4)

Choose test cases for `weighted_share(df, value, weight)` from lesson B8-L1.

Options: Four rows written out, expected value computed by hand, A 10,000-row sample of production data, expected value from the function itself.

1. Normal case
2. Edge cases

3. Float comparison

Sources

- DataFrame.equals returns False and assert_frame_equal raises for int64 vs float64 columns with equal values, while == reports True. (verified: Run on pandas 3.0.6 in this repository.)

B17-L2 Property-based tests with Hypothesis

Concept

Example tests check the cases you thought of. **Property-based tests** check a rule against hundreds of generated inputs, including ones you did not think of. **Hypothesis** generates the inputs, and when a test fails it **shrinks** the input to the smallest failing case. Examples use Hypothesis 6.168.

```
from hypothesis import given, strategies as st

@given(st.lists(st.tuples(st.floats(-1e6, 1e6), st.floats(0, 10)),
    min_size=1))
def test_weighted_mean_within_range(pairs):
    values = [v for v, _ in pairs]
    weights = [w for _, w in pairs]
    m = weighted_mean(values, weights)
    assert m is None or min(values) - 1e-6 <= m <= max(values) + 1e-6
```

Run against a `weighted_mean` without a zero guard, this test failed in this course's build with `ZeroDivisionError`, and Hypothesis shrank the input to `pairs=[(0.0, 0.0)]`: one value, one zero weight.

Properties that suit data code:

- **Bounds.** A mean lies between the minimum and maximum. A share lies in [0, 1].
- **Invariance.** Shuffling rows does not change a total. Renaming a category does not change counts.
- **Round trip.** Writing then reading a file gives back the same frame.
- **Idempotence.** Deduplicating twice equals deduplicating once.
- **Grain.** The output has exactly one row per key.
- **Agreement.** A fast implementation equals a slow, obviously correct one.

Choose strategies deliberately. `st.floats()` generates `nan` and infinities by default. Bound it (`st.floats(-1e6, 1e6)`) or pass `allow_nan=False` unless your function must handle them, and then test that behaviour on purpose. `hypothesis.extra.pandas` builds DataFrames with `data_frames` and `column`.

Use `@settings(max_examples=...)` to trade speed for coverage. Hypothesis saves failing examples in a local `.hypothesis/` directory and replays them first on the next run. Add it to `.gitignore`.

Hypothesis does not run in this app's browser runtime. The exercises check your code's structure.

Why it matters

A deduplication step kept the latest record per respondent. Its example tests used three tidy rows. In production, two records for one respondent had the **same** timestamp, and the function kept both, so the respondent was counted twice. A Hypothesis test of the grain, "the output has exactly one row per respondent", fails on this bug at once. In this course's build it shrank the failure to `rows=[('a', 0), ('a', 0)]`: two identical rows.

Worked example: weak code

```
1 @given(st.floats(), st.floats())
2 def test_share_in_range(part, total):
2     assert 0 <= share(part, total) <= 1
```

1. **Line 1.** Unbounded floats include nan, infinities and negatives the function never promised to handle.
2. **Line 3.** Ignores the None branch, so the property is false for a correct function.

Worked example: strong code

```
 1 from hypothesis import given, strategies as st
 2
 1 counts = st.integers(min_value=0, max_value=10_000)
 4
 5
 6 @given(counts, counts)
 7 def test_share_in_unit_interval(a: int, b: int) -> None:
 2     part, total = min(a, b), max(a, b)
 9     result = share(part, total)
10     if total == 0:
 3         assert result is None
12     else:
13         assert result is not None and 0.0 <= result <= 1.0
```

1. **Line 3.** Inputs the function promises to handle: non-negative counts.
2. **Line 8.** Builds valid inputs (part <= total) instead of filtering most examples away.
3. **Line 11.** The zero branch is part of the property.

State a rule that must always hold, generate inputs from the domain the function promises, and let Hypothesis find and shrink the counterexample.

Recap card

- **Property-based test** A rule checked against many generated inputs, with failing inputs shrunk to a minimal case.
- **Four data-code properties** Bounds, invariance, round trip, idempotence, grain, and agreement with a slow version.
- **st.floats() default** Includes nan and infinities. Bound it or pass allow_nan=False on purpose.
- **Hypothesis example database** .hypothesis/ replays past failures. Keep it out of Git.

Practice

Exercise 1. Multiple choice (level 1)

A Hypothesis test fails. What does Hypothesis report?

- a) The first random input that failed.
- b) A shrunk input: the simplest example it found that still fails.
- c) Every failing input it tried.
- d) Only the exception, without the input.

Why?

- r1) Shrinking turns a messy random failure into a minimal case you can read.
- r2) Hypothesis only ever generates simple inputs.

Exercise 2. Predict the output (level 2)

This property test runs with Hypothesis 6.168. Does it pass or fail?

```
from hypothesis import given, strategies as st


@given(st.floats())
def test_equal_to_itself(x):
    assert x == x
```

Exercise 3. Fill the gap (level 2)

Generate bounded floats without nan, and run 500 examples.

```
@_____(1)(max_examples=500)
@given(st.floats(min_value=-1e6, max_value=1e6, _____(2)=False))
def test_round_trip(x):
    assert parse(format(x)) == x
```

Exercise 4. Write the function (level 3)

Write a Hypothesis test `test_dedupe_idempotent` for `dedupe(rows: list[tuple[str, int]]) -> list[tuple[str, int]]`. Generate `rows` as lists of `(id, timestamp)` tuples with `st.lists(st.tuples(st.sampled_from(["a", "b", "c"]), st.integers(0, 5)))`. Assert that `dedupe(dedupe(rows)) == dedupe(rows)`.

Hypothesis does not run in the browser. The checker reads your code. Run it locally with pytest.

What you need to know.

Hypothesis builds test inputs from **strategies**:

- `st.sampled_from(["x", "y"])` picks one value from the list.
- `st.integers(0, 5)` picks an integer from 0 to 5.
- `st.tuples(a, b)` builds a tuple from two strategies, and `st.lists(s)` a list of any length.

`@given(strategy)` above a test function runs it many times. Each run passes a new generated value as the function's argument:

```
@given(st.lists(st.integers(0, 9)))
def test_sort_keeps_length(values: list[int]) -> None:
    assert len(sorted(values)) == len(values)
```

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
def test_dedupe_idempotent(
    rows: list[tuple[str, int]],
) -> None: ...
```

Starter code:

```
from hypothesis import given, strategies as st


def test_dedupe_idempotent():
    rows = [("a", 1), ("a", 1)]
    assert dedupe(rows) == [("a", 1)]
```

Exercise 5. Architecture design (level 4)

Pick a property for each function.

Options: Shuffling rows does not change the result, Write then read returns the same frame, Result lies between the minimum and maximum input, Equals a slow, obviously correct version.

1. weighted_total(df): sum of value × weight

2. save_parquet and load_parquet
3. weighted_mean(values, weights) with positive weights
4. A vectorised rewrite of a row-by-row scoring loop

Sources

- A grain property found a dedupe that keeps timestamp ties and shrank it to rows=[('a', 0), ('a', 0)]. (verified: Run with Hypothesis 6.168.1 and pytest in this repository.)
- Hypothesis 6.168 found a ZeroDivisionError in an unguarded weighted mean and shrank it to pairs=[(0.0, 0.0)]; st.floats() generated nan and failed x == x. (verified: Run with Hypothesis 6.168.1 and pytest in this repository.)

B17-L3 Measure before you optimise

Concept

Most slow data code spends its time in one or two places, and they are rarely where you guess. Measure first, change one thing, measure again. All tools here are in the Python standard library.

Profile to find where time goes. `cProfile` counts calls and time per function:

```
import cProfile, pstats
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Sort by `cumulative` to see which of your functions, including everything they call, takes longest. In this course's build, a job calling a pure-Python sum of squares and a NumPy version over 200,000 numbers showed 0.037 s cumulative for the Python one and 0.002 s for NumPy. In a notebook, `%prun main()` does the same.

Time a candidate change with `timeit`. `timeit.repeat(fn, repeat=5, number=100)` returns five totals, each for 100 calls. Report the **minimum** divided by `number`: other processes can only slow a run down, so the fastest run is the best estimate of the code's own cost. Use `time.perf_counter()`, not `time.time()`, for manual timing.

Measure memory when that is the limit. `tracemalloc` reports current and peak allocation: one million float64 values showed a peak of 8.0 MB, as

expected at 8 bytes each. For pandas, `df.memory_usage(deep=True)` counts the contents of text columns, not just the pointers.

Rules:

- Profile with realistic data sizes. A 100-row sample hides what dominates at 10 million rows.
- Keep setup out of the timed region: reading files, building inputs, copying frames.
- Fix the biggest cost first. Speeding up a function that takes 2% of the runtime cannot save more than 2%.
- Keep a test around the code you optimise (lesson B17-L1), so faster does not become wrong.

Why it matters

An analyst spent two days rewriting a weighting function in Numba because "loops are slow in Python". The job still took 40 minutes. A profile, run afterwards, showed 36 of those minutes in reading 300 CSV files one at a time with default type inference. Switching to one Parquet read with explicit dtypes cut the job to 5 minutes. The Numba rewrite saved 20 seconds.

Worked example: weak code

```
start = time.time()
for _ in range(10):
    df = pd.read_csv("big.csv")
    result = transform(df)
print("transform takes", (time.time() - start) / 10)
```

1. **Line 1.** Wall-clock time with low resolution. Use time.perf_counter().
2. **Line 3.** The file read sits inside the timed loop, so the number is mostly I/O.
3. **Line 5.** An average over runs, inflated by any background activity.

Worked example: strong code

```
import time
from collections.abc import Callable


def best_time(fn: Callable[[], object], repeat: int = 5, number: int =
    10) -> float:
    """Best per-call time in seconds over `repeat` rounds of `number`
    calls."""
    if repeat < 1 or number < 1:
        raise ValueError("repeat and number must be at least 1")
    best = float("inf")
    for _ in range(repeat):
```

```
(1)          start = time.perf_counter()
12           for _ in range(number):
13               fn()
(2)          best = min(best, (time.perf_counter() - start) / number)
15       return best
```

1. **Line 11.** A high-resolution clock meant for measuring intervals.
2. **Line 14.** Keep the fastest round: noise only ever adds time.

Profile to find the cost, time the change with perf_counter or timeit, keep setup out of the timing, and fix the biggest cost first.

Recap card

- **Find where time goes** cProfile with pstats sorted by cumulative, or %prun in a notebook.
- **Time a small change** timeit.repeat or perf_counter. Report the minimum per call.
- **Measure memory** tracemalloc peak, and df.memory_usage(deep=True) for pandas.
- **The order of work** Profile, fix the biggest cost, re-measure, keep a test around the change.

Practice

Exercise 1. Multiple choice (level 1)

A job takes 40 minutes. A profile shows 36 minutes in file reading and 1 minute in the weighting function. What should you optimise first?

- a) The weighting function, because loops are slow.
- b) File reading.
- c) Both equally.
- d) Neither: buy a faster machine.

Why?

- r1) A 1-minute function can save at most 1 minute. Reading holds 90% of the time.
- r2) I/O is always the bottleneck.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import timeit
2
3 rounds = timeit.repeat(lambda: sum(range(1000)), repeat=5, number=100)
```

```
print(len(rounds))
```

Exercise 3. Fill the gap (level 2)

Profile `main()` and print the 10 functions with the highest cumulative time.

```
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("_____(1)")._____(2)(10)
```

Exercise 4. Write the function (level 3)

Write `best_time(fn, repeat=5, number=10) -> float`. Run `repeat` rounds. In each round, call `fn()` `number` times, timed with `time.perf_counter()`, and compute the per-call time. Return the smallest per-call time. Raise `ValueError` if `repeat` or `number` is below 1.

Your code must pass these checks:

- Call fn number times in each of repeat rounds (3 × 4 = 12 calls), and return a float.
- Return the smallest round time divided by number: rounds of 2, 1 and 3 seconds over 2 calls give 0.5.
- repeat or number below 1 must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Best per-call time over several rounds.
def best_time(
    fn: Callable[[], object],
    repeat: int = 5,
    number: int = 10,
) -> float: ...
```

Starter code:

```
import time


def best_time(fn, repeat=5, number=10):
    start = time.time()
    fn()
    return time.time() - start
```

Exercise 5. Spot the bug (level 4)

This benchmark says `add_features` takes 1.8 seconds. The function itself takes about 0.1 seconds. Tap the line that distorts the measurement.

```
import time

import pandas as pd

```

```

def benchmark(path: str, rounds: int) -> float:
    """Best time of add_features over rounds."""
    best = float("inf")
    for _ in range(rounds):
        start = time.perf_counter()
        df = pd.read_parquet(path)
        add_features(df)
        best = min(best, time.perf_counter() - start)
    return best
```

Write down the line number of the bug. Then write the fix.

Sources

- cProfile showed 0.037 s cumulative for a pure-Python sum of squares over 200,000 numbers and 0.002 s for NumPy; tracemalloc peak for one million float64 values was 8.0 MB. (verified: Run on Python 3.11 with NumPy 2.4.6 in this repository.)

B19 Testing in depth

Roadmap: Prerequisite for all months. pytest from the inside: fixtures, parametrize, testing pandas, statistics and ML, mocks, coverage and CI.

B19-L1 How pytest finds, runs and reports your tests

Concept

pytest only runs what it **collects**. With default settings in pytest 9, it looks for:

- files named `test_*.py` or `*_test.py`,
- functions in those files whose names start with `test`,
- methods starting with `test` inside classes whose names start with `Test`, as long as the class has no `__init__`.

A function called `check_weights` in `validation.py` never runs. pytest does not warn you about it.

Use plain `assert`. pytest rewrites assert statements, so a failure shows the values on both sides and the call that produced them:

```
E       assert 0.3333333333333333 == 0.33
E        +  where 0.3333333333333333 = share(1, 3)
```

Test errors on purpose with `pytest.raises`. Pass `match=` a regular expression. pytest searches the error message for it, so a `ValueError` raised for a different reason fails the test:

```
with pytest.raises(ValueError, match="negative weight"):
    normalise([2.0, -1.0])
```

If nothing is raised, the test fails with "DID NOT RAISE". An exception of another type passes straight through and fails the test too.

Run less while you work:

- `pytest tests/test_share.py::test_zero_total` runs one test by its **node ID**.
- `pytest -k "zero and not slow"` runs tests whose names match the expression.
- `pytest -x` stops at the first failure. `pytest --lf` reruns only the tests that failed last time.

The exit code tells CI what happened: 0 when everything passed, 1 when a test failed, and 5 when pytest collected no tests at all. Set where tests live once, in `pyproject.toml`: `testpaths = ["tests"]` under `[tool.pytest.ini_options]`, or under `[tool.pytest]`, which pytest 9 also reads.

Why it matters

A team kept 30 data checks in `validation_checks.py`, as functions named `validate_*`. The rest of the suite lived in properly named files, so every CI run was green. pytest never collected the 30 checks. A broken weight cap shipped in four monthly reports before anyone noticed. Renaming the file and the functions turned up two failures on the first run.

Worked example: weak code

```
# validation_checks.py
def check_negative_weights():
    try:
        normalise([2.0, -1.0])
        print("no error!")
    except Exception:
        print("ok")
```

1. **Line 1.** The file name does not match test_*.py, so pytest never collects it.
2. **Line 2.** The function name does not start with test either.
3. **Line 5.** A print never fails a test. Nobody reads it in CI.
4. **Line 6.** Any error passes, including a TypeError from a typo.

Worked example: strong code

```
# tests/test_normalise.py
import pytest

from survey.weights import normalise


def test_weights_sum_to_one() -> None:
    assert sum(normalise([1.0, 3.0])) == pytest.approx(1.0)


def test_negative_weight_raises() -> None:
    with pytest.raises(ValueError, match="negative weight"):
        normalise([2.0, -1.0])
```

1. **Line 1.** A collected file name, in the folder that testpaths names.
2. **Line 11.** A collected function, named for the behaviour it checks.

3. **Line 12.** Fails if nothing is raised, if another type is raised, or if the message differs.

Name files and functions so pytest collects them, assert instead of printing, and pin expected errors with pytest.raises and match.

Recap card

- **What pytest collects by default** Files test_*.py or *_test.py, functions starting with test, and test methods in Test classes without **init**.
- **Test that code raises the right error** with pytest.raises(ValueError, match="pattern"): call(). match searches the message with a regular expression.
- **Run one test, or only failures** pytest path::test_name for one test. -k expr selects by name. -x stops at the first failure. --lf reruns last failures.
- **pytest exit codes** 0 all passed, 1 a test failed, 5 no tests collected.

Practice

Exercise 1. Multiple choice (level 1)

With default settings, which of these does pytest 9 collect and run?

- a) `def check_weights():` in `tests/test_weights.py`
- b) `def test_weights():` in `tests/weight_checks.py`
- c) `def test_weights():` in `tests/weights_test.py`
- d) `def test_total(self):` in `class WeightTests:` in `tests/test_weights.py`

Why?

- r1) Both the file name (`*_test.py`) and the function name (`test*`) match the default patterns.
- r2) pytest runs every function in the tests folder.

Exercise 2. Predict the output (level 2)

What does this print? (pytest 9)

```
 1 import pytest
 2
 3
 4 def check(total):
 5     if total < 0:
 6         raise ValueError(f"total must be >= 0, got {total}")
 7
 8
 9 with pytest.raises(ValueError, match=r"got -\d"):
10     check(-5)
```

```
print("first ok")

try:
    with pytest.raises(ValueError, match="must be > 0"):
        check(-5)
except AssertionError:
    print("pattern did not match")
```

Exercise 3. Fill the gap (level 2)

Run only the tests whose names contain `zero`, and stop at the first failure.

```
pytest -k _____(1) -_____(2)
```

Exercise 4. Write the function (level 3)

`normalise(weights)` divides each weight by the total. It must raise `ValueError` with a message containing `negative weight` for any negative weight, and `ValueError` containing `total weight is 0` when the weights sum to 0.

Write two tests with `pytest.raises` and `match=`:

- `test_negative_weight_raises()` calls `normalise([2.0, -1.0])`.
- `test_zero_total_raises()` calls `normalise([0.0, 0.0])`.

The checker runs your tests against a correct `normalise`, which must pass, and against three broken versions, which your tests must catch.

Your code must pass these checks:

- Your tests must pass for a correct normalise.
- test_negative_weight_raises must fail when normalise accepts a negative weight. A try/except that passes on no error cannot catch that.
- test_zero_total_raises must expect ValueError, not any exception. A ZeroDivisionError is a bug.
- Add `match=` with the expected message, so a ValueError raised for another reason fails the test.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Scale weights so they sum to 1.
def normalise(weights: list[float]) -> list[float]: ...

def test_negative_weight_raises() -> None: ...

def test_zero_total_raises() -> None: ...
```

Starter code:

```
import pytest

```

```
 3
 4 def normalise(weights: list[float]) -> list[float]:
 5     """Scale weights so they sum to 1."""
 6     for i, w in enumerate(weights):
 7         if w < 0:
 8             raise ValueError(f"negative weight at index {i}")
 9     total = sum(weights)
10     if total == 0:
11         raise ValueError("total weight is 0")
12     return [w / total for w in weights]
13
14
15 def test_negative_weight_raises():
16     try:
17         normalise([2.0, -1.0])
18     except ValueError:
19         pass
20
21
22 def test_zero_total_raises():
23     try:
24         normalise([0.0, 0.0])
25     except Exception:
26         pass
```

Exercise 5. Put in order (level 3)

A test fails in CI. Put the steps to fix it in order.

- Run the whole suite, then push.
- Copy the failing test's node ID from the CI log.
- Run that one test locally: `pytest tests/test_share.py::test_zero_total`.
- Change the code until that test passes.
- Run `pytest --lf` to rerun every test that failed last time.
- Read the assertion output: the values on both sides and the call that produced them.

Sources

- Default collection: test_*.py and *_test.py files, test-prefixed functions, Test-prefixed classes without **init** (with a PytestCollectionWarning). (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- pytest.raises fails with 'DID NOT RAISE' when nothing is raised, lets other exception types through, and raises AssertionError when match does not find the pattern. (verified: Run on pytest 9.0.2 (Pyodide pin) and 9.2.0.)

- Exit codes 0, 1 and 5; -k, -x, --lf and node IDs; testpaths read from [tool.pytest.ini_options] and from [tool.pytest]. (verified: Run on pytest 9.0.2 and 9.2.0.)

B19-L2 Fixtures, shared setup without shared state

Concept

A **fixture** is a function decorated with `@pytest.fixture`. A test asks for it by naming it as a parameter, and pytest calls the fixture and passes in the result:

```
@pytest.fixture
def respondents() -> pd.DataFrame:
    return pd.DataFrame({"respondent_id": ["R1", "R2"], "final_weight":
    [1.0, 3.0]})


def test_total_weight(respondents: pd.DataFrame) -> None:
    assert respondents["final_weight"].sum() == 4.0
```

By default a fixture runs once **per test**, so every test gets a fresh object. A test can change it without touching any other test.

Use `yield` when something must be cleaned up. Code before `yield` is setup. Code after it is teardown, and pytest runs it even when the test fails.

`scope="module"` or `scope="session"` builds the object once and shares it. Use that only for things that are expensive and that no test changes, such as the SparkSession in lesson A9-L1. A shared object that one test changes makes results depend on the order tests run in.

Put fixtures that several files use in `conftest.py`. Tests in that folder and its subfolders can use them without an import.

pytest ships built-in fixtures. `tmp_path` gives each test a fresh temporary directory as a `pathlib.Path`. `monkeypatch` changes settings and undoes the change afterwards (lesson B19-L7).

When tests need many slightly different inputs, write a **factory fixture**. It returns a function that builds the data, with defaults for everything, so each test states only what matters to it:

```
@pytest.fixture
def make_respondents():
    def make(weights: list[float]) -> pd.DataFrame:
        ids = [f"R{i}" for i in range(1, len(weights) + 1)]
        return pd.DataFrame({"respondent_id": ids, "final_weight":
    weights})
```

```
    return make
```

Why it matters

A QA test suite shared one session-scoped DataFrame of 500 respondents across 40 tests. One test capped the weights in place to check the cap function. On a laptop the tests ran in file order and passed. In CI a plugin ran them in random order, and three weight-total tests failed about one run in four. The team spent two days calling the failures "flaky" before they found the in-place change. Function scope fixed it in one line.

Worked example: weak code

```
import pandas as pd

RESPONDENTS = pd.DataFrame({"respondent_id": ["R1", "R2"],
    "final_weight": [1.0, 3.0]})


def test_cap_weights():
    RESPONDENTS.loc[RESPONDENTS["final_weight"] > 2, "final_weight"] = 2.0
    assert RESPONDENTS["final_weight"].max() == 2.0


def test_total_weight():
    assert RESPONDENTS["final_weight"].sum() == 4.0
```

1. **Line 3.** One module-level frame, shared by every test in the file.
2. **Line 7.** Changes the shared frame in place. Every later test sees 2.0 instead of 3.0.
3. **Line 12.** Passes alone, fails after test_cap_weights: a test that depends on order.

Worked example: strong code

```
import pandas as pd
import pytest


@pytest.fixture
def respondents() -> pd.DataFrame:
    return pd.DataFrame({"respondent_id": ["R1", "R2"], "final_weight":
    [1.0, 3.0]})


def test_cap_weights(respondents: pd.DataFrame) -> None:
    capped = cap_weights(respondents, cap=2.0)
    assert capped["final_weight"].max() == 2.0


```

```
15 def test_total_weight(respondents: pd.DataFrame) -> None:
16     assert respondents["final_weight"].sum() == 4.0
```

1. **Line 5.** Function scope by default: a fresh frame for every test.
2. **Line 10.** The test asks for the fixture by naming it as a parameter.
3. **Line 11.** The function under test returns a new frame. Nothing shared changes.

Fixtures give each test fresh data by default. Share a fixture only when it is expensive and nothing changes it.

Recap card

- **What a fixture is** A function decorated with @pytest.fixture. A test gets its value by naming it as a parameter.
- **Fixture setup and teardown** Code before yield is setup. Code after yield is teardown, and it runs even when the test fails.
- **When to widen a fixture's scope** Only for expensive objects that no test changes. A shared object that a test changes makes results depend on test order.
- **Factory fixture** A fixture that returns a function building fresh data with defaults, so each test states only what matters.
- **Share fixtures across files** Put them in conftest.py. Tests in that folder and below use them without an import.

Practice

Exercise 1. Multiple choice (level 1)

A fixture returns a three-row DataFrame. Several tests change values in it. Which scope should it have?

- a) The default, function scope
- b) `scope="module"`
- c) `scope="session"`
- d) A module-level variable instead of a fixture

Why?

- r1) Each test gets its own fresh frame, so one test's changes cannot reach another.
- r2) Function scope makes tests run faster.

Exercise 2. Put in order (level 2)

`conftest.py` holds this fixture. `test_fix.py` has two tests that both use it and print their own names. You run `pytest -s tests/test_fix.py` (`-s` shows prints). Put the printed lines in order.

```
@pytest.fixture
def weights():
    print("setup")
    yield [1.0, 2.0]
    print("teardown")


def test_sum(weights):
    print("test_sum")
    assert sum(weights) == 3.0


def test_len(weights):
    print("test_len")
    assert len(weights) == 2
```

- teardown (first time)
- test_len
- teardown (second time)
- setup (second time)
- setup (first time)
- test_sum

Exercise 3. Fill the gap (level 2)

Open one DuckDB connection for all the tests in a file, and close it after the last one.

```
@pytest._____(1)(scope="_____(2)")
def con():
    connection = duckdb.connect()
    _____(3) connection
    connection.close()
```

Exercise 4. Write the function (level 3)

Replace the shared module-level frame with a factory fixture.

- `make_respondents` is a fixture that returns a function `make(weights, in_scope=None)`. It builds a new DataFrame with `respondent_id` (`"R1"`, `"R2"`, ...), `final_weight` (the weights, as floats) and `in_scope` (the given booleans, or all `True` when `in_scope` is `None`).

- `test_in_scope_weight_skips_out_of_scope(make_respondents)` builds two rows with weights 1.0 and 3.0, where only the first is in scope, and asserts that `in_scope_weight(df)` is 1.0.

Your code must pass these checks:

- make([1.0, 2.0]) must give columns respondent_id, final_weight and in_scope, with IDs R1, R2, float weights and in_scope all True.
- Pass the in_scope list through when it is given.
- Each call must build a new frame, so a change to one never reaches another.
- Your test must pass for a correct in_scope_weight.
- Your test must fail when in_scope_weight also counts the out-of-scope row. Mark the 3.0 row out of scope and assert 1.0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Total final_weight of in-scope rows.
def in_scope_weight(df: pd.DataFrame) -> float: ...

# Factory for small respondent frames with sensible defaults.
def make_respondents(): ...

def test_in_scope_weight_skips_out_of_scope(
    make_respondents,
) -> None: ...
```

Starter code:

```
import pandas as pd
import pytest


def in_scope_weight(df: pd.DataFrame) -> float:
    """Total final_weight of in-scope rows."""
    return float(df.loc[df["in_scope"], "final_weight"].sum())


RESPONDENTS = pd.DataFrame(
    {"respondent_id": ["R1", "R2"], "final_weight": [1.0, 3.0],
   "in_scope": [True, False]}
)


def test_in_scope_weight_skips_out_of_scope():
    assert in_scope_weight(RESPONDENTS) == 1.0
```

Exercise 5. Spot the bug (level 4)

`test_total_weight` passes on its own and fails when the whole file runs. Tap the **two** lines that together make the tests depend on each other.

```
import pandas as pd
import pytest


@pytest.fixture(scope="module")
def respondents():
    return pd.DataFrame({"id": ["R1", "R2"], "w": [1.0, 3.0]})


def test_cap_weights(respondents):
    respondents.loc[respondents["w"] > 2, "w"] = 2.0
    assert respondents["w"].max() == 2.0


def test_total_weight(respondents):
    assert respondents["w"].sum() == 4.0
```

Write down the line number of the bug.

Sources

- Function scope is the default; code after yield runs after a failing test; conftest.py fixtures reach their folder and subfolders, and a test elsewhere gets "fixture not found". (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- A module-scoped fixture changed in place by one test makes a later test fail, and the same tests pass in the reverse order. (verified: Run on pytest 9.2.0 with pandas 3.0.6.)
- Calling a fixture directly fails; fixture.**wrapped** returns the undecorated function in pytest 9.0.2 and 9.2.0. (verified: Run in the dev and Pyodide-pin environments. The E4 checker relies on **wrapped**.)

B19-L3 Parametrize, one test and many readable cases

Concept

`@pytest.mark.parametrize` runs one test function once per row of a table. Each row becomes its own test, with its own pass or fail:

```
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="below-35"),
```

```
        pytest.param(35, "35-54", id="at-35"),
        pytest.param(None, None, id="missing"),
    ],
)
def test_age_band(age: int | None, expected: str | None) -> None:
    assert age_band(age) == expected
```

pytest names each case from its values, such as `test_share[1-4-0.25]`. Give a case a name with `pytest.param(..., id="missing")`, so a failure reads `test_age_band[missing]`.

Choose rows on purpose. For every threshold, test the value just below it, the value on it, and a missing value. Most banding bugs are a `<` that should be a `<=`.

Mark a known bug on one row with `pytest.param(..., marks=pytest.mark.xfail(reason="bug #41", strict=True))`. The row is expected to fail. With `strict=True`, the run fails as soon as the row starts passing, which tells you to remove the mark once the bug is fixed.

Two `parametrize` decorators on one test run every combination: two values times three values make six tests.

To run every test that uses a fixture once per value, give the fixture `params`. Read the current value from the built-in `request` fixture:

```
@pytest.fixture(params=["pandas", "polars"])
def backend(request: pytest.FixtureRequest) -> str:
    return request.param
```

Keep the test body free of `if` statements that pick a behaviour per row. When some rows should raise and others return a value, write two tables and two tests.

Why it matters

An age banding function had one test, for age 40. A refactor changed `age < 35` to `age <= 35`, and every 35-year-old moved into the younger band. In a tracker with 800 respondents a wave, about 16 people moved band each wave. The 16-34 share rose by two points, and the client asked why. A parametrized test with rows 34 and 35 fails on that change before review.

Worked example: weak code

```
def test_age_band():
    for age, expected in [(20, "16-34"), (40, "35-54"), (70, "55+")]:
        if age is None:
            assert age_band(age) is None
        else:
```

```
6             assert age_band(age) == expected
```

1. **Line 2.** One test for three cases: the first failure hides the rest, and the report does not say which case broke.
2. **Line 3.** Branching inside the test. The table never contains None, so this branch never runs.

Worked example: strong code

```
1  import pytest
2
3
4  @pytest.mark.parametrize(
5      ("age", "expected"),
6      [
7          pytest.param(34, "16-34", id="below-35"),
(1)        pytest.param(35, "35-54", id="at-35"),
9          pytest.param(54, "35-54", id="below-55"),
10         pytest.param(55, "55+", id="at-55"),
(2)        pytest.param(None, None, id="missing"),
12     ],
13 )
14 def test_age_band(age: int | None, expected: str | None) -> None:
(3)    assert age_band(age) == expected
```

1. **Line 8.** Each threshold is tested on both sides. This row catches a < that became <=.
2. **Line 11.** The missing value gets its own named row.
3. **Line 15.** One assertion, no branching. Every row runs and reports separately.

One row per case, with both sides of every threshold and the missing value, each named so a failure explains itself.

Recap card

- **Run one test over a table of cases** @pytest.mark.parametrize(("age", "expected"), [...]). Each row is its own test.
- **Name a parametrized case** pytest.param(34, "16-34", id="below-35") reports as test_age_band[below-35].
- **Mark a known bug on one row** pytest.param(..., marks=pytest.mark.xfail(reason=..., strict=True)). strict fails the run once the row passes.
- **Run every test once per implementation** @pytest.fixture(params=[...]) and return request.param.
- **Rows to pick for a threshold** Just below it, on it, and a missing value.

Practice

Exercise 1. Predict the output (level 1)

How many tests does pytest collect from this file?

```python
import pytest


@pytest.mark.parametrize("total", [1, 4, 10])
@pytest.mark.parametrize("part", [0, 1])
def test_grid(part, total):
    assert part <= total


@pytest.fixture(params=["pandas", "polars"])
def backend(request):
    return request.param


def test_backend(backend):
    assert backend in {"pandas", "polars"}
```

Exercise 2. Multiple choice (level 2)

A row is marked `pytest.param(1, 3, 0.33, marks=pytest.mark.xfail(reason="rounding bug #41", strict=True))`. A colleague fixes the rounding bug. What happens on the next run?

- a) The row passes and the suite stays green.
- b) The row is reported as XPASS(strict) and the run fails.
- c) pytest skips the row.
- d) pytest removes the mark itself.

Why?

- r1) strict=True turns an unexpected pass into a failure, so the stale mark gets removed.
- r2) xfail rows always fail.

Exercise 3. Fill the gap (level 2)

Name the row for age 34, so a failure reads `test_age_band[just-below-35]`.

```python
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest._____(1)(34, "16-34", _____(2)="just-below-35"),
        pytest.param(35, "35-54", id="at-35"),
    ],
)
```

```
 8 def test_age_band(age, expected):
 9     assert age_band(age) == expected
```

Exercise 4. Write the function (level 3)

`age_band(age)` returns `"16-34"` under 35, `"35-54"` under 55, `"55+"` from 55, and `None` for a missing age.

Replace the loop with a parametrized test `test_age_band(age, expected)`. Your table must include the ages 34, 35, 54 and 55, and `None`, each with its correct expected band.

The checker reads your table, runs every row against a correct `age_band`, then against two broken versions, which your rows must catch.

Your code must pass these checks:

- Include rows for ages 34, 35, 54, 55 and None, both sides of each threshold and the missing value.
- Every row must pass for a correct age_band. Check the expected band for each age.
- Your rows must catch a < that became <=. Ages 35 and 55 do that when their expected bands are right.
- Add a row (None, None), so a missing age that lands in 55+ fails.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
 1 # Age group for reporting. None stays None.
 2 def age_band(age: int | None) -> str | None: ...
 3
 4 def test_age_band(
 5     age: int | None,
 6     expected: str | None,
 7 ) -> None: ...
```

Starter code:

```
 1 import pytest
 2
 3
 4 def age_band(age: int | None) -> str | None:
 5     """Age group for reporting. None stays None."""
 6     if age is None:
 7         return None
 8     if age < 35:
 9         return "16-34"
10     if age < 55:
11         return "35-54"
12     return "55+"
13
```

```
14
15 def test_age_band():
16     for age, expected in [(20, "16-34"), (40, "35-54"), (70, "55+")]:
17         assert age_band(age) == expected
```

Exercise 5. Architecture design (level 4)

Pick the tool for each testing need.

Options: @pytest.mark.parametrize table, Fixture with params, Hypothesis property (lesson B17-L2), A second parametrized test for the error cases.

1. The same rule checked on 12 input and output pairs
2. Every QA test must run against both the pandas and the Polars implementation
3. Normalised weights must sum to 1 for any non-empty list of positive weights
4. Five invalid inputs must each raise ValueError

Sources

- Default case IDs join the values (test_share[1-4-0.25]); stacked parametrize runs every combination; fixture params run each test once per value; a strict xfail row that passes reports [XPASS(strict)] and fails the run. (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- test.pytestmark holds the parametrize mark, with argnames in args[0] and rows (tuples or ParameterSet with .values) in args[1], in pytest 9.0.2 and 9.2.0. (verified: Run in the dev and Pyodide-pin environments. The E4 checker reads it.)

B19-L4 Testing pandas and NumPy code

Concept

`==` on DataFrames compares cell by cell and returns another DataFrame. It treats two missing values as different and ignores dtypes. Use the pandas testing helpers instead. In pandas 3.0, `pd.testing.assert_frame_equal(result, expected)` checks:

- values, with missing values in the same place counted as equal,
- dtypes, so a count that silently became float fails,
- column names and their order,
- the index.

Float columns are compared with a relative tolerance of 1e-5 by default, so `0.1 + 0.2` matches `0.3`. Integer columns must match exactly. `pd.testing.assert_series_equal` does the same for one column.

Two options matter most:

- `check_like=True` ignores the order of columns, and of rows with the same index labels.
- `check_dtype=False` ignores dtypes. Reach for it rarely: a dtype change is often the bug.

Filtering keeps the original index labels. After `df[df["w"] > 0]` drops row 1, the index is `[0, 2]`, and a comparison with a freshly built expected frame fails on the index. Decide what the function promises. If it promises a fresh index, it calls `.reset_index(drop=True)` and the test stays strict.

To compare rows whatever their order, sort both frames by their key columns and reset both indexes before comparing.

For NumPy arrays, use `np.testing.assert_allclose(actual, expected)`. Its defaults are `rtol=1e-7` and `atol=0`, and it treats NaN as equal to NaN. A relative tolerance never accepts any difference from an expected 0, so add `atol` when an expected value is 0.

Test that a function leaves its input alone. Take `before = df.copy()`, call the function, then compare `df` with `before`. Without `.copy()`, `before` is the same object and the comparison can never fail.

Why it matters

A refactor of a respondent-count function introduced a missing value, so pandas turned the integer `n` column into float. Every downstream check still compared with `==` and passed. The report template formatted `n` with `{:,}` and printed "1,204.0 respondents" on 40 pages. `assert_frame_equal` against an expected frame with an `int64` column fails on that change.

Worked example: weak code

```
def test_drop_zero_weights():
    df = pd.DataFrame({"id": ["R1", "R2", "R3"], "w": [1.0, 0.0, 2.0]})
    result = drop_zero_weights(df)
    assert (result["w"].values == [1.0, 2.0]).all()
```

1. **Line 4.** Checks one column's values only. A dropped id column, a dtype change or a mutated input all pass.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 4 def test_drop_zero_weights() -> None:
 5     df = pd.DataFrame({"id": ["R1", "R2", "R3"], "w": [1.0, 0.0, 2.0]})
 6     before = df.copy()
 7
 8     result = drop_zero_weights(df)
 9
10     expected = pd.DataFrame({"id": ["R1", "R3"], "w": [1.0, 2.0]})
11     pd.testing.assert_frame_equal(result, expected)
12     pd.testing.assert_frame_equal(df, before)
```

1. **Line 6.** A real copy, so the last line can detect a function that changes its input.
2. **Line 10.** The whole expected frame, written out. Its fresh index states that the function resets it.
3. **Line 11.** Checks every column, the dtypes and the index, with float tolerance.

Compare whole frames with assert_frame_equal against an expected frame you write out, and copy the input to prove the function leaves it alone.

Recap card

- **What assert_frame_equal checks** Values (missing values in the same place count as equal), dtypes, column names and order, and the index. Floats with rtol 1e-5.
- **Compare rows whatever their order** Sort both frames by the key columns, reset both indexes, then assert_frame_equal(..., check_like=True).
- **Why a filtered result fails on the index** Filtering keeps the old labels, such as [0, 2]. Reset the index in the function if it promises a fresh one.
- **assert_allclose and an expected 0** Defaults are rtol=1e-7, atol=0. A relative tolerance of 0 accepts nothing, so add atol.
- **Test that a function leaves its input alone** before = df.copy(), call the function, then assert_frame_equal(df, before).

Practice

Exercise 1. Predict the output (level 1)

What does this print? (pandas 3.0)

```
1 import numpy as np
2 import pandas as pd
3
4 a = pd.DataFrame({"share": [0.5, np.nan]})
```

```
b = pd.DataFrame({"share": [0.5, np.nan]})
print((a == b).all().all())
pd.testing.assert_frame_equal(a, b)
print("equal")
```

Exercise 2. Multiple choice (level 2)

`drop_zero_weights(df)` returns `df[df["w"] > 0]`. Its test fails with `DataFrame.index are different`, although the rows look right. The function should return a fresh index. What is the right fix?

- a) End the function with `.reset_index(drop=True)` and keep the test as it is.
- b) Add `check_dtype=False` to the assertion.
- c) Compare `result.values == expected.values` instead.
- d) Add `check_like=True` to the assertion.

Why?

- r1) Filtering keeps the old labels [0, 2]. The function promises a fresh index, so the function resets it, and the strict test protects that promise.
- r2) Indexes never matter in tests.

Exercise 3. Fill the gap (level 2)

Compare an array of shares with the expected values. One expected value is exactly 0, and the result there is `1e-15`, so a relative tolerance alone fails.

```
np.testing._____(1)(shares, [0.25, 0.75, 0.0], _____(2)=1e-12)
```

Exercise 4. Write the function (level 3)

Write `assert_same_rows(actual, expected, keys)` for tests whose result order does not matter. It raises `AssertionError` unless both frames hold the same rows, values and dtypes, whatever the order of rows and columns.

Sort both frames by the `keys` columns, reset both indexes, then compare with `pd.testing.assert_frame_equal(..., check_like=True)`. Do not change the frames passed in.

Your code must pass these checks:

- Rows and columns in a different order must pass. Sort by keys, reset the index and use check_like=True.
- A different value must raise AssertionError.
- A dtype change must raise. Do not pass check_dtype=False.
- A frame with an extra row must raise.

- Leave the frames passed in unchanged. sort_values and reset_index return new frames.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Assert two frames hold the same rows, ignoring row and column order.
def assert_same_rows(
    actual: pd.DataFrame,
    expected: pd.DataFrame,
    keys: list[str],
) -> None: ...
```

Starter code:

```python
import pandas as pd


def assert_same_rows(actual, expected, keys):
    assert (actual.values == expected.values).all()
```

Exercise 5. Spot the bug (level 4)

`add_share` changes its input, which the team rule forbids. This test was written to catch exactly that, yet it passes. Tap the line in the test that hides the problem, then fix the test. Leave `add_share` as it is.

```python
import pandas as pd


def add_share(df: pd.DataFrame) -> pd.DataFrame:
    df["share"] = df["w"] / df["w"].sum()
    return df


def test_add_share_keeps_input():
    df = pd.DataFrame({"w": [1.0, 3.0]})
    before = df
    add_share(df)
    pd.testing.assert_frame_equal(df, before)
```

Write down the line number of the bug. Then write the fix.

Sources

- assert_frame_equal: NaN in the same place is equal; float columns use relative tolerance 1e-5 (1.000005 passes, 1.00002 fails); integers compare exactly; column order, index and dtype differences fail; check_like=True ignores column and row order by label. (verified: Run on pandas 3.0.6 and 3.0.2 (Pyodide pin).)

- np.testing.assert_allclose defaults rtol=1e-07, atol=0, equal_nan=True; 1e-12 against 0.0 fails without atol. (verified: Run on NumPy 2.4.6.)

B19-L5 Testing statistical and random code

Concept

You rarely know the exact right answer for a statistic by heart. Three kinds of test still pin it down.

1. Compare with an independent oracle. On a small input, check your function against a trusted library that computes the same thing. Your Wilson interval should match statsmodels 0.14:

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)
```

The tolerance covers small differences, such as a z value rounded to 1.959964.

2. Control the randomness. A function that draws random numbers takes a `np.random.Generator` as a parameter. The test passes in `np.random.default_rng(7)`, often from a fixture, so every run draws the same numbers. Assert properties and tolerances, not exact draws: a test that hardcodes `0.4123` tells a reader nothing about why that number is right.

3. Derive the tolerance from theory. A simulation estimate is itself random. The share of 2,000 simulations in which a 95% interval covers the true value has a standard error of `sqrt(0.95 * 0.05 / 2000)`, about 0.0049. Assert that the coverage lies within four standard errors, between 0.931 and 0.969. A correct method passes. Pick parameters where a broken method fails clearly: at p = 0.05 and n = 30, the Wald interval covers only about 79% of the time.

Test invariants too. A probability lies in [0, 1]. A lower bound is at most the estimate. An interval narrows as n grows.

Never assert something that fails by chance. A test that checks `p_value < 0.05` on random data fails about one run in twenty, whatever the code does, and teaches the team to ignore red builds.

Why it matters

A team's bootstrap test asserted that the 95% interval for a mean was exactly `(4.812, 5.391)`, copied from one run. It had no seed, so it failed

about once a week. The team added a retry. Months later a refactor resampled without replacement, which made every interval far too narrow. The retried test stayed green, because it now only checked that the function returned two numbers. A seeded coverage test with a theory-based tolerance would have failed at once.

Worked example: weak code

```
def test_bootstrap_ci():
    lo, hi = bootstrap_ci([4.0, 5.0, 6.0, 5.5], n_boot=1000)
    assert (round(lo, 3), round(hi, 3)) == (4.812, 5.391)
```

1. **Line 2.** No generator passed in, so every run draws different resamples.
2. **Line 3.** One run's output, copied in. It fails by chance and says nothing about why the numbers are right.

Worked example: strong code

```
import math

import numpy as np
import pytest


@pytest.fixture
def rng() -> np.random.Generator:
    return np.random.default_rng(7)


def test_wilson_matches_statsmodels() -> None:
    from statsmodels.stats.proportion import proportion_confint

    expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
    np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)


def test_wilson_coverage(rng: np.random.Generator) -> None:
    sims, p, n = 2000, 0.05, 30
    successes = rng.binomial(n, p, size=sims)
    covered = [lo <= p <= hi for lo, hi in (wilson_interval(int(x), n) for x in successes)]
    se = math.sqrt(0.95 * 0.05 / sims)
    assert abs(np.mean(covered) - 0.95) < 4 * se
```

1. **Line 9.** A seeded generator from a fixture: the same draws on every run.
2. **Line 15.** An independent oracle for the exact value on a small input.
3. **Line 20.** Parameters where a broken method, such as the Wald interval, fails clearly.

4. **Line 24.** The tolerance comes from the simulation's own standard error, not from a guess.

Check exact values against an oracle, seed every random draw, and size simulation tolerances from the standard error.

Recap card

- **Oracle test for a statistic** On a small input, compare with an independent library, such as statsmodels proportion_confint, using assert_allclose.
- **Make random code testable** Pass a np.random.Generator in. Tests pass np.random.default_rng(seed), often from a fixture.
- **Tolerance for a simulated coverage** Standard error sqrt(p(1 - p) / sims). Allow about four standard errors: 0.931 to 0.969 for 0.95 over 2,000 simulations.
- **An assertion never to write** p_value < 0.05 on random data. It fails by chance about one run in twenty.

Practice

Exercise 1. Multiple choice (level 1)

Which test for `bootstrap_ci(values, rng, n_boot)` is reliable and catches real bugs?

- a) Run it once without a seed and assert the exact interval printed by that run.
- b) Pass `np.random.default_rng(7)`, then assert that lower <= mean <= upper and that the interval is narrower for 400 values than for 40.
- c) Assert that a t-test on the bootstrap means gives p < 0.05.
- d) Run it ten times without a seed and pass if most runs look sensible.

Why?

- r1) The seed makes every run identical, and the assertions are properties any correct interval has.
- r2) Seeds make code faster.

Exercise 2. Predict the output (level 2)

What does this print?

```
import math

p, sims = 0.95, 2000
```

```
se = math.sqrt(p * (1 - p) / sims)
low, high = p - 4 * se, p + 4 * se
print(round(se, 4), round(low, 3), round(high, 3))
```

Exercise 3. Fill the gap (level 2)

Check your Wilson interval against statsmodels on one small input.

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="_____(1)")
np.testing._____(2)(wilson_interval(12, 40), expected, rtol=1e-6)
```

Exercise 4. Write the function (level 3)

Write `coverage(ci_fn, rng, p, n, sims) -> float`, the core of a coverage test. It draws `sims` binomial success counts with `rng.binomial(n, p, size=sims)`, calls `ci_fn(successes, n)` for each count (as a Python `int`), and returns the share of intervals `(lo, hi)` with `lo <= p <= hi`. Raise `ValueError` if `sims` is below 1. Use only the `rng` passed in.

Your code must pass these checks:

- For the Wilson interval at p = 0.05, n = 30 and 2,000 simulations, coverage must be close to 0.95. A result of 1.0 means you compared the interval with the estimate, not with the true p.
- For the Wald interval at p = 0.05 and n = 30, coverage is far below 0.95. Count an interval only when lo <= p <= hi.
- An interval of (0, 1) always covers p, so coverage must be 1.0.
- The same seed must give the same coverage. Draw only from the rng passed in.
- Call ci_fn once per simulation, with the count as a Python int.
- Raise ValueError when sims is below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Share of simulated samples whose interval covers the true proportion.
def coverage(
    ci_fn: Callable[[int, int], tuple[float, float]],
    rng: np.random.Generator,
    p: float,
    n: int,
    sims: int,
) -> float: ...
```

Starter code:

```
import numpy as np

```

```
3
4 def coverage(ci_fn, rng, p, n, sims):
5     return 0.95
```

Exercise 5. Architecture design (level 4)

Plan the tests for `weighted_mean_ci(values, weights, rng)`, which returns a bootstrap interval for a weighted mean.

Options: Compare with np.average(values, weights=weights) on four rows, Compare with the value the function printed last week.

1. Is the point estimate right?
2. Make the bootstrap reproducible
3. Does the interval cover often enough?
4. Edge cases

Sources

- statsmodels proportion_confint(12, 40, alpha=0.05, method="wilson") matches a Wilson interval with z = 1.959964 within rtol 1e-6. (verified: Run on statsmodels 0.14.6 (Pyodide pin) and 0.15.0.)
- With np.random.default_rng(7), 2,000 simulations at p = 0.05, n = 30: Wilson coverage 0.9415, Wald coverage 0.7865. At p = 0.3, n = 50: Wilson 0.9605, Wald 0.938. (verified: Run on NumPy 2.4.6 in the dev and Pyodide-pin environments.)

B19-L6 Testing machine learning pipelines

Concept

You cannot assert that a model's AUC is exactly 0.873. The next scikit-learn release, a new seed or one more row changes it. Test what must hold for any correct pipeline instead. With scikit-learn 1.8 and a few hundred synthetic rows, each of these tests runs in under a second.

Contracts. `predict_proba` returns one row per input and one column per class. Every row sums to 1, and every value lies between 0 and 1.

No leakage. Fit the pipeline on the training split only, then check what it learned. A `StandardScaler` inside it must hold the training means: `pipe.named_steps["standardscaler"].mean_` equals `X_train.mean(axis=0)`. If it holds the mean of all the data, the test rows leaked into preprocessing (lesson B10-L2).

Determinism. Set `random_state` in every splitter and estimator. Two fits on the same data must give the same predictions. A split without `random_state` gives different rows every run, and the test becomes flaky.

Beats a baseline. `DummyClassifier(strategy="prior")` ignores the features, so its AUC is 0.5. On fixed data from `make_classification(random_state=0)`, assert that the model clearly beats it. A fixed high bar, such as AUC above 0.9, breaks on harmless changes. A margin over the baseline catches a pipeline that has stopped learning.

Behaviour. A model's prediction for a row must not depend on where the row sits. Shuffle the rows, predict, put the predictions back in the original order, and compare. A feature that raises risk must not lower the predicted risk when it alone increases, if the model promises that.

For your own transformers, scikit-learn's `parametrize_with_checks` from `sklearn.utils.estimator_checks` runs its standard API checks as pytest tests.

Why it matters

A churn pipeline's only test asserted `auc > 0.85` on a split without `random_state`. It failed one run in eight, and the team added a retry. Later a refactor moved the scaler outside the pipeline and fitted it on all rows. Offline AUC rose by 0.03 and the test stayed green. In production, AUC fell by 0.05. A one-line test comparing the fitted scaler's means with the training means fails on that refactor.

Worked example: weak code

```
def test_model():
    X_train, X_test, y_train, y_test = train_test_split(X, y)
    model = build_pipeline().fit(X_train, y_train)
    assert roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]) > 0.85
```

1. **Line 2.** No random_state: different rows every run, so the test is flaky.
2. **Line 4.** One fixed bar. It cannot tell leakage from skill, and it fails on harmless changes.

Worked example: strong code

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split


def split():
```

```
    X, y = make_classification(n_samples=300, random_state=0)
    return train_test_split(X, y, test_size=0.3, random_state=0,
stratify=y)


def test_scaler_fitted_on_training_rows_only() -> None:
    X_train, _, y_train, _ = split()
    pipe = build_pipeline().fit(X_train, y_train)
    np.testing.assert_allclose(pipe.named_steps["standardscaler"].mean_,
X_train.mean(axis=0))


def test_probabilities_are_valid() -> None:
    X_train, X_test, y_train, _ = split()
    proba = build_pipeline().fit(X_train, y_train).predict_proba(X_test)
    assert proba.shape == (len(X_test), 2)
    np.testing.assert_allclose(proba.sum(axis=1), 1.0)


def test_beats_baseline() -> None:
    X_train, X_test, y_train, y_test = split()
    model = build_pipeline().fit(X_train, y_train)
    baseline = DummyClassifier(strategy="prior").fit(X_train, y_train)
    model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    base_auc = roc_auc_score(y_test, baseline.predict_proba(X_test)[:, 1])
    assert model_auc > base_auc + 0.2
```

1. **Line 10.** Fixed synthetic data and a seeded split: the same rows every run.
2. **Line 16.** A leakage test: the scaler must have learned from training rows only.
3. **Line 22.** A contract: one row per input, one column per class.
4. **Line 32.** A margin over a baseline that ignores the features, not a fixed bar.

Test contracts, leakage, determinism and a margin over a baseline on fixed data, not one exact score.

Recap card

- **Leakage test for a pipeline** Fit on the training split, then assert pipe.named_steps["standardscaler"].mean_ equals X_train.mean(axis=0).
- **Contract tests for a classifier** predict_proba shape is (n_rows, n_classes), each row sums to 1, values lie in [0, 1].
- **Performance test that is not fragile** On fixed data from make_classification(random_state=0), assert the model beats DummyClassifier(strategy="prior") by a margin.
- **Behavioural test: row order** Shuffle with a seeded permutation and check each row's prediction is unchanged: compare with original[perm].

Practice

Exercise 1. Multiple choice (level 1)

Which test fails when the scaler has been fitted on all the data instead of the training rows?

- a) Fit on the training split, then assert the scaler's `mean_` equals `X_train.mean(axis=0)`.
- b) Assert test AUC is above 0.85.
- c) Assert `predict_proba` rows sum to 1.
- d) Assert the pipeline fits without raising.

Why?

- r1) It checks what the pipeline learned, which differs exactly when test rows leak in.
- r2) Means are faster to compute than AUC.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train = np.array([[1.0], [3.0], [5.0], [7.0]])
y_train = np.array([0, 0, 1, 1])
X_test = np.array([[100.0]])

pipe = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train,
  y_train)
pipe.predict(X_test)
print(pipe.named_steps["standardscaler"].mean_[0])
```

Exercise 3. Fill the gap (level 2)

Compare the model with a baseline that ignores the features.

```
baseline = DummyClassifier(strategy="_____(1)").fit(X_train, y_train)
base_auc = roc_auc_score(y_test, baseline._____(2)(X_test)[:, 1])
model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
assert model_auc > base_auc + 0.2
```

Exercise 4. Write the function (level 3)

Write `order_invariant(model, X, rng) -> bool`, a behavioural check. Draw a permutation with `rng.permutation(len(X))`, predict probabilities for the original rows and for the shuffled rows, and return `True` when each row gets

the same probabilities either way (within `np.allclose`). `X` is a NumPy array. Do not change it.

What you need to know.

`np.allclose(a, b)` returns `True` when every pair of values is close, with `rtol=1e-05` and `atol=1e-08` by default. Unlike `np.testing.assert_allclose`, it returns a bool instead of raising.

Indexing an array with an array of positions picks rows in that order and returns a new array:

```
scores = np.array([[10], [20], [30]])
scores[[2, 0, 1]]   # rows 30, 10, 20
```

Your code must pass these checks:

- A logistic regression scores each row on its own, so the check must return True.
- Compare each row's probabilities with the same row's shuffled result: align them with the permutation before comparing.
- Leave X unchanged. Index with the permutation instead of shuffling in place.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# True when each row's predicted probabilities do not depend on row order.
def order_invariant(
    model: object,
    X: np.ndarray,
    rng: np.random.Generator,
) -> bool: ...
```

Starter code:

```
import numpy as np


def order_invariant(model, X, rng):
    return True
```

Exercise 5. Spot the bug (level 4)

This test fails about one run in seven, with no code change. Tap the line that makes it flaky, then fix it.

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

```

```
6
7  def test_model_beats_threshold():
8      X, y = make_classification(n_samples=300, random_state=0)
9      X_train, X_test, y_train, y_test = train_test_split(X, y,
       test_size=0.3)
10     model = RandomForestClassifier(n_estimators=50,
       random_state=0).fit(X_train, y_train)
11     auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
12     assert auc > 0.97
```

Write down the line number of the bug. Then write the fix.

Sources

- DummyClassifier with strategy prior or most_frequent gives AUC 0.5; a fitted StandardScaler in a pipeline holds the training means, unchanged by predict; train_test_split without random_state returns different rows on each call; parametrize_with_checks and check_estimator import from sklearn.utils.estimator_checks. (verified: Run on scikit-learn 1.8.0 (Pyodide pin) and 1.9.1.)
- The E5 test with an unseeded split fails (AUC <= 0.97) in 14.3% of 1,000 seeded splits; its AUC never fell to 0.9 in 1,000 splits. (verified: Run on scikit-learn 1.9.1 with make_classification(n_samples=300, random_state=0) and a 50-tree forest.)

B19-L7 Isolate I/O with fakes, monkeypatch and mocks

Concept

A unit test should not call a survey API, read a production table or depend on today's date. Lesson B2-L1 pushed side effects to the edges. Three tools handle what is left at those edges.

Pass the dependency in. A function that takes `get` or `now` as a parameter needs no tricks to test. The test passes a small fake:

```
1  class FakeResponse:
2      status_code = 200
3
4      def json(self) -> dict[str, int]:
5          return {"respondents": 812}
6
7
8  assert wave_count("W14", get=lambda url, timeout: FakeResponse()) == 812
```

`monkeypatch`, a built-in fixture, changes something for one test and undoes it afterwards. `monkeypatch.setenv("SURVEY_API_TOKEN", "test-token")` sets an

environment variable. `monkeypatch.delenv("SURVEY_API_TOKEN", raising=False)` removes it. `monkeypatch.setattr(module, "name", value)` replaces an attribute.

`unittest.mock` from the standard library builds stand-ins that record how they were called:

- `Mock(return_value=x)` returns `x` on every call.
- `Mock(side_effect=[a, ConnectionError("down")])` returns `a`, then raises on the next call.
- `fake.assert_called_once_with(url, timeout=30)` checks the arguments.
- `fake.call_count` counts calls, including calls that raised.

`patch("module.name")` swaps a name for the length of a `with` block. Patch the name **where the code looks it up**. If `reports.py` does `from requests import get`, patch `reports.get`. Patching `requests.get` leaves `reports.get` pointing at the real function, and the test makes a real call.

Use mocks sparingly. A mock checks how your code talks to the outside world, not whether its result is right. Mock the boundary, such as HTTP, the clock or a database, and test the logic with real objects. Never mock pandas.

Why it matters

A test of a weekly loader patched `requests.get`, but the loader imported `get` by name. Every CI run quietly called the supplier's live API with a test token. The supplier rate-limited the account during a delivery week, and the real nightly load failed. Patching `loader.get` instead, or passing `get` in as a parameter, would have kept the tests offline.

Worked example: weak code

```
1  import requests
2
3
4  def wave_count(wave_id: str) -> int:
1      response = requests.get(f"https://survey.example/api/waves/
       {wave_id}", timeout=30)
6      return response.json()["respondents"]
7
8
9  def test_wave_count():
2      assert wave_count("W14") > 0
```

1. **Line 5.** The network call is buried inside the logic, so the test cannot avoid it.
2. **Line 10.** Calls the live API on every run, and asserts almost nothing.

Worked example: strong code

```
from collections.abc import Callable
from typing import Any

import pytest

API = "https://survey.example/api/waves"


def wave_count(wave_id: str, get: Callable[..., Any]) -> int:
    response = get(f"{API}/{wave_id}", timeout=30)
    if response.status_code != 200:
        raise RuntimeError(f"wave API returned {response.status_code}")
    return int(response.json()["respondents"])


class FakeResponse:
    def __init__(self, status_code: int, payload: dict[str, int]) -> None:
        self.status_code = status_code
        self._payload = payload

    def json(self) -> dict[str, int]:
        return self._payload


def test_wave_count_reads_respondents() -> None:
    calls = []

    def fake_get(url: str, timeout: int) -> FakeResponse:
        calls.append((url, timeout))
        return FakeResponse(200, {"respondents": 812})

    assert wave_count("W14", fake_get) == 812
    assert calls == [(f"{API}/W14", 30)]


def test_wave_count_raises_on_error_status() -> None:
    with pytest.raises(RuntimeError, match="503"):
        wave_count("W14", lambda url, timeout: FakeResponse(503, {}))
```

1. **Line 9.** The HTTP function is a parameter. Production passes requests.get, tests pass a fake.
2. **Line 16.** A tiny fake with only what the code uses: status_code and json().
3. **Line 33.** Checks the request too: the right URL and a timeout.
4. **Line 37.** The error path is tested without a real outage.

Pass I/O in as a parameter where you can, use monkeypatch for settings, and patch a name where the code looks it up.

Recap card

- **Easiest way to test code that does I/O** Pass the dependency in as a parameter, such as get or now. Tests pass a small fake.
- **Change an environment variable for one test** monkeypatch.setenv(name, value) or monkeypatch.delenv(name, raising=False). pytest restores it afterwards.
- **Mock that fails on the second call** Mock(side_effect=[result, ConnectionError("down")]). call_count counts both calls.
- **Where to patch** Where the code looks the name up. After from requests import get in reports.py, patch reports.get.

Practice

Exercise 1. Multiple choice (level 1)

`fetch.py` starts with `from requests import get` and calls `get(url, timeout=30)`. Which patch keeps a test of `fetch.load()` offline?

- a) `patch("requests.get")`
- b) `patch("fetch.get")`
- c) `patch("get")`
- d) `patch("fetch.requests.get")`

Why?

- r1) fetch.py looks up the name get in its own module, so that is the name to replace.
- r2) Patching a module's name is faster than patching requests.

Exercise 2. Predict the output (level 2)

What does this print?

```
from unittest.mock import Mock

api = Mock(side_effect=[{"n": 10}, ConnectionError("down")])
print(api("W1")["n"])
try:
    api("W2")
except ConnectionError as err:
    print("error:", err)
print(api.call_count)
```

Exercise 3. Fill the gap (level 2)

Test both paths of `load_token()`, which reads `SURVEY_API_TOKEN` and raises `RuntimeError` when it is missing. Leave the real environment untouched.

```python
def test_missing_token_raises(monkeypatch):
    monkeypatch._____(1)("SURVEY_API_TOKEN", raising=False)
    with pytest.raises(RuntimeError, match="SURVEY_API_TOKEN"):
        load_token()


def test_token_is_read(monkeypatch):
    monkeypatch._____(2)("SURVEY_API_TOKEN", "test-token")
    assert load_token() == "test-token"
```

Exercise 4. Refactor (level 3)

Make `wave_count` testable without a network. Its new signature is `wave_count(wave_id: str, get: Callable[..., Any]) -> int`.

- Call `get(f"https://survey.example/api/waves/{wave_id}", timeout=30)`.
- Raise `RuntimeError` naming the status code when `response.status_code` is not 200.
- Return `int(response.json()["respondents"])`.
- Remove the `requests` import. The caller passes `requests.get` in production.

Your code must pass these checks:

- Call get once with the wave URL and timeout=30, and return the respondents count as an int.
- Raise RuntimeError naming the status code when it is not 200.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
API = ...

# Number of respondents in one wave, from the survey API.
def wave_count(wave_id: str, get: Callable[..., Any]) -> int: ...
```

Starter code:

```python
import requests


def wave_count(wave_id):
    response = requests.get(f"https://survey.example/api/waves/{wave_id}", timeout=30)
    return response.json()["respondents"]
```

Exercise 5. Spot the bug (level 4)

This test calls the live API on every run. Tap the line that lets the real call through, then fix it.

```python
# reports.py contains:
#     from requests import get
#
#     def wave_count(wave_id):
#         url = f"https://survey.example/api/waves/{wave_id}"
#         return get(url, timeout=30).json()["respondents"]

from unittest.mock import patch

import reports


def test_wave_count_reads_respondents():
    with patch("requests.get") as fake_get:
        fake_get.return_value.json.return_value = {"respondents": 812}
        assert reports.wave_count("W14") == 812
```

Write down the line number of the bug. Then write the fix.

Sources

- patch on the original module leaves a name imported with from ... import pointing at the real function; patching the importing module's name works; assert_called_once_with checks arguments. (verified: Run on Python 3.11 with pytest 9.2.0, using a stand-in module for requests.)
- Mock(side_effect=[value, exception]) returns then raises, and call_count counts the raising call; monkeypatch.delenv(raising=False) and setenv are undone after the test. (verified: Run on Python 3.11 and 3.13.)

B19-L8 Coverage, markers and a test suite you can trust in CI

Concept

Coverage tells you which code your tests ran. It does not tell you whether they checked anything. A test whose only assertion is `assert result` reaches every line and catches almost nothing (lesson B17-L1). Use coverage to find code no test reaches, not as proof of quality.

With pytest-cov 7.1:

```
pytest --cov=survey_stats --cov-branch --cov-report=term-missing --cov-fail-under=90
```

`--cov-branch` adds **branch coverage**. Line coverage counts a line once it runs. Branch coverage also asks whether each `if` went both ways. A function with `if cap is not None:` and a single test that passes a cap runs

every line, 100% line coverage, but never takes the path where `cap` is `None`. The report lists that missed jump as `2->4` in the Missing column, and the file drops to 83%.

`--cov-fail-under=90` makes pytest exit with an error when total coverage falls below 90%. Agree a floor with the team and raise it over time. Do not chase 100% with tests that assert nothing.

Markers label tests. Register each one in `pyproject.toml`, and turn on `--strict-markers`, so a typo such as `@pytest.mark.slwo` stops the run instead of creating a new, unused marker:

```
[tool.pytest.ini_options]
addopts = "--strict-markers"
markers = ["slow: takes more than a few seconds"]
```

Run `pytest -m "not slow"` while you work, and the full suite in CI. pytest 9 also accepts `strict = true`, which turns on strict markers along with other strict checks.

A **flaky** test passes and fails without a code change. Find the cause: a missing seed, shared state, the clock or the network. Never add a retry and move on.

Coverage misses weak assertions. **Mutation testing** finds them: mutmut 3.8 changes your code in small ways, such as `<` to `<=`, and reports every change that no test catches. It automates the habit of breaking the code once.

Why it matters

A pricing module had 96% line coverage, and the team was proud of it. Branch coverage was 71%. Every `else` path that handled a missing discount code was untested, including one that applied a 100% discount. It shipped and cost a client three days of free orders. `--cov-branch` with a 90% floor would have blocked the merge and pointed at the exact branches.

Worked example: weak code

```
# pyproject.toml has no marker settings
import pytest


@pytest.mark.slwo
def test_coverage_simulation():
    ...


def test_capped():
```

```
    assert capped(5, cap=3)
```

1. **Line 5.** A typo. Without --strict-markers it only warns, so `-m "not slow"` still runs this slow test.
2. **Line 11.** Runs every line of capped, so line coverage says 100%. It checks nothing, and the cap=None path never runs.

Worked example: strong code

```
# pyproject.toml:
#   addopts = "--strict-markers"
#   markers = ["slow: takes more than a few seconds"]
import pytest


@pytest.mark.slow
def test_coverage_simulation(rng):
    ...


@pytest.mark.parametrize(
    ("weight", "cap", "expected"),
    [(5, 3, 3), (2, 3, 2), (5, None, 5)],
)
def test_capped(weight, cap, expected):
    assert capped(weight, cap=cap) == expected
```

1. **Line 2.** An unregistered marker now stops the run.
2. **Line 7.** A registered marker. `pytest -m "not slow"` skips it locally, and CI runs it.
3. **Line 14.** Rows for both sides of the if, so branch coverage reaches 100% and each row asserts a value.

Measure branch coverage and set a floor, register markers strictly, fix flaky tests at the cause, and remember that coverage says what ran, not what was checked.

Recap card

- **Line coverage against branch coverage** Line: did each line run? Branch (--cov-branch): did each if go both ways? A missed jump shows as 2->4 in Missing.
- **Fail CI when coverage drops** pytest --cov=pkg --cov-branch --cov-fail-under=90. Start near today's level and raise it.
- **Catch marker typos** Register markers under markers = [...] and add --strict-markers to addopts. pytest 9 also offers strict = true.

- **What coverage cannot tell you** Whether tests check anything. Mutation testing (mutmut 3.8) changes the code and reports changes no test catches.
- **A flaky test** Fix the cause: seed, shared state, clock or network. A retry hides it.

Practice

Exercise 1. Multiple choice (level 1)

A module has 100% line coverage, yet a bug in the path where `cap` is `None` shipped. Which setting would have pointed at the untested path?

- a) `--cov-branch`
- b) `--cov-fail-under=100` on line coverage
- c) `-m "not slow"`
- d) `--strict-markers`

Why?

- r1) Branch coverage reports each if that never went one of its two ways, even when every line ran.
- r2) Branch coverage makes the tests run faster.

Exercise 2. Predict the output (level 2)

`survey/weights.py` holds this function, and the only test is `assert capped(5, cap=3) == 3`. pytest-cov reports the file's coverage without `--cov-branch` and with it. What are the two percentages? Answer as two numbers, such as `90 75`.

```
1 def capped(w, cap=None):
2     if cap is not None:
3         w = min(w, cap)
4     return w
```

Exercise 3. Fill the gap (level 2)

In `pyproject.toml`, register a `slow` marker and make any unregistered marker stop the run.

```
1 [tool.pytest.ini_options]
2 addopts = "_____(1)"
3 markers = ["_____(2): takes more than a few seconds"]
```

Exercise 4. Write the function (level 3)

`risk_band(score)` is already imported in your test file:

```python
def risk_band(score: float | None) -> str:
    if score is None:
        return "unknown"
    if score >= 0.8:
        return "high"
    if score >= 0.5:
        return "medium"
    return "low"
```

Write tests, as functions whose names start with `test_`, that run every branch **and** catch an off-by-one change. The checker measures which lines your tests run, then runs them against two changed versions: `>= 0.8` becomes `> 0.8`, and `>= 0.5` becomes `> 0.5`. Your tests must fail for both.

Your code must pass these checks:

- Your tests must pass for the correct risk_band. Check each expected band.
- Run every branch: include a missing score, a high, a medium and a low score.
- Coverage is not enough: add a case exactly at 0.8, which must be high.
- Add a case exactly at 0.5, which must be medium.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
def test_risk_band(
    score: float | None,
    expected: str,
) -> None: ...
```

Starter code:

```python
def test_risk_band():
    assert risk_band(0.9) == "high"
    assert risk_band(0.1) == "low"
```

Exercise 5. Architecture design (level 4)

Set up the team's test runs.

Options: pytest with --cov-branch and --cov-fail-under at the agreed floor, all tests including slow, No tests, to keep pull requests quick.

1. Every pull request
2. While you work on your laptop
3. A test fails one run in ten with no code change
4. The coverage floor

Practical: Build a complete test suite for a survey package

Platform: Laptop. Work in `labs/testing/` in this repository. It holds a small package, `survey_stats`, with age bands, weight helpers, a Wilson interval and an API client. You need the `dev` dependency group, which includes pytest 9, pytest-cov and statsmodels. Each step uses one lesson of module B19.

1. From the repository root, run `pip install -e . --group dev`, then `cd labs/testing` and `pytest`. One example test passes. Read every file in `src/survey_stats/`.
2. In `tests/test_bands.py`, write a parametrized test for `age_band` with named rows on both sides of 35 and 55, and a missing age (lesson B19-L3).
3. In `tests/conftest.py`, write a `make_respondents` factory fixture. Use it in `tests/test_weights.py` to test `cap_weights` with `assert_frame_equal`, including a missing weight and a check that the input is unchanged (lessons B19-L2 and B19-L4).
4. Test both errors of `normalise` and of `cap_weights` with `pytest.raises` and `match` (lesson B19-L1).
5. In `tests/test_intervals.py`, compare `wilson_interval` with statsmodels, then write a seeded coverage simulation with a tolerance of four standard errors. Mark it `@pytest.mark.slow` (lessons B19-L5 and B19-L8).
6. In `tests/test_api.py`, test `wave_count` with a fake `get`: one test for a 200 response that also checks the URL and timeout, one for a 503 (lesson B19-L7).
7. Run `pytest --cov=survey_stats --cov-branch --cov-report=term-missing`. Add tests until the Missing column is empty. Then run `pytest -m "not slow"` and check the simulation is deselected.
8. Break each function once, for example change `<` to `<=` in `age_band`. Confirm a test fails each time, then undo the change. Finally run `pytest reference_tests` and compare the reference suite with yours.

Sources

- With one test passing a cap, pytest-cov reports 100% line coverage and 83% with --cov-branch, listing the missed jump 2->4; --cov-fail-under=90 exits 1 with 'Coverage failure'. (verified: Run on pytest 9.2.0, pytest-cov 7.1.0, coverage 7.17.0.)
- An unregistered marker warns (PytestUnknownMarkWarning) without strict mode, and stops collection with --strict-markers or strict = true; -m

"not slow" deselects marked tests. (verified: Run on pytest 9.0.2 and 9.2.0.)

- mutmut 3.8.1 is the current release on PyPI. (search-only: pip index versions mutmut, October 2026.)

B18 Documentation and communication

Roadmap: Prerequisite for all months; Every month. Docstrings, READMEs, model cards and decision logs.

B18-L1 Docstrings and READMEs that answer the next person's questions

Concept

The next person to read your code, often you in six months, has three questions: what does this do, how do I run it, and what will surprise me? Docstrings answer them for a function. The README answers them for the project.

Docstrings. This course uses the Google style (CLAUDE.md), which tools such as Sphinx and mkdocstrings read:

```
def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of a 0/1 column. Grain: one row per respondent.

    Args:
        df: Respondent-level data.
        value: 0/1 column to average.
        weight: Non-negative weight column. Rows with a missing value or weight are dropped.

    Returns:
        The weighted share, or None when no rows remain or weights sum to 0.

    Raises:
        KeyError: If a named column is missing.
    """
```

Put in the docstring what the signature cannot say: the **grain**, **units**, how **missing values** and **zeros** are handled, and what `None` means. Leave out what the code already says plainly, such as "loops over the rows".

`help(weighted_share)` and `weighted_share.__doc__` show the docstring. Editors show it on hover.

README. One page at the project root, in this order:

1. **Purpose**: one paragraph, including who uses the output.

2. **Quick start**: the exact commands to install and run, copied from a working terminal.
3. **Inputs and outputs**: sources, tables or files, their grain and schedule.
4. **Configuration**: where settings live and which ones matter.
5. **Tests and checks**: how to run them, and what CI enforces.
6. **Owners and support**: a team, not a person who might leave.

Keep both close to the truth. Update the docstring in the same commit as the code, and test the README's quick start in CI where you can.

Why it matters

A weighting function returned `None` for empty cells. Its docstring said "Returns the weighted share." A dashboard developer read that, assumed a float, and wrote `round(share * 100)`. The first empty cell crashed the nightly refresh. One line in the docstring, "or None when no rows remain", would have told them to handle it.

Worked example: weak code

```
1 def calc(df, v, w):
2     # calculate
3     d = df[[v, w]].dropna()
4     return (d[v] * d[w]).sum() / d[w].sum() if d[w].sum() else None
```

1. **Line 1.** Name, parameters and types say nothing about purpose or inputs.
2. **Line 2.** A comment that restates the obvious and hides what matters.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 4 def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float |
     None:
 1     """Weighted mean of a 0/1 column. Grain: one row per respondent.
 6
 7     Args:
 8         df: Respondent-level data.
 9         value: 0/1 column to average.
10         weight: Non-negative weight column. Rows missing value or weight
    are dropped.
11
12     Returns:
 2         The weighted share, or None when no rows remain or weights sum to
    0.
14
15     Raises:
 3         KeyError: If a named column is missing.
```

```
    """
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)
```

1. **Line 5.** One-line summary with the grain.
2. **Line 13.** What None means, so callers handle it.
3. **Line 16.** The failure a caller might need to catch.

Docstrings state grain, units, missing values and what None means. The README states purpose, quick start, inputs, config, tests and owners.

Recap card

- **Google-style docstring sections** One-line summary, then Args:, Returns:, Raises:.
- **What a data docstring must state** Grain, units, missing and zero handling, and what None means.
- **README sections** Purpose, quick start, inputs and outputs, configuration, tests, owners.
- **Keeping docs true** Update docstrings in the same commit as the code. Test the quick start where possible.

Practice

Exercise 1. Multiple choice (level 1)

Which line adds the most value to a function's docstring?

- a) "Loops over rows and sums values."
- b) "Returns None when the weights sum to 0."
- c) "Written by the analytics team."
- d) "This is a function."

Why?

- r1) It tells callers about a behaviour the signature and name cannot show.
- r2) Short lines are better.

Exercise 2. Predict the output (level 2)

What does this print?

```
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    return None if total == 0 else part / total
```

```
4
5
6 print(share.__doc__)
```

Exercise 3. Put in order (level 2)

Order the sections of a project README.

- Tests and checks
- Inputs and outputs, with grain and schedule
- Owners and support
- Quick start: install and run commands
- Purpose and users
- Configuration

Exercise 4. Write the function (level 3)

Add a Google-style docstring to `cap_weights`. It must have a one-line summary, an `Args:` section naming `weights` and `cap`, a `Returns:` section, and a `Raises:` section with `ValueError` for a cap below 1. Mention that missing weights stay missing. Keep the code as it is.

Your code must pass these checks:

- Include Args:, Returns: and Raises: sections.
- Name both parameters under Args:, and ValueError under Raises:.
- Say what happens to missing weights.
- Start with a one-line summary.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Cap survey weights at a maximum value.
2 def cap_weights(weights: pd.Series, cap: float) -> pd.Series: ...
```

Starter code:

```
1 import pandas as pd
2
3
4 def cap_weights(weights: pd.Series, cap: float) -> pd.Series:
5     if cap < 1:
6         raise ValueError("cap must be at least 1")
7     return weights.clip(upper=cap)
```

Exercise 5. Architecture design (level 4)

Decide where each piece of information belongs.

Options: Docstring, README, Inline comment, Config file.

1. The function returns None for empty cells
2. The command to run the monthly job
3. Why one line uses a workaround for a library bug, with the issue link
4. The minimum reporting base of 50

B18-L2 Model cards that state limits as clearly as scores

Concept

A **model card** is a short document that travels with a model. It tells users what the model is for, how well it works, for whom, and where it should not be used. The format comes from Mitchell et al., "Model Cards for Model Reporting" (2019). Keep it in the repository next to the training code, and link it from the registered model version (lesson B14-L2).

Sections to include:

- **Model details**: name, version, registry alias, owners, training date, run ID.
- **Intended use**: the decision it supports and who uses it.
- **Out-of-scope uses**: uses it must not serve, stated plainly.
- **Training data**: source, period, grain, known gaps.
- **Evaluation**: the honest metric (lesson B11-L3), with the test period and prevalence.
- **Subgroup results**: rates per group with counts (lesson B13-L3), including the weakest group.
- **Limitations and risks**: known failure modes, drift risks, proxies.
- **Monitoring and review**: what is tracked, alert thresholds, next review date.

Write numbers with their context: "AUC 0.78 on October to December 2026 customers, 2.4% positive rate, 51,000 rows". A bare "AUC 0.78" invites misuse.

State limits as clearly as strengths. "Recall for customers over 70 is 0.28 against 0.62 overall. Do not use for outreach targeting of this group without human review" protects the people affected and the team that built the model.

Treat the card as code: a template with required sections, reviewed in the same pull request as the model change. A check in CI can refuse a card with missing sections.

Why it matters

A propensity model built for marketing emails was later reused by another team to prioritise collections calls. Nobody had written down that it was trained only on customers who had opted in to marketing, a group with very different payment behaviour. Collections priorities were wrong for months. An "Out-of-scope uses" line, "Not validated for credit or collections decisions", would have stopped the reuse at the first review.

Worked example: weak code

```
Churn model v4
AUC: 0.78
Owner: Sam
Works well, use for retention.
```

1. **Line 2.** No test period, prevalence or subgroup results.
2. **Line 3.** A person, not a team. Sam may leave.
3. **Line 4.** No limits, no out-of-scope uses, no monitoring.

Worked example: strong code

```
Model details: churn_model v4 (@champion), owner Retention Analytics, run 4da8..., trained 2026-09-01
Intended use: rank active customers for the monthly retention call list (top 500)
Out of scope: credit, collections or pricing decisions; customers with under 3 months' tenure
Training data: CRM extract Jan 2024 to Jun 2026, one row per customer-month
Evaluation: AUC 0.78, average precision 0.21 (prevalence 2.4%), test Oct to Dec 2026, 51,000 rows
Subgroups: recall at top 500 is 0.62 overall, 0.28 for customers over 70 (n positives = 64)
Limitations: sparse digital activity lowers recall for older customers; tenure proxies contract type
Monitoring: monthly AUC and calibration, alert below 0.72; next review 2027-03-01
```

1. **Line 3.** Stops reuse the model was never validated for.
2. **Line 5.** Each number with its period, baseline and size.
3. **Line 6.** The weakest group, stated with its count.

A model card states purpose, limits, data, honest metrics, subgroup results and monitoring, and is reviewed with every model change.

Recap card

- **Model card sections** Details, intended use, out-of-scope uses, training data, evaluation, subgroups, limitations, monitoring.
- **How to report a metric on a card** With test period, prevalence and row count.
- **Why out-of-scope uses** They stop reuse for decisions the model was never validated for.
- **Keep cards honest** Template with required sections, reviewed in the model's pull request, checked in CI.

Practice

Exercise 1. Multiple choice (level 1)

Which model card section would most directly have prevented a marketing model being reused for collections?

- a) Model details
- b) Out-of-scope uses
- c) Evaluation
- d) Monitoring

Why?

- r1) It names the uses the model was not built or validated for.
- r2) It is the longest section.

Exercise 2. Predict the output (level 2)

What does this print?

```
REQUIRED = ["intended_use", "out_of_scope", "evaluation", "subgroups",
    "limitations"]
card = {"intended_use": "retention list", "evaluation": "AUC 0.78",
    "limitations": ""}
print([s for s in REQUIRED if not str(card.get(s, "")).strip()])
```

Exercise 3. Fill the gap (level 2)

Complete the evaluation line so the metric has its context.

```
Evaluation: AUC 0.78, average precision 0.21 (_____(1) 2.4%), test Oct to
    Dec 2026, 51,000 _____(2)
```

Exercise 4. Write the function (level 3)

Write `missing_sections(card: dict[str, str], required: list[str]) -> list[str]`. Return the required section names, in the order of `required`, that

are absent from `card` or contain only whitespace. CI will fail the build when the list is not empty.

Your code must pass these checks:

- Absent sections must be listed.
- Sections with only whitespace count as missing.
- Keep the order of required.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Required model card sections that are absent or blank.
def missing_sections(
    card: dict[str, str],
    required: list[str],
) -> list[str]: ...
```

Starter code:

```python
def missing_sections(card, required):
    return [s for s in required if s not in card]
```

Exercise 5. Architecture design (level 4)

Write the subgroup and limitations parts of a card for a hardship-support model.

Options: Recall and false positive rate per age band, with counts, weakest group first, Overall recall only.

1. Subgroup results
2. Known weakness for customers over 70
3. Review

Sources

- The model card format comes from Mitchell et al. (2019), Model Cards for Model Reporting. (verified: Mitchell et al., Proceedings of FAT* 2019, pages 220 to 229.)

B18-L3 Decision logs, so the reasons outlive the meeting

Concept

Every analysis rests on choices: the weighting scheme, the outlier rule, the metric, the cut-off, the fairness property. Six months later someone asks

"why did we do it this way?". Without a record, the answer is a guess, and the choice gets reversed for the wrong reasons or kept for no reason.

A **decision log** is a list of short, dated entries, one per decision, kept in the repository (for example `docs/decisions/`). The format follows software architecture decision records:

```
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Context: Four respondents had weights above 12 and moved brand share by 4 points.
Options considered:
- No trimming
- Trim at 5, redistribute to keep the weighted total
- Rake with bounded weights
Decision: Trim at 5 with redistribution. Raking deferred until the new panel design.
Consequences: Design effect falls from 2.1 to 1.6. Small bias towards the sample profile. Review at wave 20.
```

Rules:

- **Write it when you decide**, in the same pull request as the code or config change.
- **List real options**, at least two, including "do nothing". A decision with one option was not a decision.
- **State the consequences**, including the costs you accept.
- **Never edit a past decision's reasoning.** When things change, add a new entry and mark the old one `superseded`, pointing to its replacement. The history of thinking is the value.
- **Link it**: from the config value it explains, from the model card (lesson B18-L2) and from the assumptions log (lesson B7-L3).

Keep entries short. Five to fifteen lines is enough for most analytical choices. A long, perfect document written after the fact helps less than a short one written on the day.

Why it matters

A new lead analyst removed the weight trimming from a tracker because it "looked arbitrary". The next wave's brand share jumped 5 points on four heavily weighted respondents, and the client asked why the trend broke. The original team had discussed exactly this case in a meeting two years earlier. Nothing was written down. A three-line decision entry linked from the config would have explained the rule in a minute.

Worked example: weak code

```
# config.yaml
weight_cap: 5   # don't change!!
```

1. **Line 2.** A warning with no reason invites either blind obedience or careless removal.

Worked example: strong code

```
# config.yaml
weight_cap: 5   # D-007, docs/decisions/D-007-trim-weights.md

# docs/decisions/D-007-trim-weights.md
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Options considered: no trimming; trim at 5 with redistribution; bounded
  raking.
Decision: trim at 5 with redistribution.
Consequences: design effect 2.1 to 1.6; review at wave 20.
```

1. **Line 2.** The value links to its reasons.
2. **Line 7.** Real alternatives, including doing nothing.
3. **Line 9.** The cost accepted, and when to revisit.

One short, dated entry per decision, with options, consequences and status, linked from the code it explains, and superseded rather than edited.

Recap card

- **Parts of a decision entry** ID and title, date and status, context, options considered, decision, consequences.
- **When a decision changes** Add a new entry. Mark the old one superseded and link to the new one.
- **Where and when to write** In the repository, in the same pull request as the change.
- **Make it findable** Link from the config value, the model card and the assumptions log.

Practice

Exercise 1. Multiple choice (level 1)

A decision from last year no longer holds. What should you do with its log entry?

- a) Edit it to describe the new decision.
- b) Delete it.

- c) Add a new entry, and mark the old one superseded with a link to the new one.
- d) Leave both without links.

Why?

- r1) The log keeps the history of reasoning. Each entry records what was known when it was written.
- r2) Logs must never shrink for storage reasons.

Exercise 2. Predict the output (level 2)

What does this print?

```
log = {
    "D-003": {"status": "superseded", "superseded_by": "D-007"},
    "D-007": {"status": "accepted"},
}
current = [k for k, v in log.items() if v["status"] == "accepted"]
print(current, log["D-003"]["superseded_by"] in log)
```

Exercise 3. Put in order (level 2)

Order the parts of a decision entry.

- Decision
- ID and title
- Consequences and review point
- Context: the problem and the evidence
- Date and status
- Options considered

Exercise 4. Write the function (level 3)

Write `decision_problems(entry: dict[str, object], log_ids: set[str]) -> list[str]` that checks one decision entry and returns problem messages, in this order:

1. `"status"` must be one of `"proposed"`, `"accepted"`, `"superseded"`. Otherwise add `"invalid status"`.
2. `"options"` must be a list with at least 2 items. Otherwise add `"fewer than two options"`.
3. `"consequences"` must be non-blank text. Otherwise add `"no consequences"`.
4. If status is `"superseded"`, `"superseded_by"` must be in `log_ids`. Otherwise add `"superseded without a valid replacement"`.

Your code must pass these checks:

- A complete accepted entry has no problems.
- Check status, options (at least 2) and non-blank consequences, one message each.
- A superseded entry must point to an ID that exists in the log.
- Missing keys count as problems, reported in the rule order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
VALID_STATUS = ...

# Problems with one decision log entry, in rule order.
def decision_problems(
    entry: dict[str, object],
    log_ids: set[str],
) -> list[str]: ...
```

Starter code:

```
def decision_problems(entry, log_ids):
    return []
```

Exercise 5. Architecture design (level 4)

Decide how the team handles decision records.

Options: docs/decisions/ in the repository, one file per decision, Slides from the steering meeting.

1. Where entries live
2. When an entry is written
3. How readers find it

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B0-L1 Python essentials for data work

B0-L1 exercise 1

Hints. (1) Which of these is also true for 0.0? (2) 0.0 is falsy. (3) Only one test checks for None itself.

Answer. b) `if share is None:` Because: r1) Only None is None. A real 0.0 share is a value, and `is None` leaves it alone.

`is None` is true only for a missing value. Truthiness lumps 0.0 in with None.

Common wrong answers.

- `a`: 0.0 is falsy, so `not share` is also true for a real 0% share.
- `c`: That catches a real 0.0 share, and comparing None with < would fail in the next option.
- `d`: `None < 0.01` raises a TypeError, and small real shares would count as missing.
- `b|r2`: Right condition, wrong reason. It is about meaning, not speed.

B0-L1 exercise 2

Hints. (1) `.items()` gives each key with its value. (2) Which weights are above 0? (3) The comprehension does not change the dict.

Answer.

```
1 ['R1', 'R3'] 3
```

The comprehension builds a new list of keys whose weight is positive. The dict itself still has three entries.

Common wrong answers.

- `['R1', 'R2', 'R3'] 3`: The comprehension keeps only IDs whose weight is above 0. R2 has 0.0.
- `['R1', 'R3'] 2`: `len(weights)` counts the original dict, which still has three entries.
- `[1.5, 2.0] 3`: The comprehension collects `rid`, the key, not `w`.

B0-L1 exercise 3

Hints. (1) Which value means missing? (2) A percentage format with one decimal. (3) `None` and `.1%`.

Answer.

```python
def label(name: str, share: float | None) -> str:
    if share is None:
        return f"{name}: no data"
    return f"{name}: {share:.1%}"
```

`float | None` documents that the share may be missing. `.1%` multiplies by 100, rounds to one decimal and adds `%`.

Common wrong answers.

- g1 = `None`; g2 = `.1f`: `.1f` prints 0.1. The `%` format multiplies by 100 and adds the sign.
- g1 = `0`; g2 = `.1%`: The hint names a type. A missing value has the type None.

B0-L1 exercise 4

Hints. (1) Loop with `for rid, w in weights.items():` and skip `w is None`. (2) Raise `ValueError(f"negative weight for {rid}")` before you keep a weight. (3) Compute the mean only when n > 0. Otherwise it is None.

Answer.

```python
def summarise_weights(weights: dict[str, float | None]) -> dict[str, float | int | None]:
    """Count, total and mean of the recorded weights.

    Args:
        weights: Respondent ID to weight, None when missing.

    Returns:
        A dict with n, total and mean. mean is None when no weight is recorded.

    Raises:
        ValueError: If any weight is negative.
    """
    recorded: list[float] = []
    for rid, w in weights.items():
        if w is None:
            continue
        if w < 0:
            raise ValueError(f"negative weight for {rid}: {w}")
        recorded.append(w)
    total = float(sum(recorded))
    n = len(recorded)
```

```
    mean = total / n if n > 0 else None
    return {"n": n, "total": total, "mean": mean}
```

Skipping only `None` keeps real zeros, checking n before dividing avoids a ZeroDivisionError, and an error message that names the ID points straight at the bad row.

B0-L1 exercise 5

Hints. (1) Which brands does the function count? (2) Is 0.0 truthy or falsy? (3) `if share is None:`.

Answer. The bug is on line 5.

Fixed version:

```
def count_missing(shares: dict[str, float | None]) -> int:
    """Number of brands with no share recorded."""
    missing = 0
    for brand, share in shares.items():
        if share is None:
            missing += 1
    return missing
```

`not share` is true for 0.0 as well as None. `share is None` counts only the missing value.

Common wrong answers.

- Line 4: The loop is fine: it visits every brand with its share.
- Line 6: Adding one is right. The condition that decides when is wrong.

B0-L2 pandas essentials

B0-L2 exercise 1

Hints. (1) Which operator works row by row on a Series? (2) `and` wants one True or False. (3) Brackets around each comparison.

Answer. a) `df[(df["w"] > 0) & df["in_scope"]]` Because: r1) `&` combines two masks row by row, and the brackets make the comparison happen first.

Masks combine with `&`, `|` and `~`. Bracket each comparison so it runs before the combination.

Common wrong answers.

- `b`: Python's `and` needs a single True or False. On a Series it raises "The truth value of a Series is ambiguous".
- `c`: A comma inside the brackets makes a tuple, which pandas reads as a column label, not a filter.

- `d`: DataFrame.filter selects columns by name. It does not filter rows by a condition.
- `a|r2`: Right line, wrong reason. `&` works row by row, which `and` cannot do.

B0-L2 exercise 2

Hints. (1) Group the rows by region, then add up w. (2) What does sum do with a missing value? (3) It skips it.

Answer.

```
1 {'N': 1.0, 'S': 2.0}
```

`sum` skips missing values, so region N totals 1.0 and S totals 2.0. `.to_dict()` turns the result into a dict keyed by region.

Common wrong answers.

- `{'N': nan, 'S': 2.0}`: pandas' sum skips missing values, so N adds up only 1.0.
- `{'N': 2, 'S': 1}`: That counts rows. sum adds the values.

B0-L2 exercise 3

Hints. (1) Which column do both tables have? (2) Which merge keeps every row of the left table? (3) `region_code` and `left`.

Answer.

```
1 joined = respondents.merge(regions, on="region_code", how="left")
```

`on` names the shared key. `how="left"` keeps every respondent and fills the region with a missing value when there is no match.

Common wrong answers.

- g1 = `region_code`; g2 = `inner`: An inner merge drops respondents with no match, which is what the task forbids.
- g1 = `region`; g2 = `left`: Join on the column both tables share, the code. The name is what you want to add.

B0-L2 exercise 4

Hints. (1) `df.groupby("region", as_index=False)` keeps region as a column. (2) `.agg(n=("w", "size"), recorded=("w", "count"), total_weight=("w", "sum"))`. (3) Finish with `.sort_values("region").reset_index(drop=True)`.

Answer.

```
1 import pandas as pd
2
3
```

```
def region_summary(df: pd.DataFrame) -> pd.DataFrame:
    """Rows, recorded weights and weight total per region.

    Args:
        df: Respondents with region and w columns.

    Returns:
        One row per region, sorted, with a default index.
    """
    return (
        df.groupby("region", as_index=False)
        .agg(n=("w", "size"), recorded=("w", "count"), total_weight=("w", "sum"))
        .sort_values("region")
        .reset_index(drop=True)
    )
```

`size` counts rows, `count` counts non-missing values and `sum` skips missing values. Named aggregations produce all three in one pass.

B0-L2 exercise 5

Hints. (1) Which runs first, `>` or `&`? (2) `&` binds tighter than `>`, so Python computes `0 & df["in_scope"]` first. (3) Bracket the comparison: `(df["w"] > 0) & df["in_scope"]`.

Answer. The bug is on line 6.

Fixed version:

```
import pandas as pd


def in_scope_positive(df: pd.DataFrame) -> pd.DataFrame:
    """Rows that are in scope and have a positive weight."""
    return df[(df["w"] > 0) & df["in_scope"]]
```

`&` binds more tightly than `>`, so the line compares `w` with `0 & in_scope`, which is all zeros. The in-scope condition disappears without an error. Brackets fix it.

Common wrong answers.

- Line 5: The docstring describes the right behaviour. The code does not match it.

B0-L3 The statistics you need for this track

B0-L3 exercise 1

Hints. (1) What does the calculation assume? (2) It assumes there is no real difference. (3) It measures how surprising the data would be then.

Answer. b) If the pages truly converted equally, a difference this large or larger would happen about 3% of the time. Because: r1) A p-value is computed assuming no real difference, and measures how surprising the data would be then.

p = 0.03 says the observed difference would be unusual if there were no real difference. It does not give the probability of any hypothesis or the size of the effect.

Common wrong answers.

- `a`: A p-value is not the probability of a hypothesis. It assumes no difference and asks how unusual the data are.
- `c`: The p-value says nothing about the size of the difference. Report the effect and its interval.
- `d`: There is no such certainty in a p-value.
- `b|r2`: Right statement, wrong reason. The p-value is about the data, given no difference, not about the hypothesis.

B0-L3 exercise 2

Hints. (1) Both compute a standard deviation. Do they divide by the same number? (2) NumPy divides by n. pandas divides by n - 1. (3) The sum of squared deviations is 32, and n is 8.

Answer.

```
1 2.0 2.138
```

The squared deviations sum to 32. NumPy divides by 8 (SD 2.0), pandas by 7 (SD 2.138). Pass `ddof=1` to NumPy for the sample SD.

Common wrong answers.

- `2.0 2.0`: pandas' std divides by n - 1 by default, the sample SD, so it is larger than NumPy's.
- `2.138 2.138`: NumPy's std divides by n by default. It gives 2.0 here.

B0-L3 exercise 3

Hints. (1) p(1 - p) divided by the sample size, then the square root. (2) The multiplier for 95%. (3) `n` and `1.96`.

Answer.

```
se = math.sqrt(p * (1 - p) / n)
low = p - 1.96 * se
```

The SE of a share is `sqrt(p(1 - p) / n)`. A 95% interval stretches 1.96 SEs either side of the estimate.

Common wrong answers.

- g1 = `n`; g2 = `2.58`: 2.58 gives a 99% interval. A 95% interval uses 1.96.
- g1 = `n - 1`; g2 = `1.96`: The SE of a share divides p(1 - p) by n.

B0-L3 exercise 4

Hints. (1) Handle `n == 0` first, then check `0 <= successes <= n`. (2) `se = math.sqrt(p * (1 - p) / n)`. (3) Clip with `max(0.0, low)` and `min(1.0, high)`.

Answer.

```
import math


def share_ci(successes: int, n: int, z: float = 1.96) -> tuple[float, float] | None:
    """Normal-approximation confidence interval for a share.

    Args:
        successes: Number of successes, from 0 to n.
        n: Sample size.
        z: Normal multiplier, 1.96 for 95%.

    Returns:
        (lower, upper) clipped to [0, 1], or None when n is 0.

    Raises:
        ValueError: If successes is outside 0 to n.
    """
    if n == 0:
        return None
    if not 0 <= successes <= n:
        raise ValueError(f"successes must be between 0 and {n}, got {successes}")
    p = successes / n
    se = math.sqrt(p * (1 - p) / n)
    return max(0.0, p - z * se), min(1.0, p + z * se)
```

Guarding n and the count first, then clipping, gives an interval that never crashes or leaves [0, 1]. Lesson B8-L2 replaces this approximation with the Wilson interval, which behaves better near 0 and 1.

B0-L3 exercise 5

Hints. (1) A mean or a share? (2) A range or a precision? (3) Significance is not size.

Answer.

- Standard error of a mean: SD / sqrt(n), here 4 / 10 = 0.4.
- Standard error of a share: sqrt(p(1 - p) / n), here about 0.016.
- A 95% interval: the estimate plus or minus 1.96 SE.
- Whether it matters: the size of the effect, which a tiny p-value in a big sample does not show.

Use the SE formula that matches the estimate, turn it into an interval for a range, and judge importance by the size of the effect.

Common wrong answers.

- matters = `interval`: An interval helps, but the question is whether the size of the change matters. Look at the effect itself.
- se_share = `sd_root_n`: A share is a mean of 0s and 1s, but its SD comes from p itself: use sqrt(p(1 - p) / n).

B1-L1 One pyproject.toml and a src layout

B1-L1 exercise 1

Hints. (1) Which table never ships in the built package? (2) It was added by PEP 735. (3) `[dependency-groups]`.

Answer. b) `[dependency-groups]` Because: r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.

`[dependency-groups]` holds development-only requirements. `pip install --group dev` (pip 25.1+) installs them.

Common wrong answers.

- `a`: `[project] dependencies` install for every user of the package, in production too.
- `c`: That table configures how pytest runs. It does not install anything.
- `d`: A comment installs nothing and drifts from reality.

- `b|r2`: Right table, wrong reason. Groups can hold any development tools: tests, linting, docs, notebooks.

B1-L1 exercise 2

Hints. (1) The runtime key is plural. (2) The development table has a hyphenated name. (3) `dependencies` and `dependency-groups`.

Answer.

```
[project]
name = "churn"
requires-python = ">=3.11"
dependencies = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]

[dependency-groups]
dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

`[project].dependencies` lists what users need to run the package. `[dependency-groups]` lists what developers need to work on it.

Common wrong answers.

- g1 = `requires`: The runtime list under [project] is called `dependencies`.
- g2 = `tool.dev`: `[tool.*]` tables configure tools. Development requirements go in `[dependency-groups]`.
- g2 = `optional-dependencies`: Optional dependencies (extras) ship with the package for users to opt into. Development-only tools belong in `[dependency-groups]`.

B1-L1 exercise 3

Hints. (1) `tomllib.loads(text)` returns a dict. (2) Use `.get` with defaults for the project table and the dependencies list. (3) Keep entries that contain none of = < > ~ !.

Answer.

```
import tomllib

CONSTRAINT_CHARS = set("=<>~!")


def unpinned(pyproject_text: str) -> list[str]:
    """Runtime dependencies with no version constraint.

    Args:
        pyproject_text: Contents of pyproject.toml.

    Returns:
        Sorted requirement strings without =, <, >, ~ or !.
```

```
    """
    project = tomllib.loads(pyproject_text).get("project", {})
    deps = project.get("dependencies", [])
    return sorted(d for d in deps if not CONSTRAINT_CHARS & set(d))
```

`tomllib` ships with Python 3.11+. Checking characters is enough to spot a missing constraint, and `.get` handles projects that declare no dependencies.

B1-L1 exercise 4

Hints. (1) Why can Python not see src/churn? (2) What makes a package importable everywhere in the environment? (3) An editable install.

Answer. b) Run `pip install -e .` in the project's virtual environment. Because: r1) The src layout is designed so that only the installed package is importable, which makes tests use it like any user would.

`pip install -e .` installs the package in editable mode, so imports resolve to `src/churn` and edits take effect without reinstalling.

Common wrong answers.

- `a`: Path hacks in every file defeat the point of the layout and break when files move.
- `c`: That removes the protection the layout gives: tests would import the working directory instead of the installed package.
- `d`: Copies drift. Tests must exercise the real package.
- `b|r2`: Right fix, wrong reason. pytest can import from folders. The src layout deliberately hides the package until it is installed.

B1-L1 exercise 5

Hints. (1) What must more than one program import? (2) What should CI run on every change? (3) What must never be cloned?

Answer.

- Shared logic in the package, imported by notebook and jobs.
- One-off exploration in notebooks, which import the package.
- Known-answer checks in tests/.
- Real data in governed storage, never in Git. Keep only small anonymised samples in the repository.

Package for shared logic, notebooks for exploration, tests for checks, governed storage for data.

Common wrong answers.

- features = `notebook`: The scoring job cannot import a notebook cell, so it would copy it, and the copies would drift.
- extract = `src`: Client data in the repository leaks with every clone. Keep it in governed storage.
- known_answer = `notebook`: A check in a notebook runs only when someone reruns the notebook. Put it in tests/ so CI runs it.

B1-L2 One virtual environment per project, and python -m pip

B1-L2 exercise 1

Hints. (1) How many Pythons might be on your PATH? (2) Which pip does `python -m pip` use? (3) The one belonging to that python.

Answer. b) It installs into the interpreter that `python` runs, whatever is first on PATH. Because: r1) `python -m pip` runs pip as a module of that exact interpreter.

`python -m pip` ties the install to the interpreter you will run. Bare `pip` is whichever pip script PATH finds first.

Common wrong answers.

- `a`: Speed is the same. The benefit is installing into the right interpreter.
- `c`: It installs into the current interpreter's environment, which inside a venv is the project only.
- `d`: Resolution is identical. Only the target interpreter is guaranteed.
- `b|r2`: Right answer, wrong reason. Bare pip still works. It may simply belong to a different Python.

B1-L2 exercise 2

Hints. (1) What does sys.prefix point to inside a venv? (2) And sys.base_prefix? (3) They are different folders.

Answer.

```
1 True
```

`sys.prefix` is the virtual environment, `sys.base_prefix` is the Python it was created from. They differ only inside a venv.

Common wrong answers.

- `False`: Inside a venv, sys.prefix points at the venv folder and sys.base_prefix at the base installation, so they differ.

B1-L2 exercise 3

Hints. (1) The standard library module that creates environments. (2) The flag for an editable install. (3) `venv` and `-e`.

Answer.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e . --group dev
```

`python -m venv .venv` creates the environment. `pip install -e .` installs the project so edits under src/ take effect immediately.

Common wrong answers.

- g1 = `virtualenv`: virtualenv is a separate package. The standard library module is `venv`.
- g2 = `-r`: `-r` reads a requirements file. Editable installs of the project use `-e .`.

B1-L2 exercise 4

Hints. (1) Inside a venv the two prefixes differ. (2) Raise when they are equal. (3) Read them inside the function body.

Answer.

```
import sys


def require_virtualenv() -> None:
    """Stop when not running inside a virtual environment.

    Raises:
        RuntimeError: If sys.prefix equals sys.base_prefix.
    """
    if sys.prefix == sys.base_prefix:
        raise RuntimeError("activate the project's virtual environment
    first")
```

Equal prefixes mean the system or base interpreter is running. Reading them at call time keeps the check correct in tests and in any process that changes them.

B1-L2 exercise 5

Hints. (1) Can one environment hold both constraints? (2) Where will the next developer look? (3) CI must match how each environment is used.

Answer.

- Two environments, because the constraint is real and tested.
- Two dependency groups in pyproject.toml, with the reason written next to the pin.
- Two CI jobs, so each environment is tested exactly as it is used.

When two tools conflict, give each its own pinned environment, record why in pyproject.toml, and test both in CI.

Common wrong answers.

- envs = `one_latest`: PySpark 4.0.4 fails with pandas 3.x: `assertDataFrameEqual` raises ImportError. One environment cannot satisfy both.
- envs = `one_old`: That works, but the analysis code then runs on an older pandas than intended. Separate environments keep each part current.
- ci = `one_job`: Installing both groups together recreates the conflict in CI.
- declare = `readme`: Chat messages are lost. Record the pin and its reason in pyproject.toml.

B1-L3 ruff and mypy in daily work

B1-L3 exercise 1

Hints. (1) How long is each list? (2) What does zip do when lengths differ? (3) It stops at the shortest.

Answer.

```
1 [(1, 2.0), (0, 1.0)]
```

`zip` stops at the shortest input, so the third respondent disappears. ruff's B905 asks for `strict=True`, which raises `ValueError` on unequal lengths (Python 3.10+).

Common wrong answers.

- `ValueError`: Only `zip(..., strict=True)` raises. Plain zip stops at the shorter input without a word.
- `[(1, 2.0), (0, 1.0), (1, None)]`: That is `itertools.zip_longest`. zip drops the extra element.

B1-L3 exercise 2

Hints. (1) When can share be None? (2) Is that case possible in real data? (3) Handle it.

Answer. b) Handle the None case before the multiplication. Because: r1) mypy found a real path where the value is None. Handling it removes the crash.

mypy found a reachable crash. Handle `None` explicitly, for example by returning `"n/a"`.

Common wrong answers.

- `a`: The ignore hides the warning and keeps the crash.
- `c`: The hint would then lie, and mypy would flag the `return None` instead. The crash stays.
- `d`: Weakening the checker hides this bug and every future one like it.
- `b|r2`: Right fix, wrong reason. This is a real crash on zero weight, not style.

B1-L3 exercise 3

Hints. (1) The flag that applies safe fixes. (2) The formatter subcommand. (3) The type checker's command.

Answer.

```
ruff check --fix .
ruff format .
mypy
```

`ruff check --fix .` applies safe fixes, `ruff format .` formats, and `mypy` type-checks with the settings in pyproject.toml.

Common wrong answers.

- g1 = `--unsafe-fixes`: Unsafe fixes can change behaviour. Use --fix for the safe ones, and review the rest by hand.
- g3 = `python -m pytest`: Tests matter, but the step after formatting in this routine is the type checker.

B1-L3 exercise 4

Hints. (1) Annotate: `flags: list[int], weights: list[float]`. (2) `zip(flags, weights, strict=True)`. (3) In report, store the share and return 'n/a' when it is None.

Answer.

```python
def weighted_share(flags: list[int], weights: list[float]) -> float | None:
    """Weighted share of flagged rows, or None when the total weight is 0."""
    total = sum(weights)
    if total == 0:
        return None
    return sum(w for f, w in zip(flags, weights, strict=True) if f == 1) / total


def report(flags: list[int], weights: list[float]) -> str:
    """Share as a percentage string, or 'n/a' when undefined."""
    share = weighted_share(flags, weights)
    if share is None:
        return "n/a"
    return f"{share * 100:.1f}%"
```

The types make the None case visible, `strict=True` stops silent truncation, and report handles None on purpose.

B1-L3 exercise 5

Hints. (1) Prefer real type information. (2) When there is none, narrow the exception. (3) Never global.

Answer.

- Stubs exist: install them in the dev group, and mypy checks your use of the library.
- No stubs: silence only that module with a targeted override.
- Never switch it off globally, or you lose the check for libraries that do ship types.

Install stubs where they exist, override per module where they do not, and keep the global check on.

Common wrong answers.

- stubs_exist = `override`: Stubs exist, so installing them gives you real checks for free.
- many = `global`: A global setting hides missing stubs you could have installed, for every library, forever.

B2-L1 Pure functions, side effects at the edges

B2-L1 exercise 1

Hints. (1) Does the result depend on anything except the arguments? (2) Does the function change anything outside itself? (3) Only one answers no to both.

Answer. b) `def share(n, total): return n / total if total else None` Because: r1) Its result depends only on its arguments, and it changes nothing outside itself.

`share` depends only on `n` and `total` and changes nothing. The others mutate, read the clock, or write a file.

Common wrong answers.

- `a`: append mutates the caller's list, a side effect.
- `c`: It reads the clock, so the same input gives a different result each call.
- `d`: Writing a file is a side effect. It belongs at the edge.
- `b|r2`: Right function, wrong reason. Purity is about inputs and side effects, not length.

B2-L1 exercise 2

Hints. (1) Is `df` a copy or the caller's object? (2) Does column assignment change the object it is called on? (3) It does.

Answer.

```
1 ['x', 'flag']
```

`df` is the same object as `original`. Assigning a column changes it, so the caller sees `flag` appear. Use `df.assign(flag=...)` to return a new frame.

Common wrong answers.

- `['x']`: Copy-on-Write protects against changes through derived objects, but `df["flag"] = ...` changes df itself, and df is the caller's object.

B2-L1 exercise 3

Hints. (1) Add a parameter for the year. (2) Use `df.assign(...)`, which returns a new frame. (3) `return df.assign(age=as_of_year - df["birth_year"])`.

Answer.

```
1 import pandas as pd
2
3
```

```
4 def add_age(df: pd.DataFrame, as_of_year: int) -> pd.DataFrame:
5     """Return a copy of df with age = as_of_year - birth_year."""
6     return df.assign(age=as_of_year - df["birth_year"])
```

The year is an input, so tests fix it. `assign` returns a new frame, so callers keep their data.

B2-L1 exercise 4

Hints. (1) When is the default list created? (2) Is it shared between calls? (3) Build a fresh list inside the function.

Answer. The bug is on line 1.

Fixed version:

```
1 def flag_low_bases(bases: dict[str, float], minimum: float) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     return [name for name, base in bases.items() if base < minimum]
```

The default list is created once and shared by every call, a hidden state that makes the function impure. Build the list inside, here with a comprehension.

Common wrong answers.

- Line 5: append is where the state leaks, but the shared list is created on line 1.

B2-L1 exercise 5

Hints. (1) Does the step touch anything outside Python's memory? (2) Does its result depend only on its arguments? (3) I/O at the edges, logic in the core.

Answer.

- Reading and writing, logging and choosing the reference date: edge.
- Features and scoring, given all inputs as arguments: pure core.

A thin edge does I/O. A pure core does the thinking and gets the tests.

Common wrong answers.

- read = `core`: Reading storage depends on the world outside the function. Keep it at the edge.
- write = `core`: Writing and logging are side effects. Keep them at the edge.
- features = `edge`: Feature logic given its inputs is deterministic. Make it pure so it can be tested and reused.

B2-L2 Dataclasses as configuration objects

B2-L2 exercise 1

Hints. (1) What does frozen=True promise? (2) Is the failure silent or loud? (3) The exception is a subclass of AttributeError.

Answer. b) Raises dataclasses.FrozenInstanceError. Because: r1) frozen=True makes the generated **setattr** raise on every field assignment.

Frozen dataclasses raise `dataclasses.FrozenInstanceError` on assignment. Build a changed copy with `dataclasses.replace`.

Common wrong answers.

- `a`: That is the behaviour without frozen=True. Frozen instances reject assignment.
- `c`: Assignment never returns a copy. Use dataclasses.replace(cfg, min_base=30) for that.
- `d`: It fails loudly, which is the point: settings cannot drift during a run.
- `b|r2`: Right answer, wrong reason. Without frozen=True every field, int or not, accepts assignment.

B2-L2 exercise 2

Hints. (1) Does a dataclass check or convert types? (2) What does the generated **repr** show for a string? (3) It uses repr() of each value, so strings keep their quotes.

Answer.

```
1 Settings(min_base='30')
```

The hint `int` is for your editor and mypy. At runtime the dataclass stores `'30'`. Check or convert in `__post_init__`.

Common wrong answers.

- `Settings(min_base=30)`: Dataclasses do not convert values. The string '30' is stored as it is.
- `TypeError`: Type hints are not checked at runtime. The call succeeds and stores a string.

B2-L2 exercise 3

Hints. (1) A plain `= []` raises ValueError at class definition. (2) You need a function that builds a fresh list per instance. (3) The keyword is `default_factory`.

Answer.

```python
from dataclasses import dataclass, field


@dataclass
class Run:
    waves: list[str] = field(default_factory=list)
```

`field(default_factory=list)` calls `list()` for each new instance, so no two instances share one list.

Common wrong answers.

- g1 = `default`: `field(default=list)` stores the list class itself, not a new list.

B2-L2 exercise 4

Hints. (1) `ReportConfig(**raw)` passes each key as a keyword argument. (2) The generated **init** raises TypeError on an unknown or missing keyword. (3) Check `isinstance(self.min_base, int)` and `self.min_base > 0` in **post_init**.

Answer.

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class ReportConfig:
    """Settings for one reporting run."""

    weight_col: str
    min_base: int = 50

    def __post_init__(self) -> None:
        if not isinstance(self.min_base, int) or self.min_base <= 0:
            raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")


def load_config(raw: dict[str, object]) -> ReportConfig:
    """Build a ReportConfig from a parsed YAML or JSON mapping.

    Args:
        raw: Keys must match ReportConfig fields.

    Returns:
        A validated, frozen config.

    Raises:
        TypeError: On an unknown or missing key.
        ValueError: On an invalid min_base.
```

```
    """
    return ReportConfig(**raw)  # type: ignore[arg-type]
```

Unpacking the dict into the constructor lets the generated `__init__` reject unknown and missing keys. `__post_init__` catches bad values. `frozen=True` stops later edits.

B2-L2 exercise 5

Hints. (1) Which shape rejects a misspelt key? (2) When would you rather see a bad value: now or in hour three? (3) Globals make functions impure (lesson B2-L1).

Answer.

- A frozen dataclass: named fields, unknown keys rejected, no edits during the run.
- Check once at load time in **post_init**, so a bad file fails before any work starts.
- Pass the object as a parameter, so tests build their own config.

Build a frozen, validated object at the edge and pass it into the pure core.

Common wrong answers.

- shape = `dict`: A dict accepts any key, including typos, and any function can change it.
- shape = `globals`: Reading a file at import is a module-level side effect, and every test shares the same values.
- when = `use`: Checks spread across functions fail late, after hours of work, and drift apart.
- pass = `import`: A shared import is a global. Tests cannot give one function a different config.

B2-L3 When a class earns its place

B2-L3 exercise 1

Hints. (1) Which task has two phases? (2) What must survive between those phases? (3) Learned frequencies must be stored between fit and transform.

Answer. b) Learn category frequencies on training data and encode test data with them. Because: r1) State learned in one call must be reused in a later call.

Fit then transform needs stored state, the classic case for a class, as in scikit-learn's encoders.

Common wrong answers.

- `a`: One step with no state. A function taking the mapping is enough.
- `c`: One step with no state. A function is enough.
- `d`: Settings with no behaviour belong in a dataclass, not a full class.
- `b|r2`: Right answer, wrong reason. Size does not decide it. State across calls does.

B2-L3 exercise 2

Hints. (1) What is in self.seen after the first call? (2) Which of 2, 3 and 4 is not in it? (3) Only 4.

Answer.

```
1 [4]
```

The object keeps state between calls. Reuse it on a new wave and it drops rows the new wave needs.

Common wrong answers.

- `[2, 3, 4]`: self.seen still holds 1, 2 and 3 from the first call, so 2 and 3 are dropped.

B2-L3 exercise 3

Hints. (1) Move self.col into a parameter. (2) Compute the total once and test it for zero. (3) `df.assign(share=0.0)` when the total is zero.

Answer.

```
1 import pandas as pd
2
3
4 def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame:
5
    """Return df with share = col / column total, or 0.0 when the total is
    zero."""
6     total = df[col].sum()
7     if total == 0:
8         return df.assign(share=0.0)
9     return df.assign(share=df[col] / total)
```

The class held one value and did one thing. A function with two parameters is shorter, and the zero guard stops a NaN column.

B2-L3 exercise 4

Hints. (1) Which state should transform read? (2) Does transform change that state? (3) Refitting inside transform discards what fit learned.

Answer. The bug is on line 11.

Fixed version:

```
1  class Scaler:
2      """Standardise with the mean and sd learned in fit."""
3
4      def fit(self, values: list[float]) -> "Scaler":
5          n = len(values)
6          self.mean_ = sum(values) / n
7          self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8          return self
9
10     def transform(self, values: list[float]) -> list[float]:
11         return [(v - self.mean_) / self.sd_ for v in values]
```

Refitting in `transform` throws away the learned state, the only reason the class exists. It also leaks test data into the scaling.

Common wrong answers.

- Line 6: The mean formula is correct. The problem is when it is recomputed.
- Line 12: The scaling formula is correct. It uses values that were just overwritten on line 11.

B2-L3 exercise 5

Hints. (1) Does it hold data, do something, or both? (2) Do several operations share the same data? (3) Delete the class: what breaks?

Answer.

- Recoding is one stateless step: a function.
- Settings with no behaviour: a frozen dataclass.
- A survey design shares the same data across several operations and must keep weights valid: a class.

Match the form to the need: function for a step, dataclass for settings, class for shared state with behaviour.

Common wrong answers.

- recode = `class`: One step, no state that outlives the call. A class adds construction with no benefit.
- settings = `class`: No behaviour to attach. A frozen dataclass gives fields, equality and immutability for free.
- design = `function`: Every function would need weights, strata and clusters passed again. A class keeps them together and valid.

B3-L1 Dtypes in pandas 3, nullable and string

B3-L1 exercise 1

Hints. (1) Can NumPy's int64 hold NaN? (2) What does pandas use for NaN by default? (3) A float.

Answer. c) float64 Because: r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.

NaN is a float, so a missing value upcasts integers to `float64`. Ask for `Int64` to keep integers.

Common wrong answers.

- `a`: int64 has no way to represent a missing value.
- `b`: Int64 is used only when you ask for it, for example with dtype="Int64" or convert_dtypes().
- `d`: Numbers with None infer a numeric dtype, not object.
- `c|r2`: Right answer, wrong reason. pd.Series([4, 2]) is int64. The None forces the float.

B3-L1 exercise 2

Hints. (1) What dtype does read_csv infer for 01234? (2) What does the blank row add? (3) A number with a missing value is float64.

Answer.

```
1 1234.0
```

`read_csv` reads `01234` as a number and the blank as NaN, so the column is `float64` and the leading zero is gone. Pass `dtype={"postcode": "str"}`.

Common wrong answers.

- `01234`: Without a dtype map, read_csv infers a number. The blank row makes it float64.
- `1234`: The blank row forces float64, so the value prints with .0.

B3-L1 exercise 3

Hints. (1) IDs are labels, not quantities. (2) Which integer dtype holds pd.NA? (3) `str` and `Int64`.

Answer.

```
1 df = pd.read_csv(path, dtype={"respondent_id": "str", "household_size":
    "Int64"})
```

`str` keeps IDs as written. `Int64` is the nullable integer dtype.

Common wrong answers.

- g2 = `int64`: Lower-case int64 is NumPy's integer and fails on a blank value. Use capital-I Int64.
- g1 = `int`: IDs are labels. Reading them as numbers drops leading zeros.

B3-L1 exercise 4

Hints. (1) read_csv takes a dtype mapping. (2) IDs as str, counts as Int64. (3) `pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)`.

Answer.

```
1  import io
2
3  import pandas as pd
4
5  PANEL_DTYPES: dict[str, str] = {"respondent_id": "str", "household_size":
     "Int64"}
6
7
8  def read_panel(text: str) -> pd.DataFrame:
9      """Read panel CSV text with explicit dtypes.
10
11     Args:
12         text: CSV content with respondent_id and household_size.
13
14     Returns:
15         A frame with str IDs and nullable integer counts.
16     """
17     return pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)
```

The dtype map fixes both columns at load time. In a real pipeline it comes from config, not code.

B3-L1 exercise 5

Hints. (1) What does mean() do with <NA>? (2) What does it do with 0? (3) Filling blanks with 0 adds fake answers.

Answer. The bug is on line 7.

Fixed version:

```
1  import pandas as pd
2
3
4  def mean_household(df: pd.DataFrame) -> float:
5      """Mean household size among respondents who answered."""
6      sizes = df["household_size"].astype("Int64")
7      return float(sizes.mean())
```

`mean()` skips `<NA>` by default. Filling with 0 turns non-answers into households of size zero and pulls the mean down.

Common wrong answers.

- Line 6: Int64 is right: it keeps blanks as <NA>, which mean() skips. The problem is what happens next.
- Line 8: mean() is fine on its own. It skips <NA>, but by then the blanks are zeros.

B3-L2 Vectorise instead of apply

B3-L2 exercise 1

Hints. (1) Look for the option with no per-row Python code. (2) A vectorised call takes whole columns as arguments. (3) `np.where(cond, a, b)` picks from `a` or `b` element by element in compiled code.

Answer. c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)` Because: r1) It evaluates the condition and the choice on whole arrays in compiled code.

`np.where` takes whole arrays and runs one compiled loop. The other three options call Python code once per row.

Common wrong answers.

- `a`: `apply(axis=1)` calls the lambda once per row in Python. It measured about 500 times slower than `np.where` on one million rows.
- `b`: A list comprehension beats `apply`, but it still runs a Python loop over every value.
- `d`: `iterrows()` builds a Series for each row. It is the slowest option here.
- `c|r2`: Right choice, wrong reason. Length has nothing to do with speed. `np.where` works on whole arrays in compiled code.
- `c|r3`: Right choice, wrong reason. There is no lambda, and `np.where` runs on one core. It is fast because it avoids a Python call per row.

B3-L2 exercise 2

Hints. (1) `None` in a float Series becomes `NaN`. (2) What does any comparison with `NaN` return? (3) Every comparison with `NaN` is `False`.

Answer.

```
1 [False, True, False]
```

`None` becomes `NaN` in a float64 Series, and `NaN > 1` is `False`. So the missing value is treated as "not above 1". This is the missing-value trap from the concept.

Common wrong answers.

- `[False, True, None]`: A comparison on a float Series returns plain booleans. `NaN > 1` evaluates to `False`, not to a missing value.
- `[False, True, nan]`: Comparisons do not propagate `NaN` on float64 data. They return `False`, which is why missing values get silently mislabelled.
- `[True, True, False]`: `0.0 > 1` is `False`. Only 1.5 exceeds 1.

B3-L2 exercise 3

Hints. (1) The function takes a condition, a value for True and a value for False. (2) Its name is the SQL keyword for filtering rows. (3) `np.where(cond, value_if_true, value_if_false)`, with zero as the false value.

Answer.

```
df["weighted_reach"] = np.where(df["reach_flag"] == 1,
    df["final_weight"], 0.0)
```

`np.where(cond, df["final_weight"], 0.0)` keeps the weight where the flag is 1 and returns 0.0 elsewhere, as one float array.

Common wrong answers.

- g1 = `select`: `np.select` takes a list of conditions and a list of choices. For one condition and two outcomes, use `np.where`.
- g1 = `if`: There is no `np.if`. The vectorised if-else in NumPy is `np.where`.
- g2 = `None`: `None` turns the result into an object array. Use `0.0` so the column stays numeric.

B3-L2 exercise 4

Hints. (1) Compute the total weight first, and return early when it is zero. (2) `df['final_weight'].where(df['reach_flag'] == 1, 0.0)` keeps weights for reached rows only. (3) Sum the masked weights, divide by the total, and wrap the result in `float()`.

Answer.

```
import numpy as np
import pandas as pd


def weighted_reach_pct(df: pd.DataFrame) -> float | None:
    """Weighted share of respondents with reach_flag == 1.

```

```
    Args:
        df: Rows with reach_flag and final_weight.

    Returns:
        The weighted share, or None when the total weight is zero.
    """
    total = df["final_weight"].sum()
    if total == 0:
        return None
    reached = df["final_weight"].where(df["reach_flag"] == 1, 0.0).sum()
    return float(reached / total)
```

Sum the weights once for the denominator and return `None` when it is zero, which covers the empty frame too. Mask weights by the flag, sum, divide and convert to `float`.

B3-L2 exercise 5

Hints. (1) `pd.cut` turns numbers into labelled bands and keeps missing values missing. (2) Bins `[-np.inf, 35, 55, np.inf]` with `right=False` give [.., 35), [35, 55), [55, ..). (3) Wrap it in `df.assign(age_group=pd.cut(...))` so you return a new frame.

Answer.

```
import numpy as np
import pandas as pd

AGE_BINS = [-np.inf, 35, 55, np.inf]
AGE_LABELS = ["16-34", "35-54", "55+"]


def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
    """Return a copy of df with an age_group column. Missing ages stay missing."""
    return df.assign(
        age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS, right=False)
    )
```

`pd.cut` with left-closed bins (`right=False`) matches the original `<` comparisons exactly. It keeps `NaN` as missing, which fixes a bug the original had. `assign` returns a new frame.

B3-L2 exercise 6

Hints. (1) What is `NaN >= 60`? (2) `np.where` has only two outcomes, so a missing value gets one of them. (3) Build the band as a Series, then keep it only where `minutes.notna()`.

Answer. The bug is on line 4.

Fixed version:

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
    minutes stay missing."""
3     minutes = df["minutes"]
4     band = pd.Series(np.where(minutes >= 60, "heavy", "light"),
    index=df.index)
5     return df.assign(viewing_band=band.where(minutes.notna()))
```

`NaN >= 60` is `False`, so `np.where` returns `"light"` for missing minutes. Wrap the result in a Series on the same index and apply `.where(minutes.notna())` to restore the missing values.

Common wrong answers.

- Line 3: Line 3 only selects the column. The missing value is lost where the condition is evaluated.
- Line 5: `assign` is right. It writes whatever `band` holds, and `band` is already wrong.

B3-L3 Method chaining and Copy-on-Write

B3-L3 exercise 1

Hints. (1) What does df[mask] return under Copy-on-Write? (2) Where does the second step write? (3) To a copy that is thrown away.

Answer. b) Nothing. pandas emits a ChainedAssignmentError warning. Because: r1) The first indexing step returns a new object, so the write lands on a temporary copy.

Two indexing steps mean two objects. The write hits the temporary one. Write in one step with `.loc`, or use `assign`.

Common wrong answers.

- `a`: Under Copy-on-Write the assignment goes to a temporary copy. Use df.loc[mask, "weight"] = 0.
- `c`: ChainedAssignmentError is a warning, not an exception. The code keeps running with df unchanged.
- `d`: The filter only selects rows. And the write never reaches df anyway.
- `b|r2`: Right answer, wrong reason. df.loc[mask, col] = value still changes df.

B3-L3 exercise 2

Hints. (1) Is s a view you can write through? (2) When does Copy-on-Write copy? (3) On the first write to shared data.

Answer.

```
[1, 2, 3]
```

`s` shares memory with `df` until you write to it. The write triggers a copy, so `df` keeps its values.

Common wrong answers.

- `[100, 2, 3]`: Under Copy-on-Write, s behaves as a separate object. Writing to it copies the data first, so df is unchanged.

B3-L3 exercise 3

Hints. (1) Which column names does each step need? (2) Which step depends on which rows are present? (3) Rename, filter, normalise, band.

Answer.

1. .rename(columns={"wt": "weight"})
2. .loc[lambda d: d["age"] >= 18]
3. .assign(weight=lambda d: d["weight"] / d["weight"].mean())
4. .pipe(add_age_band, bins=BINS, labels=LABELS)

Each step sees the output of the step above. Rename first so later steps use canonical names. A step that aggregates, such as a mean, must follow the filter that defines its population.

Common wrong answers.

- The normalising step reads the weight column, which does not exist until the rename.
- Normalising before the filter averages over minors too, so adult weights no longer average 1.0.

B3-L3 exercise 4

Hints. (1) Filter with .loc[lambda d: ...]. (2) Cap with Series.clip(upper=cap). (3) Join them: `df.loc[...].assign(weight=lambda d: d["weight"].clip(upper=cap))`.

Answer.

```python
import pandas as pd


def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
```

```
    """Keep adults and cap their weights at cap."""
    return df.loc[lambda d: d["age"] >= 18].assign(
        weight=lambda d: d["weight"].clip(upper=cap)
    )
```

`loc` then `assign` builds a new frame in two steps, with no temporary copies to write to.

B3-L3 exercise 5

Hints. (1) How many indexing steps does line 7 take before the =? (2) Where does the value land? (3) Use .loc in one step, or Series.where inside assign.

Answer. The bug is on line 7.

Fixed version:

```
import pandas as pd


def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
    """Set weight to 0 where valid is False."""
    return df.assign(weight=df["weight"].where(df["valid"], 0.0))
```

`out[mask]["weight"] = 0.0` writes to a temporary copy. `df.assign(weight=df["weight"].where(df["valid"], 0.0))` keeps weights where valid and puts 0.0 elsewhere.

Common wrong answers.

- Line 6: copy() is harmless. Under Copy-on-Write it is also cheap until a write.
- Line 8: Returning out is fine. out never received the zeros.

B4-L1 Polars expressions and lazy frames

B4-L1 exercise 1

Hints. (1) scan_ versus read_. (2) When does a LazyFrame run? (3) On collect().

Answer. b) Nothing is read yet. lf holds a query plan. Because: r1) A LazyFrame records steps and runs them only on collect().

`scan_parquet` builds a plan. `collect()` optimises and runs it, pushing the filter to the reader.

Common wrong answers.

- `a`: That is what pl.read_parquet followed by filter does. scan_parquet is lazy.
- `c`: Close, but no rows are read until collect(). Then only w9 rows are read.
- `d`: Nothing is written. A LazyFrame is a plan in memory.
- `b|r2`: Right answer, wrong reason. Polars reads Parquet itself.

B4-L1 exercise 2

Hints. (1) Which values are in group w2? (2) What is the sum of no non-null values? (3) Polars returns the additive identity.

Answer.

```
1 [3.0, 0.0]
```

The sum of an empty set of values is 0 in Polars. A real zero and "no data" look the same, so report `null_count()` alongside totals.

Common wrong answers.

- `[3.0, None]`: Polars sums an all-null group to 0, not null. Check null_count() when that matters.
- `[3.0]`: Every group appears in the output, including w2.

B4-L1 exercise 3

Hints. (1) A window over groups that keeps every row. (2) The method that runs a LazyFrame. (3) `over` and `collect`.

Answer.

```
1 out = (
2     lf.with_columns(share=pl.col("weight") /
    pl.col("weight").sum().over("wave"))
3     .collect()
4 )
```

`.over("wave")` broadcasts the wave total to each row. `collect()` runs the plan.

Common wrong answers.

- g1 = `group_by`: group_by collapses rows. over computes the group total and keeps every row.
- g2 = `execute`: A LazyFrame runs with collect().

B4-L1 exercise 4

Hints. (1) Aggregate to wave and brand first. (2) Get the wave total with `.sum().over("wave")`. (3) Guard with `pl.when(total > 0).then(...).otherwise(None)`.

Answer.

```python
import polars as pl


def brand_share(df: pl.DataFrame) -> pl.DataFrame:
    """Weighted share of each brand within its wave. Grain: one row per wave and brand.

    Args:
        df: Columns wave, brand, weight.

    Returns:
        Columns wave, brand, share. share is null when the wave total is 0.
    """
    return (
        df.group_by("wave", "brand")
        .agg(brand_weight=pl.col("weight").sum())
        .with_columns(total=pl.col("brand_weight").sum().over("wave"))
        .with_columns(
            share=pl.when(pl.col("total") > 0)
            .then(pl.col("brand_weight") / pl.col("total"))
            .otherwise(None)
        )
        .select("wave", "brand", "share")
        .sort("wave", "brand")
    )
```

Aggregate to the output grain, broadcast the wave total with `over`, and guard the division so a zero total gives null instead of NaN.

B4-L1 exercise 5

Hints. (1) What does pl.len() count? (2) Does it count rows with a null score? (3) mean() skips nulls.

Answer. The bug is on line 8.

Fixed version:

```python
import polars as pl


def mean_score(df: pl.DataFrame) -> pl.DataFrame:
    """Mean score per region among respondents who answered."""
    return (
        df.group_by("region")
```

```
        .agg(mean_score=pl.col("score").mean())
        .sort("region")
    )
```

`pl.len()` counts every row, including skipped answers, so the denominator is too big. `pl.col("score").mean()` divides by the non-null count.

Common wrong answers.

- Line 7: Grouping by region is right. The formula inside agg is the problem.
- Line 9: Sorting does not change values.

B4-L2 DuckDB, SQL over files and DataFrames

B4-L2 exercise 1

Hints. (1) Which options build a new SQL string from the value? (2) Which one keeps the SQL fixed? (3) The placeholder.

Answer. b) `con.execute("... WHERE region = ?", [region])` Because: r1) The value is sent as data, separate from the SQL text.

A `?` placeholder keeps the SQL text fixed and passes the value as data.

Common wrong answers.

- `a`: An f-string pastes the value into the SQL. A quote in the value changes the query.
- `c`: Concatenation pastes the value into the SQL, the same risk as an f-string.
- `d`: % formatting is string pasting too, not a parameter.
- `b|r2`: Right answer, wrong reason. Safety comes from sending the value separately.

B4-L2 exercise 2

Hints. (1) How many rows does an aggregate without GROUP BY return? (2) What is SQL sum over no rows? (3) NULL, returned as None inside a tuple.

Answer.

```
(None,)
```

One row, holding NULL. Check `row[0] is None` before using the value.

Common wrong answers.

- `(0.0,)`: SQL sum over zero rows is NULL, which DuckDB returns as None.
- `None`: An aggregate without GROUP BY always returns one row. fetchone() gives a tuple holding None.

B4-L2 exercise 3

Hints. (1) The DuckDB table function for Parquet files. (2) The positional placeholder. (3) `read_parquet` and `?`.

Answer.

```
con.execute(
    "SELECT count(*) FROM read_parquet('extracts/*.parquet') WHERE wave
    = ?",
    [wave],
).fetchone()
```

`read_parquet` with a glob reads all files as one table. `?` binds the first value in the list.

Common wrong answers.

- g2 = `'{wave}'`: That is f-string pasting, and the string is not even an f-string here. Use a ? placeholder.
- g1 = `read_csv`: The files are Parquet. read_csv would fail to parse them.

B4-L2 exercise 4

Hints. (1) Replace the f-string with a ? placeholder and a list of values. (2) What does sum return when no rows match? (3) Return 0.0 when `row[0] is None`, else `float(row[0])`.

Answer.

```
import duckdb

REGION_TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE region = ?"


def region_total(con: duckdb.DuckDBPyConnection, region: str) -> float:
    """Total weight for one region.

    Args:
        con: Connection holding a responses table.
        region: Region name, passed as a query parameter.

    Returns:
        The weight total, or 0.0 when no rows match.
    """
    row = con.execute(REGION_TOTAL_SQL, [region]).fetchone()
    if row is None or row[0] is None:
        return 0.0
    return float(row[0])
```

The placeholder keeps the SQL fixed, so quotes in the value are just characters. The None check turns SQL's NULL into the 0.0 the caller expects.

B4-L2 exercise 5

Hints. (1) Can a placeholder stand for a table name? (2) What stops arbitrary text becoming SQL? (3) Globals make testing hard (lesson B2-L1).

Answer.

- Values such as the wave go in as parameters.
- Identifiers such as table names cannot be parameters. Check them against an allowlist before building the SQL.
- Pass one connection in, so tests supply an in-memory database.

Parameters for values, an allowlist for identifiers, and a connection passed in.

Common wrong answers.

- wave = `fstring`: A quote in the wave changes the SQL. Use a parameter.
- table = `param`: Placeholders bind values, not identifiers. `FROM ?` is not valid SQL for a table name.
- table = `raw`: Any text the user types becomes SQL. Check it against the allowed tables first.
- con = `default`: The default connection is a shared global. Tests cannot swap in their own.

B4-L3 Choosing pandas, Polars, DuckDB or Spark

B4-L3 exercise 1

Hints. (1) Can the whole dataset sit in RAM? (2) Which tools aggregate while reading? (3) Scan-based engines on one machine.

Answer. b) DuckDB or Polars lazy, aggregating while reading Because: r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.

DuckDB and Polars lazy read in chunks and aggregate as they go. The result is small.

Common wrong answers.

- `a`: pandas loads the whole selection into memory first. 40 GB does not fit in 16 GB.
- `c`: A cluster works, but a one-off aggregate on one machine does not need one.
- `d`: Manual chunking re-implements what DuckDB and Polars do for you, with more room for mistakes.

- `b|r2`: Right answer, wrong reason. Age is not the point. Out-of-memory scanning is.

B4-L3 exercise 2

Hints. (1) Where is respondent_id after set_index? (2) Does Polars have an index? (3) The default is include_index=False.

Answer.

```
['score']
```

The ID lives in the pandas index, and `from_pandas` drops the index by default. Call `reset_index()` first.

Common wrong answers.

- `['respondent_id', 'score']`: Polars has no index. from_pandas drops it unless you pass include_index=True.

B4-L3 exercise 3

Hints. (1) DuckDB relations have .df(), .pl() and .fetchall(). (2) Polars converts to pandas with a to_ method. (3) `pl` and `to_pandas`.

Answer.

```
top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand").pl()
for_model = top_brands.to_pandas()
```

`.pl()` returns a Polars DataFrame from DuckDB. `.to_pandas()` converts Polars to pandas through Arrow.

Common wrong answers.

- g1 = `df`: .df() returns pandas. The first step asks for Polars.
- g2 = `df`: Polars DataFrames convert with to_pandas().

B4-L3 exercise 4

Hints. (1) How much data exists after line 7? (2) Where should the filter run? (3) Scan, filter, aggregate, collect.

Answer. The bug is on line 7.

Fixed version:

```
import polars as pl


def wave_mean(path: str, wave: str) -> float:
    """Mean weight for one wave."""
    result = (
```

```
7         pl.scan_parquet(path)
8         .filter(pl.col("wave") == wave)
9         .select(pl.col("weight").mean())
10        .collect()
11        .item()
12    )
13    return float(result)
```

Line 7 reads every wave eagerly, then copies it into pandas. A lazy scan pushes the wave filter into the reader and returns one number.

Common wrong answers.

- Line 8: The filter is correct, but by then every wave is already in memory twice.

B4-L3 exercise 5

Hints. (1) Where does the data live? (2) How big is it next to memory? (3) Which library does the next step need?

Answer.

- Small data feeding the Python modelling stack: pandas.
- SQL across many local files: DuckDB.
- Data already in the lakehouse, scheduled: Spark.
- Large local files with a transform pipeline: Polars lazy.

Size against memory, location of the data, and the next library in line decide the engine.

Common wrong answers.

- model = `spark`: 50,000 rows fit in memory many times over. Spark adds start-up time and a cluster for no gain.
- nightly = `pandas`: Pulling lakehouse tables into one pandas process moves the data away from the compute that already sits next to it.
- transform = `pandas`: pandas would load most of 25 GB into 32 GB of RAM. A lazy scan keeps memory in check.

B5-L1 DataFrame schemas with pandera

B5-L1 exercise 1

Hints. (1) Without lazy, when does validation stop? (2) What does the exception hold with lazy? (3) failure_cases, one row per failure.

Answer. b) Every failure is collected and raised together as SchemaErrors. Because: r1) You see all problems in one run, with failure_cases listing each one.

Without `lazy`, the first failing check raises `SchemaError`. With it, pandera runs every check and raises `SchemaErrors` with all failures.

Common wrong answers.

- `a`: Validation runs at once. lazy refers to collecting errors, not delaying them.
- `c`: pandera raises. Dropping rows needs an explicit choice, not a flag on validate.
- `d`: All checks run. lazy changes how failures are reported.
- `b|r2`: Right answer, wrong reason. lazy runs every check, so it is not faster.

B5-L1 exercise 2

Hints. (1) Which values fail gt(0)? (2) Does lazy stop at the first? (3) All failing values are listed.

Answer.

```
[-2.0, -3.0]
```

Both negative weights fail. `lazy=True` lists both in `failure_cases`.

Common wrong answers.

- `[-2.0]`: With lazy=True pandera reports every failing value, not only the first.
- `[1, 2]`: failure_case holds the failing values. The row positions are in the index column.

B5-L1 exercise 3

Hints. (1) A Column argument about repeated values. (2) A schema argument about extra columns. (3) `unique` and `strict`.

Answer.

```
schema = pa.DataFrameSchema(
    {"respondent_id": pa.Column(str, unique=True), "score":
    pa.Column(int)},
    strict=True,
)
```

`unique=True` enforces the grain. `strict=True` fails on any unexpected column.

Common wrong answers.

- g2 = `coerce`: coerce converts dtypes. strict rejects unknown columns.
- g1 = `nullable`: nullable allows nulls. unique rejects duplicates.

B5-L1 exercise 4

Hints. (1) Each column is `pa.Column(dtype, checks..., options)`. (2) Columns reject nulls unless nullable=True. (3) strict and coerce go on the DataFrameSchema, lazy on validate.

Answer.

```python
import pandas as pd
import pandera.pandas as pa

SCORES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "nps": pa.Column(int, pa.Check.in_range(0, 10)),
        "comment": pa.Column(str, nullable=True),
    },
    strict=True,
    coerce=True,
)


def validate_scores(df: pd.DataFrame) -> pd.DataFrame:
    """Validate an NPS extract. Grain: one row per respondent.

    Raises:
        pa.errors.SchemaErrors: With every failing check.
    """
    return SCORES.validate(df, lazy=True)
```

The schema names every column, enforces the grain with `unique`, bounds the score, and allows nulls only where they are expected.

B5-L1 exercise 5

Hints. (1) What does lazy=True raise? (2) Is SchemaErrors a subclass of SchemaError? (3) Even if it were, what does the except branch return?

Answer. The bug is on line 12, 13.

Fixed version:

```python
import pandas as pd
import pandera.pandas as pa

WEIGHTS = pa.DataFrameSchema({"weight": pa.Column(float,
    pa.Check.gt(0))}, strict=True)

```

```
6
7  def load_weights(path: str) -> pd.DataFrame:
8      """Read and validate weights.
9
10     Raises:
11         pa.errors.SchemaErrors: When any check fails.
12     """
13     return WEIGHTS.validate(pd.read_csv(path), lazy=True)
```

Two faults. `lazy=True` raises `SchemaErrors`, which `except SchemaError` does not catch, so the handler is dead code for value failures. And when it does catch, it returns the unvalidated frame. Let the error raise, so the run stops.

Common wrong answers.

- Line 4: The schema is sound: positive, non-null weights, no extra columns.
- Line 11: Validating with lazy=True is right. What happens when it fails?

B5-L2 pydantic models for config and input records

B5-L2 exercise 1

Hints. (1) What is the default value of the extra setting? (2) Does ignore warn? (3) Change it with model_config.

Answer. b) The key is ignored silently. Because: r1) The default extra setting is "ignore". Set extra="forbid" to reject unknown keys.

Default `extra="ignore"` drops unknown keys. Config models should use `extra="forbid"`.

Common wrong answers.

- `a`: Only with ConfigDict(extra="forbid"). The default ignores extras.
- `c`: pydantic does not guess spellings. minbase is a different key.
- `d`: No warning. That is why extra="forbid" matters for config files.
- `b|r2`: Right answer, wrong reason. It checks keys when you set extra="forbid".

B5-L2 exercise 2

Hints. (1) Is "30" accepted in lax mode? (2) Does model_copy run validators? (3) It does not.

Answer.

```
1 -1
```

Lax mode converts `"30"` to `30`. `model_copy(update=...)` then sets `-1` without validation. Rebuild with `model_validate` instead.

Common wrong answers.

- `30`: model_copy applies the update. It just does not validate it.
- `ValidationError`: model_copy(update=...) skips validation, so gt=0 is never checked.

B5-L2 exercise 3

Hints. (1) The extra setting that raises. (2) The config key that blocks assignment. (3) Less than or equal.

Answer.

```python
class Response(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    respondent_id: str
    nps: int = Field(ge=0, le=10)
```

`extra="forbid"`, `frozen=True` and `le=10` encode the rules in the model.

Common wrong answers.

- g1 = `ignore`: ignore is the default and drops unknown keys silently.
- g3 = `lt`: lt=10 excludes 10. NPS allows 10, so use le.

B5-L2 exercise 4

Hints. (1) Bounds go in Field(gt=...). (2) Unknown keys and assignment are controlled in model_config. (3) `ConfigDict(extra="forbid", frozen=True)` and `model_validate(raw)`.

Answer.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    """Settings for one weighting run."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


def load_config(raw: dict[str, object]) -> WeightingConfig:
    """Validate a parsed YAML or JSON mapping.

    Raises:
        pydantic.ValidationError: On any invalid, missing or unknown key.
    """
```

```
    return WeightingConfig.model_validate(raw)
```

Every rule lives in the model: required fields, bounds, no extras, no changes. `model_validate` reports all failures at once.

B5-L2 exercise 5

Hints. (1) Which pydantic methods validate? (2) Does model_copy? (3) Rebuild with model_validate from model_dump().

Answer. The bug is on line 14.

Fixed version:

```
from pydantic import BaseModel, ConfigDict, Field


class Cfg(BaseModel):
    """Run settings."""

    model_config = ConfigDict(extra="forbid", frozen=True)

    min_base: int = Field(default=50, gt=0)


def with_min_base(cfg: Cfg, value: int) -> Cfg:
    """Return a validated copy of cfg with a new min_base."""
    return Cfg.model_validate({**cfg.model_dump(), "min_base": value})
```

`model_copy(update=...)` skips validation. Dump, merge and `model_validate` so the bound applies to the new value.

Common wrong answers.

- Line 9: The bound is right. It is never checked on the copy.
- Line 7: frozen=True is right. model_copy builds a new object, so frozen does not stop it.

B5-L3 Validate at the boundaries, with severity

B5-L3 exercise 1

Hints. (1) Where does data you do not control arrive? (2) Where do you make promises to someone else? (3) In and out.

Answer. b) After reading, and before writing. Because: r1) Those are the boundaries where data changes owner. Tests cover the pure middle.

Validate what you receive and what you deliver. Test the logic in between.

Common wrong answers.

- `a`: Checks inside a tested pure core add noise. The core's inputs are already validated.
- `c`: A bad input would pass through every transform first, and the error would name the wrong step.
- `d`: Unit tests check your code on test data. They say nothing about this week's supplier file.
- `b|r2`: Right answer, wrong reason. The point is ownership boundaries, not speed.

B5-L3 exercise 2

Hints. (1) Is 0.0 truthy? (2) Which branch runs? (3) The fallback.

Answer.

```
1 no data
```

`0.0` is falsy, so the best possible result is reported as missing data. Test with `is not None`.

Common wrong answers.

- `pass`: 0.0 is falsy, so `if share:` skips the comparison. Use `if share is not None:`.

B5-L3 exercise 3

Hints. (1) Check input before using it. (2) Check output before delivering it. (3) Log before raising.

Answer.

1. Read the supplier file
2. Run input checks and collect results
3. Log all results, then raise if any CRITICAL failed
4. Run the pure transforms
5. Run output checks on the result
6. Write the client table

Read, check, enforce, transform, check, write. Each boundary gets its checks before the data moves on.

Common wrong answers.

- Input checks must run before the transforms, so bad input never reaches them.
- Output checks after writing are too late. The client already has the table.

B5-L3 exercise 4

Hints. (1) Compute n once. The share is None when n is 0. (2) Build one QaResult per check. Use `share is not None and share <= max_missing`. (3) enforce filters on severity == "CRITICAL" and not passed.

Answer.

```python
from dataclasses import dataclass
from typing import Literal

import pandas as pd

Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    """One check outcome."""

    check: str
    severity: Severity
    passed: bool
    observed: float | None


def run_checks(df: pd.DataFrame, min_rows: int, max_missing: float) -> list[QaResult]:
    """Evaluate input checks. Never raises.

    Args:
        df: Input with a weight column.
        min_rows: Minimum acceptable row count.
        max_missing: Largest acceptable share of null weights.

    Returns:
        One QaResult per check.
    """
    n = len(df)
    share = None if n == 0 else float(df["weight"].isna().sum()) / n
    return [
        QaResult("min_rows", "CRITICAL", n >= min_rows, float(n)),
        QaResult("missing_weight", "WARN", share is not None and share <= max_missing, share),
    ]


def enforce(results: list[QaResult]) -> None:
    """Stop the run when any CRITICAL check failed.

    Raises:
        RuntimeError: Naming each failed CRITICAL check.
    """
```

```
    failed = [r.check for r in results if r.severity == "CRITICAL" and not r.passed]
    if failed:
        raise RuntimeError(f"critical checks failed: {failed}")
```

`run_checks` measures and never raises, so every result reaches the log. `enforce` decides, and only CRITICAL failures stop the run.

B5-L3 exercise 5

Hints. (1) Does the failure make every output wrong? (2) Or does it affect one cell, handled by a known rule? (3) Grain and weights are foundations.

Answer.

- Duplicate respondents and broken weights make every figure wrong: CRITICAL.
- A small base is a reporting flag, handled by suppression: WARN.
- A rise in missing age is worth a look but does not invalidate the run: WARN.

CRITICAL for failures that invalidate the whole output. WARN for local issues with a known handling rule.

Common wrong answers.

- grain = `warn`: Duplicates double-count respondents in every total. Nothing downstream is correct.
- weights_sum = `warn`: If weights do not sum to the sample, every weighted figure is off. Stop the run.
- base = `critical`: Small bases are expected in some cells. Flag and suppress them. Do not stop the whole run.

B6-L1 Seeds and random generators

B6-L1 exercise 1

Hints. (1) Which option has no shared global state? (2) Which one can a test control? (3) default_rng.

Answer. b) `rng = np.random.default_rng(42)`, passed to the functions that need it Because: r1) Each Generator owns its state, so no other code can change the stream it gives your function.

`default_rng(seed)` gives an independent, seeded generator. Pass it in so each function's randomness is explicit.

Common wrong answers.

- `a`: This seeds a global generator that every library shares. Any extra call anywhere shifts your numbers.
- `c`: The standard library seed does not affect NumPy's generators.
- `d`: Unseeded runs cannot be reproduced or debugged.
- `b|r2`: Right answer, wrong reason. np.random.seed still exists as legacy API. Its global state is the problem.

B6-L1 exercise 2

Hints. (1) Does the generator's state change after a draw? (2) Is the second draw from the same position? (3) It continues the stream.

Answer.

```
1 False
```

The generator advances. Two draws differ, and rerunning the script repeats both.

Common wrong answers.

- `True`: One generator moves on after each draw. Only a fresh generator with the same seed repeats the first draw.

B6-L1 exercise 3

Hints. (1) The function that returns a Generator. (2) The Generator method that makes child generators. (3) `default_rng` and `spawn`.

Answer.

```
1 rng = np.random.default_rng(20260925)
2 worker_rngs = rng.spawn(4)
```

`default_rng` builds the parent. `spawn(4)` returns four independent children.

Common wrong answers.

- g1 = `seed`: np.random.seed sets the global generator and returns None.

B6-L1 exercise 4

Hints. (1) `hashlib.sha256(data)` needs bytes: `respondent_id.encode("utf-8")`. (2) `.hexdigest()` returns hex text. `int(text, 16)` reads it. (3) Finish with `% n_buckets`.

Answer.

```
1 import hashlib
2
3
```

```
def bucket(respondent_id: str, n_buckets: int) -> int:
    """Stable bucket for an ID, the same in every process.

    Args:
        respondent_id: The ID to bucket.
        n_buckets: Number of buckets.

    Returns:
        An integer from 0 to n_buckets - 1.
    """
    digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
    return int(digest, 16) % n_buckets
```

SHA-256 of the ID's bytes is the same everywhere, so the bucket is too. `hash()` is salted per process for strings.

B6-L1 exercise 5

Hints. (1) Which generator does np.random.choice use? (2) Who else can move that generator? (3) Use a local default_rng(seed).

Answer. The bug is on line 6, 9.

Fixed version:

```
import numpy as np


def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
    list[float]:
    """Bootstrap means of values, reproducible from seed."""
    rng = np.random.default_rng(seed)
    data = np.asarray(values)
    return [float(rng.choice(data, size=len(data), replace=True).mean())
    for _ in range(n_boot)]
```

`np.random.seed` and `np.random.choice` use one global generator that any imported code advances. A local `default_rng(seed)` belongs to this function alone.

Common wrong answers.

- Line 10: Taking the mean is fine. The draws come from a shared global generator.

B6-L2 Record the data, config and code behind every result

B6-L2 exercise 1

Hints. (1) Which option changes when one value in the file changes? (2) Which stays the same when the file is copied? (3) A content hash.

Answer. c) The SHA-256 of the file's bytes Because: r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.

A content hash is tied to the bytes, not to the name or the file system.

Common wrong answers.

- `a`: Suppliers resend changed files under the same name.
- `b`: Copying a file changes its modified time without changing content, and edits can keep it.
- `d`: A file with changed values and the same number of rows has the same count.
- `c|r2`: Right answer, wrong reason. Length does not matter. Content identity does.

B6-L2 exercise 2

Hints. (1) What does sort_keys do? (2) Which comes first alphabetically? (3) c before m.

Answer.

```
{"cap": 5.0, "min_base": 50}
```

Sorted keys make the text, and so the hash, independent of the order you wrote the config in.

Common wrong answers.

- `{"min_base": 50, "cap": 5.0}`: sort_keys=True orders keys alphabetically: cap before min_base.

B6-L2 exercise 3

Hints. (1) Hashes work on bytes. (2) Python 3.11 added a hashlib helper for file objects. (3) `rb` and `file_digest`.

Answer.

```
with path.open("rb") as f:
    digest = hashlib.file_digest(f, "sha256").hexdigest()
```

Binary mode gives the exact bytes. `hashlib.file_digest` reads them in chunks.

Common wrong answers.

- g1 = `r`: Text mode decodes and may change line endings. Hash the raw bytes with rb.

B6-L2 exercise 4

Hints. (1) Open the file in binary mode. (2) `hashlib.file_digest(f, "sha256")` or `hashlib.sha256(path.read_bytes())`. (3) For the config, `json.dumps(config, sort_keys=True).encode("utf-8")`.

Answer.

```python
import hashlib
import json
from pathlib import Path


def file_sha256(path: Path) -> str:
    """SHA-256 of a file's bytes, read in chunks.

    Args:
        path: File to hash.

    Returns:
        The hex digest.
    """
    with path.open("rb") as f:
        return hashlib.file_digest(f, "sha256").hexdigest()


def config_sha256(config: dict[str, object]) -> str:
    """SHA-256 of a config, independent of key order.

    Args:
        config: A JSON-serialisable mapping.

    Returns:
        The hex digest of the sorted JSON text.
    """
    text = json.dumps(config, sort_keys=True)
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

Hash content, not names, and serialise configs in a fixed key order before hashing.

B6-L2 exercise 5

Hints. (1) What changes without changing its name? (2) What would you compare between two runs? (3) Store it where the next analyst will look.

Answer.

- Content hash of an immutable input.
- Hash of the loaded config, so defaults and key order are handled.
- Git commit plus a dirty flag.
- Stored with the output and in the run log, where comparisons are easy.

Record identities, not names: content hashes and commits. Keep them where a diff between runs is one query away.

Common wrong answers.

- data = `name`: A resent file keeps its name. Only a content hash shows the data changed.
- config = `file_path`: The file at that path changes later, and defaults are not in it.
- code = `notebook`: The notebook changes under the same name. The commit pins the code.
- store = `chat`: Chat is hard to search and compare. Keep manifests with the output.

B6-L3 Pinned dependencies and lock files

B6-L3 exercise 1

Hints. (1) How many versions does a range accept? (2) What about packages you did not list, such as numpy under pandas? (3) Exact versions plus hashes.

Answer. a) The exact version of every direct and indirect dependency, with file hashes. Because: r1) Every install then gets identical packages, on every machine.

Ranges declare what you support. The lock pins what you run, down to every indirect dependency.

Common wrong answers.

- `b`: Neither file is generated from imports. The lock records what the resolver chose.
- `c`: Locks affect installation, not import speed.

- `d`: A range accepts many versions. The lock records exactly one of each, including indirect dependencies.
- `a|r2`: Right answer, wrong reason. The benefit is identical installs, not skipped work.

B6-L3 exercise 2

Hints. (1) Does uv.lock include polars? (2) What does --locked assert? (3) That the lock would stay unchanged.

Answer.

```
1 The lockfile at `uv.lock` needs to be updated, but `--locked` was
    provided. To update the lockfile, run `uv lock`.
```

`--locked` asserts the lock matches pyproject.toml. It does not, so the command exits with an error and tells you to run `uv lock`.

Common wrong answers.

- `Installed polars`: --locked forbids changing the lock. The build fails instead of resolving on the runner.

B6-L3 exercise 3

Hints. (1) pip-compile has a flag that adds sha256 lines. (2) pip install has a flag that enforces them. (3) `generate-hashes` and `require-hashes`.

Answer.

```
1 pip-compile --generate-hashes requirements.in
2 python -m pip install --require-hashes -r requirements.txt
```

Hashes turn a pin into a guarantee: a different file with the same version number is rejected.

Common wrong answers.

- g1 = `upgrade`: --upgrade moves pins to newer versions. The task asks for hashes.

B6-L3 exercise 4

Hints. (1) The lock respects the range in pyproject.toml. (2) Tests come before the commit. (3) Branch, bound, lock, test, commit.

Answer.

1. Create a branch
2. Raise the bound in pyproject.toml if the new version is outside the range
3. uv lock --upgrade-package scikit-learn
4. Run the test suite and compare key outputs with the last release

5. Commit pyproject.toml and uv.lock with the reason

Widen the range, re-lock one package, prove it with tests, then commit both files together.

Common wrong answers.

- Locking before raising the bound resolves within the old range, so the upgrade you want is not allowed yet.
- Test before you commit, so a broken upgrade never lands.

B6-L3 exercise 5

Hints. (1) Which file says what you support, and which says what you run? (2) Where do indirect dependencies get pinned? (3) What makes CI fail on a stale lock?

Answer.

- Ranges in pyproject.toml say what the code supports.
- The committed lock pins every package with hashes.
- uv sync --locked installs exactly that, and fails on a stale lock.

Ranges for intent, a lock for fact, and a locked install everywhere the code runs.

Common wrong answers.

- declare = `none`: No bounds means any future major version is acceptable, which is rarely true.
- declare = `exact`: Exact pins in pyproject.toml miss indirect dependencies and block security fixes. Pin exactly in the lock.
- lock = `freeze`: freeze captures whatever is on one laptop, one-off installs included, with no hashes.
- install = `fresh`: Fresh resolution picks up new releases silently. That is how a job breaks with no code change.

B7-L1 Profile a dataset before you analyse it

B7-L1 exercise 1

Hints. (1) Which call reports the maximum? (2) What would the maximum be? (3) 99.

Answer. b) `df["score"].describe()` Because: r1) describe() reports min and max, so a max of 99 stands out.

`describe()` gives count, mean, min, quartiles and max. A max of 99 on a 1 to 10 scale is a code, not a score.

Common wrong answers.

- `a`: head() shows five rows. The 99s are likely elsewhere.
- `c`: 99 is a valid integer. The dtype looks fine.
- `d`: The shape counts rows and columns, not values.
- `b|r2`: Right answer, wrong reason. describe() reports values. It does not clean them.

B7-L1 exercise 2

Hints. (1) Does nunique count None? (2) Which b does duplicated() mark? (3) keep="first" is the default.

Answer.

```
1 2 1
```

Two distinct non-missing values. One repeat, because the first `b` is not marked.

Common wrong answers.

- `3 1`: nunique() excludes missing values by default. Only a and b count.
- `2 2`: duplicated() keeps the first b unmarked. Only the second b is True.

B7-L1 exercise 3

Hints. (1) Which keep value marks every copy? (2) The mean of True/False is the share of True. (3) `False` and `mean`.

Answer.

```
dupes = df.duplicated(subset=["respondent_id", "wave"], keep=False).sum()
missing = df.isna().mean()
```

`keep=False` flags all copies. `isna().mean()` gives the missing share per column.

Common wrong answers.

- g1 = `"first"`: keep="first" leaves one copy unmarked. keep=False marks all of them.
- g2 = `sum`: sum() gives counts. The mean of a boolean column is the share.

B7-L1 exercise 4

Hints. (1) Loop over columns and build a dict per column. (2) `pd.api.types.is_numeric_dtype(s)` tells you whether min and max apply. (3) `pd.DataFrame.from_dict(rows, orient="index")` builds the table.

Answer.

```python
import pandas as pd


def profile(df: pd.DataFrame) -> pd.DataFrame:
    """Profile each column: dtype, missing share, distinct values, numeric range.

    Args:
        df: Any DataFrame.

    Returns:
        One row per input column.
    """
    rows = {}
    for col in df.columns:
        s = df[col]
        numeric = pd.api.types.is_numeric_dtype(s)
        rows[col] = {
            "dtype": str(s.dtype),
            "missing_share": float(s.isna().mean()),
            "n_unique": int(s.nunique()),
            "min": float(s.min()) if numeric else None,
            "max": float(s.max()) if numeric else None,
        }
    return pd.DataFrame.from_dict(rows, orient="index")
```

A per-column table is easy to save and to compare with the next delivery's profile.

B7-L1 exercise 5

Hints. (1) Which rows does duplicated() mark by default? (2) Is the first b marked? (3) keep=False.

Answer. The bug is on line 6.

Fixed version:

```python
import pandas as pd


def rows_with_duplicate_ids(df: pd.DataFrame) -> pd.DataFrame:
    """Every row whose respondent_id appears more than once."""
    mask = df.duplicated(subset=["respondent_id"], keep=False)
    return df[mask]
```

The default `keep="first"` leaves the first copy unmarked. `keep=False` marks both.

Common wrong answers.

- Line 7: Filtering with the mask is right. The mask misses the first copy.

B7-L2 Plot before you summarise

B7-L2 exercise 1

Hints. (1) What does one number summarise? (2) Remember Anscombe. (3) Shape needs a plot.

Answer. c) Very little about its shape. Because: r1) Anscombe's quartet has identical correlations with a line, a curve and an outlier-driven fit.

Correlation measures linear association. Curves, clusters and single points give the same value. Plot first.

Common wrong answers.

- `a`: Anscombe's set II is a curve with the same correlation.
- `b`: Same correlation, different shapes: that is the point of the quartet.
- `d`: Only some of the sets have an outlier. The number cannot tell you which.
- `c|r2`: Right answer, wrong reason. Correlation is useful. It does not describe shape.

B7-L2 exercise 2

Hints. (1) These are Anscombe's sets I and III. (2) What correlation does the quartet share? (3) 0.816.

Answer.

```
1 True
```

Both correlations round to 0.82. Only a plot shows that set III is a perfect line bent by one point.

Common wrong answers.

- `False`: Both are 0.82 to two places. Set III is a straight line plus one outlier, and still matches set I.

B7-L2 exercise 3

Hints. (1) The pyplot function that returns (fig, ax). (2) Axes methods that set things start with set_. (3) `subplots` and `set_xlabel`.

Answer.

```
fig, ax = plt.subplots()
ax.hist(values, bins=30)
ax.set_xlabel("Monthly spend (GBP)")
```

`plt.subplots()` returns a figure and axes. Axes setters are `set_title`, `set_xlabel` and `set_ylabel`.

Common wrong answers.

- g2 = `xlabel`: plt.xlabel is the global pyplot function. On an Axes object the method is set_xlabel.

B7-L2 exercise 4

Hints. (1) Start with `fig, ax = plt.subplots()`. (2) `ax.hist(values, bins=bins)`, then the three set_ calls. (3) Return fig.

Answer.

```
import matplotlib.pyplot as plt
from matplotlib.figure import Figure


def plot_distribution(values: list[float], title: str, xlabel: str, bins: int = 30) -> Figure:
    """Histogram of values with labelled axes.

    Args:
        values: Numbers to plot.
        title: Figure title.
        xlabel: X axis label, with units.
        bins: Number of histogram bins.

    Returns:
        The figure. The caller shows or saves it.
    """
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(values, bins=bins)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Respondents")
    return fig
```

Returning a labelled figure makes the plot testable and lets the caller choose to show or save it.

B7-L2 exercise 5

Hints. (1) How many variables does the question involve? (2) Is time part of the question? (3) Missingness is its own variable.

Answer.

- Codes and spikes: a histogram.
- A relationship and its outliers: a scatter plot.
- A break in a series: a line over waves.
- Where answers are missing: a bar chart of missing share by column and device.

Match the plot to the question: one variable, a pair, a series or missingness.

Common wrong answers.

- codes = `scatter`: A spike at 999 in one variable shows best as a tall, isolated bar in a histogram.
- driver = `hist`: A histogram shows one variable. Influential points need both variables: a scatter plot.
- break = `hist`: A histogram pools the waves. A line over waves shows the step at wave 9.

B7-L3 Write assumptions down, then turn them into checks

B7-L3 exercise 1

Hints. (1) What happens when the analyst leaves? (2) What happens when the supplier changes the code? (3) Written, applied, checked.

Answer. b) In the assumptions log, in config for the handling, and in a QA check. Because: r1) Written, applied and checked in code, it survives staff changes and new deliveries.

The log states it, config applies it, and a check detects when it stops being true.

Common wrong answers.

- `a`: A comment explains one line. It does not run, and nothing fails when the coding changes.
- `c`: Personal notes leave with the person.
- `d`: New analysts and new pipelines do not know. That is how 99s reach a mean.
- `b|r2`: Right answer, wrong reason. The point is surviving change, not compliance.

B7-L3 exercise 2

Hints. (1) Which columns does df.replace touch? (2) Is 99 a valid age? (3) It is blanked anyway, and the column becomes float.

Answer.

```
[nan, 40.0, 30.0]
```

Whole-frame `replace` blanks the 99-year-old too. Recode per column with a mapping of column to codes.

Common wrong answers.

- `[99, 40, 30]`: replace on the whole frame changes every column, including a real age of 99.

B7-L3 exercise 3

Hints. (1) Evidence before belief. (2) Belief before code. (3) Handling before the check that tests its result.

Answer.

1. Observe the coding in the questionnaire and the profile
2. Write the assumption, evidence and owner in the log
3. Implement the handling in code, driven by config
4. Add a QA check that runs on every delivery
5. Update last_verified when the check or the source is reviewed

Observe, write, implement, check, review. Each step leaves a record the next analyst can follow.

Common wrong answers.

- Code written before the evidence encodes a guess.
- The check runs on recoded data, so the handling comes first.

B7-L3 exercise 4

Hints. (1) Work column by column: `df[col].isin(vals)` marks the codes. (2) `Series.mask(cond)` sets values to missing where cond is True. (3) Wrap it in `df.assign(**{col: ... for col, vals in codes.items()})`.

Answer.

```
import pandas as pd


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) ->
    pd.DataFrame:
    """Set listed codes to missing, per column.
```

```
    Args:
        df: Input frame.
        codes: Column name to the values that mean missing in that column.

    Returns:
        A new frame. Unlisted columns are unchanged.
    """
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col,
vals in codes.items()})
```

Per-column masking changes only what the assumption covers, and `assign` returns a new frame.

B7-L3 exercise 5

Hints. (1) Is it about rows, values or people? (2) Rows: structure. Values: coding. People: scope. (3) Completes are about who is in, not how answers are coded.

Answer.

- Grain: structure, checked with duplicated(keep=False) on the key.
- Region codes: coding, checked with isin(lookup).
- Age limit and completes only: scope, checked with a filter count that must be zero.

Naming the kind of assumption tells you the kind of check to write.

Common wrong answers.

- grain = `coding`: Grain is about how rows relate to each other. Check it with key uniqueness.
- region = `scope`: Valid region codes are a coding rule: every value must appear in the lookup.
- adults = `coding`: Age is coded correctly. The rule is about who is included: scope.

B8-L1 Survey weights and the effective sample size

B8-L1 exercise 1

Hints. (1) What do unequal weights do to precision? (2) Which number measures the remaining information? (3) Kish's n_eff.

Answer. b) 71 Because: r1) Unequal weights reduce precision, and n_eff measures the information that remains.

Use `n_eff` for standard errors, intervals and tests. Report the raw base alongside.

Common wrong answers.

- `a`: Raw n overstates precision when weights vary, so intervals come out too narrow.
- `c`: The weight total depends on the scaling of the weights, not on the information in the sample.
- `d`: That difference has no statistical meaning.
- `b|r2`: Right answer, wrong reason. n_eff is the correct amount of information, not a cautious guess.

B8-L1 exercise 2

Hints. (1) Which respondents answered 1, and what are their weights? (2) Sum of those weights over the total weight. (3) 5.5 / 8.

Answer.

```
1 0.6875 0.6
```

Weighted: 5.5 / 8 = 0.6875. Unweighted: 3 / 5 = 0.6.

Common wrong answers.

- `0.6 0.6`: np.average with weights gives (0.5 + 1 + 4) / 8 = 0.6875.
- `1.1 0.6`: That is sum(y * w) / n. A weighted mean divides by sum(w) = 8.

B8-L1 exercise 3

Hints. (1) Sum of weights, squared, over the sum of squared weights. (2) Design effect = raw n / n_eff. (3) `2` and `len(w)`.

Answer.

```
1 n_eff = w.sum() ** 2 / (w ** 2).sum()
2 deff = len(w) / n_eff
```

`n_eff = (Σw)² / Σw²`. `deff = n / n_eff`. Equal weights give 1.

Common wrong answers.

- g2 = `w.sum()`: The design effect compares the raw count with n_eff: len(w) / n_eff.

B8-L1 exercise 4

Hints. (1) Drop rows missing either column first. (2) Divide by the weight total, and return None when it is 0. (3) n_eff = w.sum() ** 2 / (w ** 2).sum().

Answer.

```python
import pandas as pd


def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of value over rows with both value and weight present.

    Returns:
        The weighted mean, or None for no rows or a zero weight total.
    """
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)


def effective_n(weights: pd.Series) -> float | None:
    """Kish effective sample size, (sum w)^2 / sum(w^2), ignoring missing weights.

    Returns:
        n_eff, or None when there are no non-zero weights.
    """
    w = weights.dropna()
    sum_sq = (w**2).sum()
    if sum_sq == 0:
        return None
    return float(w.sum() ** 2 / sum_sq)
```

Weighted means divide by the weight total. `n_eff` tells you how much information the weighted sample holds.

B8-L1 exercise 5

Hints. (1) Which describes the population? (2) Which measures information? (3) Which tells readers how many answered?

Answer.

- Chart: the weighted share, which estimates the population.
- Interval: n_eff, so precision reflects the weights.
- Footnote: the raw base, so readers know how many people answered.
- Suppression: n_eff, so a heavily weighted small group is not reported as if it were large.

Weighted for estimates, n_eff for precision and suppression, raw n for transparency.

Common wrong answers.

- estimate = `unweighted`: The unweighted share describes the sample, not the population.
- ci = `raw_n`: Raw n overstates precision when weights vary.
- suppress = `raw_n`: A group of 60 with n_eff 25 carries the information of 25 people. Compare n_eff with the threshold.

B8-L2 Confidence intervals and effect sizes

B8-L2 exercise 1

Hints. (1) What is p(1-p) when p = 0? (2) What is the half-width then? (3) Zero.

Answer. a) [0, 0] Because: r1) With p = 0 the standard error is 0, so the interval collapses.

Wald claims certainty from 20 people. Wilson gives [0, 0.161].

Common wrong answers.

- `b`: That is the Wilson interval. Wald has zero width at p = 0.
- `c`: p(1-p) is 0, so the half-width is 0, not 0.05.
- `d`: sqrt(0) is 0. No error, just a misleading interval.
- `a|r2`: Right answer, wrong reason. Wilson gives a sensible interval for 20. Wald fails at the edge.

B8-L2 exercise 2

Hints. (1) How large is each group? (2) What happens to standard errors as n grows? (3) Tiny differences become significant.

Answer.

```
1 True
```

p = 0.025. The difference is 0.5 points, Cohen's h about 0.01. Significant is not the same as important.

Common wrong answers.

- `False`: With 100,000 per group, a 0.5-point difference gives p = 0.025. Significant, and still tiny.

B8-L2 exercise 3

Hints. (1) alpha is 1 minus the confidence level. (2) The method named after Edwin Wilson. (3) `0.05` and `wilson`.

Answer.

```
low, high = proportion_confint(count=40, nobs=100, alpha=0.05,
    method="wilson")
```

`alpha=0.05, method="wilson"` gives (0.309, 0.498).

Common wrong answers.

- g1 = `0.95`: alpha is the error rate. A 95% interval uses alpha=0.05.
- g2 = `normal`: normal is the Wald interval, which fails near 0 and 1.

B8-L2 exercise 4

Hints. (1) Guard first: n <= 0 or p outside [0, 1] returns None. (2) Compute denom, centre and half in that order. (3) Clip with max(0.0, ...) and min(1.0, ...).

Answer.

```python
import math


def wilson_interval(p: float, n: float, z: float = 1.959964) ->
    tuple[float, float] | None:
    """Wilson score interval for a share.

    Args:
        p: Observed share, weighted or not.
        n: Sample size, or n_eff for a weighted share.
        z: Normal quantile. 1.959964 gives 95%.

    Returns:
        (low, high) clipped to [0, 1], or None for invalid input.
    """
    if n <= 0 or not 0 <= p <= 1:
        return None
    denom = 1 + z**2 / n
    centre = (p + z**2 / (2 * n)) / denom
    half = z / denom * math.sqrt(p * (1 - p) / n + z**2 / (4 * n**2))
    return (max(0.0, centre - half), min(1.0, centre + half))
```

Wilson shifts the centre towards 0.5 and widens the interval when information is thin, so it never collapses to zero width.

B8-L2 exercise 5

Hints. (1) Is 0.04 bigger than 0.2? (2) What does a reader need to judge size? (3) Set the threshold before looking.

Answer.

- Headline: no meaningful change. 0.04 is far below the agreed 0.2.

- Detail: the difference and its interval, next to the threshold, so readers see the size.
- Rule: decide the meaningful size in advance and judge against it.

Size decides meaning. Significance only says the size is not zero.

Common wrong answers.

- headline = `sig_up`: True, and misleading. With 90,000 respondents almost any change is significant.
- detail = `p_only`: A p-value says nothing about size. Show the difference and its interval.
- rule = `p_rule`: In large samples this flags trivial changes every quarter.

B8-L3 Many tests, and the false positives they bring

B8-L3 exercise 1

Hints. (1) What is the chance one test is not significant? (2) And all 20? (3) 1 - 0.95**20.

Answer. b) About 64% Because: r1) 1 - 0.95^20 = 0.64.

The chance none of 20 fire is 0.95^20 = 0.36, so at least one fires with probability 0.64.

Common wrong answers.

- `a`: 5% is the rate for one test. The family has 20 chances.
- `c`: 20 × 5% is the expected number of false positives (1), not a probability.
- `d`: True nulls still produce p < 0.05 in 5% of tests, by definition.
- `b|r2`: Right number, shaky reasoning. The exact calculation is 1 minus the chance of no false positives.

B8-L3 exercise 2

Hints. (1) Holm: 0.01×5, 0.02×4, ... (2) BH: compare the k-th smallest p with k × 0.05 / 5. (3) BH is less strict.

Answer.

```
1 1 4
```

Holm rejects only 0.01 (adjusted 0.05). BH rejects the four p-values up to 0.04, controlling the share of false discoveries instead.

Common wrong answers.

- `4 4`: Holm multiplies the smallest p by 5, the next by 4, and so on: 0.02 × 4 = 0.08 fails.
- `1 1`: Benjamini-Hochberg compares p_(k) with k/m × alpha: 0.04 ≤ 4/5 × 0.05, so four pass.

B8-L3 exercise 3

Hints. (1) The statsmodels function in stats.multitest. (2) The method name is the author's surname. (3) `multipletests` and `holm`.

Answer.

```
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="holm")
```

`multipletests(..., method="holm")` returns reject flags and adjusted p-values.

Common wrong answers.

- g2 = `fdr_bh`: fdr_bh is Benjamini-Hochberg. Holm's method is "holm".

B8-L3 exercise 4

Hints. (1) `order = sorted(range(m), key=lambda i: p_values[i])` gives positions from smallest p. (2) For the k-th position, `running = max(running, min(1.0, (m - k) * p))`. (3) Store at `adjusted[i]`, the original position.

Answer.

```
def holm_adjust(p_values: list[float]) -> list[float]:
    """Holm step-down adjusted p-values, in input order.

    Args:
        p_values: Raw p-values for one family of tests.

    Returns:
        Adjusted p-values, same order as the input.
    """
    m = len(p_values)
    order = sorted(range(m), key=lambda i: p_values[i])
    adjusted = [0.0] * m
    running = 0.0
    for k, i in enumerate(order):
        running = max(running, min(1.0, (m - k) * p_values[i]))
        adjusted[i] = running
    return adjusted
```

Holm scales the smallest p by m, the next by m - 1, and forces the sequence to be non-decreasing. Keeping positions lets the caller match results to tests.

B8-L3 exercise 5

Hints. (1) Is the goal to avoid any false positive, or a high share of them? (2) How many tests are in the family? (3) One test needs no adjustment.

Answer.

- Few decisive tests: Holm, which controls any false positive in the family.
- Screening many cells: Benjamini-Hochberg, which controls the share of false leads.
- A single pre-registered test: no family, so no adjustment.

Match the error rate to the decision: family-wise for confirmation, false discovery rate for screening.

Common wrong answers.

- screen = `holm`: Holm across 400 tests is so strict it flags almost nothing. For screening, control the false discovery rate.
- confirm = `none`: Three tests driving one decision form a family. Without adjustment the error rate rises to about 14%.
- single = `holm`: One test is a family of one. Holm leaves its p-value unchanged.

B9-L1 Power and sample size, before fieldwork

B9-L1 exercise 1

Hints. (1) What does power 0.25 mean? (2) How often would a real effect be missed? (3) Three times in four.

Answer. b) Very little. The test was unlikely to detect the effect even if it were real. Because: r1) Power 0.25 means a real effect of that size is missed 75% of the time.

The test could not answer the question. Report it as inconclusive and, next time, size the sample first.

Common wrong answers.

- `a`: An underpowered test misses real effects most of the time. No significance is not no effect.
- `c`: Nothing in the result points to a negative effect.
- `d`: Power is not a confidence level about the effect being zero.
- `b|r2`: Right answer, wrong reason. A well-powered non-significant result is informative.

B9-L1 exercise 2

Hints. (1) The effect sits squared in the denominator. (2) Halve the effect. (3) (1/0.5)^2.

Answer.

```
4
```

Half the effect, about four times the sample: 1,534 becomes roughly 6,000 per group.

Common wrong answers.

- `2`: n grows with the inverse square of the effect. Half the effect needs about four times the sample.

B9-L1 exercise 3

Hints. (1) The method that solves for whichever argument you leave out. (2) Which input is 0.8? (3) `solve_power` and `power`.

Answer.

```
h = proportion_effectsize(0.45, 0.40)
n = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.8)
```

`solve_power` leaves `nobs1` unset and solves for it: 1,533.05, so 1,534 per group.

Common wrong answers.

- g1 = `power`: power() returns power for a given n. solve_power finds the missing quantity, here n.

B9-L1 exercise 4

Hints. (1) Two-sided: z_a = NormalDist().inv_cdf(1 - alpha / 2). (2) Square (top / (p2 - p1)), then multiply by design_effect. (3) math.ceil last.

Answer.

```
import math
from statistics import NormalDist


def n_per_group(
    p1: float, p2: float, alpha: float = 0.05, power: float = 0.8,
    design_effect: float = 1.0
) -> int:
    """Sample per group to detect p1 vs p2 with a two-sided test.

    Args:
        p1: Baseline share.
        p2: Share under the minimum detectable effect.
```

```
        alpha: Two-sided significance level.
        power: Chance of detecting the effect if real.
        design_effect: Multiplier for weighted or clustered samples.

    Returns:
        Respondents per group, rounded up.

    Raises:
        ValueError: If p1 equals p2 or a share is outside (0, 1).
    """
    if p1 == p2 or not (0 < p1 < 1 and 0 < p2 < 1):
        raise ValueError("shares must differ and lie strictly between 0 and 1")
    z_a = NormalDist().inv_cdf(1 - alpha / 2)
    z_b = NormalDist().inv_cdf(power)
    p_bar = (p1 + p2) / 2
    top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 * (1 - p1) + p2 * (1 - p2))
    return math.ceil((top / (p2 - p1)) ** 2 * design_effect)
```

The formula turns the four decisions into a number. The design effect scales it for the information weights remove.

B9-L1 exercise 5

Hints. (1) Two groups, 1,534 each, times 1.5. (2) Does a region need less sample to detect the same effect? (3) Decide before fieldwork.

Answer.

- Overall: about 4,600 needed with the design effect. 4,000 is close but short.
- Regions: each needs the full sample for the same MDE.
- Settle the trade-off with the client before fieldwork, in writing.

Power maths turns a budget conversation into explicit trade-offs, agreed before any data exists.

Common wrong answers.

- total = `plenty`: Two cells of 1,534, times a design effect of 1.5, need about 4,600.
- regions = `quarter`: A quarter of the sample cannot detect the same 5-point lift. Each region is its own test.
- decision = `run`: Underpowered regional reads produce noise dressed as findings.

B9-L2 Analyse an A/B test the way it was randomised

B9-L2 exercise 1

Hints. (1) Is a 1% gap on 100,000 users chance? (2) Which users went missing? (3) Find the cause first.

Answer. b) Stop and find why B lost users before trusting any result. Because: r1) A split that differs from the design means something removed users unevenly, and they may differ in outcome.

SRM means the arms no longer differ only by chance. Find the cause, fix it, and rerun.

Common wrong answers.

- `a`: With 100,000 users, a 1% gap is far beyond chance. The missing users may be the story.
- `c`: Reweighting assumes the missing users resemble the rest. They rarely do.
- `d`: Random deletion balances counts but not the kind of user lost from B.
- `b|r2`: Right answer, wrong reason. The concern is a specific cause, not a p-value rule.

B9-L2 exercise 2

Hints. (1) Same proportions, ten times the users. (2) The statistic grows with n for the same share. (3) p = 0.23, then p = 0.00015.

Answer.

```
1 True False
```

The same imbalance in share becomes decisive as the sample grows. Check SRM on counts, not on how the percentages look.

Common wrong answers.

- `True True`: The same 50.6% split is chance at 10,000 users (p = 0.23) but not at 100,000 (p = 0.00015).
- `False False`: At 10,000 users, 60 either side of 5,000 is well within chance: p = 0.23.

B9-L2 exercise 3

Hints. (1) Validity before results. (2) Units before estimates. (3) Primary before secondary.

Answer.

1. Check the split against the design
2. Aggregate to one outcome per randomised user
3. Estimate the primary metric's lift with its interval
4. Compare the interval with the planned minimum effect
5. Report secondary metrics as exploratory

Trust, then units, then the one planned answer, then everything else labelled as exploration.

Common wrong answers.

- Checking the split after seeing the lift invites excuses. Check it first.
- Estimate on per-user outcomes, so aggregate before estimating.

B9-L2 exercise 4

Hints. (1) Validate inputs first. (2) Expected counts: n * share and n * (1 - share). (3) Sum (observed - expected)^2 / expected, then erfc(sqrt(stat / 2)).

Answer.

```python
import math


def srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float:
    """Sample ratio mismatch p-value: chi-square with 1 degree of freedom.

    Args:
        n_a: Users observed in arm A.
        n_b: Users observed in arm B.
        expected_share_a: Planned share of users in A.

    Returns:
        The p-value. Below 0.001 signals a mismatch to investigate.

    Raises:
        ValueError: On negative counts, a zero total or an invalid share.
    """
    if n_a < 0 or n_b < 0 or n_a + n_b == 0 or not 0 < expected_share_a < 1:
        raise ValueError("counts must be non-negative with a positive total, share in (0, 1)")
    n = n_a + n_b
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))
```

The statistic compares observed with planned counts. For 1 degree of freedom the chi-square p-value equals `erfc(sqrt(stat / 2))`, so no SciPy is needed.

B9-L2 exercise 5

Hints. (1) Which users does line 6 drop? (2) Is the same rule applied to A? (3) Keep everyone as assigned.

Answer. The bug is on line 6.

Fixed version:

```
import pandas as pd


def conversion_by_arm(users: pd.DataFrame) -> dict[str, float]:
    """Conversion rate per assigned arm. One row per randomised user."""
    return users.groupby("arm")["converted"].mean().to_dict()
```

Filtering B on `saw_new_page` keeps its most engaged users and drops the rest, while A keeps everyone. The arms are no longer comparable. Analyse everyone as assigned.

Common wrong answers.

- Line 7: Grouping by arm is right. The rows reaching it are the problem.

B9-L3 Peeking, novelty and other ways tests mislead

B9-L3 exercise 1

Hints. (1) How many chances does noise get? (2) One look: 5%. Ten looks: more. (3) The simulation gave 18.4%.

Answer. b) About 18% Because: r1) Each look is another chance for noise to cross the threshold.

Repeated looks multiply the chances of a false positive. Fix the horizon or use a planned sequential design.

Common wrong answers.

- `a`: 5% holds for one planned look. Ten looks gave 18.4% in the simulation.
- `c`: Too high for 10 looks. Unlimited looking pushes it higher still.
- `d`: Identical arms still produce p < 0.05 by chance. That is what a 5% level means.
- `b|r2`: Right number, wrong reason. More users do not create significance when there is no effect.

B9-L3 exercise 2

Hints. (1) Half of 3.1 is 1.55. (2) Is 0.4 below that? (3) Yes.

Answer.

```
novelty suspected
```

A lift that fades fast is often novelty. Report the later weeks, or run longer before deciding.

Common wrong answers.

- `stable`: 0.4 is less than half of 3.1, so the effect is fading.

B9-L3 exercise 3

Hints. (1) Both, not either. (2) The plan field for duration. (3) `and` and `min_days`.

Answer.

```
return n_per_arm_now >= plan.n_per_arm and days_run >= plan.min_days
```

Sample size gives power. Duration covers weekly cycles and novelty. The analysis needs both.

Common wrong answers.

- g1 = `or`: or lets a big day-2 sample through before a full weekly cycle. Both conditions must hold.

B9-L3 exercise 4

Hints. (1) Draw conversions for all users at once, then take cumulative sums along each row. (2) Pick the columns at the end of each look: `np.arange(1, looks + 1) * n_per_look - 1`. (3) A simulation counts if `(np.abs(z) > z_crit).any(axis=1)`.

Answer.

```python
import numpy as np


def peeking_false_positive_rate(
    rng: np.random.Generator,
    n_sims: int,
    n_per_look: int,
    looks: int,
    rate: float,
    z_crit: float = 1.959964,
) -> float:
    """Share of A/A simulations declared significant at any of the looks.

    Args:
```

```
        rng: Seeded generator.
        n_sims: Number of simulated tests.
        n_per_look: New users per arm between looks.
        looks: Number of analyses.
        rate: True conversion rate in both arms.
        z_crit: Two-sided threshold for each look.

    Returns:
        The false positive rate across simulations.
    """
    total = looks * n_per_look
    a = (rng.random((n_sims, total)) < rate).cumsum(axis=1)
    b = (rng.random((n_sims, total)) < rate).cumsum(axis=1)
    idx = np.arange(1, looks + 1) * n_per_look - 1
    n = idx + 1
    xa, xb = a[:, idx], b[:, idx]
    pooled = (xa + xb) / (2 * n)
    se = np.sqrt(pooled * (1 - pooled) * 2 / n)
    diff = (xb - xa) / n
    z = np.divide(diff, se, out=np.zeros_like(diff), where=se > 0)
    return float((np.abs(z) > z_crit).any(axis=1).mean())
```

Cumulative sums give every look's data from one draw. Counting a simulation when any look crosses the threshold shows how peeking inflates false positives.

B9-L3 exercise 5

Hints. (1) What should people see before the answer is ready? (2) How many looks, at what threshold? (3) Who shares the treatment?

Answer.

- Daily views of health, not of the answer.
- One planned analysis, or a group-sequential design agreed before launch.
- Randomise by store, because visitors in the same store share prices.

Separate monitoring from deciding, plan the looks, and randomise at the level where treatment is shared.

Common wrong answers.

- dashboard = `lift`: A daily lift invites stopping on noise. Show health metrics until the horizon.
- looks = `daily`: 28 looks at 5% each inflate the false positive rate far above 5%.
- unit = `visitor`: Visitors in one store see the same shelf price. Randomising visitors mixes arms within a store.

B10-L1 Target leakage, features that know the answer

B10-L1 exercise 1

Hints. (1) When is the prediction made? (2) Which value is measured after it? (3) March, during the month you predict.

Answer. c) Number of support calls in March Because: r1) March calls happen after the prediction is made, and may be caused by leaving.

Features must come from before the prediction date. March calls are not available on 1 March.

Common wrong answers.

- `a`: Tenure on 1 March is known on 1 March.
- `b`: February is before the prediction date. Safe to use.
- `d`: Plan type on 1 March is known at prediction time.
- `c|r2`: Right answer, wrong reason. February calls are fine. Timing decides.

B10-L1 exercise 2

Hints. (1) Does days_inactive separate the classes perfectly? (2) And contract_months_left, in reverse? (3) An AUC of 0 is as informative as 1.

Answer.

```
1 1.0 1.0
```

Both separate the classes perfectly, one in reverse. A screen must check `max(auc, 1 - auc)`.

Common wrong answers.

- `1.0 0.0`: b is 0.0, a perfect inverse relationship. max(b, 1 - b) turns it into 1.0.

B10-L1 exercise 3

Hints. (1) Everything is judged against the prediction date. (2) Timing, then data-driven checks. (3) Explanation needs a suspect.

Answer.

1. Write down the prediction date and what the model will see then
2. Find the date each feature value becomes known
3. Drop features known only after the prediction date
4. Run the single-feature AUC screen on what remains
5. Explain any high-AUC feature with the data owner, or drop it

Define the moment of prediction, check each feature's timing, then use the screen as a second line of defence.

Common wrong answers.

- Screening first finds the obvious leaks but misses timing leaks with modest AUC. Start from the prediction date.
- Screen first, then explain what the screen flags.

B10-L1 exercise 4

Hints. (1) `X.select_dtypes("number").columns` limits the loop to numeric features. (2) Filter rows with `X[col].notna()` and check `y[known].nunique() == 2`. (3) Flag when `max(auc, 1 - auc) >= threshold`.

Answer.

```python
import pandas as pd
from sklearn.metrics import roc_auc_score


def leakage_screen(X: pd.DataFrame, y: pd.Series, threshold: float = 0.95) -> list[str]:
    """Numeric features that alone separate the classes suspiciously well.

    Args:
        X: Candidate features.
        y: Binary target aligned with X.
        threshold: AUC strength at or above which a feature is flagged.

    Returns:
        Suspect column names, in column order.
    """
    suspects = []
    for col in X.select_dtypes("number").columns:
        known = X[col].notna()
        if known.sum() == 0 or y[known].nunique() < 2:
            continue
        auc = roc_auc_score(y[known], X.loc[known, col])
        if max(auc, 1 - auc) >= threshold:
            suspects.append(col)
    return suspects
```

The screen looks for any single feature that nearly separates the classes. Missingness itself can leak, as `late_only` shows: filled with zeros it would look perfect.

B10-L1 exercise 5

Hints. (1) Would you know it at the moment of the transaction? (2) Can it be recomputed from earlier data? (3) Who fills it in, and when?

Answer.

- Amount: known at transaction time.
- Chargeback: a consequence of fraud, filed weeks later.
- Transaction count: useful, but must count only transactions before this one.
- Reviewer: assigned only to suspected cases, a proxy for the label.

Keep what is known at prediction time, rebuild what can be made point in time, drop the rest.

Common wrong answers.

- chargeback = `keep`: A chargeback is filed after the fraud is found. It is the label in disguise.
- txn_count = `keep`: Computed on the extract date, it includes transactions after this one. Rebuild it point in time.
- reviewer = `keep`: Reviewers are assigned to flagged cases. The feature encodes the labelling process.

B10-L2 Fit on training data only

B10-L2 exercise 1

Hints. (1) Which step computes something from the data? (2) Which uses only a fixed constant? (3) 2.54 cm per inch.

Answer. b) Converting height from inches to centimetres Because: r1) A unit conversion uses a fixed constant. It learns nothing from the rows.

Fixed transformations are safe anywhere. Anything that computes a statistic from rows must be fitted on training rows only.

Common wrong answers.

- `a`: The median is learned from the rows, including test rows.
- `c`: Selection by target correlation learns from test labels. It is the strongest leak of the four.
- `d`: Mean and sd are learned from the rows.
- `b|r2`: Right answer, wrong reason. Timing does not make a step safe. Learning nothing from the data does.

B10-L2 exercise 2

Hints. (1) Did the selection see the rows used for testing? (2) Can it pick features that fit those labels by chance? (3) Far above 0.5.

Answer.

```
0.85
```

With 5,000 features, 20 fit any 100 labels by chance. Selecting them on all rows leaks the test labels. Inside the folds, the same code scores 0.48.

Common wrong answers.

- `0.5`: That is the honest answer for noise. Selection on all rows has seen the test labels, so CV reports 0.85.

B10-L2 exercise 3

Hints. (1) Which rows does the scaler learn from? (2) What must the test set not do? (3) `X_train` and `transform`.

Answer.

```
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
```

Fit once on training rows. Transform everything with the same fitted object.

Common wrong answers.

- g2 = `fit_transform`: fit_transform on the test set refits the scaler on test rows.
- g1 = `X`: Fitting on all rows leaks test statistics into training.

B10-L2 exercise 4

Hints. (1) fit_prep: median first, then fill, then mean and std. (2) apply_prep reads only from params. (3) Guard `params["sd"] == 0`.

Answer.

```
import pandas as pd


def fit_prep(train: pd.DataFrame) -> dict[str, float]:
    """Learn imputation and scaling parameters for income from training rows.

    Returns:
        median, mean and sd of income after median imputation.
    """
    median = float(train["income"].median())
    filled = train["income"].fillna(median)
    return {"median": median, "mean": float(filled.mean()), "sd": float(filled.std())}


```

```
15 def apply_prep(df: pd.DataFrame, params: dict[str, float]) ->
     pd.DataFrame:
16     """Impute and scale income with parameters learned from training
     rows."""
17     filled = df["income"].fillna(params["median"])
18     if params["sd"] == 0:
19         return df.assign(income=0.0)
20     return df.assign(income=(filled - params["mean"]) / params["sd"])
```

All learning sits in `fit_prep`. `apply_prep` is a pure function of the data and the learned parameters, so test rows can never influence them.

B10-L2 exercise 5

Hints. (1) Which rows does `both` contain? (2) Whose labels feed the rates? (3) Group the training rows only, and give unseen regions the training mean.

Answer. The bug is on line 6, 7.

Fixed version:

```
1  import pandas as pd
2
3
4  def encode_region(train: pd.DataFrame, test: pd.DataFrame) ->
     tuple[pd.DataFrame, pd.DataFrame]:
5      """Replace region with its training churn rate. Unseen regions get the
       training mean."""
6      rates = train.groupby("region")["churned"].mean()
7      prior = float(train["churned"].mean())
8      return (
9          train.assign(region=train["region"].map(rates)),
10         test.assign(region=test["region"].map(rates).fillna(prior)),
11     )
```

Concatenating before grouping puts test labels into the encoding. Learn rates from training rows, and fall back to the training mean for regions the training data never saw. Inside training, cross-fitted encodings reduce the remaining leak, as scikit-learn's TargetEncoder does.

Common wrong answers.

- Line 8: Mapping both sets with one table is right. The table was built from test labels.

B10-L3 Time-safe features and point-in-time joins

B10-L3 exercise 1

Hints. (1) How will the model be used? (2) Can it see the future in production? (3) Then not in validation either.

Answer. b) Train on months up to a cut-off, test on later months Because: r1) It matches production, where the model predicts a future it has not seen.

A time split reproduces the production situation: learn from the past, predict the future.

Common wrong answers.

- `a`: A random split trains on rows from after the test rows, which production never allows.
- `c`: Backwards in time: the model would learn from the future of its test set.
- `d`: Training accuracy rewards memorising and says nothing about new data.
- `b|r2`: Right answer, wrong reason. The point is mirroring the direction of time, not data quality.

B10-L3 exercise 2

Hints. (1) Is there a score on the prediction date itself? (2) What is the default of allow_exact_matches? (3) True.

Answer.

```
1 9
```

The same-day score matches by default. If it was recorded after the prediction was made, it leaks. Use `allow_exact_matches=False`.

Common wrong answers.

- `3`: allow_exact_matches defaults to True, so the score from the same day matches. Set it to False to exclude it.

B10-L3 exercise 3

Hints. (1) The pandas join for 'last value before'. (2) Exclude exact timestamp matches. (3) `merge_asof` and `False`.

Answer.

```
1 pd.merge_asof(preds, scores, left_on="as_of", right_on="scored_at",
     by="customer", allow_exact_matches=False)
```

`merge_asof` takes the last earlier match per `by` group.
`allow_exact_matches=False` makes "earlier" strict.

Common wrong answers.

- g1 = `merge`: A plain merge joins every score for the customer, past and future.

B10-L3 exercise 4

Hints. (1) Sort by customer and date first. (2) Running total with groupby(...).cumsum(), minus the current spend. (3) groupby(...).cumcount() counts earlier rows from 0.

Answer.

```
1  import pandas as pd
2
3
4  def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame:
5      """Spend and order count before each order, per customer.
6
7      Args:
8          orders: Columns customer, ordered_at, spend.
9
10     Returns:
11         Orders sorted by customer and date, with prior_spend and
       prior_orders.
12     """
13     o = orders.sort_values(["customer", "ordered_at"])
14     return o.assign(
15         prior_spend=o.groupby("customer")["spend"].cumsum() - o["spend"],
16         prior_orders=o.groupby("customer").cumcount(),
17     )
```

Sorting fixes the time order. Subtracting the current spend and counting from 0 make both features strictly prior.

B10-L3 exercise 5

Hints. (1) Does the merge filter scores by date? (2) Which score does tail(1) pick? (3) The latest overall, even from the future. Use merge_asof.

Answer. The bug is on line 6, 7.

Fixed version:

```
1  import pandas as pd
2
3
4  def latest_nps(preds: pd.DataFrame, scores: pd.DataFrame) -> pd.DataFrame:
5      """Most recent NPS strictly before each prediction date."""
6      joined = pd.merge_asof(
```

```
        preds.sort_values("as_of"),
        scores.sort_values("scored_at"),
        left_on="as_of",
        right_on="scored_at",
        by="customer",
        allow_exact_matches=False,
    )
    return joined[["customer", "as_of", "nps"]].sort_values(["customer",
"as_of"]).reset_index(drop=True)
```

A merge on customer followed by "latest" picks scores from after the prediction date. `merge_asof` with `allow_exact_matches=False` picks the last strictly earlier score.

Common wrong answers.

- Line 8: Selecting and sorting columns is fine. The rows chosen before it include future scores.

B11-L1 Train, validation and test, split by the right unit

B11-L1 exercise 1

Hints. (1) What did the test set do here? (2) Is the best of 40 lucky? (3) Use validation for choosing.

Answer. b) An optimistic estimate, because the test set chose the model. Because: r1) Picking the best of 40 on the same data selects for luck on that data.

Choose with validation or CV. Keep the test set for one final measurement.

Common wrong answers.

- `a`: It would be, if the test set were used once. Selection on it adds optimism.
- `c`: Selection favours settings that got lucky on this data, so the bias is upwards.
- `d`: Test rows were not used for fitting, so it differs from the training score.
- `b|r2`: Right answer, wrong reason. Used once, a test set is unbiased.

B11-L1 exercise 2

Hints. (1) test_size counts groups, not rows. (2) How many respondents is 25% of 4? (3) Each has 2 rows.

Answer.

```
2 1
```

One of four respondents goes to test, with both rows. No respondent is split across sets.

Common wrong answers.

- `2 2`: A group split moves whole groups. 25% of 4 respondents is 1 respondent, with both of their rows.
- `1 1`: The test respondent brings every row: 2 rows.

B11-L1 exercise 3

Hints. (1) The argument that preserves class proportions. (2) The argument that fixes the seed. (3) `stratify` and `random_state`.

Answer.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    stratify=y, random_state=42)
```

`stratify=y` preserves the class balance. A fixed `random_state` makes the split repeatable.

Common wrong answers.

- g1 = `shuffle`: shuffle controls whether rows are shuffled. stratify keeps class shares.

B11-L1 exercise 4

Hints. (1) `GroupShuffleSplit(n_splits=1, test_size=..., random_state=seed)`. (2) `next(splitter.split(df, groups=df[group_col]))` gives index arrays. (3) Select with `df.iloc[...]`.

Answer.

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit


def split_by_group(
    df: pd.DataFrame, group_col: str, test_size: float, seed: int
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Split rows so no group appears in both train and test.

    Args:
        df: Rows to split.
        group_col: Column identifying the repeated unit, such as
    respondent.
        test_size: Share of groups in the test set.
        seed: Random seed for reproducibility.

    Returns:
        (train, test).
```

```
    """
    splitter = GroupShuffleSplit(n_splits=1, test_size=test_size,
random_state=seed)
    train_idx, test_idx = next(splitter.split(df, groups=df[group_col]))
    return df.iloc[train_idx], df.iloc[test_idx]
```

Group splitting keeps each respondent on one side, so validation measures performance on new people, as production will.

B11-L1 exercise 5

Hints. (1) What will be new in production? (2) Are rows independent? (3) Is time part of the use?

Answer.

- Panel members will be new in production: group by member.
- Independent rows with a rare class: stratify on the target.
- Forecasting: train on the past, test on later months.

The split mirrors production: new people, rare classes kept, or the future.

Common wrong answers.

- panel = `stratified`: Members' waves would land on both sides. The model would recognise people, not patterns.
- fraud = `group`: Rows are independent here, so grouping adds nothing. The risk is a test set with too few frauds.
- forecast = `stratified`: A random split lets the model learn from months after the ones it is tested on.

B11-L2 Pipeline and ColumnTransformer

B11-L2 exercise 1

Hints. (1) What does cross_val_score do with the estimator in each fold? (2) Which rows does the scaler see when fitted? (3) Only that fold's training rows.

Answer. a) The Pipeline refits the scaler on each fold's training rows only. Because: r1) cross_val_score clones and fits the whole pipeline per fold.

Each fold gets a fresh copy of the pipeline, fitted on that fold's training rows. The test rows are only transformed.

Common wrong answers.

- `b`: Test rows are scaled, with the parameters learned from that fold's training rows.

- `c`: Fitting once on all rows is exactly the leak the pipeline avoids.
- `d`: Pipelines only run the steps you put in them.
- `a|r2`: Right answer, wrong reason. The scaler learns mean and scale, per fold.

B11-L2 exercise 2

Hints. (1) Was region e seen in fit? (2) What is the default handle_unknown? (3) "error".

Answer.

```
ValueError
```

The default raises on unseen categories. Use `handle_unknown="ignore"` for production inputs.

Common wrong answers.

- `encoded`: The default handle_unknown is "error". An unseen category raises ValueError.

B11-L2 exercise 3

Hints. (1) The handle_unknown value that does not raise. (2) The remainder value that keeps columns. (3) `ignore` and `passthrough`.

Answer.

```
prep = ColumnTransformer(
    [("cat", OneHotEncoder(handle_unknown="ignore"), ["region"])],
    remainder="passthrough",
)
```

`handle_unknown="ignore"` avoids errors on new categories. `remainder="passthrough"` keeps other columns. Use it on purpose, not by accident.

Common wrong answers.

- g2 = `drop`: drop is the default, and it discards every unlisted column.

B11-L2 exercise 4

Hints. (1) `make_pipeline(SimpleImputer(strategy="median"), StandardScaler())` for numbers. (2) `make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore"))` for categories. (3) `Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])`.

Answer.

```python
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


def build_pipeline(numeric: list[str], categorical: list[str]) -> Pipeline:
    """Preprocessing and classifier as one estimator.

    Args:
        numeric: Numeric feature columns.
        categorical: Categorical feature columns.

    Returns:
        An unfitted Pipeline with steps prep and clf.
    """
    prep = ColumnTransformer(
        [
            ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numeric),
            (
                "cat",
                make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore")),
                categorical,
            ),
        ]
    )
    return Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
```

One estimator holds every learned step. Cross-validation refits it per fold, and production runs exactly what was validated.

B11-L2 exercise 5

Hints. (1) Which columns does the ColumnTransformer list? (2) What happens to the others by default? (3) remainder="drop".

Answer. The bug is on line 9.

Fixed version:

```python
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


def build(categorical: list[str]) -> Pipeline:
```

```
8     """Encode categorical columns and fit a classifier on every
      feature."""
9     prep = ColumnTransformer(
10        [("cat", OneHotEncoder(handle_unknown="ignore"), categorical)],
11        remainder="passthrough",
12    )
13    return Pipeline([("prep", prep), ("clf",
      LogisticRegression(max_iter=1000))])
```

`ColumnTransformer` drops unlisted columns by default. Pass them through on purpose, or better, list and preprocess them explicitly.

Common wrong answers.

- Line 10: The classifier uses whatever prep outputs. prep never outputs tenure.

B11-L3 Cross-validation and honest hyperparameter search

B11-L3 exercise 1

Hints. (1) How was the best chosen? (2) On which data was it scored? (3) Selection on the same folds.

Answer. b) The mean CV score of the best setting, optimistic because it was selected as the best Because: r1) Selecting the maximum of many noisy scores favours settings that were lucky on these folds.

`best_score_` is the winner's CV score. Use nested CV or an untouched test set for the estimate you report.

Common wrong answers.

- `a`: It is the maximum over 80 candidates on the same folds, so it carries selection optimism.
- `c`: GridSearchCV does not use a separate test set.
- `d`: It is a CV score on held-out folds, not a training score.
- `b|r2`: Right answer, wrong reason. The search is fine. Its best score needs an outer check.

B11-L3 exercise 2

Hints. (1) How does make_pipeline name its steps? (2) Lower-case class names. (3) Parameter keys are step__param.

Answer.

```
1 False True
```

`make_pipeline` names steps `standardscaler` and `logisticregression`. Use `Pipeline([("clf", ...)])` when you want short, stable names for your grids.

Common wrong answers.

- `True True`: make_pipeline names steps after their class in lower case. There is no step called clf.
- `True False`: Step names come from make_pipeline: logisticregression, so the key is logisticregression__C.

B11-L3 exercise 3

Hints. (1) step__param. (2) The argument for the metric. (3) `clf__C` and `scoring`.

Answer.

```
search = GridSearchCV(pipe, {"clf__C": [0.01, 0.1, 1, 10]}, cv=inner,
    scoring="roc_auc")
```

Pipeline parameters are addressed as `step__param`. `scoring` sets the metric the search optimises.

Common wrong answers.

- g1 = `C`: Inside a Pipeline, prefix the parameter with the step name and two underscores: clf__C.
- g1 = `clf_C`: Two underscores separate the step name from the parameter: clf__C.

B11-L3 exercise 4

Hints. (1) Build the Pipeline, then the GridSearchCV around it. (2) Pass the search object, unfitted, to cross_val_score. (3) Both loops use scoring="roc_auc".

Answer.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold,
    cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


def nested_cv_auc(X: np.ndarray, y: np.ndarray, c_values: list[float],
    seed: int) -> float:
    """Nested CV estimate of ROC AUC for a tuned logistic regression.

    Args:
        X: Features.
```

```
        y: Binary target.
        c_values: Candidate regularisation strengths.
        seed: Base seed for the inner and outer splitters.

    Returns:
        Mean outer-fold ROC AUC of the whole search.
    """
    pipe = Pipeline([("scale", StandardScaler()), ("clf",
LogisticRegression(max_iter=1000))])
    inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
    outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed +
1)
    search = GridSearchCV(pipe, {"clf__C": c_values}, cv=inner,
scoring="roc_auc")
    return float(cross_val_score(search, X, y, cv=outer,
scoring="roc_auc").mean())
```

The outer loop treats the search as one model. Its folds never influence which C is chosen, so the mean is an honest estimate.

B11-L3 exercise 5

Hints. (1) Which data never influences a choice? (2) Which metric is tuned? (3) Which number is honest?

Answer.

- Hold out a test set at the start.
- Search on the rest with CV, a fixed seed and the reported metric.
- Report one test-set score, plus the fold spread from CV as a guide to uncertainty.

Keep one set of data out of every decision, and report its score once.

Common wrong answers.

- search = `best_test`: Choosing on the test set makes its score optimistic.
- report = `best_score`: best_score_ is the best of many tries on the same folds.
- holdout = `use_all`: Without a holdout, use nested CV for the estimate. Otherwise the number is optimistic.

B12-L1 Choose the metric from the decision

B12-L1 exercise 1

Hints. (1) What does a constant prediction score? (2) Compare with 1 - prevalence. (3) 0.976.

Answer. b) Nothing useful yet. Predicting all negatives scores the same. Because: r1) Accuracy is dominated by the 97.6% majority class.

With 2.4% positives, 0.976 is the no-skill baseline. Look at recall, precision and average precision.

Common wrong answers.

- `a`: A constant 'negative' prediction scores 0.976 too.
- `c`: That would be recall. Accuracy mixes both classes.
- `d`: Accuracy says nothing about probability calibration.
- `b|r2`: Right answer, wrong reason. Accuracy works for balanced classes with equal costs.

B12-L1 exercise 2

Hints. (1) A random ranking's precision at every recall level is about the prevalence. (2) What is the prevalence? (3) About 5%.

Answer.

```
0.05
```

The no-skill average precision equals the prevalence. Always report it beside the model's score.

Common wrong answers.

- `0.5`: 0.5 is the random baseline for ROC AUC. For average precision the baseline is the prevalence, here 5%.

B12-L1 exercise 3

Hints. (1) np.sort sorts ascending. (2) Count from the end. (3) `-200`, then compare with `threshold`.

Answer.

```
threshold = np.sort(scores)[-200]
flagged = scores >= threshold
```

The threshold comes from capacity, not from 0.5. Ties at the threshold can flag a few more than 200.

Common wrong answers.

- g1 = `200`: np.sort is ascending. The 200th largest value is at index -200.

B12-L1 exercise 4

Hints. (1) flagged = scores >= threshold. tp counts flagged positives. (2) precision = tp / flagged count. recall = tp / positive count. (3) Return None when the denominator is 0.

Answer.

```python
import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score


def evaluate(y_true: np.ndarray, scores: np.ndarray, threshold: float) -> dict[str, float | None]:
    """Ranking metrics, threshold metrics and the prevalence baseline.

    Args:
        y_true: Binary labels.
        scores: Model scores, higher means more likely positive.
        threshold: Flag cases with score >= threshold.

    Returns:
        prevalence, roc_auc, average_precision, precision, recall.
    """
    flagged = scores >= threshold
    tp = int((flagged & (y_true == 1)).sum())
    n_flagged = int(flagged.sum())
    n_pos = int((y_true == 1).sum())
    return {
        "prevalence": float(y_true.mean()),
        "roc_auc": float(roc_auc_score(y_true, scores)),
        "average_precision": float(average_precision_score(y_true, scores)),
        "precision": tp / n_flagged if n_flagged > 0 else None,
        "recall": tp / n_pos if n_pos > 0 else None,
    }
```

Counts make precision and recall transparent. Returning None for an empty denominator avoids a misleading 0.0 and a crash.

B12-L1 exercise 5

Hints. (1) Is capacity fixed? (2) Are costs known and unequal? (3) Is there a threshold yet?

Answer.

- Fraud with fixed review capacity: precision in the top 150, the share of analyst time spent on real fraud.
- Comparing rankers: ROC AUC, with average precision beside it for rare positives.

- Unequal, known costs: total cost at the threshold, which picks the threshold too.

Metrics follow the decision: capacity, costs, or pure ranking.

Common wrong answers.

- fraud = `accuracy`: At 1% prevalence, flagging nothing is 99% accurate.
- credit = `auc`: AUC ignores the threshold and the cost difference, which decide profit.
- compare = `cost`: Without a threshold or agreed costs, compare rankings first.

B12-L2 Calibration, when the probability itself matters

B12-L2 exercise 1

Hints. (1) What does AUC measure? (2) What would a calibrated model show in the 0.8 to 0.9 bin? (3) About 85% observed.

Answer. a) It ranks well but is poorly calibrated. Because: r1) AUC measures ordering only. The probabilities can be far off while the order is right.

Good ranking, poor calibration. Fine for a top-k list, wrong for expected-value sums.

Common wrong answers.

- `b`: Calibrated would mean about 85% churn in that bin, not 20%.
- `c`: Nothing contradicts the AUC. Ranking and calibration are separate properties.
- `d`: 0.85 AUC is good ranking.
- `a|r2`: Right answer, wrong reason. AUC says nothing about calibration.

B12-L2 exercise 2

Hints. (1) Brier = mean of (p - y)^2. (2) For p = 0.25: three rows of 0.0625 and one of 0.5625. (3) For p = 0.5 every row is 0.25.

Answer.

```
1 0.1875 0.25
```

The prevalence is the best constant prediction. A model worse than 0.1875 here is worse than knowing only the base rate.

Common wrong answers.

- `0.25 0.25`: Predicting the prevalence (0.25) gives (3 × 0.0625 + 0.5625) / 4 = 0.1875.

B12-L2 exercise 3

Hints. (1) The scikit-learn wrapper in sklearn.calibration. (2) Platt scaling's other name. (3) `CalibratedClassifierCV` and `sigmoid`.

Answer.

```
calibrated = CalibratedClassifierCV(model, method="sigmoid",
    cv=5).fit(X_train, y_train)
```

`CalibratedClassifierCV` fits the model and a calibration map on separate folds, so calibration is learned on data the model did not train on.

Common wrong answers.

- g2 = `isotonic`: Isotonic works, but overfits small samples. The prompt asks for sigmoid.

B12-L2 exercise 4

Hints. (1) `edges = np.linspace(0.0, 1.0, n_bins + 1)`. (2) Group a DataFrame of bin, y and p by bin. (3) Named aggregation: mean of p, mean of y, size.

Answer.

```
import numpy as np
import pandas as pd


def calibration_table(y: np.ndarray, p: np.ndarray, n_bins: int) ->
    pd.DataFrame:
    """Mean predicted probability and observed rate per equal-width bin.

    Args:
        y: Binary outcomes.
        p: Predicted probabilities.
        n_bins: Number of equal-width bins on [0, 1].

    Returns:
        One row per non-empty bin: mean_predicted, observed, n.
    """
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    idx = np.searchsorted(edges[1:-1], p, side="right")
    df = pd.DataFrame({"bin": idx, "y": y, "p": p})
    table = df.groupby("bin").agg(mean_predicted=("p", "mean"),
    observed=("y", "mean"), n=("y", "size"))
    return table.reset_index(drop=True)
```

Observed is the positive rate **within** each bin, the number the predicted mean should match. groupby leaves empty bins out.

B12-L2 exercise 5

Hints. (1) Does the use read the order or the number? (2) What does summing probabilities assume? (3) What do balanced weights do to probabilities?

Answer.

- A top-k list needs order, not calibrated values.
- A count forecast sums probabilities, so they must be calibrated.
- Reweighting distorts probabilities. Recalibrate on held-out data and check the Brier score against the constant baseline.

Deliver ranking where order is enough. Deliver calibrated probabilities where the number is used.

Common wrong answers.

- calllist = `calibrated`: Calibration does not change the order, so it does not change who is in the top 500.
- forecast = `ranking`: A sum of probabilities is only right if the probabilities are right.
- training = `balanced_only`: In this lesson's example, balanced weights moved the mean probability from 0.024 to 0.42.

B12-L3 Imbalanced classes and error analysis by segment

B12-L3 exercise 1

Hints. (1) Is the problem the ranking or the cut-off? (2) What does 0.5 mean at 2% prevalence? (3) Choose the threshold from the decision.

Answer. b) Lower the threshold to match review capacity or costs. Because: r1) The model's ranking may be fine. Only the cut-off is wrong for rare positives.

At 2% prevalence few cases score above 0.5. Pick the threshold that matches capacity or cost, and check precision and recall there.

Common wrong answers.

- `a`: Never resample the test set. Its balance must match production.
- `c`: A different model does not fix a threshold chosen by default.

- `d`: Accuracy would look excellent while the model finds nothing.
- `b|r2`: Right answer, wrong reason. A lower threshold trades precision for recall. It does not improve the model.

B12-L3 exercise 2

Hints. (1) Recall is found over positives. (2) Overall pools the counts. (3) 84 / 120.

Answer.

```
email 0.8
chat 0.2
overall 0.7
```

An overall 0.7 hides a segment at 0.2. Pooled rates weight large segments more, so small segments disappear from the headline.

Common wrong answers.

- `email 0.8 chat 0.2 overall 0.5`: Overall recall pools the counts: 84 of 120 is 0.7, not the mean of the two rates.

B12-L3 exercise 3

Hints. (1) What must exist before training? (2) What do segment metrics depend on? (3) Reading examples comes last.

Answer.

1. Split train and test, stratified, test left at its natural balance
2. Train the model normally
3. Choose the threshold from capacity or cost on validation data
4. Break test results down by segment, with counts
5. Read the most confident false positives and false negatives

Split, train, threshold, then look for where and why the model fails.

Common wrong answers.

- Split before training, so the test set never influences the model.
- Segment precision and recall depend on the threshold. Choose it first.

B12-L3 exercise 4

Hints. (1) Loop over `df.groupby(segment, sort=True)`. (2) tp counts rows where y and flagged are both 1, within the segment. (3) Build a list of dicts and return `pd.DataFrame(rows)`.

Answer.

```
import pandas as pd
```

```python


def metrics_by_segment(df: pd.DataFrame, segment: str) -> pd.DataFrame:
    """Count, positives, precision and recall per segment.

    Args:
        df: Columns y and flagged (0/1) plus the segment column.
        segment: Name of the segment column.

    Returns:
        One row per segment, sorted by segment.
    """
    rows = []
    for name, g in df.groupby(segment, sort=True):
        tp = int(((g["y"] == 1) & (g["flagged"] == 1)).sum())
        n_flag = int(g["flagged"].sum())
        n_pos = int(g["y"].sum())
        rows.append(
            {
                segment: name,
                "n": len(g),
                "positives": n_pos,
                "precision": tp / n_flag if n_flag > 0 else None,
                "recall": tp / n_pos if n_pos > 0 else None,
            }
        )
    return pd.DataFrame(rows)
```

Per-segment counts and rates show where the model fails. Missing values for empty denominators stop a segment with no positives looking like a failure.

B12-L3 exercise 5

Hints. (1) Which set must look like production? (2) What does line 7 do to it? (3) Return test unchanged.

Answer. The bug is on line 7, 8.

Fixed version:

```python
import pandas as pd


def prepare_sets(train: pd.DataFrame, test: pd.DataFrame, seed: int) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Oversample training positives. The test set keeps its natural balance."""
    pos_train = train[train["y"] == 1].sample(frac=5.0, replace=True, random_state=seed)
    return pd.concat([train, pos_train]), test
```

Oversampling the test set inflates positives and duplicates them, so recall is measured on a world that does not exist. Resample training data only.

Common wrong answers.

- Line 6: Oversampling training positives is a legitimate choice. The problem is the other set.

B13-L1 Permutation importance on held-out data

B13-L1 exercise 1

Hints. (1) How many split points does a unique ID offer? (2) Which data is impurity importance computed on? (3) Training data.

Answer. b) Impurity importance favours features with many distinct values, measured on training data. Because: r1) Many split points let trees memorise training rows, which impurity importance rewards.

Unique values give trees endless ways to split training rows. Impurity importance counts those splits. Permutation on held-out data does not.

Common wrong answers.

- `a`: Permutation importance on held-out data would show a drop if it did. In this lesson's example it was 0.004.
- `c`: More trees make the bias more stable, not smaller.
- `d`: Trees are not affected by scaling.
- `b|r2`: Right answer, wrong reason. Forests handle integers. Cardinality causes the bias.

B13-L1 exercise 2

Hints. (1) What does the model still have when only age is shuffled? (2) The same information via birth_year. (3) So the single drop is small.

Answer.

```
1 True
```

Correlated features share credit. Permute them as a group to see the importance of the information they carry.

Common wrong answers.

- `False`: With birth_year intact, the model still has the information, so shuffling age alone costs little.

B13-L1 exercise 3

Hints. (1) Held-out rows. (2) The spread across repeats. (3) `X_test` and `importances_std`.

Answer.

```
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
    n_repeats=10, random_state=0)
print(result.importances_mean, result.importances_std)
```

Held-out rows show what generalises. The standard deviation shows whether a drop is distinguishable from zero.

Common wrong answers.

- g1 = `X_train`: On training data, memorised features look important. Use held-out rows.

B13-L1 exercise 4

Hints. (1) Base AUC first, on the unshuffled data. (2) `X.assign(**{col: rng.permutation(X[col].to_numpy())})` shuffles a copy. (3) Average the drops with np.mean.

Answer.

```
import numpy as np
import pandas as pd
from sklearn.base import ClassifierMixin
from sklearn.metrics import roc_auc_score


def permutation_drop(model: ClassifierMixin, X: pd.DataFrame, y:
    pd.Series, col: str, rng: np.random.Generator, n_repeats: int) ->
    float:
    """Mean fall in ROC AUC when col is shuffled.

    Args:
        model: Fitted classifier with predict_proba.
        X: Held-out features.
        y: Held-out labels.
        col: Column to shuffle.
        rng: Seeded generator.
        n_repeats: Number of shuffles.

    Returns:
        Mean of base AUC minus shuffled AUC.
    """
    base = roc_auc_score(y, model.predict_proba(X)[:, 1])
    drops = []
    for _ in range(n_repeats):
        shuffled = X.assign(**{col: rng.permutation(X[col].to_numpy())})
        drops.append(base - roc_auc_score(y, model.predict_proba(shuffled)
    [:, 1]))
    return float(np.mean(drops))
```

Shuffling keeps the column's distribution but breaks its link with the target. The fall in AUC is how much the model relied on it.

B13-L1 exercise 5

Hints. (1) Which method uses held-out data? (2) What happens when one of two twins is shuffled? (3) Does reliance imply cause?

Answer.

- Permutation importance on held-out data, with the metric the model is judged on.
- Correlated features grouped, so shared information is not split into two small numbers.
- Words that describe the model, not causes in the world.

Deliver reliance, measured honestly and worded honestly.

Common wrong answers.

- method = `impurity`: Impurity importance favours high-cardinality features and reflects training data.
- correlated = `separate`: Each would look small, because the other carries the same information.
- wording = `cause`: Importance shows what the model uses. Causes need a different design, such as an experiment.

B13-L2 SHAP values, and what they do not tell you

B13-L2 exercise 1

Hints. (1) What does SHAP attribute? (2) The model's output. (3) Not the outcome in the world.

Answer. b) The model uses the feature to push churn scores up. Because: r1) SHAP values attribute the model's output. They say nothing about intervening in the world.

SHAP describes the model's reliance. Test interventions separately.

Common wrong answers.

- `a`: Causal claims need causal evidence, such as an experiment.
- `c`: Nothing in a SHAP value indicates measurement error.
- `d`: Large contributions are normal for useful features.
- `b|r2`: Right answer, wrong reason. The values are real attributions of the model's output.

B13-L2 exercise 2

Hints. (1) Add the base value and all contributions. (2) That is in log-odds. (3) p = 1 / (1 + exp(-log_odds)).

Answer.

```
-0.2 0.45
```

Additivity holds in log-odds for this model. The probability is 0.45 after the logistic transform, and the contributions do not add up in probability space.

Common wrong answers.

- `-0.2 -0.2`: The contributions sum to log-odds. Convert with the logistic function to get a probability.
- `0.8 0.69`: Include the base value: -1.2 + 0.8 + 0.5 - 0.3 = -0.2.

B13-L2 exercise 3

Hints. (1) The explainer for tree ensembles. (2) The attribute with one number per feature per row. (3) `TreeExplainer` and `values`.

Answer.

```
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)
contributions = explanation.values
```

Calling the explainer returns a `shap.Explanation`. `.values` holds contributions, `.base_values` the baseline.

Common wrong answers.

- g2 = `base_values`: base_values is the starting point. The per-feature contributions are .values.

B13-L2 exercise 4

Hints. (1) Check the lengths first. (2) `sorted(..., key=lambda pair: abs(pair[1]), reverse=True)`. (3) Slice the first k.

Answer.

```
def top_contributions(names: list[str], values: list[float], k: int) ->
    list[tuple[str, float]]:
    """The k largest contributions to one prediction, by absolute size.

    Args:
        names: Feature names.
        values: SHAP values for one row, same order as names.
        k: Number of pairs to return.

    Returns:
```

```
        (name, value) pairs, largest absolute value first, sign kept.

    Raises:
        ValueError: If names and values differ in length.
    """
    if len(names) != len(values):
        raise ValueError("one SHAP value per feature is required")
    ranked = sorted(zip(names, values, strict=True), key=lambda pair:
abs(pair[1]), reverse=True)
    return ranked[:k]
```

A large negative contribution matters as much as a large positive one. Rank by size, report with sign.

B13-L2 exercise 5

Hints. (1) Do log-odds add as percentages? (2) Local or global? (3) Model or world?

Answer.

- Name the output space and convert only the final score.
- Give the local top contributions for this case, not the global ranking.
- Describe the model's reasoning, not a claim about the person.

Local, space-aware and modest wording keeps explanations accurate.

Common wrong answers.

- space = `mixed`: Log-odds contributions do not add up in probability points. Labelling them so is wrong.
- content = `global`: The global ranking says what the model uses in general, not why this case was declined.
- wording = `cause`: SHAP shows what moved the model's score. It is not evidence about the person.

B13-L3 Performance by subgroup, and the fairness trade-offs

B13-L3 exercise 1

Hints. (1) Which other features correlate with sex? (2) How would you measure a gap without the column? (3) Keep it for measurement where lawful.

Answer. b) They can persist through proxies, and you lose the ability to measure them if you discard the column. Because: r1) Other features correlate with sex, and measurement needs the attribute.

Exclusion is not neutrality. Keep the attribute for measuring gaps, and decide separately whether the model uses it.

Common wrong answers.

- `a`: Proxies such as occupation or purchase history can carry the same information.
- `c`: They may change either way. The point is you can no longer see them.
- `d`: Removing a feature says nothing about calibration.
- `b|r2`: Right answer, wrong reason. Removing a feature does change the model, just not necessarily the disparity.

B13-L3 exercise 2

Hints. (1) TPR = tp / positives. (2) FPR = fp / negatives. (3) 150 / 390.

Answer.

```
A 0.65 0.38
B 0.15 0.03
```

Same threshold, very different error rates, because the groups' base rates differ.

Common wrong answers.

- `A 0.65 0.15 B 0.15 0.03`: FPR divides false positives by negatives: 150 / 390 = 0.38.

B13-L3 exercise 3

Hints. (1) True positives are flagged positives. (2) False positives are flagged negatives. (3) `pos` and `neg`.

Answer.

```
tpr = (flag & pos).sum() / pos.sum()
fpr = (flag & neg).sum() / neg.sum()
```

Both rates condition on the true class: TPR on positives, FPR on negatives.

Common wrong answers.

- g2 = `pos`: False positives are flagged negatives: flag & neg.

B13-L3 exercise 4

Hints. (1) Loop over `df.groupby(group, sort=True)`. (2) Boolean masks: pos, neg and flag. (3) Guard each denominator.

Answer.

```
import pandas as pd

```

```python

def group_rates(df: pd.DataFrame, group: str) -> pd.DataFrame:
    """Selection rate, TPR and FPR per group, with counts.

    Args:
        df: Columns y and flagged (0/1) and the group column.
        group: Name of the group column.

    Returns:
        One row per group, sorted by group.
    """
    rows = []
    for name, g in df.groupby(group, sort=True):
        pos, neg = g["y"] == 1, g["y"] == 0
        flag = g["flagged"] == 1
        rows.append(
            {
                group: name,
                "n": len(g),
                "positives": int(pos.sum()),
                "selection_rate": float(flag.mean()),
                "tpr": float((flag & pos).sum() / pos.sum()) if pos.sum() > 0 else None,
                "fpr": float((flag & neg).sum() / neg.sum()) if neg.sum() > 0 else None,
            }
        )
    return pd.DataFrame(rows)
```

Rates conditioned on the true class show who bears which errors. Counts beside them show how much evidence there is.

B13-L3 exercise 5

Hints. (1) Who is harmed, and by which error? (2) Is the score used as a number? (3) Match the property to the harm.

Answer.

- Help should reach genuine cases in every group at the same rate: TPR.
- Wrong referrals harm people, so wrong flags should not fall more on one group: FPR.
- Prices use the probability itself, which must mean the same in every group: calibration.

No single property fits all uses. Pick the one that protects against the harm this decision causes, and document why.

Common wrong answers.

- support = `fpr`: An unneeded offer of help costs little. Missing people who need it is the harm.

- investigate = `tpr`: The harm is to people wrongly referred. Balance false positive rates.
- pricing = `tpr`: Pricing uses probabilities as numbers. They must be calibrated in each group.

B14-L1 Track every run with MLflow

B14-L1 exercise 1

Hints. (1) Params and metrics are stored differently. (2) Which one is numeric? (3) Metrics.

Answer. b) str Because: r1) MLflow stores parameters as strings, so compare and convert carefully.

Parameters are strings. `run.data.params["C"]` is `'1.0'`. Convert before comparing numerically.

Common wrong answers.

- `a`: Metrics are floats. Parameters come back as strings: '1.0'.
- `c`: 1.0 is a float in Python, and MLflow stores it as the string '1.0'.
- `d`: MLflow does not use Decimal for parameters.
- `b|r2`: Right answer, wrong reason. Databases store floats. MLflow chooses strings for parameters.

B14-L1 exercise 2

Hints. (1) Can a run's parameter change after logging? (2) What would that do to reproducibility? (3) MLflow refuses.

Answer.

```
1 MlflowException
```

"Changing param values is not allowed." Start a new run for new settings.

Common wrong answers.

- `C is 2.0`: Parameters are immutable within a run. Changing a logged value raises MlflowException.
- `C is 1.0`: The second call does not pass silently. It raises.

B14-L1 exercise 3

Hints. (1) The function for one tag. (2) MLflow 3 names logged models with a keyword. (3) `set_tag` and `name`.

Answer.

```
mlflow.set_tag("git_commit", commit)
mlflow.sklearn.log_model(model, name="model")
```

Tags hold run metadata such as commits. In MLflow 3, `log_model` takes `name=` for the model's name in the run.

Common wrong answers.

- g1 = `log_param`: A commit is metadata about the run, not a setting of the model. Use a tag.

B14-L1 exercise 4

Hints. (1) Open the run with a with block. (2) log_params for settings, set_tags for commit and data hash, log_metric for the score. (3) log_model with name="model", then return run.info.run_id.

Answer.

```
import mlflow
import numpy as np
from sklearn.linear_model import LogisticRegression


def tracked_fit(X_train: np.ndarray, y_train: np.ndarray, c: float, seed: int, commit: str, data_hash: str, cv_auc: float) -> str:
    """Fit a logistic regression and record the run in MLflow.

    Returns:
        The MLflow run ID.
    """
    with mlflow.start_run(run_name=f"lr-c{c}") as run:
        mlflow.log_params({"C": c, "seed": seed})
        mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
        model = LogisticRegression(C=c, random_state=seed, max_iter=1000).fit(X_train, y_train)
        mlflow.log_metric("cv_auc", cv_auc)
        mlflow.sklearn.log_model(model, name="model")
        return run.info.run_id
```

A run that records settings, code, data, score and model can be found, compared and reproduced.

B14-L1 exercise 5

Hints. (1) Does it change the result? (2) Is it a number to compare? (3) Is it a file?

Answer.

- Seed: a parameter, because it changes the result.
- Commit: a tag, metadata that links the run to code.

- AUC: a metric, numeric and comparable across runs.
- Plot: an artifact file.

Parameters for settings, tags for identity, metrics for numbers, artifacts for files.

Common wrong answers.

- seed = `tag`: The seed is a setting that changes the output. Log it as a parameter so searches can filter on it.
- auc = `param`: Parameters are strings. Metrics are numeric and sortable.
- commit = `metric`: A commit hash is not a number to compare. Tag it.

B14-L2 Registry versions and aliases, not stages

B14-L2 exercise 1

Hints. (1) Which reference changes without editing the job? (2) What replaced stages? (3) Aliases.

Answer. b) `models:/churn_model@champion` Because: r1) The alias moves on promotion and rollback, so the job's code never changes.

`@champion` resolves to whichever version currently holds the alias.

Common wrong answers.

- `a`: A fixed version number means editing the job for every release.
- `c`: Stages are deprecated since MLflow 2.9 and not supported in Unity Catalog.
- `d`: Files have no version history, run link or access control.
- `b|r2`: Right answer, wrong reason. Speed is the same. Indirection is the benefit.

B14-L2 exercise 2

Hints. (1) Follow each assignment in order. (2) The last line is a rollback. (3) Both point at 3.

Answer.

```
1 {'champion': '3', 'previous_champion': '3'}
```

Promotion saved the old champion, and rollback restored it in one assignment. Versions 3 and 5 are both unchanged.

Common wrong answers.

- `{'champion': '5', 'previous_champion': '3'}`: The last line rolls back: champion takes previous_champion's version, 3.

B14-L2 exercise 3

Hints. (1) The MlflowClient method that sets an alias. (2) The URI separator for aliases. (3) `set_registered_model_alias` and `@`.

Answer.

```
client.set_registered_model_alias("churn_model", "champion", "4")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

`set_registered_model_alias(name, alias, version)` moves the label. `models:/name@alias` loads through it.

Common wrong answers.

- g2 = `/`: models:/name/X reads X as a version or a stage. Aliases use @.

B14-L2 exercise 4

Hints. (1) Resolve the champion through its alias, then read both scores from their runs. (2) Return False early when cand_score < champ_score + min_gain. (3) Set previous_champion before moving champion.

Answer.

```
from __future__ import annotations

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from mlflow import MlflowClient


def promote_if_better(client: MlflowClient, name: str, candidate: str, metric: str, min_gain: float) -> bool:
    """Move the champion alias to candidate if it beats the champion by min_gain.

    Args:
        client: MLflow client, or any object with the same methods.
        name: Registered model name.
        candidate: Candidate version number, as a string.
        metric: Run metric to compare, higher is better.
        min_gain: Smallest improvement that justifies promotion.

    Returns:
        True if the alias moved.
    """
    champ = client.get_model_version_by_alias(name, "champion")
```

```
    champ_score = client.get_run(champ.run_id).data.metrics[metric]
    cand_score = client.get_run(client.get_model_version(name,
candidate).run_id).data.metrics[metric]
    if cand_score < champ_score + min_gain:
        return False
    client.set_registered_model_alias(name, "previous_champion",
champ.version)
    client.set_registered_model_alias(name, "champion", candidate)
    return True
```

Promotion is a comparison plus two alias moves. The margin guards against noise, and the saved alias makes rollback trivial.

B14-L2 exercise 5

Hints. (1) How are Unity Catalog objects named? (2) What replaced stages? (3) Grant to groups (lesson A4-L2).

Answer.

- Three-level names place the model in a catalog and schema, governed like tables.
- Jobs load by alias. Unity Catalog models do not support stages.
- Grant the right to move aliases to a release group, not to every trainer.

Govern models like tables: three-level names, aliases for releases, group grants for who can release.

Common wrong answers.

- pointer = `stage`: Stages are deprecated and not supported for models in Unity Catalog.
- who = `anyone`: Moving the alias changes production. Limit it to a group, as with any production change.
- name = `plain`: Unity Catalog models use catalog.schema.model names.

B14-L3 Reproduce a run from its record

B14-L3 exercise 1

Hints. (1) What does a different hash mean? (2) Can you reproduce on different data? (3) Find the original.

Answer. b) Stop. You would be reproducing on different data. Find the original input. Because: r1) Any changed byte changes the hash, so the data is not the data the run used.

A hash mismatch means different data. Reproduction needs the original input, which is why raw inputs are kept immutable (lesson B6-L2).

Common wrong answers.

- `a`: Same name, different content. The hash exists for exactly this check.
- `c`: Rewriting the record destroys the evidence of what the run used.
- `d`: Averaging runs on the wrong data does not recover the right result.
- `b|r2`: Right answer, wrong reason. The hash is reliable. That is why the mismatch matters.

B14-L3 exercise 2

Hints. (1) What does + do with two strings? (2) And with two ints? (3) Concatenation, then addition.

Answer.

```
1 71 8
```

Unconverted parameters behave like text. Convert them back to their types before use.

Common wrong answers.

- `8 8`: '7' + '1' joins strings: '71'. Convert with int() first.

B14-L3 exercise 3

Hints. (1) Which step provides the lock file? (2) Which check should stop you early? (3) Compare last.

Answer.

1. Check out the logged git commit
2. Build a fresh environment from the lock file at that commit
3. Load the input and verify its SHA-256 against input_sha256
4. Convert the logged parameters back to their types
5. Rerun and compare the metric within a tolerance

Code, then environment, then data, then settings, then the comparison.

Common wrong answers.

- The lock file that defines the environment lives at the commit. Check it out first.
- Verify the data before rerunning, or a mismatch wastes the run and confuses the comparison.

B14-L3 exercise 4

Hints. (1) Find missing keys with a set difference. (2) Raise KeyError with the sorted list. (3) `{key: cast(params[key]) for key, cast in types.items()}`.

Answer.

```python
def parse_params(params: dict[str, str], types: dict[str, type]) ->
    dict[str, object]:
    """Convert logged string parameters back to their types.

    Args:
        params: Parameters as MLflow returns them.
        types: Expected keys and the type to convert each to.

    Returns:
        Typed parameters for the expected keys.

    Raises:
        KeyError: If an expected key is missing.
        ValueError: If a value cannot be converted.
    """
    missing = sorted(set(types) - set(params))
    if missing:
        raise KeyError(f"run is missing parameters: {missing}")
    return {key: cast(params[key]) for key, cast in types.items()}
```

Explicit types turn the run record back into the exact settings. Failing on a missing key beats silently rerunning with a default.

B14-L3 exercise 5

Hints. (1) Is 0.004 rounding? (2) What does the record not contain? (3) Fix the record, not the check.

Answer.

- A gap of 0.004 is a real difference, not rounding.
- Compare every recorded identity, then look for settings that were never logged.
- Log what was missing, so the next check passes for the right reason.

A failed reproduction points at a gap in the record. Close the gap, keep the check.

Common wrong answers.

- verdict = `fine`: 0.004 AUC is larger than many model improvements you would ship. Something differs.
- next = `relog`: Editing the old run erases the evidence of what it produced.
- fix = `widen`: A wide tolerance hides incomplete records instead of fixing them.

B15-L1 Sentence embeddings for open-ended answers

B15-L1 exercise 1

Hints. (1) What defines the meaning of a direction? (2) The model that produced it. (3) Use one model for everything you compare.

Answer. b) No. Re-encode both years with the same model. Because: r1) Each model defines its own space. The same position means different things in each.

Re-encode all texts with one model and record which one.

Common wrong answers.

- `a`: Matching dimensions do not make the spaces the same.
- `c`: Normalising fixes length, not the meaning of directions.
- `d`: Averaging vectors from different spaces produces meaningless numbers.
- `b|r2`: Right answer, wrong reason. The issue is consistency, not quality.

B15-L1 exercise 2

Hints. (1) Dot product: 3×1 + 4×0. (2) Lengths: 5 and 1. (3) 3 / 5.

Answer.

```
3.0 0.6
```

A dot product equals cosine similarity only for unit-length vectors. Normalise or divide by the lengths.

Common wrong answers.

- `0.6 0.6`: The raw dot product is 3.0. Only after dividing by the lengths (5 and 1) do you get 0.6.

B15-L1 exercise 3

Hints. (1) One length per row. (2) keepdims makes the shape broadcast. (3) `1` and `norms`.

Answer.

```
norms = np.linalg.norm(matrix, axis=1, keepdims=True)
unit = matrix / norms
```

`axis=1` gives each row's length, and `keepdims=True` keeps a column shape that divides row-wise. Guard rows with zero length.

Common wrong answers.

- g1 = `0`: axis=0 gives one norm per column. Each row needs its own length: axis=1.

B15-L1 exercise 4

Hints. (1) Divide dot products by both lengths. (2) Start similarities at -inf, fill only rows with non-zero length. (3) `np.argsort(-sims, kind="stable")[:k]`, then convert to int.

Answer.

```python
import numpy as np


def top_k_similar(query: np.ndarray, matrix: np.ndarray, k: int) -> list[int]:
    """Row indices of the k rows most cosine-similar to query.

    Args:
        query: Vector of length d.
        matrix: Array of shape (n, d).
        k: Number of indices to return.

    Returns:
        Indices, most similar first. Zero-length rows rank last.

    Raises:
        ValueError: If query has zero length.
    """
    q_norm = np.linalg.norm(query)
    if q_norm == 0:
        raise ValueError("query vector has zero length")
    norms = np.linalg.norm(matrix, axis=1)
    sims = np.full(len(matrix), -np.inf)
    ok = norms > 0
    sims[ok] = (matrix[ok] @ query) / (norms[ok] * q_norm)
    return [int(i) for i in np.argsort(-sims, kind="stable")[:k]]
```

Cosine similarity ignores length, so a long vector pointing elsewhere does not win. Zero rows are kept but ranked last.

B15-L1 exercise 5

Hints. (1) One space for all comparisons. (2) What makes a cached vector reusable? (3) Mixing models breaks comparisons.

Answer.

- One multilingual model puts all languages in one space. Test it on your data.

- Store the identity of text and model, so vectors can be reused and audited.
- Reuse vectors and keep the model fixed. Changing model means re-encoding everything.

Fix the model, record identities, and reuse vectors only within one model.

Common wrong answers.

- model = `per_lang`: Different models give different spaces. Answers in different languages could not be compared.
- model = `english`: An English-only model maps other languages poorly. Test before you trust it.
- update = `newest`: New and old vectors would come from different spaces.
- store = `vector_only`: Without the model name and text hash, nobody knows which vectors can be reused.

B15-L2 HDBSCAN, clusters with a noise label

B15-L2 exercise 1

Hints. (1) What does -1 mean in HDBSCAN? (2) Where do those points sit? (3) In sparse regions.

Answer. b) Those answers are in sparse regions and share no dense theme with enough others. Because: r1) -1 is the noise label for points that no dense cluster claims.

Noise answers are real answers without a shared theme. Report their share and read some of them.

Common wrong answers.

- `a`: Noise is a result, not a failure.
- `c`: -1 is not a cluster. Counting it as one mixes unrelated answers.
- `d`: Duplicates are close together, so they would usually form or join a cluster.
- `b|r2`: Right answer, wrong reason. The noise share depends on the data and the settings.

B15-L2 exercise 2

Hints. (1) Remove -1 before counting clusters. (2) Noise: 2 of 10. (3) Largest cluster: 5 of the 8 clustered points.

Answer.

```
2 0.2 0.625
```

Two clusters, 20% noise, and the largest cluster holds 62.5% of clustered answers.

Common wrong answers.

- `3 0.2 0.5`: -1 is noise, not a cluster. Two clusters, and the largest (5 of 8) is 0.625 of clustered points.

B15-L2 exercise 3

Hints. (1) The smallest group you would call a theme. (2) Copy the input instead of changing it. (3) `min_cluster_size` and `True`.

Answer.

```
labels = HDBSCAN(min_cluster_size=20, copy=True).fit_predict(vectors)
```

`min_cluster_size` sets the smallest theme. `copy=True` protects your array and silences the scikit-learn 1.8 warning.

Common wrong answers.

- g1 = `n_clusters`: HDBSCAN has no n_clusters. The main setting is min_cluster_size.

B15-L2 exercise 4

Hints. (1) Handle the empty array first. (2) `clustered = labels[labels != -1]`. (3) `np.unique(clustered, return_counts=True)` gives cluster sizes.

Answer.

```
import numpy as np


def summarise_labels(labels: np.ndarray) -> dict[str, float | int | None]:
    """Cluster count, noise share and largest cluster share.

    Args:
        labels: Cluster labels, -1 for noise.

    Returns:
        n_clusters, noise_share (of all points) and largest_share (of clustered points).
    """
    n = len(labels)
    if n == 0:
        return {"n_clusters": 0, "noise_share": None, "largest_share": None}
    clustered = labels[labels != -1]
    sizes = np.unique(clustered, return_counts=True)[1]
```

```
    return {
        "n_clusters": int(len(sizes)),
        "noise_share": float((labels == -1).mean()),
        "largest_share": float(sizes.max() / len(clustered)) if
    len(clustered) > 0 else None,
     }
```

Noise is reported, not clustered. Guarded denominators keep all-noise and empty results honest.

B15-L2 exercise 5

Hints. (1) What does an unbalanced result tell you? (2) Which setting controls the smallest theme? (3) Reproducibility (lesson B6-L2).

Answer.

- Read before tuning: the samples show whether the big cluster is one theme or a mix.
- Adjust min_cluster_size (and min_samples) on purpose, one change at a time.
- Record every setting, so the themes can be reproduced next quarter.

Diagnose with samples, tune deliberately, and record everything that shaped the clusters.

Common wrong answers.

- diagnose = `publish`: One dominant cluster and 40 tiny ones rarely reflect real themes. Look first.
- tune = `kmeans`: k-means forces all 45% noise into themes, inflating their shares.
- record = `final`: Without the settings, nobody can rebuild or compare the themes.

B15-L3 Validate clusters before you name them

B15-L3 exercise 1

Hints. (1) Would the same clusters appear on a new sample? (2) Can people name them? (3) Three checks, not one.

Answer. b) No. Also check stability across reruns and read samples from each cluster. Because: r1) Separation in one run says nothing about whether clusters recur or mean anything.

Separation, stability and meaning together. One number is a start, not a conclusion.

Common wrong answers.

- `a`: No threshold turns one number into proof. Check stability and meaning too.
- `c`: 1 is almost never reached on real data. Good separation is not the only test.
- `d`: A dashboard layout is not evidence about the data.
- `b|r2`: Right answer, wrong reason. Silhouette is useful. It is not sufficient.

B15-L3 exercise 2

Hints. (1) Does ARI care what the labels are called? (2) Are the groupings the same in the first case? (3) The second is worse than chance.

Answer.

```
1.0 -0.5
```

ARI compares groupings, not names. It can go below 0 when agreement is worse than chance.

Common wrong answers.

- `0.0 -0.5`: ARI ignores label names. Swapping 0 and 1 gives the same grouping: 1.0.
- `1.0 0.0`: The second grouping splits every pair the first one keeps together, which scores below chance: -0.5.

B15-L3 exercise 3

Hints. (1) Exclude the noise label. (2) Use the same mask for labels. (3) `!=` and `keep`.

Answer.

```
keep = labels != -1
score = silhouette_score(X[keep], labels[keep])
```

One mask for both arrays keeps points and labels aligned.

Common wrong answers.

- g1 = `==`: That keeps only the noise. Keep points whose label is not -1.

B15-L3 exercise 4

Hints. (1) Mask out -1 first. (2) Count clusters among the remaining labels. (3) silhouette_score needs 2 <= clusters <= n - 1.

Answer.

```
import numpy as np
from sklearn.metrics import silhouette_score
```

```python


def silhouette_without_noise(X: np.ndarray, labels: np.ndarray) -> float | None:
    """Silhouette over clustered points only.

    Args:
        X: Feature matrix.
        labels: Cluster labels, -1 for noise.

    Returns:
        The silhouette score, or None when it is undefined.
    """
    keep = labels != -1
    kept = labels[keep]
    n_clusters = len(np.unique(kept))
    if n_clusters < 2 or n_clusters >= len(kept):
        return None
    return float(silhouette_score(X[keep], kept))
```

Noise is not a cluster, so it stays out. Guarding the cluster count turns an exception into a clear "undefined".

B15-L3 exercise 5

Hints. (1) What does noise do to silhouette? (2) Would the themes come back on new data? (3) Which examples are honest?

Answer.

- Silhouette on clustered points, with noise reported beside it.
- Stability across reruns, naming the themes that do not recur.
- Random verbatims, so readers see what a theme typically contains.

Honest methods show separation, stability and representative evidence.

Common wrong answers.

- separation = `sil_all`: Noise is scattered by definition. Counting it as a cluster drags the score down and mixes two messages.
- stability = `skip`: A clustering can be well separated in one run and different in the next.
- examples = `best`: Hand-picked examples make every theme look cleaner than it is.

B16-L1 Choose the device once, move model and data to it

B16-L1 exercise 1

Hints. (1) Is Tensor.to in place? (2) What happens to an unassigned return value? (3) Reassign.

Answer. b) Where it was before. .to() on a tensor returns a new tensor. Because: r1) Tensor.to returns a new tensor. Only nn.Module.to changes the object in place.

`x = x.to(device)`. For modules, `model.to(device)` is in place and returns the same model.

Common wrong answers.

- `a`: That is true for models, not tensors. Write x = x.to(device).
- `c`: The new tensor is discarded because it is not assigned.
- `d`: x is unchanged. The moved copy is what disappears.
- `b|r2`: Right answer, wrong reason. .to() works on tensors, it just does not modify them.

B16-L1 exercise 2

Hints. (1) Which call is in place? (2) Module.to. (3) Tensor.to returns a copy.

Answer.

```
cpu meta
```

The asymmetry catches everyone once. Reassign tensors, and do not rely on reassigning models.

Common wrong answers.

- `meta meta`: Tensor.to returns a new tensor. t itself stays on the CPU.
- `cpu cpu`: Module.to moves parameters in place, so the model is on meta.

B16-L1 exercise 3

Hints. (1) The device-agnostic method. (2) NumPy needs CPU memory. (3) `to` and `cpu`.

Answer.

```
x = x.to(device)
preds = model(x).detach().cpu().numpy()
```

`.to(device)` keeps code portable. `.cpu()` before `.numpy()` avoids an error on GPU tensors.

Common wrong answers.

- g1 = `cuda`: .cuda() hardcodes the GPU. .to(device) works on any machine.

B16-L1 exercise 4

Hints. (1) Check prefer_gpu in every GPU branch. (2) CUDA before MPS. (3) Fall through to cpu.

Answer.

```
def pick_device_name(prefer_gpu: bool, cuda_ok: bool, mps_ok: bool) ->
    str:
    """Name of the device to use: cuda, then mps, then cpu.

    Args:
        prefer_gpu: False forces the CPU, for tests and CI.
        cuda_ok: Result of torch.cuda.is_available().
        mps_ok: Result of torch.backends.mps.is_available().

    Returns:
        "cuda", "mps" or "cpu".
    """
    if prefer_gpu and cuda_ok:
        return "cuda"
    if prefer_gpu and mps_ok:
        return "mps"
    return "cpu"
```

Passing availability in makes the decision a pure function (lesson B2-L1): easy to test on a laptop with no GPU.

B16-L1 exercise 5

Hints. (1) Which line tries to move x? (2) Does Tensor.to change x? (3) Reassign it.

Answer. The bug is on line 12.

Fixed version:

```
from collections.abc import Iterable

import torch


def validate(model: torch.nn.Module, loader: Iterable[tuple[torch.Tensor,
    torch.Tensor]], device: torch.device) -> float:
    """Accuracy on the validation loader."""
    model.eval()
    correct, total = 0, 0
    with torch.inference_mode():
        for x, y in loader:
            x = x.to(device)
```

```
            preds = model(x).argmax(dim=1)
            correct += int((preds.cpu() == y).sum())
            total += len(y)
    return correct / total if total > 0 else float("nan")
```

On the CPU, x and the model already share a device, so the bug hides. On the GPU, x never moved. Reassign: `x = x.to(device)`.

Common wrong answers.

- Line 14: preds.cpu() is right: y stays on the CPU, so the comparison happens there.
- Line 8: model.eval() is correct for validation.

B16-L2 A training loop that does what you think

B16-L2 exercise 1

Hints. (1) Does backward overwrite or add? (2) It adds. (3) zero_grad resets.

Answer. b) Gradients accumulate across batches, so steps use the sum of all earlier gradients. Because: r1) backward() adds into .grad. Only zero_grad() resets it.

Call `optimizer.zero_grad()` before each backward pass.

Common wrong answers.

- `a`: PyTorch accumulates on purpose, for gradient accumulation. Clearing is your job.
- `c`: No error: training runs with growing, wrong updates.
- `d`: It keeps changing, with updates that grow each batch.
- `b|r2`: Right answer, wrong reason. The optimizer uses the accumulated gradients.

B16-L2 exercise 2

Hints. (1) The gradient of 2w is 2. (2) How many backward calls? (3) They add up.

Answer.

```
6.0
```

Three accumulations of 2.0. With `zero_grad()` each iteration it would print 2.0.

Common wrong answers.

- `2.0`: Each backward adds 2.0 to w.grad. Three calls without zeroing give 6.0.

B16-L2 exercise 3

Hints. (1) Data first. (2) Clear gradients before computing new ones. (3) Compute, then apply.

Answer.

1. x, y = x.to(device), y.to(device)
2. optimizer.zero_grad()
3. loss = loss_fn(model(x), y)
4. loss.backward()
5. optimizer.step()

Move, clear, forward, backward, step. zero_grad can also sit at the end of the iteration, as long as it runs between steps.

Common wrong answers.

- zero_grad after backward erases the gradients the step needs.
- step before backward uses stale or empty gradients.

B16-L2 exercise 4

Hints. (1) Split the list: before the window and the window. (2) Compare the window's best with the earlier best. (3) Subtract min_delta from the earlier best.

Answer.

```python
def should_stop(val_losses: list[float], patience: int, min_delta: float = 0.0) -> bool:
    """Early-stopping rule on validation losses, one per epoch.

    Args:
        val_losses: Validation loss per epoch, oldest first.
        patience: Epochs allowed without improvement.
        min_delta: Smallest decrease that counts as improvement.

    Returns:
        True when training should stop.

    Raises:
        ValueError: If patience is below 1.
    """
    if patience < 1:
        raise ValueError("patience must be at least 1")
```

```
    if len(val_losses) <= patience:
        return False
    best_before = min(val_losses[:-patience])
    return min(val_losses[-patience:]) > best_before - min_delta
```

Comparing best against best ignores noisy single epochs. min_delta stops training from continuing on improvements too small to matter.

B16-L2 exercise 5

Hints. (1) What does CrossEntropyLoss expect as input? (2) What does forward return? (3) Logits, not probabilities.

Answer. The bug is on line 13.

Fixed version:

```
import torch
from torch import nn


class Classifier(nn.Module):
    """Two-layer classifier. Train with nn.CrossEntropyLoss."""

    def __init__(self, n_in: int, n_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, 32), nn.ReLU(),
    nn.Linear(32, n_classes))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

Softmax squeezes outputs into [0, 1], and CrossEntropyLoss then applies log-softmax again. The loss and its gradients are wrong. Apply softmax only when you need probabilities for reporting.

Common wrong answers.

- Line 10: The architecture is fine. The output passed to the loss is the problem.

B16-L3 Reproducible PyTorch runs and safe checkpoints

B16-L3 exercise 1

Hints. (1) What does weights-only loading allow? (2) What does a state dict contain? (3) Tensors and plain values.

Answer. b) Save model.state_dict() instead, rebuild the model class in code, and load the weights into it. Because: r1) State dicts hold only tensors and plain values, which load safely under the weights-only default.

Save `state_dict()`, keep the model class in version-controlled code, and load weights into a fresh instance.

Common wrong answers.

- `a`: That loads arbitrary pickles, which can run code. Only do it for files you created and trust, and prefer fixing how you save.
- `c`: Downgrading removes a safety default and freezes you on an old version.
- `d`: The extension changes nothing about the content.
- `b|r2`: Right answer, wrong reason. Objects can be saved. Loading them safely is the problem.

B16-L3 exercise 2

Hints. (1) Which generator does torch.rand use here? (2) Do g1 and g2 share state with it? (3) No.

Answer.

```
True
```

Dedicated generators isolate the shuffle order from other random calls. That is why DataLoaders get their own.

Common wrong answers.

- `False`: torch.rand(100) uses the global generator. g1 and g2 are separate and unaffected.

B16-L3 exercise 3

Hints. (1) The Generator method that seeds it. (2) The DataLoader argument that takes it. (3) `manual_seed` and `generator`.

Answer.

```
loader_gen = torch.Generator().manual_seed(seed)
loader = DataLoader(ds, batch_size=64, shuffle=True, generator=loader_gen)
```

`torch.Generator().manual_seed(seed)` returns the generator, ready to pass as `generator=`.

Common wrong answers.

- g2 = `seed`: DataLoader has no seed argument. Pass a torch.Generator as generator=.

B16-L3 exercise 4

Hints. (1) Three global generators: random, NumPy, torch. (2) One dedicated generator for the loader. (3) `return torch.Generator().manual_seed(seed)`.

Answer.

```python
import random

import numpy as np
import torch


def seed_everything(seed: int) -> torch.Generator:
    """Seed Python, NumPy and PyTorch global state at the entry point.

    Args:
        seed: The run's seed, logged with the run.

    Returns:
        A seeded generator to pass to shuffling DataLoaders.
    """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    return torch.Generator().manual_seed(seed)
```

Seeding every global generator at the entry point covers code you do not control. The returned generator keeps batch order independent of everything else.

B16-L3 exercise 5

Hints. (1) What must be restored to continue exactly? (2) Which two checkpoints serve different needs? (3) Keep the safe default.

Answer.

- State dicts plus epoch, RNG state and best score: everything needed to resume exactly.
- Save the last epoch for resuming, and the best epoch for release.
- Load with the safe default into a model defined in code.

Resume from state, release the best epoch, and never unpickle files you did not create.

Common wrong answers.

- content = `object`: A pickled object ties the file to the exact code layout and fails the weights-only default.

- when = `end`: An interruption at hour 11 loses everything, and the last epoch is rarely the best.
- load = `unsafe`: Unpickling files from shared storage can run arbitrary code.

B17-L1 Unit tests for data code that catch real bugs

B17-L1 exercise 1

Hints. (1) Which buggy values would each assertion let through? (2) Is 0.0 falsy? (3) Only one rejects 0.0.

Answer. b) `assert share(3, 0) is None` Because: r1) It fails for every result except None, including 0.0.

Write assertions that pass only for the correct value.

Common wrong answers.

- `a`: 0.0 is falsy too, so a buggy 0.0 would pass.
- `c`: That asserts the bug.
- `d`: Almost any wrong value passes.
- `b|r2`: Right answer, wrong reason. Precision matters, not speed.

B17-L1 exercise 2

Hints. (1) What are the dtypes? (2) Does == compare dtypes? (3) assert_frame_equal does.

Answer.

```
False True
AssertionError
```

`==` compares values only. `assert_frame_equal` also checks dtypes, which catches a column that silently became float.

Common wrong answers.

- `True True equal`: Values match, but dtypes differ (int64 and float64). assert_frame_equal checks dtypes by default.

B17-L1 exercise 3

Hints. (1) The pytest marker for a table of cases. (2) Float comparison with tolerance. (3) `parametrize` and `approx`.

Answer.

```python
@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
    (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == pytest.approx(expected)
```

`parametrize` runs the test once per row. `approx` compares floats with a sensible tolerance.

Common wrong answers.

- g1 = `parameterize`: pytest spells it parametrize, without the extra e.

B17-L1 exercise 4

Hints. (1) A test must fail for wrong values, not only for crashes. (2) Compare with the exact expected value: `== 0.25`. (3) Test for None with `is None`.

Answer.

```python
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    if total == 0:
        return None
    return part / total


def test_share_basic() -> None:
    assert share(1, 4) == 0.25


def test_share_zero_total() -> None:
    assert share(3, 0) is None
```

The checker plays the role of a mutation test: it breaks the code and checks your tests notice. Precise assertions catch both bugs.

B17-L1 exercise 5

Hints. (1) Where does the expected value come from? (2) Which inputs break data code? (3) Floating point.

Answer.

- Tiny explicit inputs with hand-computed answers.
- Every edge the function promises to handle.
- approx for floats that come from arithmetic.

Independent expected values, every edge, and tolerant float comparison make tests that catch bugs.

Common wrong answers.

- normal = `sample`: An expected value produced by the function under test only checks that it agrees with itself.
- edges = `skip`: Line coverage is not behaviour coverage. Edge branches are where bugs ship.
- compare = `exact`: 0.1 + 0.2 != 0.3 in floating point. Use approx.

B17-L2 Property-based tests with Hypothesis

B17-L2 exercise 1

Hints. (1) What makes a failure easy to debug? (2) A small input. (3) Shrinking.

Answer. b) A shrunk input: the simplest example it found that still fails. Because: r1) Shrinking turns a messy random failure into a minimal case you can read.

Shrinking reduced a weighted-mean failure to one value with one zero weight.

Common wrong answers.

- `a`: Hypothesis keeps simplifying the failing input before reporting it.
- `c`: It reports one minimal example, not the whole search.
- `d`: The failing example is shown, such as pairs=[(0.0, 0.0)].
- `b|r2`: Right answer, wrong reason. It generates complex inputs, then shrinks the failure.

B17-L2 exercise 2

Hints. (1) What special values can st.floats() produce? (2) Is nan equal to itself? (3) No.

Answer.

```
fail
```

The test fails on `x=nan`. Bound the strategy or pass `allow_nan=False` unless the function must handle nan.

Common wrong answers.

- `pass`: st.floats() generates nan by default, and nan == nan is False.

B17-L2 exercise 3

Hints. (1) The decorator that configures a test. (2) The floats argument for nan. (3) `settings` and `allow_nan`.

Answer.

```
@settings(max_examples=500)
@given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False))
def test_round_trip(x):
    assert parse(format(x)) == x
```

`@settings(max_examples=500)` raises the search budget. `allow_nan=False` keeps nan out when the function does not promise to handle it.

Common wrong answers.

- g2 = `allow_infinity`: Bounds already exclude infinities. The gap is for nan.

B17-L2 exercise 4

Hints. (1) Decorate with @given and the strategy from the prompt. (2) The test receives rows as its argument. (3) Compare dedupe applied twice with dedupe applied once.

Answer.

```
from hypothesis import given, strategies as st


@given(st.lists(st.tuples(st.sampled_from(["a", "b", "c"]),
   st.integers(0, 5))))
def test_dedupe_idempotent(rows: list[tuple[str, int]]) -> None:
    assert dedupe(dedupe(rows)) == dedupe(rows)
```

A small id alphabet and small timestamps make duplicates and ties likely. Idempotence is one property. Pair it with a grain property, one row per id, which catches a dedupe that keeps ties: that bug is idempotent, so idempotence alone misses it.

B17-L2 exercise 5

Hints. (1) What must never change? (2) What must come back? (3) What can be compared?

Answer.

- Totals do not depend on row order.
- Saving and loading must not lose data or dtypes.
- A weighted mean stays within the range of its inputs.
- A rewrite must agree with the code it replaces.

Good properties come from what the function promises: invariance, round trips, bounds and agreement.

Common wrong answers.

- total = `bounds`: A total can exceed every input value. Order invariance is the property that holds.
- io = `invariance`: For I/O the key promise is that nothing is lost: a round trip.
- fast = `bounds`: Bounds would pass many wrong rewrites. Compare with the trusted version.

B17-L3 Measure before you optimise

B17-L3 exercise 1

Hints. (1) What is the most a speed-up of each part can save? (2) Compare 36 with 1. (3) Follow the profile.

Answer. b) File reading. Because: r1) A 1-minute function can save at most 1 minute. Reading holds 90% of the time.

Fix the biggest measured cost first. Guesses about "slow loops" waste days.

Common wrong answers.

- `a`: Even an infinitely fast weighting function saves only 1 minute of 40.
- `c`: Effort should follow cost. One of them holds 36 of 40 minutes.
- `d`: Hardware is a last resort once the code's own costs are known.
- `b|r2`: Right answer, wrong reason. Here the profile says I/O. Elsewhere it may not.

B17-L3 exercise 2

Hints. (1) What does repeat control? (2) What does number control? (3) One total per round.

Answer.

```
5
```

Five rounds, each the total time for 100 calls. Report `min(rounds) / 100` as the per-call time.

Common wrong answers.

- `100`: number is the calls per round. repeat is the number of rounds returned.

- `500`: repeat returns one total per round, not one per call.

B17-L3 exercise 3

Hints. (1) Time including everything a function calls. (2) The Stats method that prints rows. (3) `cumulative` and `print_stats`.

Answer.

```
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Cumulative time points to the high-level function that owns the cost. Then drill into it.

Common wrong answers.

- g1 = `calls`: Sorting by call count shows busy functions, not expensive ones. Sort by cumulative time.

B17-L3 exercise 4

Hints. (1) Validate repeat and number first. (2) Time each round with perf_counter before and after the inner loop. (3) Keep the minimum of round_time / number.

Answer.

```
import time
from collections.abc import Callable


def best_time(fn: Callable[[], object], repeat: int = 5, number: int = 10) -> float:
    """Best per-call time over several rounds.

    Args:
        fn: Function to time, called with no arguments.
        repeat: Number of rounds.
        number: Calls per round.

    Returns:
        The smallest per-call time in seconds.

    Raises:
        ValueError: If repeat or number is below 1.
    """
    if repeat < 1 or number < 1:
        raise ValueError("repeat and number must be at least 1")
    best = float("inf")
    for _ in range(repeat):
        start = time.perf_counter()
        for _ in range(number):
```

```
            fn()
        best = min(best, (time.perf_counter() - start) / number)
    return best
```

The minimum over rounds estimates the code's own cost, because interference only adds time. Dividing by `number` gives a per-call figure you can compare.

B17-L3 exercise 5

Hints. (1) What runs between start and the end of the round? (2) Is reading the file part of add_features? (3) Move setup out of the timed region.

Answer. The bug is on line 11.

Fixed version:

```
import time

import pandas as pd


def benchmark(path: str, rounds: int) -> float:
    """Best time of add_features over rounds."""
    df = pd.read_parquet(path)
    best = float("inf")
    for _ in range(rounds):
        start = time.perf_counter()
        add_features(df)
        best = min(best, time.perf_counter() - start)
    return best
```

The file read is setup, and it dominates the timing. Read once before the loop, then time only the function.

Common wrong answers.

- Line 13: Taking the minimum is right. The timed region includes more than the function.

B19-L1 How pytest finds, runs and reports your tests

B19-L1 exercise 1

Hints. (1) Check the file name and the function name separately. (2) Files: `test_*.py` or `*_test.py`. Functions: `test*`. (3) Only one option matches both.

Answer. c) `def test_weights():` in `tests/weights_test.py` Because: r1) Both the file name (`*_test.py`) and the function name (`test*`) match the default patterns.

Collection needs a matching file name and a matching function name. A class needs a `Test` prefix and no `__init__`.

Common wrong answers.

- `a`: The file matches, but the function name must start with `test`.
- `b`: The function matches, but `weight_checks.py` matches neither `test_*.py` nor `*_test.py`.
- `d`: Test classes must start with `Test`. `WeightTests` does not.
- `c|r2`: Right choice, wrong reason. pytest collects only names that match its patterns.

B19-L1 exercise 2

Hints. (1) `match` is searched in the message with a regular expression. (2) Does `total must be >= 0, got -5` contain `got -` and a digit? (3) Does it contain `must be > 0`?

Answer.

```
1 first ok
2 pattern did not match
```

Both blocks raise a ValueError. Only the first message matches its pattern. A failed match is reported as an AssertionError.

Common wrong answers.

- `first ok`: The second block raises a ValueError, but its message does not contain `must be > 0`. A failed match raises AssertionError, which the `except` catches.
- `pattern did not match`: `match` is a regular expression searched in the message. `got -\d` finds `got -5`, so the first block passes.

B19-L1 exercise 3

Hints. (1) `-k` takes an expression matched against test names. (2) The stop-early flag is one letter. (3) `-k zero -x`.

Answer.

```
1 pytest -k zero -x
```

`-k zero` selects tests whose names contain `zero`. `-x` stops the run at the first failure.

Common wrong answers.

- g1 = `zero`; g2 = `s`: `-s` shows print output. The flag that stops at the first failure is `-x`.

- g1 = `zero`; g2 = `q`: `-q` makes the output shorter. Stopping early is `-x`.

B19-L1 exercise 4

Hints. (1) A try/except that ignores the 'no error' case passes for broken code. (2) `with pytest.raises(ValueError, match="..."):` around the call. (3) Use `negative weight` and `total weight is 0` as the patterns.

Answer.

```python
import pytest


def normalise(weights: list[float]) -> list[float]:
    """Scale weights so they sum to 1."""
    for i, w in enumerate(weights):
        if w < 0:
            raise ValueError(f"negative weight at index {i}")
    total = sum(weights)
    if total == 0:
        raise ValueError("total weight is 0")
    return [w / total for w in weights]


def test_negative_weight_raises() -> None:
    with pytest.raises(ValueError, match="negative weight"):
        normalise([2.0, -1.0])


def test_zero_total_raises() -> None:
    with pytest.raises(ValueError, match="total weight is 0"):
        normalise([0.0, 0.0])
```

`pytest.raises` fails when nothing is raised and lets other exception types through. `match` rules out a ValueError raised for the wrong reason.

B19-L1 exercise 5

Hints. (1) Start from what CI tells you. (2) Reproduce and read before you change anything. (3) Widen the run step by step: one test, last failures, everything.

Answer.

1. Copy the failing test's node ID from the CI log.
2. Run that one test locally: `pytest tests/test_share.py::test_zero_total`.
3. Read the assertion output: the values on both sides and the call that produced them.
4. Change the code until that test passes.
5. Run `pytest --lf` to rerun every test that failed last time.

6. Run the whole suite, then push.

Reproduce the single failure, read pytest's output, fix, then widen the run: last failures first, the whole suite last.

Common wrong answers.

- Read the failure before you change code. The assertion output often names the bug.
- `--lf` is the quick check before the full run, not after it.

B19-L2 Fixtures, shared setup without shared state

B19-L2 exercise 1

Hints. (1) What happens to a shared object when one test changes it? (2) Which scope builds a new object for every test? (3) The default.

Answer. a) The default, function scope Because: r1) Each test gets its own fresh frame, so one test's changes cannot reach another.

Function scope gives every test its own object. Wider scopes are for expensive objects that no test changes.

Common wrong answers.

- `b`: Module scope shares one frame across the file. A change in one test leaks into the next.
- `c`: Session scope shares one frame across the whole run. Save it for expensive, read-only objects.
- `d`: A module-level variable is shared state, the problem fixtures solve.
- `a|r2`: Right scope, wrong reason. A three-row frame is cheap either way. Isolation is the point.

B19-L2 exercise 2

Hints. (1) The fixture has function scope. (2) Code after `yield` runs when that test finishes. (3) Setup, test, teardown, then the same again.

Answer.

1. setup (first time)
2. test_sum
3. teardown (first time)
4. setup (second time)
5. test_len
6. teardown (second time)

For each test, pytest runs the code before `yield`, runs the test, then runs the code after `yield`.

Common wrong answers.

- A function-scoped fixture is torn down after each test and built again for the next.
- pytest builds a function-scoped fixture right before each test, not all at the start.

B19-L2 exercise 3

Hints. (1) The decorator that makes a fixture. (2) One per file is one per module. (3) Hand the value over and come back later for teardown.

Answer.

```
@pytest.fixture(scope="module")
def con():
    connection = duckdb.connect()
    yield connection
    connection.close()
```

`@pytest.fixture(scope="module")` builds one connection per file. `yield` gives it to the tests, and the line after it runs once they finish.

Common wrong answers.

- g1 = `fixture`; g2 = `session`; g3 = `yield`: Session scope shares the connection across every file in the run. The task asks for one per file: module scope.
- g1 = `fixture`; g2 = `module`; g3 = `return`: `return` ends the function, so `connection.close()` never runs. `yield` hands over the value and resumes for teardown.

B19-L2 exercise 4

Hints. (1) The fixture defines an inner function `make` and returns it, without calling it. (2) Inside `make`, build the IDs with `f"R{i}"` and default `in_scope` to `[True] * len(weights)`. (3) In the test: `df = make_respondents([1.0, 3.0], in_scope=[True, False])`, then assert 1.0.

Answer.

```
import pandas as pd
import pytest


def in_scope_weight(df: pd.DataFrame) -> float:
    """Total final_weight of in-scope rows."""
    return float(df.loc[df["in_scope"], "final_weight"].sum())
```

```
 8
 9
10 @pytest.fixture
11 def make_respondents():
12     """Factory for small respondent frames with sensible defaults."""
13
14     def make(weights: list[float], in_scope: list[bool] | None = None) ->
       pd.DataFrame:
15         ids = [f"R{i}" for i in range(1, len(weights) + 1)]
16         flags = in_scope if in_scope is not None else [True] *
       len(weights)
17         return pd.DataFrame(
18             {
19                 "respondent_id": ids,
20                 "final_weight": [float(w) for w in weights],
21                 "in_scope": flags,
22             }
23         )
24
25     return make
26
27
28 def test_in_scope_weight_skips_out_of_scope(make_respondents) -> None:
29     df = make_respondents([1.0, 3.0], in_scope=[True, False])
30     assert in_scope_weight(df) == 1.0
```

The factory builds a new frame on every call, with defaults, so each test states only the values it cares about and no test can change another's data.

B19-L2 exercise 5

Hints. (1) Which test sees different data depending on what ran before? (2) One line shares the frame. Another changes it. (3) The scope argument and the in-place assignment.

Answer. The bug is on line 5, 11.

`scope="module"` builds one frame for the file, and `test_cap_weights` changes it in place. Use function scope, or have the code under test return a new frame.

Common wrong answers.

- Line 5: Module scope shares the frame, which is only a problem because a test changes it. Find the change too.
- Line 11: That line changes the frame. It leaks into the next test only because the fixture is shared. Find what shares it.
- Line 16: This assertion is correct for the fixture's data. It fails because of what happened to the frame earlier.

B19-L3 Parametrize, one test and many readable cases

B19-L3 exercise 1

Hints. (1) Two decorators on one test combine every value with every value. (2) A fixture with params runs each test that uses it once per value. (3) 6 + 2.

Answer.

```
1 8
```

`test_grid` runs for 2 x 3 = 6 combinations and `test_backend` once per param: 8 tests.

Common wrong answers.

- `7`: Stacked parametrize decorators multiply: 2 parts times 3 totals is 6, plus 2 backends.
- `3`: Each row and each fixture param becomes its own test. Count the combinations.
- `5`: Two stacked decorators give every combination, 2 x 3 = 6, not 2 + 3.

B19-L3 exercise 2

Hints. (1) The row now passes. Is that expected? (2) What does strict change? (3) An unexpected pass.

Answer. b) The row is reported as XPASS(strict) and the run fails. Because: r1) strict=True turns an unexpected pass into a failure, so the stale mark gets removed.

With strict=True, a row expected to fail that passes is reported as XPASS(strict) and fails the run, so you remove the mark.

Common wrong answers.

- `a`: That is what happens without strict=True: an xpass is reported, and the stale mark stays forever.
- `c`: xfail runs the row. skip does not.
- `d`: pytest never edits your code. Failing the run is how it tells you.
- `b|r2`: Right outcome, wrong reason. An xfail row can pass. strict=True makes that pass fail the run.

B19-L3 exercise 3

Hints. (1) The next row shows the pattern. (2) One row with options. (3) `pytest.param(..., id=...)`.

Answer.

```python
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="just-below-35"),
        pytest.param(35, "35-54", id="at-35"),
    ],
)
def test_age_band(age, expected):
    assert age_band(age) == expected
```

`pytest.param` wraps one row so it can carry an `id` and marks.

Common wrong answers.

- g1 = `param`; g2 = `name`: The keyword is `id`, as in the next row.
- g1 = `mark`; g2 = `id`: `pytest.mark` holds markers. A single row with options is `pytest.param`.

B19-L3 exercise 4

Hints. (1) One row per age, with its expected band: `(34, "16-34")`. (2) At 35 the band is already 35-54. At 55 it is already 55+. (3) Add `(None, None)` and keep a single assertion in the test body.

Answer.

```python
import pytest


def age_band(age: int | None) -> str | None:
    """Age group for reporting. None stays None."""
    if age is None:
        return None
    if age < 35:
        return "16-34"
    if age < 55:
        return "35-54"
    return "55+"


@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="below-35"),
        pytest.param(35, "35-54", id="at-35"),
        pytest.param(54, "35-54", id="below-55"),
        pytest.param(55, "55+", id="at-55"),
        pytest.param(None, None, id="missing"),
    ],
)
def test_age_band(age: int | None, expected: str | None) -> None:
```

```
26      assert age_band(age) == expected
```

Rows on both sides of each threshold catch an off-by-one change, and the missing-value row catches None falling into the last band.

B19-L3 exercise 5

Hints. (1) Known pairs, or any input? (2) Which tool applies to every test that uses it? (3) Keep raising and non-raising cases apart.

Answer.

- A table of known cases: parametrize.
- The same tests against several implementations: a fixture with params, used by every test.
- A rule for all inputs: a Hypothesis property.
- Invalid inputs: their own parametrized test with pytest.raises, not an if inside the value test.

Tables for known cases, fixture params for implementations, properties for general rules, and a separate table for errors.

Common wrong answers.

- backends = `parametrize`: Parametrizing each test by hand repeats the backend list in every test. A fixture with params applies it to all of them.
- errors = `parametrize`: Mixing raising and non-raising rows in one table forces an if into the test body. Give errors their own test.
- property = `parametrize`: A table checks the cases you thought of. A property checks generated cases, including the ones you did not.

B19-L4 Testing pandas and NumPy code

B19-L4 exercise 1

Hints. (1) Is NaN == NaN? (2) How does assert_frame_equal treat missing values in the same place? (3) One check says different, the other says equal.

Answer.

```
1 False
2 equal
```

`==` follows NaN != NaN. `assert_frame_equal` treats missing values in matching positions as equal, which is what a test needs.

Common wrong answers.

- `True equal`: NaN is never equal to NaN under `==`, so the second cell is False and so is `.all().all()`.
- `False`: `assert_frame_equal` counts missing values in the same place as equal, so it does not raise and `equal` prints.

B19-L4 exercise 2

Hints. (1) What index does a filter keep? (2) Which side should change: the test or the function? (3) The function promises a fresh index.

Answer. a) End the function with `.reset_index(drop=True)` and keep the test as it is. Because: r1) Filtering keeps the old labels [0, 2]. The function promises a fresh index, so the function resets it, and the strict test protects that promise.

Filtering keeps the original labels. When the contract says the index is fresh, the function resets it and the test stays strict.

Common wrong answers.

- `b`: The dtypes match. The failure is about the index.
- `c`: That hides the index, the dtypes and the column names, so later bugs pass too.
- `d`: check_like reorders by label. It cannot turn labels [0, 2] into [0, 1].
- `a|r2`: Right fix, wrong reason. The index matters: a caller that uses .loc[1] gets an error with labels [0, 2].

B19-L4 exercise 3

Hints. (1) The NumPy helper that compares with a tolerance. (2) Relative tolerance times 0 is 0. (3) An absolute tolerance.

Answer.

```
np.testing.assert_allclose(shares, [0.25, 0.75, 0.0], atol=1e-12)
```

`assert_allclose` passes when `|actual - expected| <= atol + rtol * |expected|`. For an expected 0, only `atol` helps.

Common wrong answers.

- g1 = `assert_allclose`; g2 = `rtol`: A relative tolerance is a share of the expected value. A share of 0 is 0, so 1e-15 still fails. Use an absolute tolerance.
- g1 = `assert_array_equal`; g2 = `atol`: assert_array_equal checks exact equality and has no tolerance.

B19-L4 exercise 4

Hints. (1) `actual.sort_values(keys)` returns a new, sorted frame. (2) After sorting, `.reset_index(drop=True)` gives both frames the index 0, 1, 2. (3) Finish with `pd.testing.assert_frame_equal(left, right, check_like=True)`.

Answer.

```python
import pandas as pd


def assert_same_rows(actual: pd.DataFrame, expected: pd.DataFrame, keys: list[str]) -> None:
    """Assert two frames hold the same rows, ignoring row and column order.

    Args:
        actual: Frame produced by the code under test.
        expected: Frame the test expects.
        keys: Columns that identify a row, used to sort both frames.

    Raises:
        AssertionError: If rows, values or dtypes differ.
    """
    left = actual.sort_values(keys).reset_index(drop=True)
    right = expected.sort_values(keys).reset_index(drop=True)
    pd.testing.assert_frame_equal(left, right, check_like=True)
```

Sorting by the keys and resetting the index makes row order irrelevant. `check_like=True` makes column order irrelevant. Everything else stays strict.

B19-L4 exercise 5

Hints. (1) What object does `before` refer to? (2) Assignment copies a reference, not the data. (3) `before = df.copy()`.

Answer. The bug is on line 11.

Fixed version:

```python
import pandas as pd


def add_share(df: pd.DataFrame) -> pd.DataFrame:
    df["share"] = df["w"] / df["w"].sum()
    return df


def test_add_share_keeps_input():
    df = pd.DataFrame({"w": [1.0, 3.0]})
    before = df.copy()
    add_share(df)
    pd.testing.assert_frame_equal(df, before)
```

`before = df` makes a second name for the same frame, so the comparison checks the frame against itself. `df.copy()` keeps the original values.

Common wrong answers.

- Line 5: That line does change the input, but the task is the test: why does it not notice?
- Line 13: The comparison is fine. It compares df with itself, because of how `before` was made.

B19-L5 Testing statistical and random code

B19-L5 exercise 1

Hints. (1) Will the test give the same result every run? (2) Does it assert something a correct interval must satisfy? (3) Seed plus invariants.

Answer. b) Pass `np.random.default_rng(7)`, then assert that lower <= mean <= upper and that the interval is narrower for 400 values than for 40. Because: r1) The seed makes every run identical, and the assertions are properties any correct interval has.

A seeded generator removes chance from the run. Invariants such as ordering and narrowing hold for every correct implementation.

Common wrong answers.

- `a`: Without a seed the draws change every run, so the copied interval fails by chance.
- `c`: A p-value on random data falls below 0.05 about one time in twenty, whatever the code does.
- `d`: Repeating an unseeded test only hides how often it fails. 'Looks sensible' is not an assertion.
- `b|r2`: Right test, wrong reason. A seed buys reproducibility, not speed.

B19-L5 exercise 2

Hints. (1) 0.95 x 0.05 = 0.0475. (2) 0.0475 / 2000 = 0.00002375. Take the square root. (3) Four times 0.0049 is about 0.0195.

Answer.

```
1 0.0049 0.931 0.969
```

The standard error of a share from 2,000 simulations is about 0.0049, so four standard errors give the band 0.931 to 0.969.

Common wrong answers.

- `0.0049 0.94 0.96`: The band is four standard errors wide on each side: 4 x 0.0049 is about 0.0195.
- `0.0024 0.94 0.96`: 0.95 x 0.05 / 2000 is 0.00002375, and its square root is about 0.0049.

B19-L5 exercise 3

Hints. (1) Name the same method as your function. (2) Compare with a tolerance. (3) `wilson` and `assert_allclose`.

Answer.

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)
```

An independent library computing the same interval is an oracle. `assert_allclose` with a small `rtol` absorbs rounding differences.

Common wrong answers.

- g1 = `normal`; g2 = `assert_allclose`: `method="normal"` is the Wald interval. Compare like with like: `wilson`.
- g1 = `wilson`; g2 = `assert_array_equal`: Exact equality fails on tiny floating-point differences. Use a tolerance.

B19-L5 exercise 4

Hints. (1) `counts = rng.binomial(n, p, size=sims)` gives one success count per simulation. (2) For each count, `lo, hi = ci_fn(int(count), n)`, then check the **true** p: `lo <= p <= hi`. (3) Return the number covered divided by sims. Check sims first.

Answer.

```
from collections.abc import Callable

import numpy as np


def coverage(
    ci_fn: Callable[[int, int], tuple[float, float]],
    rng: np.random.Generator,
    p: float,
    n: int,
    sims: int,
) -> float:
```

```
    """Share of simulated samples whose interval covers the true
proportion.

    Args:
        ci_fn: Interval function taking (successes, n) and returning (lo,
hi).
        rng: Seeded generator.
        p: True proportion.
        n: Sample size per simulation.
        sims: Number of simulations.

    Returns:
        The coverage, between 0 and 1.

    Raises:
        ValueError: If sims is below 1.
    """
    if sims < 1:
        raise ValueError("sims must be at least 1")
    counts = rng.binomial(n, p, size=sims)
    covered = 0
    for count in counts:
        lo, hi = ci_fn(int(count), n)
        if lo <= p <= hi:
            covered += 1
    return covered / sims
```

Coverage asks how often the interval contains the true value. An interval almost always contains its own estimate, so checking the estimate instead of p hides a broken method.

B19-L5 exercise 5

Hints. (1) Where does each expected value come from? (2) Remove chance rather than retrying around it. (3) Size the tolerance from theory.

Answer.

- An independent oracle for the exact value.
- A seeded generator, never retries.
- A coverage band sized from the standard error.
- Every edge case the function documents.

An oracle, a seed, a theory-based tolerance and the documented edge cases make a statistical test both stable and sharp.

Common wrong answers.

- exact = `copy`: A value copied from the function checks only that it agrees with itself.

- random = `retry`: Retries hide failures. Remove chance with a seed instead.
- coverage = `pvalue`: A p-value threshold fails by chance one run in twenty, or passes a broken method when the simulation is small.

B19-L6 Testing machine learning pipelines

B19-L6 exercise 1

Hints. (1) Which assertion changes when the scaler sees test rows? (2) What does a fitted StandardScaler store? (3) Its means.

Answer. a) Fit on the training split, then assert the scaler's `mean_` equals `X_train.mean(axis=0)`. Because: r1) It checks what the pipeline learned, which differs exactly when test rows leak in.

A scaler fitted on all rows stores the overall mean. Comparing its `mean_` with the training mean detects that directly.

Common wrong answers.

- `b`: Leakage usually raises offline AUC, so this test passes more easily, not less.
- `c`: Probabilities sum to 1 with or without leakage.
- `d`: A leaky pipeline fits without errors.
- `a|r2`: Right test, wrong reason. It works because it inspects the fitted state, not because of speed.

B19-L6 exercise 2

Hints. (1) Which rows does fit see? (2) Does predict change what the scaler learned? (3) (1 + 3 + 5 + 7) / 4.

Answer.

```
4.0
```

`fit` learns the mean from the four training rows, 4.0. `predict` only applies it, so the stored mean stays 4.0.

Common wrong answers.

- `23.2`: That is the mean of all five values. The scaler learned from the four training rows only. predict uses it and changes nothing.
- `100.0`: predict transforms the test row with the training mean. It does not refit the scaler.

B19-L6 exercise 3

Hints. (1) A DummyClassifier strategy that predicts the class shares. (2) AUC needs probabilities. (3) `prior` and `predict_proba`.

Answer.

```
baseline = DummyClassifier(strategy="prior").fit(X_train, y_train)
base_auc = roc_auc_score(y_test, baseline.predict_proba(X_test)[:, 1])
model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
assert model_auc > base_auc + 0.2
```

`DummyClassifier(strategy="prior")` gives every row the same scores, so its AUC is 0.5. The model must beat that by a clear margin.

Common wrong answers.

- g1 = `prior`; g2 = `predict`: AUC needs scores, not class labels. Use predict_proba and take column 1.

B19-L6 exercise 4

Hints. (1) `perm = rng.permutation(len(X))`, then `X[perm]` is a shuffled copy. (2) Row i of `model.predict_proba(X[perm])` belongs to original row `perm[i]`. (3) Compare with `original[perm]` using `np.allclose`, and return a bool.

Answer.

```
import numpy as np


def order_invariant(model: object, X: np.ndarray, rng:
    np.random.Generator) -> bool:
    """True when each row's predicted probabilities do not depend on row
    order.

    Args:
        model: Fitted estimator with predict_proba.
        X: Feature matrix.
        rng: Seeded generator for the permutation.

    Returns:
        Whether shuffling the rows leaves each row's probabilities
    unchanged.
    """
    perm = rng.permutation(len(X))
    original = model.predict_proba(X)
    shuffled = model.predict_proba(X[perm])
    return bool(np.allclose(shuffled, original[perm]))
```

Indexing the original predictions with the same permutation lines every row up with its shuffled twin. Sorting the scores instead would hide a model that swaps them between rows.

B19-L6 exercise 5

Hints. (1) What differs between two runs of this test? (2) Check every call that draws random numbers for a seed. (3) One of them has none.

Answer. The bug is on line 9.

Fixed version:

```
1  from sklearn.datasets import make_classification
2  from sklearn.ensemble import RandomForestClassifier
3  from sklearn.metrics import roc_auc_score
4  from sklearn.model_selection import train_test_split
5
6
7  def test_model_beats_threshold():
8      X, y = make_classification(n_samples=300, random_state=0)
9      X_train, X_test, y_train, y_test = train_test_split(X, y,
       test_size=0.3, random_state=0, stratify=y)
10     model = RandomForestClassifier(n_estimators=50,
       random_state=0).fit(X_train, y_train)
11     auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
12     assert auc > 0.97
```

The split draws a new random set of test rows each run, so the AUC varies. Seed the split, and stratify it so both classes appear in the same shares.

Common wrong answers.

- Line 10: The forest has random_state=0, so it is deterministic for a given split.
- Line 8: make_classification has random_state=0, so the data is the same every run.
- Line 12: A fixed bar is fragile, but it would fail every run or none. Something changes between runs.

B19-L7 Isolate I/O with fakes, monkeypatch and mocks

B19-L7 exercise 1

Hints. (1) `from requests import get` copies the name into fetch.py. (2) Which name does load() look up when it runs? (3) The one in fetch.

Answer. b) `patch("fetch.get")` Because: r1) fetch.py looks up the name get in its own module, so that is the name to replace.

`from requests import get` creates the name `fetch.get`. Replace that name, where the code looks it up.

Common wrong answers.

- `a`: fetch.get still points at the original function, so the test makes a real request.
- `c`: patch needs a full dotted path to an importable name.
- `d`: fetch.py never imports the requests module itself, so there is no fetch.requests to patch.
- `b|r2`: Right patch, wrong reason. It works because that is where the code looks the name up.

B19-L7 exercise 2

Hints. (1) side_effect as a list: one item per call, in order. (2) An exception in the list is raised instead of returned. (3) Does a call that raises still count?

Answer.

```
10
error: down
2
```

The first call returns the dict, the second raises the ConnectionError, and the mock records both calls.

Common wrong answers.

- `10 error: down 1`: call_count counts every call, including the one that raised.
- `{'n': 10} error: down 2`: The first call returns the dict, and `["n"]` takes its value, 10.

B19-L7 exercise 3

Hints. (1) One test removes the variable, the other sets it. (2) Both methods end in `env`. (3) `delenv` and `setenv`.

Answer.

```python
def test_missing_token_raises(monkeypatch):
    monkeypatch.delenv("SURVEY_API_TOKEN", raising=False)
    with pytest.raises(RuntimeError, match="SURVEY_API_TOKEN"):
        load_token()


def test_token_is_read(monkeypatch):
    monkeypatch.setenv("SURVEY_API_TOKEN", "test-token")
    assert load_token() == "test-token"
```

`monkeypatch.delenv` and `monkeypatch.setenv` change the environment for one test, and pytest restores it afterwards.

Common wrong answers.

- g1 = `setenv`; g2 = `setenv`: The first test needs the variable gone. `delenv` removes it, and `raising=False` allows it to be missing already.

B19-L7 exercise 4

Hints. (1) Add `get` as a parameter and call it where `requests.get` was. (2) Check `response.status_code != 200` before reading the body. (3) Wrap the count in `int(...)`, and import `Callable` from `collections.abc` and `Any` from `typing`.

Answer.

```python
from collections.abc import Callable
from typing import Any

API = "https://survey.example/api/waves"


def wave_count(wave_id: str, get: Callable[..., Any]) -> int:
    """Number of respondents in one wave, from the survey API.

    Args:
        wave_id: Wave to look up.
        get: HTTP GET function, such as requests.get.

    Returns:
        The respondent count.

    Raises:
        RuntimeError: If the API does not answer with status 200.
    """
    response = get(f"{API}/{wave_id}", timeout=30)
    if response.status_code != 200:
        raise RuntimeError(f"wave API returned {response.status_code}")
    return int(response.json()["respondents"])
```

With `get` as a parameter, a test passes a fake response for each case: a normal count and an error status. Production passes `requests.get`.

B19-L7 exercise 5

Hints. (1) Read the first line of reports.py. (2) Which name does wave_count look up when it runs? (3) `reports.get`, not `requests.get`.

Answer. The bug is on line 14.

Fixed version:

```
# reports.py contains:
#     from requests import get
#
```

```
#     def wave_count(wave_id):
#         url = f"https://survey.example/api/waves/{wave_id}"
#         return get(url, timeout=30).json()["respondents"]

from unittest.mock import patch

import reports


def test_wave_count_reads_respondents():
    with patch("reports.get") as fake_get:
        fake_get.return_value.json.return_value = {"respondents": 812}
        assert reports.wave_count("W14") == 812
```

`from requests import get` binds `reports.get` to the real function at import. Patching `requests.get` changes a different name. Patch `reports.get`, or pass `get` in as a parameter.

Common wrong answers.

- Line 15: Setting the fake's return value is fine. The fake is never used, because of which name it replaced.
- Line 10: Importing reports is needed. The patch target is the problem.

B19-L8 Coverage, markers and a test suite you can trust in CI

B19-L8 exercise 1

Hints. (1) Every line ran. What did not happen? (2) An if has two ways out. (3) Which report counts both?

Answer. a) `--cov-branch` Because: r1) Branch coverage reports each if that never went one of its two ways, even when every line ran.

`--cov-branch` reports a missed jump such as `2->4` for an if whose false path never ran, even at 100% line coverage.

Common wrong answers.

- `b`: Line coverage was already 100%, so the floor passes. Lines ran. A path did not.
- `c`: That deselects slow tests. It measures nothing.
- `d`: Strict markers catch marker typos, not untested paths.
- `a|r2`: Right setting, wrong reason. Branch coverage adds measurement. It does not speed anything up.

B19-L8 exercise 2

Hints. (1) Which lines run when cap is 3? (2) The if has two branches. How many were taken? (3) Branch coverage counts statements and branches together: (4 + 1) / (4 + 2).

Answer.

```
100 83
```

All four statements run, so line coverage is 100%. With branches, 5 of 6 items are covered, 83%, and the report lists the missed jump `2->4`.

Common wrong answers.

- `75 67`: Every line runs when cap=3: the if, the assignment and the return. Line coverage is 100%.
- `100 100`: The if never takes its false path, the jump from line 2 straight to line 4. Branch coverage counts that.
- `100 50`: pytest-cov counts statements and branches together: 4 of 4 statements plus 1 of 2 branches is 5 of 6, 83%.

B19-L8 exercise 3

Hints. (1) The flag names what it makes strict. (2) The marker name goes before the colon. (3) `--strict-markers` and `slow`.

Answer.

```
[tool.pytest.ini_options]
addopts = "--strict-markers"
markers = ["slow: takes more than a few seconds"]
```

Registered markers document themselves, and `--strict-markers` turns a typo into an error instead of a silently unused marker.

Common wrong answers.

- g1 = `--strict`; g2 = `slow: --strict` is not the option name here. The command-line flag is `--strict-markers`. In config, pytest 9 also offers `strict = true`.
- g1 = `-m slow`; g2 = `slow: -m slow` would run only slow tests on every run. Registering markers strictly is `--strict-markers`.

B19-L8 exercise 4

Hints. (1) Four return statements need at least four cases, including None. (2) A score of 0.9 runs the high branch but cannot tell `>=` from `>`. (3) Test exactly 0.8 and exactly 0.5, the values where the two versions disagree.

Answer.

```python
import pytest


@pytest.mark.parametrize(
    ("score", "expected"),
    [
        pytest.param(None, "unknown", id="missing"),
        pytest.param(0.8, "high", id="at-0.8"),
        pytest.param(0.79, "medium", id="below-0.8"),
        pytest.param(0.5, "medium", id="at-0.5"),
        pytest.param(0.49, "low", id="below-0.5"),
    ],
)
def test_risk_band(score: float | None, expected: str) -> None:
    assert risk_band(score) == expected
```

Cases at 0.9, 0.6, 0.1 and None reach every line, yet miss both off-by-one changes. Only values on the thresholds tell `>=` from `>`. Coverage shows what ran. Mutants show what was checked.

B19-L8 exercise 5

Hints. (1) Where must everything run? (2) What keeps the local loop fast? (3) Fix causes, raise floors gradually.

Answer.

- Pull requests run everything, with branch coverage and a floor.
- Locally, deselect slow tests and iterate with -x and --lf.
- Flaky tests get a root cause, never a retry.
- The floor starts where the team is and only rises.

CI runs the whole suite with branch coverage and a rising floor. Developers run the fast subset. Flaky tests are fixed at the cause.

Common wrong answers.

- flaky = `retry`: A retry hides the failure and whatever causes it. Seeds, isolation and fakes remove the cause.
- floor = `hundred`: A 100% demand produces tests that run lines and assert nothing. Set a floor and raise it.
- local = `full`: A slow loop makes people stop running tests. Keep the full run for CI.

B18-L1 Docstrings and READMEs that answer the next person's questions

B18-L1 exercise 1

Hints. (1) What would surprise a caller? (2) What can the signature not say? (3) Edge behaviour.

Answer. b) "Returns None when the weights sum to 0." Because: r1) It tells callers about a behaviour the signature and name cannot show.

Document what a caller must know and cannot see: edge cases, units, grain.

Common wrong answers.

- `a`: The code already shows that. Docstrings explain what the code cannot.
- `c`: Ownership belongs in the README or CODEOWNERS, not every docstring.
- `d`: It adds nothing.
- `b|r2`: Right answer, wrong reason. Value comes from information, not length.

B18-L1 exercise 2

Hints. (1) Where does Python store a function's docstring? (2) In **doc**. (3) It prints the string.

Answer.

```
part / total, or None when total is 0.
```

The first string in the body is the docstring. `help()`, editors and documentation tools read it from `__doc__`.

Common wrong answers.

- `None`: A string literal as the first statement of a function becomes its **doc**.

B18-L1 exercise 3

Hints. (1) What does a newcomer need first? (2) Then how to run it. (3) Details after.

Answer.

1. Purpose and users
2. Quick start: install and run commands
3. Inputs and outputs, with grain and schedule
4. Configuration

5. Tests and checks
6. Owners and support

Purpose, run, data, settings, checks, contacts: the order a newcomer's questions arrive in.

Common wrong answers.

- Say what the project is before telling people to run it.
- The quick start is what most readers came for. Put it near the top.

B18-L1 exercise 4

Hints. (1) Summary line, blank line, then Args:, Returns:, Raises:. (2) Indent each entry under its section. (3) Mention missing weights under weights.

Answer.

```python
import pandas as pd


def cap_weights(weights: pd.Series, cap: float) -> pd.Series:
    """Cap survey weights at a maximum value.

    Args:
        weights: One weight per respondent. Missing weights stay missing.
        cap: Largest allowed weight, at least 1.

    Returns:
        A new Series with every weight above cap set to cap.

    Raises:
        ValueError: If cap is below 1.
    """
    if cap < 1:
        raise ValueError("cap must be at least 1")
    return weights.clip(upper=cap)
```

The docstring now tells a caller the bounds, the edge behaviour and the failure mode, none of which the signature shows.

B18-L1 exercise 5

Hints. (1) Who reads it, and where? (2) Will the value change? (3) How local is the explanation?

Answer.

- Behaviour a caller must handle: the docstring.
- How to run the project: the README.
- Why a specific line is unusual: an inline comment next to it.

- A setting that may change: config (lesson A1-L3), described in the README.

Put information where its reader looks, and settings where they can change without editing text.

Common wrong answers.

- none_meaning = `readme`: Callers read the function's docstring, often in their editor, not the README.
- threshold = `docstring`: A value that can change belongs in config. Hardcoding it in prose invites drift.
- why_line = `docstring`: A workaround on one line is explained best right beside that line.

B18-L2 Model cards that state limits as clearly as scores

B18-L2 exercise 1

Hints. (1) Which section says 'do not use for'? (2) Reuse is a scope question. (3) Out of scope.

Answer. b) Out-of-scope uses Because: r1) It names the uses the model was not built or validated for.

Out-of-scope uses turn assumptions in the builders' heads into a rule readers see.

Common wrong answers.

- `a`: Details identify the model. They do not say where it must not be used.
- `c`: Evaluation shows how it performs on its intended task, not on others.
- `d`: Monitoring tracks the model in use. It does not stop misuse.
- `b|r2`: Right answer, wrong reason. Length does not matter. Its purpose does.

B18-L2 exercise 2

Hints. (1) Which keys are absent? (2) Which are present but blank? (3) Both count.

Answer.

```
1 ['out_of_scope', 'subgroups', 'limitations']
```

Empty sections are as unhelpful as missing ones. A CI check should reject both.

Common wrong answers.

- `['out_of_scope', 'subgroups']`: limitations exists but is empty. The check treats blank text as missing.

B18-L2 exercise 3

Hints. (1) The baseline for average precision. (2) How much evidence stands behind the number? (3) Prevalence and row count.

Answer.

```
Evaluation: AUC 0.78, average precision 0.21 (prevalence 2.4%), test Oct to Dec 2026, 51,000 rows
```

Prevalence gives the baseline and the row count gives the evidence behind the metric.

Common wrong answers.

- g1 = `accuracy`: Accuracy is another metric. Average precision needs the prevalence as its baseline.

B18-L2 exercise 4

Hints. (1) Loop over required, not over card. (2) card.get(name, "") handles absent keys. (3) Use .strip() to treat whitespace as empty.

Answer.

```python
def missing_sections(card: dict[str, str], required: list[str]) -> list[str]:
    """Required model card sections that are absent or blank.

    Args:
        card: Section name to text.
        required: Section names that must have content.

    Returns:
        Missing section names, in the order of required.
    """
    return [name for name in required if not card.get(name, "").strip()]
```

A small, strict check turns the template into a gate: no model ships with an empty limitations section.

B18-L2 exercise 5

Hints. (1) What must users know to use the model safely? (2) What happens to hidden weaknesses? (3) Who reviews, and when?

Answer.

- Per-group rates with counts, so the weakest group is visible.

- The gap stated plainly with a mitigation.
- A team and a date that make review happen.

A card is useful when it makes weaknesses and responsibilities impossible to miss.

Common wrong answers.

- subgroups = `overall`: An overall rate hides a group at half the recall (lesson B12-L3).
- limit = `omit`: Hiding a known weakness moves the harm onto the people the model serves.
- review = `adhoc`: Without a date and an owner, reviews do not happen.

B18-L3 Decision logs, so the reasons outlive the meeting

B18-L3 exercise 1

Hints. (1) Why keep past reasoning? (2) How would a reader know it no longer applies? (3) Supersede and link.

Answer. c) Add a new entry, and mark the old one superseded with a link to the new one. Because: r1) The log keeps the history of reasoning. Each entry records what was known when it was written.

Supersede, never rewrite. The chain of entries shows how and why thinking changed.

Common wrong answers.

- `a`: Editing erases why the first choice made sense at the time.
- `b`: Deleting loses the history that explains past results.
- `d`: Without links, readers find the old entry and think it still applies.
- `c|r2`: Right answer, wrong reason. The point is preserving reasoning, not storage.

B18-L3 exercise 2

Hints. (1) Which entries are accepted? (2) Does the superseded entry point to an existing one? (3) Yes: D-007.

Answer.

```
1 ['D-007'] True
```

Current decisions are the accepted ones. Every superseded entry should point to an existing replacement.

Common wrong answers.

- `['D-003', 'D-007'] True`: D-003 is superseded, so it is not a current decision.

B18-L3 exercise 3

Hints. (1) Problem first. (2) Then the choices. (3) Then the pick and its cost.

Answer.

1. ID and title
2. Date and status
3. Context: the problem and the evidence
4. Options considered
5. Decision
6. Consequences and review point

Context, options, decision, consequences: the order of the reasoning itself.

Common wrong answers.

- State the problem and options before the choice, so the reader can judge it.
- Options come before the decision that chooses among them.

B18-L3 exercise 4

Hints. (1) Use entry.get(...) so missing keys do not raise. (2) Check the rules in the order given and append a message for each failure. (3) For superseded entries, test membership in log_ids.

Answer.

```
VALID_STATUS = {"proposed", "accepted", "superseded"}


def decision_problems(entry: dict[str, object], log_ids: set[str]) ->
    list[str]:
    """Problems with one decision log entry, in rule order.

    Args:
        entry: Parsed decision entry.
        log_ids: IDs of all entries in the log.

    Returns:
        Problem messages. Empty when the entry is complete.
    """
    problems = []
    status = entry.get("status")
    if status not in VALID_STATUS:
```

```
        problems.append("invalid status")
    options = entry.get("options")
    if not isinstance(options, list) or len(options) < 2:
        problems.append("fewer than two options")
    consequences = entry.get("consequences")
    if not isinstance(consequences, str) or not consequences.strip():
        problems.append("no consequences")
    if status == "superseded" and entry.get("superseded_by") not in
log_ids:
        problems.append("superseded without a valid replacement")
    return problems
```

A validator in CI keeps the log useful: every entry has real options and consequences, and every superseded entry leads somewhere.

B18-L3 exercise 5

Hints. (1) Where does the next analyst look? (2) When are the reasons clearest? (3) How does a reader meet the decision?

Answer.

- In the repository, versioned with the code it explains.
- Written with the change, while the reasons are fresh.
- Linked from where people meet the decision.

Records are useful when they are versioned, timely and linked.

Common wrong answers.

- where = `slides`: Slides are hard to find, hard to search and not versioned with the code.
- when = `later`: By the end, the options and reasons are half forgotten.
- link = `search`: People do not search for a record they do not know exists. Link it from the value.